Project 2 – Acme Analytics Linux Platform

Complete report so far · Days 1–14 · 13 October – 7 November 2026

Muhammad Bilal Javed

Goal	A hardened, observable and recoverable Linux platform for a small analytics company, built by hand in VMware
Servers	gw01 192.168.60.1 (gateway) · app01 .10 (application) · stor01 . 20 (storage) · svc01 .40 (service desk)
Repository	github.com/muhammadbilaljaved684-dotcom/Linux-Platform

Where the project stands

Phase	Topic	Status
0	Know the system: kernel, boot, targets	🟡 gw01 done
1	Network foundation: static IPs, routing, NAT, names, SSH, users, inodes	✅ complete – snapshot `20-Network`
1A	Service desk: GLPI tickets and inventory	✅ GLPI in Docker, inventory of all 4 servers, first Change and Request tickets
2	Storage: LVM, NFS, backups, snapshots	✅ LVM pool, NFS share, hard-linked backup versions, restore via ticket, snapshot rollback
3	Automation: systemd services and timers	✅ job control, disk-full alert timer, socket activation, custom target, cron vs timer, nightly backup by timer
4	Processes, signals, cgroups	🟡 Step 4.1 ✅ runaway process · Step 4.2 ✅ pipes · Step 4.3 ✅ queue system and worker (L3 Problem open) · Step 4.4 ✅ processes and zombies · Step 4.5 🟡 cgroups concept
5 / 5A	Logging · monitoring and alerting	⏳
6 / 6A	Security hardening · WireGuard VPN	⏳
7 / 7A	Publish the web service · Ansible	⏳
8	Acceptance tests and disaster drill	⏳

Contents

Part	What is inside
Day 1 – 13 Oct	The Acme story · Phase 0 baseline (kernel, drivers, memory, boot, server mode) · Phase 1 part 1: static addresses, gw01 as router with NAT and firewall
Day 2 – 15 Oct	Steps 1.5–1.9: SSH keys, hard and soft links, inodes, users and groups, service account · Linux NAT vs cloud NAT · end of Phase 1
Day 3 – 16–17 Oct	Phase 1A: why a service desk · svc01 settings and install · LVM disk growth · the network story · Docker + GLPI · VMnet2 host adapter · GLPI agent inventory
Day 4 – 18 Oct	GLPI clean-up, Change #1, ITIL and SLA · LVM from empty disks, fstab · NFS · tar/rsync backups with versions · restore drill via ticket
Day 5 – 20 Oct	NFS access test · LVM snapshot rollback and stale-mount fix · job control with real uploads · disk-full alert as systemd service + timer · socket activation
Day 6 – 21 Oct	systemd units revised in my own words · Step 3.4 custom `acme-stack.target` · a look inside `multi-user.target` · cron vs timer · nightly backup by timer – Phase 3 complete
Day 7 – 24 Oct	A question from industry: backup and log growth · incident-drill plan · Phase 4 Step 4.1: primary/replica, ETL, files vs records · hands-on on app01: runaway process, load average, PID/PPID, SIGTERM · gw01-off incident · GLPI ticket #2 · Step 4.2 pipes: tar
Day 8 – 25 Oct	Step 4.3: signals, lock, trap · queue systems, producers and workers · the Acme archive queue system on stor01 (queue, producer, worker service at nice 19, FIFO, stop mid-job without loss) · L1 → L2 escalation with GLPI ticket #3
Day 9 – 26 Oct + 1 Nov	Step 4.3 clean-up (nightly backup OK, /backups 26 %) · Step 4.4: the PID pool, fork / exec / wait, boot chain systemd → getty → login → bash, hands-on on app01 (family tree, background child, grandchild, exec, orphan) · normal vs orphan vs zombie
Day 10 – 1 Nov	Step 4.5 concept: nice vs cgroup limits · systemd creates one box per service · CPUQuota throttling · MemoryHigh (cache, swap) vs MemoryMax (OOM killer) · TasksMax · the same limits in Docker and Kubernetes · who creates the box (systemd, dockerd/ kubelet → containerd → runc) · container networking vs gw01 NAT · lab Step 1 on app01: the cgroup tree (init.scope, system.slice, user.slice), the 7 places a limit can go, where limits are stored (drop-in files, set once – every login covered)
Day 11 – 3 Nov	Step 4.5 lab Step 2: hard memory limit on app01 – systemd-run box, stress-ng, OOM killer, CONSTRAINT_MEMCG · how Linux uses disk and RAM: kernel copied at boot, systemd reads /etc and builds the live things, page cache, mount = connect a folder to a source · live test vs permanent – where I write it decides · evening: how the OOM killer chooses (oom_score, oom_score_adj, OOMScoreAdjust, OOMPolicy) · test #2 soft limit MemoryHigh – slowed into swap, not killed
Day 12 – 4 Nov	Step 4.5 lab test #3: CPU limit CPUQuota=20% on app01 – 100 % → 20.5 %, cpu.max 20000 100000, throttled not killed, each box its own limit · console tricks (& sleep 2; top, sudo -v) · revision sheet of the 3 limit tests
Day 13 – 4 Nov	Step 4: permanent limits on stor01 (set-property → drop-in files, proven after reboot) · first incident drill: deleted file still open (df vs du, lsof +L1) · GLPI Incident #4 + L3 Problem behind ticket #3 · snapshot 50-processes – Phase 4 complete

Next

1. Phase 5: logging – journald, rsyslog, logrotate.
2. Phase 2 tickets in GLPI.

Day 1 – 13 October 2026

The story · Phase 0: know the system · Phase 1: network foundation (part 1)

Project 2 – Acme Analytics Linux Platform: the Day 1 story

Build story and history · Day 1 · 13 October 2026 · Muhammad Bilal Javed

> The same story as a document to download: Project-2-Day-1-story.odt. The detailed technical reports are part 1 and part 2.

How to read this document

This is the story of the first day of building a small company's Linux platform in a home lab. It follows the order in which the work really happened, including the mistakes. Every chapter answers four questions:

- **Where are we?** The situation before the step.
- **What did I do, and why?** The commands, with a plain-language explanation.
- **What does the screenshot prove?** The real output from my lab.
- **Why do we go to the next step?** What this step made possible, or what problem it revealed.

You do not need to know Linux to follow it. Technical words are explained when they first appear, and a short glossary is at the end.

The story in one paragraph

Acme Analytics is a small company whose server setup has grown messily: nobody controls what reaches the internet, nothing is documented, and nobody could rebuild a server the same way twice. My job is to build a clean, controlled platform. On Day 1 I first **recorded how the servers looked** before touching anything. Then I built the **network foundation**: every server got a fixed address, the two internal servers lost their own internet connection, and the gateway server gw01 became the **one and only door** to the internet. At the end of the day, app01 – a server with no internet card of its own – could reach the internet through gw01, the outside world saw only gw01's address, and the servers could find each other by name. Along the way I hit and solved ten real problems.

Chapter 0 – The plan and the machines

Where are we?

The goal is the platform shown below. It is built in phases (P0, P1, …); Day 1 covers Phase 0 and most of Phase 1 – the boxes **gw01**, the **VMnet2 lab network**, **app01** and **stor01**.

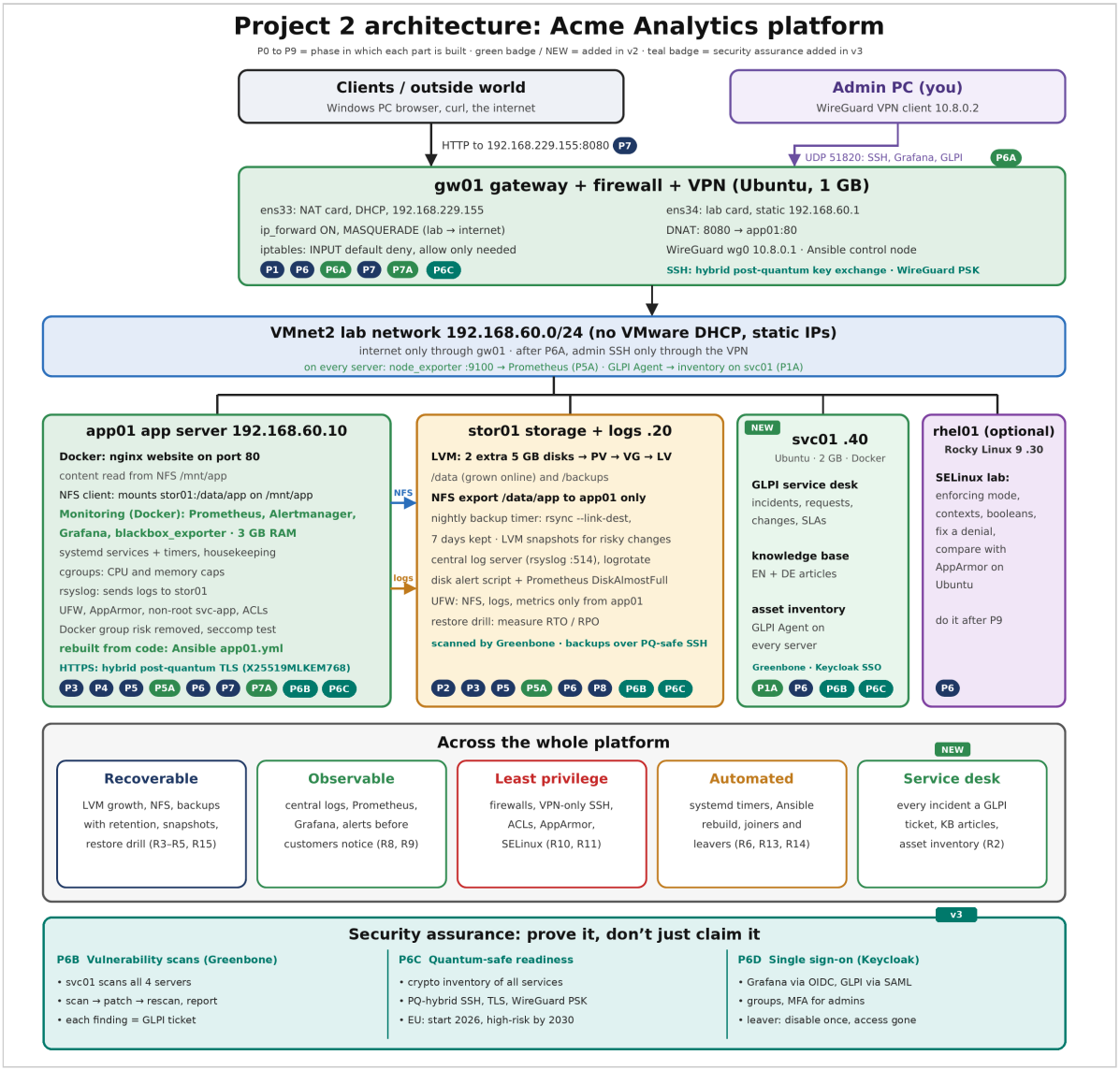


Figure 1 – Target architecture of the whole project. Day 1 builds the gateway, the lab network and the addresses of app01 and stor01.

Instead of buying servers, I run three **virtual machines** (computers simulated by software) in VMware. Think of them as three employees with different jobs:

Server	Job in plain words	Job in technical words
gw01	The gatekeeper – the only door to the internet	Gateway: router, NAT, firewall, later VPN
app01	The shop window – will run the website	Application server
stor01	The archive room – files, backups, logs	Storage and log server

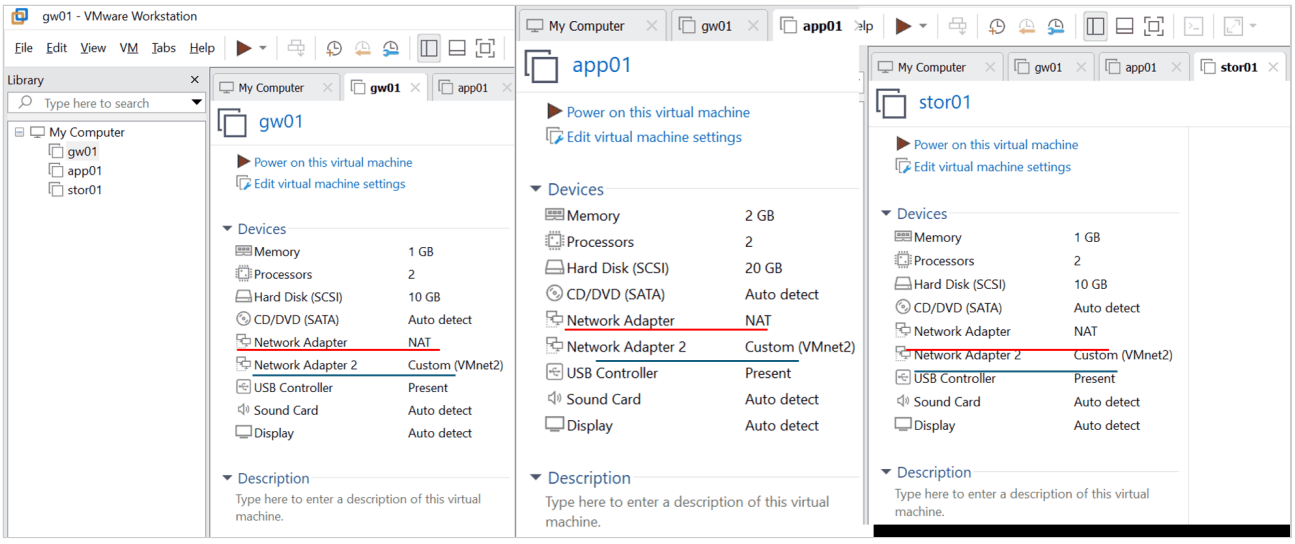


Figure 2 – The three VMs in VMware. Each has two network cards: "Network Adapter" (NAT = internet through Windows) and "Network Adapter 2" (VMnet2 = the private lab network).

Why do we go to the next step?

Before changing anything, a good engineer writes down how the system looks *now*. Otherwise, when something breaks later, there is nothing to compare against.

Chapter 1 – Know the system (Phase 0)

Step 0.1 – Engine, drivers and memory

What I did and why. The **kernel** is the engine of Linux. **Modules** are drivers – small add-ons that let the engine use hardware. I recorded the engine version, the loaded drivers, the memory, and one important switch: whether gw01 may pass traffic between networks.

```
uname -r                    # kernel version only
uname -a                    # everything: name, kernel, build date, CPU type
cat /proc/version           # the kernel's own description (compiler, build)
lsmod | head                # first 10 loaded drivers
free -h                     # memory and swap, in human-readable units
sysctl net.ipv4.ip_forward  # may this machine route traffic? (0 = no, 1 = yes)
```

```
bilal_001@gw01:~$ uname -r
5.15.0-198-generic
bilal_001@gw01:~$ uname -a
Linux gw01 5.15.0-198-generic #208-Ubuntu SMP Fri Sep 4 10:22:38 UTC 2026 x86_64 x86_64 x86_64 GNU/L
inux
bilal_001@gw01:~$ cat /proc/version
Linux version 5.15.0-198-generic (buildd@lcy02-amd64-040) (gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4
.0, GNU ld (GNU Binutils for Ubuntu) 2.38) #208-Ubuntu SMP Fri Sep 4 10:22:38 UTC 2026
bilal_001@gw01:~$ lsmod | head
Module                  Size  Used by
vsock_loopback         16384  0
vmw_vsock_virtio_transport_common    40960  1 vsock_loopback
vmw_vsock_vmci_transport    32768  1
vsock                  49152  5 vmw_vsock_virtio_transport_common,vsock_loopback,vmw_vsock_vmci_tran
sport
intel_rapl_msr         20480  0
binfmt_misc            24576  1
vmw_balloon            24576  0
intel_rapl_common      40960  1 intel_rapl_msr
rapl                   20480  0
bilal_001@gw01:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       299Mi       1.3Gi       1.0Mi       331Mi       1.4Gi
Swap:          1.3Gi          0B       1.3Gi
bilal_001@gw01:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
bilal_001@gw01:~$ _
```

Figure 3 – Baseline of gw01: kernel 5.15.0-198, VMware drivers loaded, 1.9 GiB memory, and ip_forward = 0.

What the screenshot proves.

- Kernel `5.15.0-198-generic` – the standard Ubuntu 22.04 engine.
- Drivers `vmw_vsock`, `vmw_balloon` – the system knows it is a VMware virtual machine.
- Memory: 1.9 GiB, only 299 MiB used – plenty for a gateway.
- **`ip_forward = 0`** – gw01 is still an ordinary computer, *not* a router. Remember this number: changing it to 1 is the turning point of the afternoon.

Step 0.2 – How the machine starts

What I did and why. Starting a computer is a relay race: firmware → bootloader (GRUB) → kernel → systemd (the manager) → services → login. I measured how long it takes and saved the details to files as evidence.

```
systemd-analyze                           # total boot time
systemd-analyze blame >> boot-gw01.txt    # time per service, saved to a file
journalctl -b | head -15 >> journal-gw01.txt  # first log lines of this boot, saved
ls /boot                                  # the files used to start Linux
```

```
bilal_001@gw01:~$ systemd-analyze
Startup finished in 17.022s (kernel) + 21.095s (userspace) = 38.117s
graphical.target reached after 19.809s in userspace
bilal_001@gw01:~$ systemd-analyze blame >>boot-gw01.txt
bilal_001@gw01:~$ journalctl -b | head -15 >>journal-gw01.txt
bilal_001@gw01:~$ ls /boot
config-5.15.0-198-generic       initrd.img.old                  vmlinuz-5.15.0-198-generic
grub                            lost+found                      vmlinuz.old
initrd.img                      System.map-5.15.0-198-generic
initrd.img-5.15.0-198-generic   vmlinuz
bilal_001@gw01:~$ _
```

Figure 4 – Boot takes 38.1 s (17.0 s kernel + 21.1 s services). /boot holds the current kernel and the previous one as a fallback.

What the screenshot proves. The machine boots in **38.1 seconds** and ends in `graphical.target` (desktop mode). `/boot` keeps an **older kernel (`vmlinuz.old`) as a safety net** if an update fails.

Step 0.3 – Server mode instead of desktop mode

What I did and why. Linux can start in different modes, called **targets**. A desktop (`graphical.target`) is wasted on a server nobody sits in front of. Server mode (`multi-user.target`) runs everything needed – network, services, logins – without the desktop: faster, lighter and with fewer things an attacker could abuse.

```
systemctl get-default                          # which mode now?
sudo systemctl set-default multi-user.target   # switch to server mode
systemctl get-default                          # check
```

```
bilal_001@gw01:~$ systemctl get default
Unknown command verb get.
bilal_001@gw01:~$ systemctl get-default
graphical.target
bilal_001@gw01:~$ sudo systemctl set-default multi-user.target
[sudo] password for bilal_001:
Created symlink /etc/systemd/system/default.target → /lib/systemd/system/multi-user.target.
bilal_001@gw01:~$ systemctl get-default
multi-user.target
```

Figure 5 – gw01 switched from graphical.target to multi-user.target. The first line shows my typo "get default" (the command is get-default).

Why do we go to the next step?

I now know how gw01 looks. The next question is: **how are the three machines connected?** That is the network – and without a working network nothing else in the project (storage, monitoring, VPN, automation) can work.

Chapter 2 – The starting point of the network

Where are we?

Before configuring anything, I looked at each machine's network cards (`ip -br a`) and disks (`lsblk`).

```
hostname    # which machine am I on?
ip -br a    # network cards: name, state, address (brief form)
lsblk       # disks and partitions
```

```
bilal_001@gw01:~$ hostname
gw01
bilal_001@gw01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.155/24 metric 100 fe80::20c:29ff:feac:fa08/64
ens34            DOWN
bilal_001@gw01:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0 63.9M  1 loop /snap/core20/2318
loop1                       7:1    0 63.8M  1 loop /snap/core20/2922
loop2                       7:2    0   87M  1 loop /snap/lxd/29351
loop3                       7:3    0 38.8M  1 loop /snap/snapd/21759
loop4                       7:4    0 44.7M  1 loop /snap/snapd/28254
sda                         8:0    0   10G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0  1.8G  0 part /boot
└─sda3                      8:3    0  8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0  8.2G  0 lvm  /
sr0                        11:0    1    2G  0 rom
bilal_001@gw01:~$ _
```

Figure 6 – gw01 at the start: ens33 (internet side) has 192.168.229.155; ens34 (lab side) is DOWN with no address.

```
bilal_002@app01:~$ hostname
app01
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.152/24 metric 100 fe80::20c:29ff:fee7:59c4/64
ens34            DOWN
bilal_002@app01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0  63.8M  1 loop /snap/core20/2922
loop2                       7:2    0    74M  1 loop /snap/core22/2955
loop3                       7:3    0    87M  1 loop /snap/lxd/29351
loop4                       7:4    0 115.2M  1 loop /snap/lxd/40911
loop5                       7:5    0  38.8M  1 loop /snap/snapd/21759
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    20G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0  18.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0    10G  0 lvm  /
sr0                        11:0    1  1024M  0 rom
bilal_002@app01:~$ _
```

Figure 7 – app01 at the start: it ALSO has its own internet address (192.168.229.152) on ens33; the lab card ens34 is DOWN.

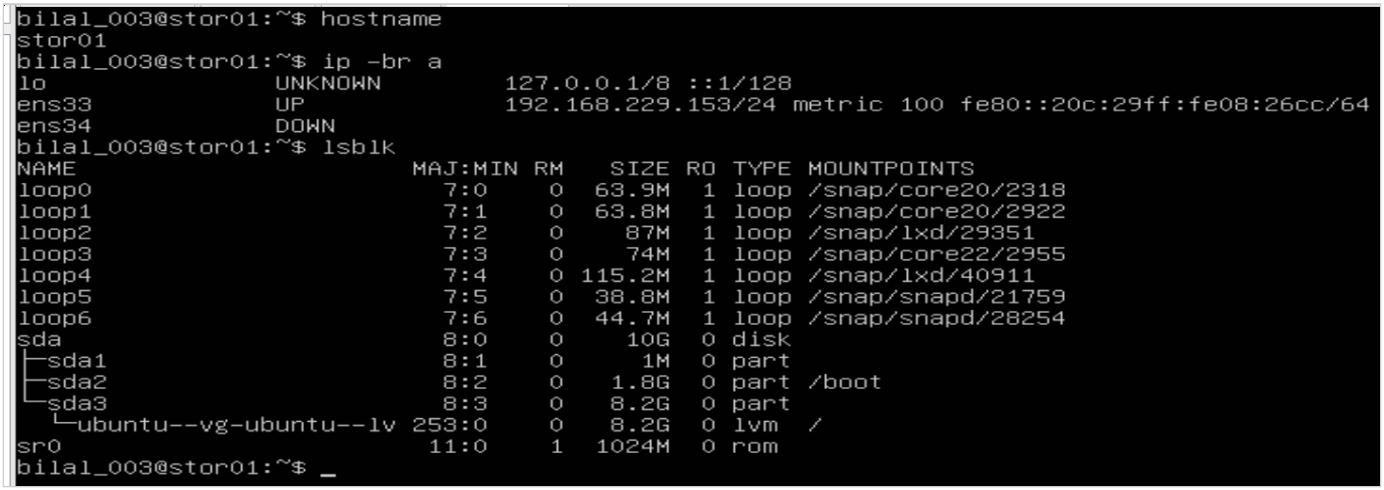


Figure 8 – stor01 at the start: its own internet address (192.168.229.153), lab card DOWN, and only one disk (sda) – the two extra data disks are missing.

What the screenshots reveal – three problems

What I saw	Why it is a problem
ens34 is **DOWN** on all three machines	The lab network is unused – the servers cannot talk to each other
app01 and stor01 have their **own internet address** on ens33	Breaks the design: every server has its own uncontrolled door to the internet
stor01 has only **sda**	The storage phase needs two extra empty disks

`lo` (loopback) is the machine talking to itself and is always there – it can be ignored.

Why do we go to the next step?

The servers need **fixed addresses** on the lab network, like permanent desk-phone numbers. Every later part of the project – firewall rules, file sharing, monitoring, automation – refers to these numbers. If they changed, everything would break.

Chapter 3 – Fixed addresses for every server (Step 1.1)

The method (same on every server)

On Ubuntu, network settings are stored in a small text file in `/etc/netplan/`. The safe method has six moves:

1. **Find the file** – `ls /etc/netplan`
2. **Back it up** – `sudo cp … ~/netplan-backup.yaml`, so I can always go back.
3. **Stop cloud-init from overwriting it.** `cloud-init` is a helper that configures the network at first boot – and may silently *rewrite my file* at a later boot. I tell it "network: hands off".
4. **Write the new settings** in the file. The format is **YAML**, where indentation (spaces) is part of the meaning.
5. **Apply with an automatic undo** – `sudo netplan try`. It applies the change and **undoes it after 120 seconds** unless I press Enter. If I can still type, the change works.
6. **Reboot and check** – a setting that disappears after a restart is worthless.

3.1 gw01 – the gatekeeper gets its inside address

```
ls /etc/netplan                                                  # -> 50-cloud-init.yaml
sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml  # backup
echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
```

Figure 9 – gw01: the real file is 50-cloud-init.yaml, backed up, and cloud-init's network handling disabled.

First surprise: my guide expected a file called `00-installer-config.yaml`. The real file is `50-cloud-init.yaml`. Lesson: *check, don't assume.*

The new settings for gw01: the internet card keeps getting its address automatically; the lab card gets the fixed address **192.168.60.1**. gw01 needs no gateway on the lab side – it *is* the gateway.

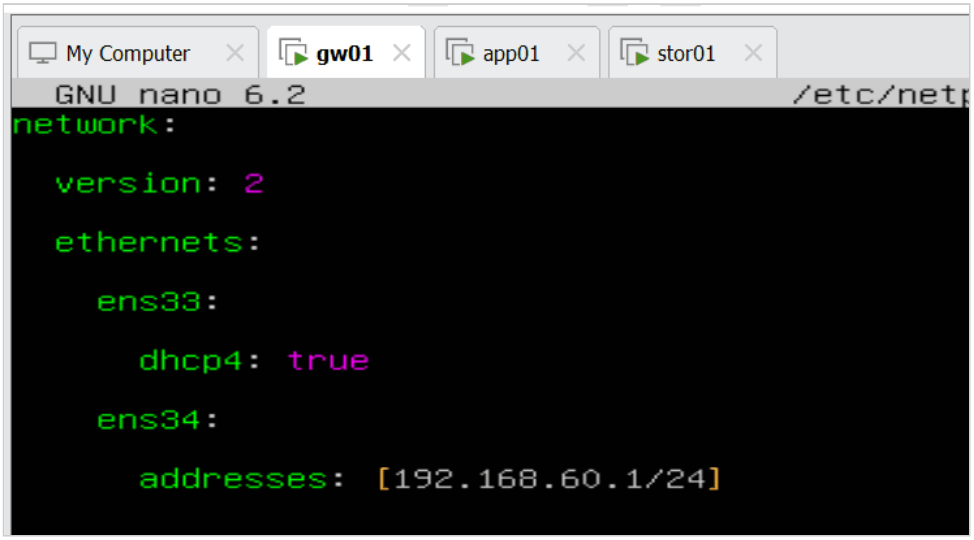


Figure 10 – gw01's netplan file, typed in the nano editor.

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml   # only the administrator may read/write it
sudo netplan try                                 # apply with automatic undo; Enter = keep
ip -br a                                         # check
```

Figure 11 – netplan try accepted; ens34 on gw01 now has 192.168.60.1/24. The Open vSwitch warning is harmless (that software is not installed).

Result: gw01 now has two addresses – one facing the internet, one facing the lab.

3.2 app01 – the shop window, and an unplugged cable

app01 gets the fixed address **192.168.60.10**, and a **default route**: "for any address you don't know, ask gw01 (192.168.60.1)". It also gets DNS servers (1.1.1.1 and 8.8.8.8) – the internet's phone book, which turns names into numbers.

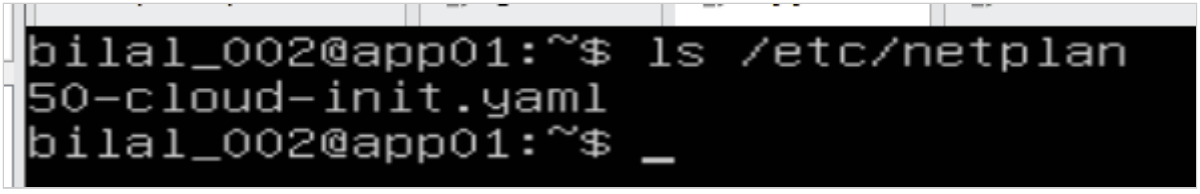


Figure 12 – app01 also uses 50-cloud-init.yaml.

```
50-cloud-init.yaml
bilal_002@app01:~$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml
bilal_002@app01:~$ echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-n
etwork-config.cfg
#network: {config: disabled}
bilal_002@app01:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:

  version: 2

  ethernets:

    ens34:

      addresses: [192.168.60.10/24]

      routes:

        - to: default

          via: 192.168.60.1

      nameservers:

        addresses: [1.1.1.1, 8.8.8.8]
bilal_002@app01:~$ sudo chmod 600 /etc/netplan/50-cloud-init.yaml
bilal_002@app01:~$ sudo netplan try
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
Do you want to keep these settings?


Press ENTER before the timeout to accept the new configuration


Changes will revert in 119 seconds
Configuration accepted.
```

Figure 13 – app01: backup, cloud-init line, the new settings (192.168.60.10, default route via gw01, DNS servers) and netplan try. Note the "#" at the start of the cloud-init line – a typing slip that turns the line into a comment.

```
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.152/24 metric 100 fe80::20c:29ff:fee7:59c4/64
ens34            UP             192.168.60.10/24 fe80::20c:29ff:fee7:59ce/64
bilal_002@app01:~$ ping -c 3 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=10.8 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=0.755 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=1.33 ms

--- 192.168.60.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 0.755/4.287/10.779/4.596 ms
bilal_002@app01:~$
```

Figure 14 – app01 can now reach gw01 (3 of 3 replies) – the lab network works. BUT ens33 still has its own internet address 192.168.229.152.

Second surprise. The lab network worked, but app01 still had its own internet line. Configuring Linux alone was not enough: the **virtual cable was still plugged in** in VMware. The real fix is to unplug it in the VM's settings – untick **Connected** and **Connect at power on**.

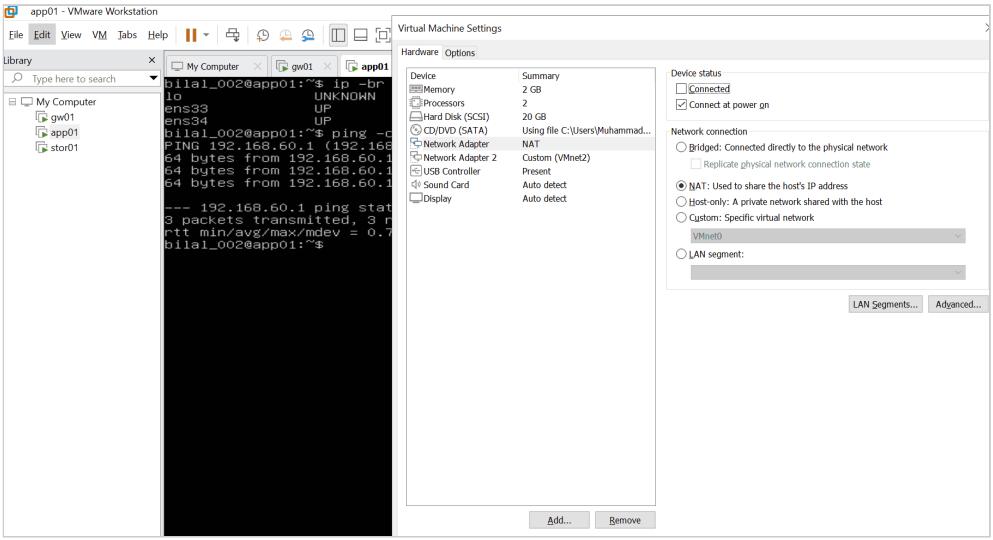


Figure 15 – Unplugging app01's internet adapter in VMware (Connected unticked).

Before rebooting I also checked the cloud-init file, because of the # seen in Figure 13. It now reads correctly, without the #:

```
bilal_002@app01:~$ sudo cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
[sudo] password for bilal_002:
network: {config: disabled}
bilal_002@app01:~$ sudo reboot_
```

Figure 16 – The cloud-init file is correct: "network: {config: disabled}", no "#".

```
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            DOWN
ens34            UP             192.168.60.10/24 fe80::20c:29ff:fee7:59ce/64
bilal_002@app01:~$ ip route
default via 192.168.60.1 dev ens34 proto static
192.168.60.0/24 dev ens34 proto kernel scope link src 192.168.60.10
bilal_002@app01:~$ sudo cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
network: {config: disabled}
```

Figure 17 – app01 after reboot: ens33 DOWN (unplugged), ens34 = 192.168.60.10, default route via gw01, cloud-init file still correct.

What Figure 17 proves:

Line	Meaning
`ens33 DOWN`	The private internet line is gone
`ens34 UP 192.168.60.10/24`	The fixed address survived the reboot
`default via 192.168.60.1`	Everything not local is sent to gw01

A note on VMnet2. The lab network VMnet2 does not appear in VMware's Virtual Network Editor (only VMnet1 and VMnet8 are listed). That is correct: VMnet2 is used as a **private, isolated switch** – no connection to Windows and no automatic address service. The lab is closed; only gw01 connects it to the outside.

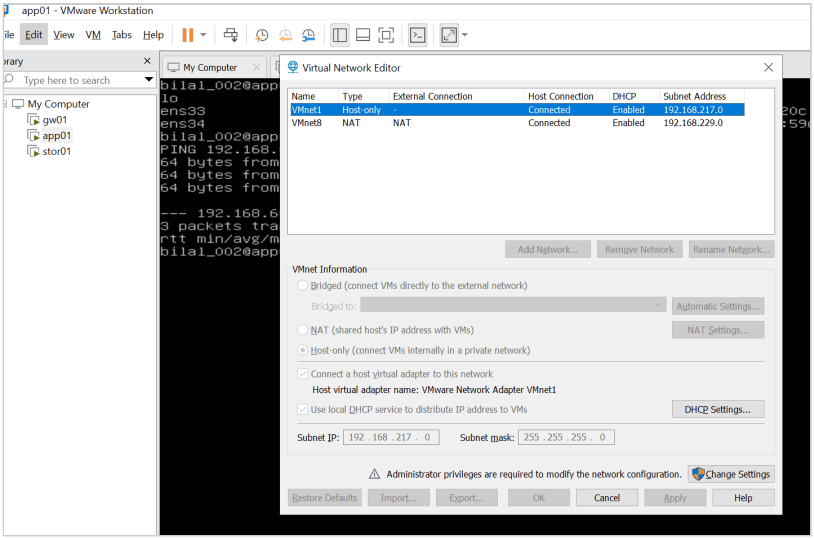


Figure 18 – VMware's Virtual Network Editor lists only VMnet1 and VMnet8 – VMnet2 is an isolated switch, as intended.

3.3 stor01 – the archive room, and the missing disks

stor01 got the same treatment, now done in the right order from the start – lesson learned from app01.

```
bilal_003@stor01:~$ ls /etc/netplan
50-cloud-init.yaml
bilal_003@stor01:~$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml
bilal_003@stor01:~$ echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-
network-config.cfg
network: {config: disabled}
bilal_003@stor01:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:

  version: 2

  ethernets:

    ens34:

      addresses: [192.168.60.20/24]

      routes:

        - to: default

          via: 192.168.60.1

      nameservers:

        addresses: [1.1.1.1, 8.8.8.8]
bilal_003@stor01:~$ sudo chmod 600 /etc/netplan/50-cloud-init.yaml
bilal_003@stor01:~$
```

Figure 19 – stor01: file found, backed up, cloud-init disabled (correctly this time), settings with 192.168.60.20, permissions set.

```
bilal_003@stor01:~$ sudo netplan try
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 108 seconds
Configuration accepted.
bilal_003@stor01:~$ _
```

Figure 20 – stor01: netplan try accepted.

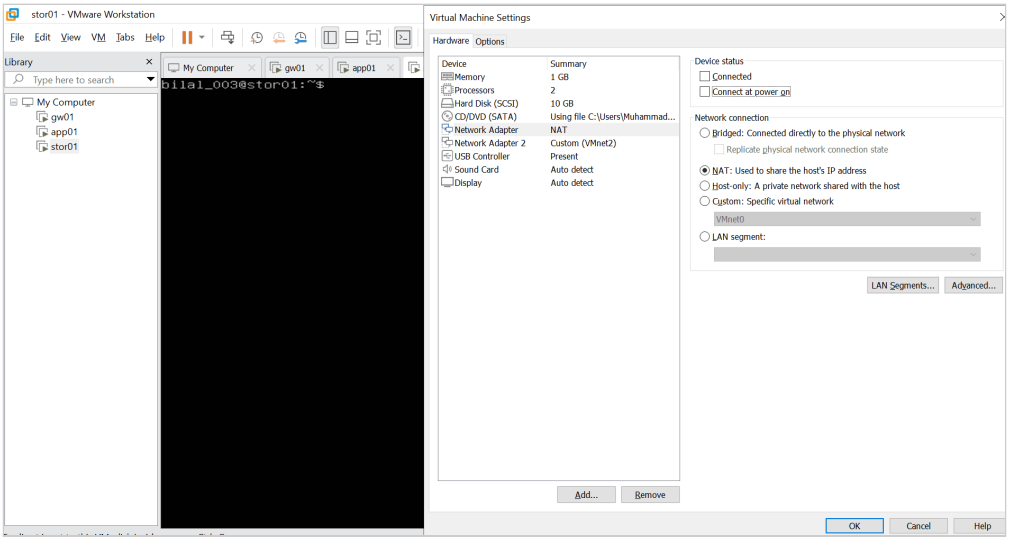


Figure 21 – stor01's internet adapter unplugged in VMware: both "Connected" and "Connect at power on" unticked.

```
bilal_003@stor01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            DOWN
ens34            UP             192.168.60.20/24 fe80::20c:29ff:fe08:26d6/64
bilal_003@stor01:~$ ip route
default via 192.168.60.1 dev ens34 proto static
192.168.60.0/24 dev ens34 proto kernel scope link src 192.168.60.20
bilal_003@stor01:~$ ping -c 3 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=5.36 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=4.76 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=2.04 ms

--- 192.168.60.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.039/4.050/5.357/1.443 ms
bilal_003@stor01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0    74M  1 loop /snap/core22/2955
loop2                       7:2    0  63.8M  1 loop /snap/core20/2922
loop3                       7:3    0    87M  1 loop /snap/lxd/29351
loop4                       7:4    0  38.8M  1 loop /snap/snapd/21759
loop5                       7:5    0 115.2M  1 loop /snap/lxd/40911
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    10G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0   8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   8.2G  0 lvm  /
sr0                        11:0    1  1024M  0 rom
bilal_003@stor01:~$
```

Figure 22 – stor01 after reboot: ens33 DOWN, 192.168.60.20, default via gw01, 3/3 pings to gw01 – but lsblk still shows only sda.

Third surprise. The network was perfect, but `lsblk` showed that the two extra data disks planned for the storage phase had never been added. I powered stor01 off, added two 5 GB disks in VMware, and checked again:

```
bilal_003@stor01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0  63.8M  1 loop /snap/core20/2922
loop2                       7:2    0    87M  1 loop /snap/lxd/29351
loop3                       7:3    0    74M  1 loop /snap/core22/2955
loop4                       7:4    0 115.2M  1 loop /snap/lxd/40911
loop5                       7:5    0  38.8M  1 loop /snap/snapd/21759
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    10G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0   8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   8.2G  0 lvm  /
sdb                         8:16   0     5G  0 disk
sdc                         8:32   0     5G  0 disk
sr0                        11:0    1     2G  0 rom
bilal_003@stor01:~$ _
```

Figure 23 – After adding them: sdb and sdc, 5 GB each, empty and ready for Phase 2 (LVM).

Why do we go to the next step?

All three servers now have fixed addresses and can reach gw01. app01 and stor01 have **no internet line of their own** – so right now they have **no internet at all**. They know they should ask gw01, but gw01 does not pass anything on yet (`ip_forward = 0`, Figure 3). That is the next step.

Chapter 4 – gw01 becomes a router (Step 1.2)

What I did and why. By default, Linux refuses to pass on traffic that is not meant for itself – like a receptionist who only answers her own calls. Switching **IP forwarding** on turns gw01 into a switchboard between the lab and the internet.

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf  # permanent setting
sudo sysctl --system                                                   # load it now
sysctl net.ipv4.ip_forward                                             # check: must be 1
```

```
bilal_001@gw01:~$ echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf
net.ipv4.ip_forward=1
bilal_001@gw01:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
bilal_001@gw01:~$
```

Figure 24 – ip_forward = 1. In the Phase 0 baseline (Figure 3) it was 0.

Why do we go to the next step?

gw01 now *passes traffic on*, but the internet still cannot answer app01. Addresses like 192.168.60.10 are **private**: they are not valid on the internet, so replies would have nowhere to go. gw01 must translate them – that is NAT.

Chapter 5 – Sharing the internet safely: NAT (Step 1.3)

What I did and why. NAT (Network Address Translation) works like a company mailroom. When app01 sends a letter out, gw01 crosses out the sender "Room 10" and writes the **company's street address** – its own. When the reply arrives, gw01 remembers who asked and delivers it to app01. The firewall rules decide **who may pass**: the lab may go out, and replies come back in. (Strictly blocking everything else needs a DROP default policy, which follows in Phase 6; today the lab is protected because its private addresses cannot be reached from the internet.)

```
# NAT: lab traffic leaving through ens33 gets gw01's address as sender
sudo iptables -t nat -A POSTROUTING -o ens33 -s 192.168.60.0/24 -j MASQUERADE
# allow: lab -> internet
sudo iptables -A FORWARD -i ens34 -o ens33 -j ACCEPT
# allow back in ONLY replies to conversations the lab started
sudo iptables -A FORWARD -i ens33 -o ens34 -m state --state ESTABLISHED,RELATED -j ACCEPT
# keep the rules after a reboot
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
```

Rule	Plain words
MASQUERADE	"Write the company's return address on outgoing mail"
FORWARD lab → internet	"Staff may make outgoing calls"
FORWARD internet → lab, only ESTABLISHED,RELATED	"Only call-backs are put through"

The proof – from a server that has no internet card

```
ping -c 3 8.8.8.8      # can app01 reach a server on the internet?
curl -s ifconfig.me    # ask a website: which address do you see me coming from?
```

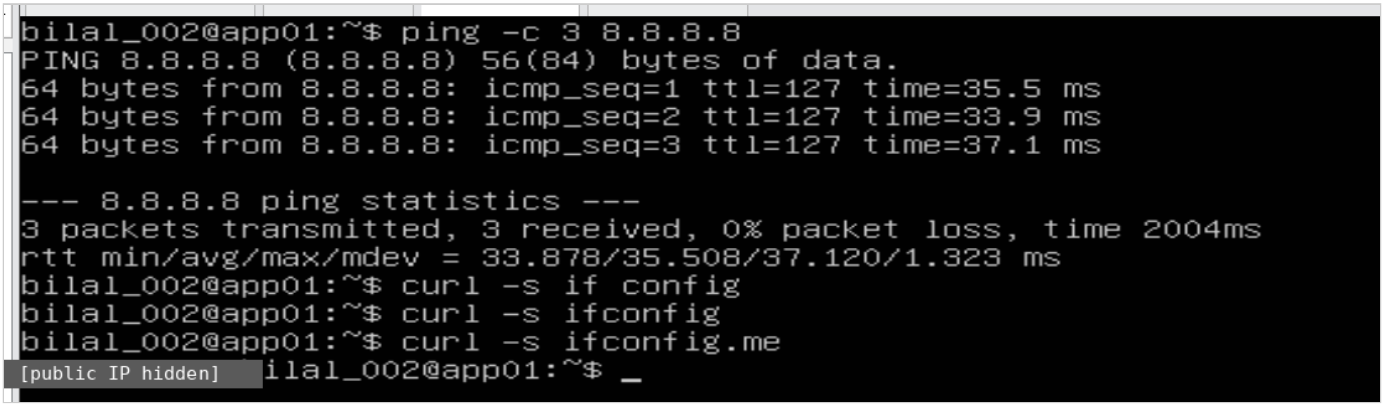


Figure 25 – app01 reaches the internet (3/3 replies from 8.8.8.8), and the website sees the home public address (hidden here), not 192.168.60.10. The two empty curl lines before were typos.

What Figure 25 proves.

- app01 has no internet card (Figure 17), yet it reaches 8.8.8.8 – **the only possible path is through gw01**.
- The website sees the public address, **not** 192.168.60.10 – NAT replaced the sender address.
- The reply TTL is 127 instead of 128. TTL is a counter that drops by one at every router – **the packets passed through one extra router: gw01**.

And from gw01's side – the NAT rule's counter shows the traffic it translated:

```
sudo iptables -t nat -L -n -v --line-numbers   # list NAT rules with packet counters
```

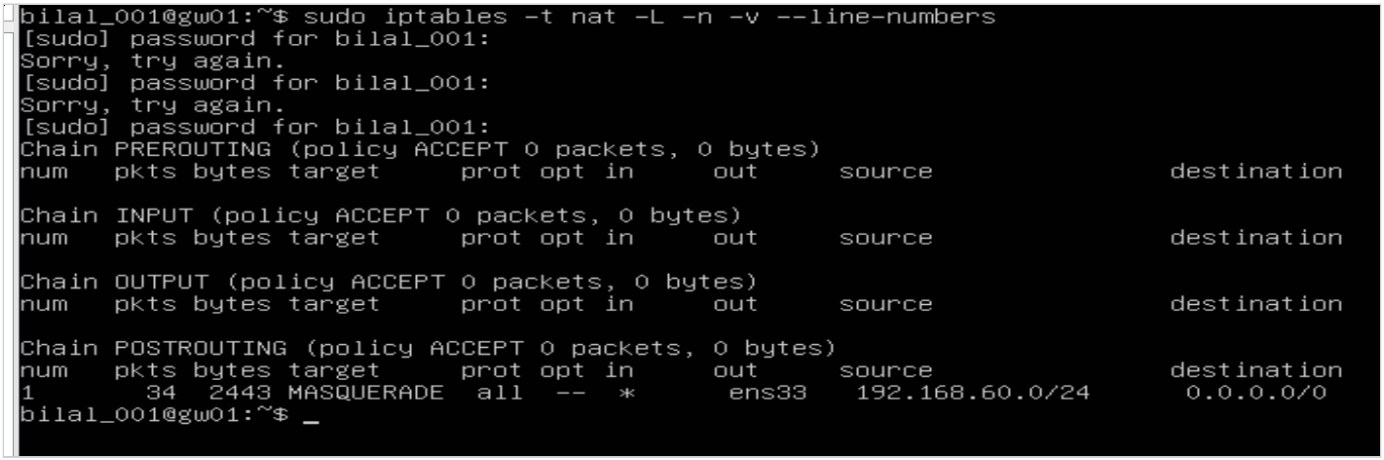


Figure 26 – On gw01 the MASQUERADE rule has translated 34 packets (2443 bytes) from 192.168.60.0/24 out through ens33. ("Sorry, try again" = mistyped sudo password.)

Why do we go to the next step?

The servers can now talk to each other and to the internet – but only by **number**. People and tools (SSH, file sharing, automation) use **names** like `stor01`. The next step teaches the servers to find each other by name.

Chapter 6 – Names instead of numbers (Step 1.4)

What I did and why. Computers connect only to numbers. Before connecting to a name, Linux looks it up – like a phone looking up a contact before dialing. It checks two places, in this order:

1. **`/etc/hosts`** – a small **personal contact list** on each machine. Checked first.
2. **DNS** – the internet's **public phone book**. Used only if the name is not in the contact list.

`getent` ("get entry") asks the system exactly this: *what is the address for this name?*

```
getent hosts stor01                                  # unknown -> prints nothing
echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts  # add stor01 to app01's contact list
getent hosts stor01                                  # now: 192.168.60.20
ping -c 1 stor01                                     # reach the server BY NAME
```

```
bilal_002@app01:~$ getent hosts stor01
bilal_002@app01:~$ echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts
[sudo] password for bilal_002:
192.168.60.20 stor01
bilal_002@app01:~$ getent hosts stor01
192.168.60.20   stor01
bilal_002@app01:~$ pinc -c 1 stor01
Command 'pinc' not found, did you mean:
  command 'ping' from deb iputils-ping (3:20211215-1ubuntu0.1)
  command 'ping' from deb inetutils-ping (2:2.2-2ubuntu0.2)
  command 'inc' from deb mailutils-mh (1:3.14-1)
  command 'inc' from deb mmh (0.4-4)
  command 'inc' from deb nmh (1.7.1-11)
  command 'minc' from deb mblaze (1.1-1)
  command 'pic' from deb groff-base (1.22.4-8build1)
Try: sudo apt install <deb name>
bilal_002@app01:~$ ping -c 1 stor01
PING stor01 (192.168.60.20) 56(84) bytes of data.
64 bytes from stor01 (192.168.60.20): icmp_seq=1 ttl=64 time=5.30 ms

--- stor01 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 5.296/5.296/5.296/0.000 ms
bilal_002@app01:~$ _
```

Figure 27 – On app01: the first lookup is empty; after adding the entry the name resolves and "ping stor01" works. ("pinc" was a typo.)

```
grep '^hosts' /etc/nsswitch.conf   # the lookup ORDER
dig +short google.com              # ask DNS directly
```

```
bilal_002@app01:~$ grep '^hosts' /etc/nsswitch.conf
hosts:          files dns
bilal_002@app01:~$ resolvectl status | head -5
Global
       Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
resolv.conf mode: stub

Link 2 (ens33)
bilal_002@app01:~$ sudo apt install -y dnsutils
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
dnsutils is already the newest version (1:9.18.39-0ubuntu0.22.04.6).
0 upgraded, 0 newly installed, 0 to remove and 66 not upgraded.
bilal_002@app01:~$ dig +short google.com
142.251.110.100
142.251.110.102
142.251.110.113
142.251.110.139
142.251.110.138
142.251.110.101
bilal_002@app01:~$
```

Figure 28 – "hosts: files dns" – the contact list is checked before DNS. dig returns Google's addresses, so DNS works through gw01.

Every server gets the same contact list

Each machine has its **own** `/etc/hosts`, so app01's entry did not help gw01 or stor01. I gave every server the names of the other two:

Machine	Added
gw01	app01 (.10), stor01 (.20)
app01	gw01 (.1) – stor01 was already there
stor01	gw01 (.1), app01 (.10)

The accidental "break it" test. On gw01 I first typed `192.168.60.200 stor01` by mistake, then the correct `.20` line. `/etc/hosts` is read top to bottom and **the first match wins**, so gw01 would have sent everything for stor01 to the non-existent .200 – exactly the classic fault the guide wanted me to provoke on purpose. I removed the wrong line in nano. (A similar slip on stor01 – `192.168.60.20 app01` – was caught and corrected the same way.)

```
My Computer | gw01 | app01 | stor01
bilal_001@gw01:~$ echo "192.168.60.1  gw01" | sudo tee -a /etc/hosts
[sudo] password for bilal_001:
192.168.60.1  gw01
bilal_001@gw01:~$ echo "192.168.60.10  app01" | sudo tee -a /etc/hosts
192.168.60.10  app01
bilal_001@gw01:~$ echo "192.168.60.200  stor01" | sudo tee -a /etc/hosts
192.168.60.200  stor01
bilal_001@gw01:~$ echo "192.168.60.20  stor01" | sudo tee -a /etc/hosts
192.168.60.20  stor01
bilal_001@gw01:~$ _
```

Figure 29 – The typo on gw01: a wrong 192.168.60.200 line for stor01 before the correct .20 line. The first match would have won.

The proof – every server reaches the other two by name:

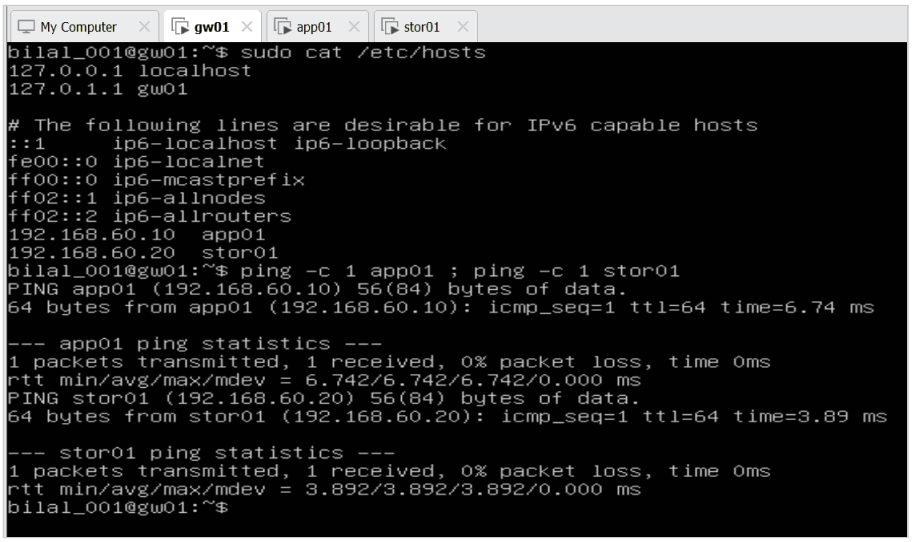


Figure 30 – gw01: correct contact list; ping app01 and ping stor01 by name, 1/1 received each.

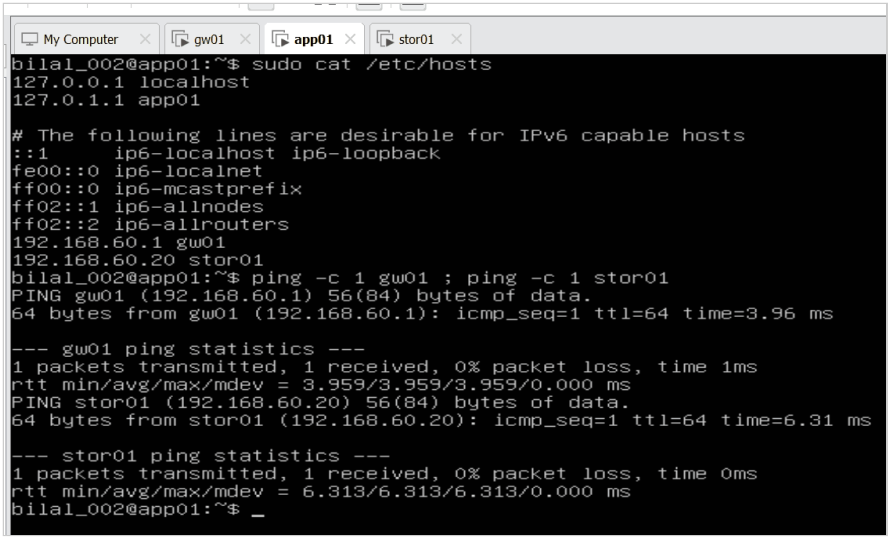


Figure 31 – app01: gw01 and stor01 by name, both reply.

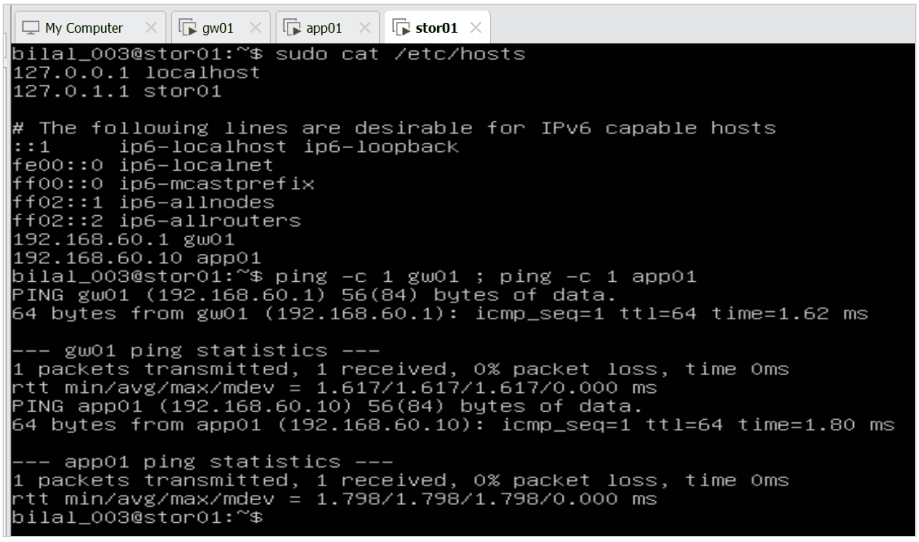


Figure 32 – stor01: gw01 and app01 by name, both reply.

Step 1.4 is complete. It also showed the weakness of `/etc/hosts`: the same list had to be typed by hand on three machines, with two typos along the way. That is exactly why companies use a central DNS server – or automation such as Ansible (Phase 7A).

Chapter 7 – Problems I met and how I solved them

#	What happened	Why	How I solved it
1	`systemctl get default` failed	Typo – the command is `get-default`	Retyped
2	Unsure whether `>> tee file` saves output	>> writes to a file; a pipe sends output to a program	Pipe into `tee file` (screen + file), or `>> file` (file only)
3	The netplan file in the guide did not exist	Ubuntu names it `50-cloud-init.yaml`	Checked with `ls`, used the real file
4	Pasted YAML arrived scrambled	Different keyboard layout in the VM; vim auto-indent	Quit without saving, typed by hand in nano
5	Warning “Cannot call Open vSwitch”	netplan checks for software that is not installed	Harmless – ignored
6	app01 still had its own internet address	Virtual cable still plugged in	Unplugged in VMware settings, rebooted
7	cloud-init line started with # on app01	Typing slip – # makes it a comment	Rewrote the file, verified with `cat`
8	stor01 had no data disks	Never added at VM creation	Added 2 × 5 GB disks
9	`curl ifconfig` printed nothing	Incomplete name; `-s` hides errors	Used `ifconfig.me`
11	Typos in `/etc/hosts`: `.200` for stor01 on gw01, `.20` for app01 on stor01	Typing slips; the first matching line wins	Checked with `cat` and `getent`, corrected in nano
12	stor01 flooded the console with `EXT4-fs error … Detected aborted journal` and `Remounting filesystem read-only`	The virtual disk was interrupted on the Windows side (host disk space, sleep or a forced stop); Linux switched the filesystem to read-only to protect the data	Restarted the VM from VMware; the filesystem recovered and became writable again (verified by editing `/etc/hosts`)
10	SSH from Windows to gw01 timed out, although SSH was running	Not needed for the design – admin access comes through a VPN later	Deferred; worked in the VMware console

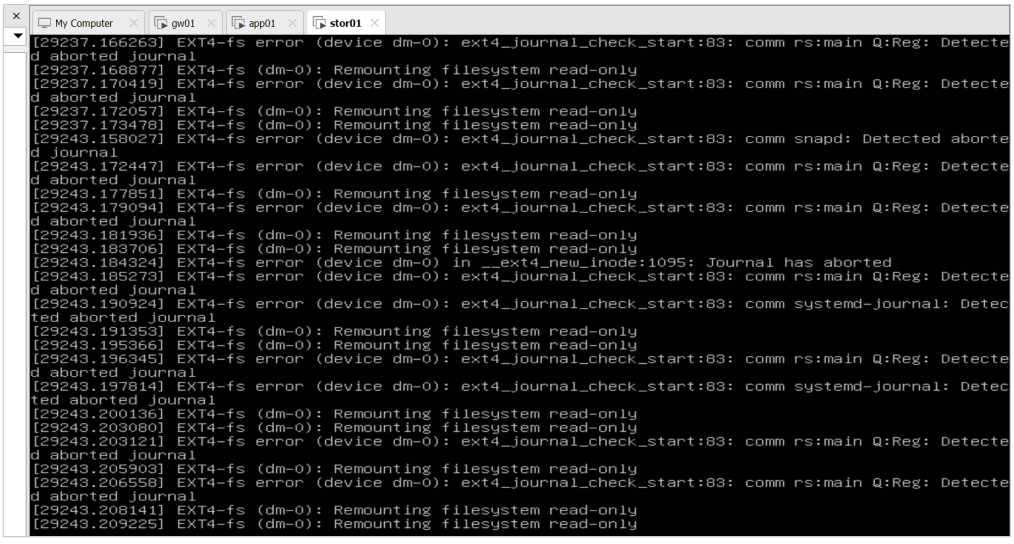


Figure 33 – Problem 12: stor01’s system disk hit errors and Linux switched it to read-only to protect the data.

Chapter 8 – Where we stand at the end of Day 1

Goal	Status
Baseline of gw01 recorded (kernel, memory, boot)	Done
gw01 in server mode	Done (app01, stor01 to follow)
Fixed addresses on all three servers, surviving reboot	Done
app01 and stor01 without their own internet line	Done
gw01 routes and translates (IP forwarding + NAT)	Done – 34 packets translated
Only replies allowed back into the lab	Rule in place; strict enforcement (DROP policy) in Phase 6
Servers find each other by name	Done – all three, by name
Data disks ready for storage	Done (sdb, sdc)

The journey in one picture:

```
Morning:   3 separate machines, each with its own internet line, lab network unused
              |
Step 1.1:  fixed addresses  ->  gw01 .1   app01 .10   stor01 .20
           internet cables of app01 and stor01 unplugged
              |
Step 1.2:  gw01 becomes a router (ip_forward 0 -> 1)
              |
Step 1.3:  gw01 translates addresses (NAT) and lets only replies back in
              |
Step 1.4:  servers find each other by name
              |
Evening:   one controlled door to the internet - gw01
```

How I would explain it in an interview: *"I built the network foundation of a three-server Linux lab: static addressing with netplan, internal servers with no direct internet access, and a Linux gateway doing NAT with stateful forwarding, so all outbound traffic passes one controlled point. I verified every change after reboot and documented ten issues with their root causes."*

Next steps: Step 1.5 (ports – which programs listen on which "door") and Step 1.6 (SSH key login from gw01 to the other servers, the basis for automation later).

Chapter 9 – Key understanding (in my own words)

These are the conclusions I reached by questioning each step until it made sense. They are written so I can explain them in an interview.

1. The network file and cloud-init

- `/etc/netplan/50-cloud-init.yaml` is the main network settings file: IP address, gateway and DNS. It was written once by **cloud-init** when Ubuntu was installed. At every boot **netplan reads it** and configures the network cards.
- cloud-init is the **writer**, netplan is the **reader**. cloud-init could rewrite the file on a later boot and erase my settings.
- So I created `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` with `network: {config: disabled}`. This only stops **cloud-init from managing the network** – the network itself and netplan keep working.
- Files in a `.d` folder are read in alphabetical order and **the last one wins**, so a `99-` file has the final word. The numbers only matter between files in the **same folder** – 50 (netplan) and 99 (cloud-init) are never compared.
- In Proxmox or the cloud, cloud-init is used the other way round: the admin enters IP, user and SSH key in the hypervisor, and cloud-init writes them into the server at first boot – ideal for cloned templates. In my VMware lab nothing feeds cloud-init, so I configure netplan by hand and keep cloud-init away from the network.

2. What the network file says

- `addresses: [192.168.60.10/24]` – **I am** 192.168.60.10; `/24` means 192.168.60.0–255 are my direct neighbours.
- `routes: - to: default via: 192.168.60.1` – "default" is not a machine, it means **everything I don't know**; `via` names who to hand it to: gw01.
- `nameservers: [1.1.1.1, 8.8.8.8]` – the internet's phone books. A name is first looked up in `/etc/hosts` (my own contact list – that is how `stor01` is found), and only then asked from DNS (that is how `google.com` is found).
- netplan only sets **addresses, gateway and DNS**. Being a router (ip_forward), NAT and firewall (iptables) and VPN (WireGuard) are set elsewhere.

3. The path to the internet

> app01 is 192.168.60.10 and has no internet adapter. For anything outside the lab it goes via gw01 (192.168.60.1), its default gateway. gw01 translates app01's private address into its own (NAT) and passes the traffic on through VMware and the home router to the internet. stor01 works the same way with 192.168.60.20.

4. NAT, forwarding and conntrack

> NAT recognises lab traffic by its IP range (`-s 192.168.60.0/24`) and changes the sender address as it leaves via `-o ens33`. Forwarding decides who may pass using the cards (`-i ens34 -o ens33`). Replies are let back in only if conntrack – gw01's memory of every conversation – confirms they are ESTABLISHED or RELATED to a conversation the lab started. conntrack also remembers which lab machine asked, so NAT can return each reply to the right one.

- **Two jobs, two tables:** the NAT table *changes the address* (`-j MASQUERADE`); the filter table's FORWARD chain *gives permission* (`-j ACCEPT`). A packet is checked by FORWARD first and rewritten in POSTROUTING just before it leaves – whatever order the commands were typed in.
- **`-s`/`-d` take IP addresses, `-i`/`-o` take card names.** `-s ens34` is always wrong. In POSTROUTING only the exit card is known, so `-i` is not allowed there – the lab must be identified by its IP range.
- **States:** NEW = someone starts a conversation; ESTABLISHED = a reply in a conversation the lab started; RELATED = a helper message (e.g. an error) belonging to it. `sudo conntrack -L` on gw01 shows the live list.
- **Honest limit today:** Ubuntu's default FORWARD policy is still ACCEPT, so the "only replies come back" rule is not yet enforced strictly. Right now the lab is protected because its private addresses sit behind two layers of NAT and cannot be reached from the internet. The strict version comes in Phase 6, when the default policy becomes DROP.
- `netfilter-persistent save` must run **last** – it saves only the rules that exist at that moment.

5. Names: /etc/hosts, DNS, getent and ping

> **getent** ("get entries") is like searching the phone book. It looks up a name or IP first in `/etc/hosts`, then in **DNS** if one is configured. It works both ways: name → IP (forward lookup) and IP → name (reverse lookup, e.g. `getent hosts 192.168.60.20` → `stor01`). It only **reads** – it never contacts the machine.
>
> **ping** is like calling. It uses the **same lookup** (`/etc/hosts`, then DNS) to find the number, then really contacts the machine with **ICMP** messages – "are you there?" (echo request) and "yes, I'm here" (echo reply). ICMP has **no ports**. So getent can succeed while ping fails – if the machine is off, the network is broken, or a **firewall blocks ICMP**.
>
> In our lab there is no internal DNS server, so we put the names in `/etc/hosts`. The servers could already reach each other **by IP**; `/etc/hosts` lets them reach each other **by name**.

- **Phone picture:** the contacts app inside the phone = the system resolver (`/etc/hosts` → DNS); getent = opening the contacts and reading a number; ping = pressing *call* – it looks up the same contact, then dials; ICMP = the phone network that carries the call.
- **Troubleshooting rule:** getent fails → a **name** problem (missing/wrong entry or DNS). getent works but ping fails → a **network** problem (machine off, cabling, firewall).
- `/etc/hosts` is a file on **each** machine – fine for a small lab, but 100 servers means 100 files to edit. Companies use an **internal DNS server** for their own names (as the Windows Server does in Project 1) and **public DNS** (1.1.1.1, 8.8.8.8) for internet names.
- **The DNS helper and loopback:** on Ubuntu, programs do not ask 1.1.1.1 directly – they ask a small helper on the same machine, **systemd-resolved**, at **127.0.0.53**. Every **127.x.x.x** address is **loopback** – the machine talking to itself, like putting a note in your own pocket: the traffic never leaves the machine (`lo` in `ip -br a`). The helper answers from its **DNS cache** – short-term memory of name lookups for the whole machine – or asks the real DNS servers through gw01. The cache speeds things up, expires by itself (**TTL**), and can be cleared with `sudo resolvectl flush-caches`. `resolvectl status` shows which DNS servers are in use.
- **DNS troubleshooting in three commands:** `getent hosts <name>` (does the name resolve at all?) → `resolvectl status` (which DNS servers?) → `dig +short <name>` (does DNS itself answer? dig skips `/etc/hosts`).
- `hostname` shows the machine's own name; `getent hosts <name>` asks what address a name has; `tee -a /etc/hosts` appends a new entry.

Glossary

Word	Meaning
Virtual machine (VM)	A computer simulated by software inside a real computer
Kernel	The engine of Linux that talks to the hardware
Module / driver	Add-on that lets the kernel use specific hardware
systemd / target	The manager that starts services / the start-up mode (desktop or server)
IP address	A machine's "phone number" on a network
Private address	Valid only inside a home or company network (e.g. 192.168.x.x)
/24	Network size: 256 addresses sharing the first three number blocks
Network card (ens33, ens34)	The socket a network cable plugs into; `lo` = the machine talking to itself
netplan / YAML	Ubuntu's network settings / the file format where indentation matters
cloud-init	Helper that sets up a new server and may overwrite manual settings
Default route / gateway	"Where to send traffic for unknown destinations" – here gw01
IP forwarding	The switch that lets Linux act as a router
NAT / MASQUERADE	Replacing private sender addresses with the gateway's own address
Firewall / iptables	Rules that decide which traffic may pass
TTL	Counter in each packet, lowered by 1 at every router
DNS	The internet's phone book: names to numbers
/etc/hosts	A machine's personal contact list, checked before DNS
getent	Asks the system what a name resolves to
lsblk / sda, sdb, sdc	Lists disks / the system disk and two data disks
SSH	Secure remote login to a server's command line

Day 1 (part 1) – 2026-10-13: Lab setup and Phase 0 (system baseline)

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	0 – System baseline
Hosts worked on	gw01 (app01 and stor01 prepared)
Status	Phase 0 complete on gw01 · app01 and stor01: boot target still to set
Author	Muhammad Bilal Javed

> **How to read this report.** Every step has three layers: 💡 **In plain words** – the idea, for any reader, no Linux knowledge needed. 🔧 **Technical** – the exact commands (with a comment on every line) and what they do. 📸 **Evidence** – the real screenshot of the output from my lab. Unfamiliar words are explained in the glossary.

---

1. Objective

💡 **In plain words:** before a mechanic repairs a car, they write down its condition: mileage, oil level, warning lights. Then, if something goes wrong later, they can compare against "how it was before". This phase does the same for a server: **write down how it looks before I change anything.**

🔧 **Technical:** record a **baseline** of the gateway server gw01: kernel version, loaded drivers, memory, IP forwarding state, boot time and boot target. The baseline is the reference point for every later change, for troubleshooting ("what was it like before?") and for audits.

2. The lab environment

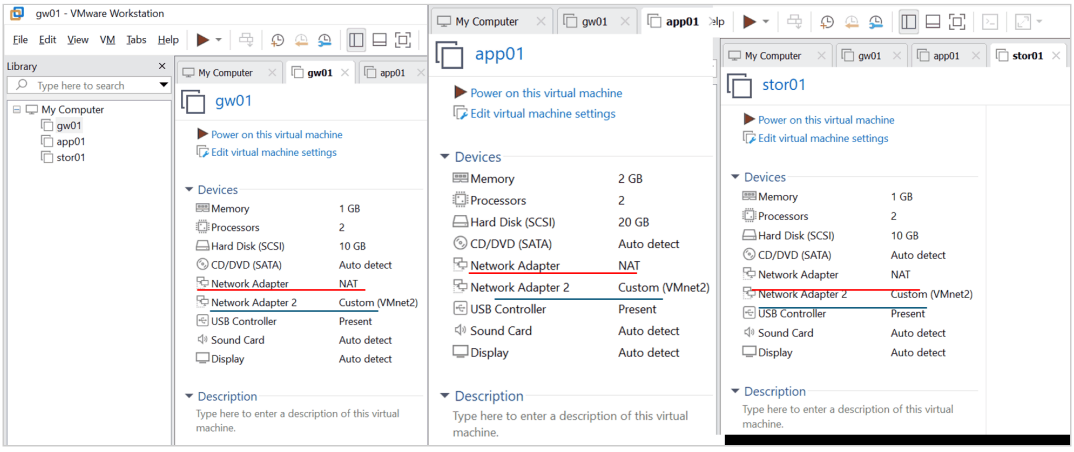
💡 **In plain words:** instead of buying real servers, I run three "computers inside my computer" (virtual machines) with VMware. Each has its own job, like employees in a small company:

- **gw01** – the *gatekeeper*. The only machine allowed to talk to the internet. Everything in and out passes through it.
- **app01** – the *shop window*. Runs the company's web application.
- **stor01** – the *archive room*. Stores files, backups and logs.

Item	Value
Hypervisor (software that runs the virtual machines)	VMware Workstation on Windows
Lab network	VMnet2, isolated custom switch, 192.168.60.0/24, no DHCP, no host adapter
Operating system	Ubuntu Server 22.04 LTS
Kernel	5.15.0-198-generic (#208-Ubuntu SMP, built 4 Sep 2026)

VM	Role	vCPU	RAM	Disks	Network adapters
gw01	Gateway: NAT, firewall, VPN	2	1.9 GiB visible to the OS	10 GB	NAT (internet) + VMnet2 (lab)
app01	Application server	2	2 GB (3 GB planned before monitoring phase)	20 GB	VMnet2 only (NAT adapter disconnected, see part 2)
stor01	Storage and log server	2	1 GB	10 GB + 2 × 5 GB (for LVM)	VMnet2 only (NAT adapter disconnected, see part 2)

📸 **VM hardware as created in VMware** (Network Adapter = NAT, Network Adapter 2 = VMnet2 lab network):



VM hardware of gw01, app01 and stor01

Design decision: only gw01 keeps an internet-facing adapter. app01 and stor01 reach the internet exclusively through gw01 (built in part 2). This makes gw01 the single controlled entry and exit point, so NAT, firewall and VPN controls can be enforced and tested in one place.

---

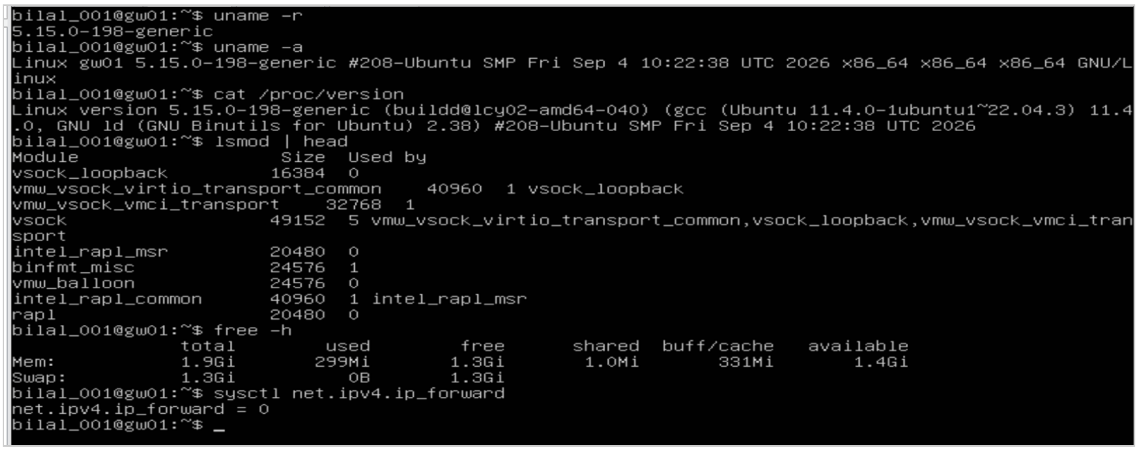
3. Step 0.1 – Kernel, drivers and memory

💡 **In plain words:** the **kernel** is the "engine" of Linux – the core program that talks to the hardware. **Drivers (modules)** are small add-ons that let the engine use specific hardware. Here I check *which engine version* the server runs, *which add-ons are loaded* and *how much memory it has*. I also check one important switch: whether this machine is allowed to pass traffic between networks (like a router) – right now it is not.

🔧 **Commands (run on gw01):**

```
uname -r                    # print only the kernel version
uname -a                    # print everything: hostname, kernel, build date, CPU architecture
cat /proc/version           # the kernel's own description: compiler and build details
lsmod | head                # list loaded kernel modules (drivers); "| head" shows only the first 10 lines
free -h                     # memory and swap usage, "-h" = human-readable (GiB/MiB)
sysctl net.ipv4.ip_forward  # read the kernel setting that decides whether this host routes packets
```

📸 **Evidence:**



Kernel, modules, memory and ip_forward on gw01

Check	Result	What it means
Kernel version	`5.15.0-198-generic`	Ubuntu 22.04 standard kernel; noted so future updates can be tracked
Build info	gcc 11.4.0, binutils 2.38	Standard Ubuntu build, nothing custom
Loaded modules	`vmw_vsock_*`, `vmw_balloon`, `vsock`, `intel_rapl_*`	VMware guest drivers are loaded – the OS knows it runs as a VM. `vmw_balloon` lets VMware take back unused memory
Memory	1.9 GiB total, 299 MiB used, 1.4 GiB available; swap 1.3 GiB, 0 used	Plenty of headroom for a gateway
IP forwarding	`net.ipv4.ip_forward = 0`	**gw01 does not route packets yet** – it behaves like a normal PC. Phase 1 switches this to 1

---

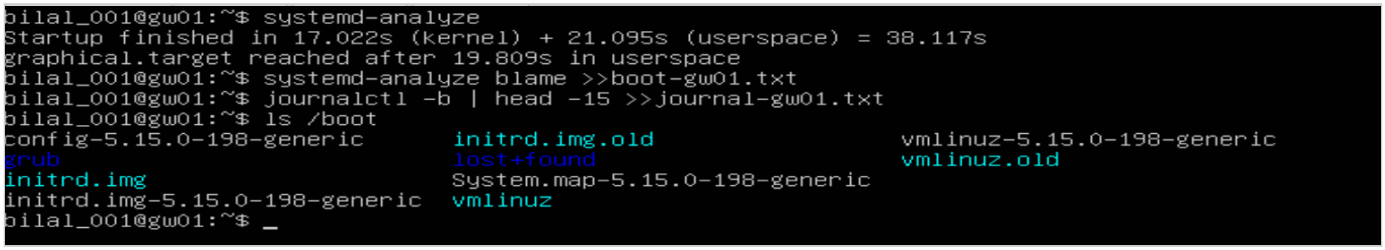
4. Step 0.2 – Boot process

💡 **In plain words:** starting a computer is a relay race – each runner hands over to the next: the **firmware** wakes the hardware → the **bootloader (GRUB)** picks and loads the engine → the **kernel** starts → **systemd**, the "manager", starts all the services (network, SSH, ...) → you get a login. Here I measure **how long the race takes** and save the details as evidence.

🔧 **Commands (run on gw01):**

```
systemd-analyze                          # total boot time, split into kernel and userspace
systemd-analyze blame >> boot-gw01.txt   # time per service, appended (>>) to a file as evidence
journalctl -b | head -15 >> journal-gw01.txt  # first 15 log lines of this boot (-b), appended to a file
ls /boot                                 # files the bootloader uses to start Linux
```

📸 **Evidence:**



systemd-analyze and /boot contents on gw01

Check	Result
Total boot time	**38.117 s** = 17.022 s kernel + 21.095 s userspace (services)
Default target reached	`graphical.target` after 19.809 s in userspace
`/boot` contents	`vmlinuz-5.15.0-198-generic` (the kernel), `initrd.img-5.15.0-198-generic` (initial RAM disk – a mini system that helps the kernel find the real disk), `vmlinuz.old` / `initrd.img.old` (**previous kernel kept as a fallback** if an update fails), `grub/` (bootloader), `config-*` and `System.map-*` (kernel build settings and symbol map)

Boot chain: firmware (BIOS/UEFI) → GRUB bootloader → Linux kernel → initramfs → systemd (PID 1) → services → login.

---

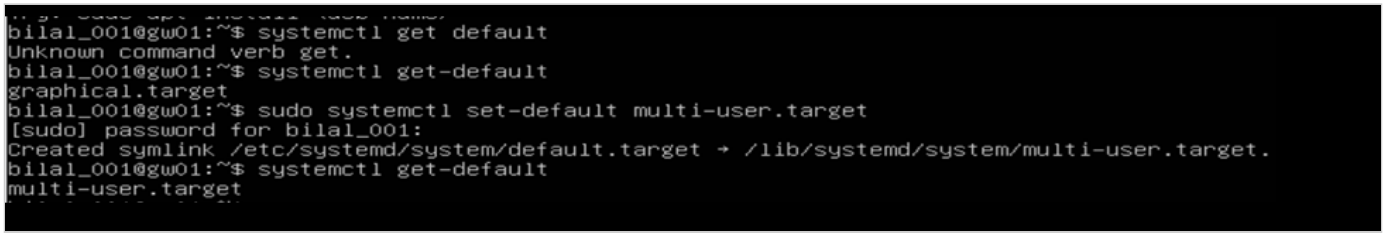
5. Step 0.3 – Boot target (desktop mode vs server mode)

💡 **In plain words:** Linux can start in different "modes", called **targets**. `graphical.target` starts a desktop with windows and mouse – useful for a laptop, wasted on a server nobody sits in front of. `multi-user.target` starts everything a server needs (network, services, logins) **without** the desktop. Less software running = faster start, less memory used and fewer things an attacker could abuse.

🔧 **Commands (run on gw01):**

```
systemctl get-default                          # which target the system boots into now
sudo systemctl set-default multi-user.target   # change it to server mode (needs admin rights: sudo)
systemctl get-default                          # check that the change took effect
```

📸 **Evidence** (including my first typo, see issues):



Changing the default boot target on gw01

Before	After
`graphical.target`	`multi-user.target`

systemd confirmed the change by creating the link `/etc/systemd/system/default.target → /lib/systemd/system/multi-user.target`.

Old "runlevel" (classic Linux)	systemd target today
0	poweroff.target
1	rescue.target (repair mode)
3	multi-user.target (server)
5	graphical.target (desktop)
6	reboot.target

---

6. Issues encountered

#	Symptom	Cause	Resolution
1	`systemctl get default` → `Unknown command verb get.`	Typo: the command is `get-default`, one word with a hyphen	Re-ran as `systemctl get-default`
2	Unsure whether `>> tee file` saves output	`>>` sends output to a **file**; `|` (pipe) sends it to a **program**. `>> tee` would create a file literally named `tee`	Use `command | tee file` (show on screen **and** save) or `command >> file` (append to file only)

7. Lessons learned

- A baseline captured **before** changes makes later troubleshooting faster and provable.
- `ip_forward` is the single kernel setting that decides whether a Linux host routes traffic.
- Servers should boot into `multi-user.target`; the previous kernel in `/boot` is the safety net if an update fails.
- Saving command output to files turns “I did it” into evidence for documentation and audits.

8. Next steps

1. Set `multi-user.target` on app01 and stor01; take snapshot `10-baseline` of all VMs.
2. **Phase 1 – Network foundation** → continued in Day 1, part 2.

Day 1 (part 2) – 2026-10-13: Phase 1, network foundation

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1 – Network foundation: Steps 1.1–1.4
Hosts worked on	gw01, app01, stor01
Status	1.1 static IPs ✅ · 1.2 routing ✅ · 1.3 NAT ✅ · 1.4 name resolution ✅
Author	Muhammad Bilal Javed

> **How to read this report.** Every step has three layers: 💡 **In plain words** – the idea, for any reader. 🔧 **Technical** – the exact commands, with a comment on every line. 📸 **Evidence** – the real screenshot from my lab. Unfamiliar words are explained in the glossary.

---

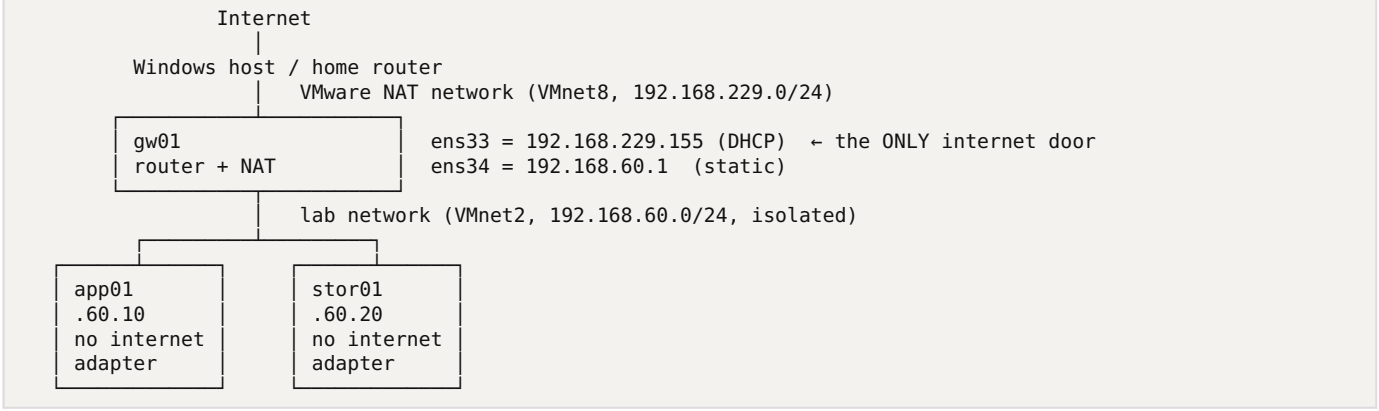
1. Objective

💡 **In plain words:** at the end of Phase 0 the three servers were like three new employees with no desk phones: they could not reliably reach each other, and two of them had their own uncontrolled line to the outside world. In this phase I:

1. give every server a **fixed address** (like a permanent desk-phone number),
2. **cut the private internet lines** of app01 and stor01,
3. turn gw01 into the **company switchboard** – the one and only way out to the internet,
4. teach the servers to find each other **by name** instead of by number.

🔧 **Technical:** static IPv4 addressing with netplan, IP forwarding and source NAT (MASQUERADE) with stateful forwarding on gw01, and name resolution via `/etc/hosts` and DNS.

2. Target network design



Host	External card (ens33)	Lab card (ens34)	Way to the internet (“default gateway”)
gw01	DHCP, 192.168.229.155	192.168.60.1/24	VMware NAT, directly
app01	disconnected	192.168.60.10/24	through gw01 (192.168.60.1)
stor01	disconnected	192.168.60.20/24	through gw01 (192.168.60.1)

💡 **Why this design?** One door is easier to guard than three. All firewall rules, logs and later the VPN live on gw01. The internet can never start a connection to app01 or stor01 directly.

---

3. Step 1.1 – Fixed (static) addresses with netplan

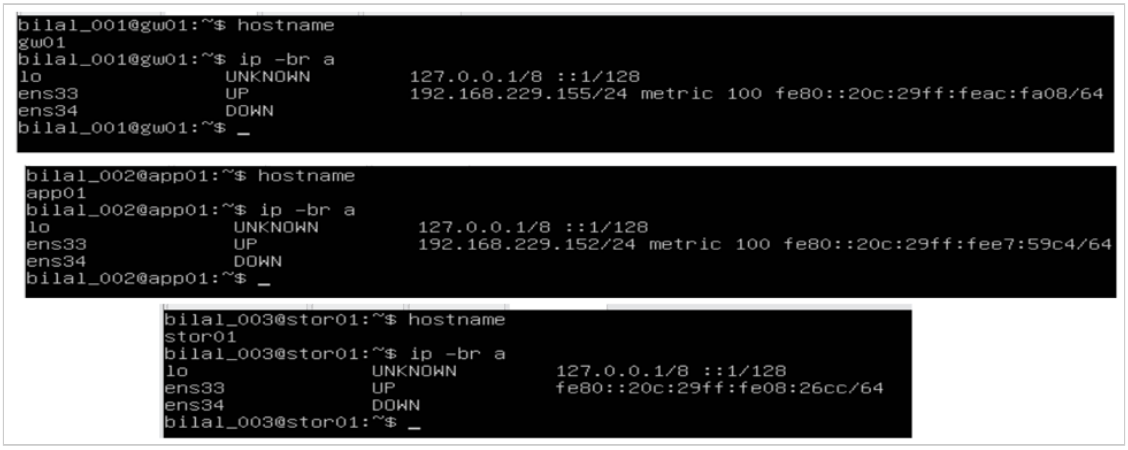
3.1 Look before you change: which network cards exist?

💡 **In plain words:** each VM has two “network sockets” (cards). Before configuring them I check their names and current addresses.

🔧 **Commands:**

```
hostname          # confirm which machine I am on
ip -br a          # list all network cards, brief (-br) form: name, state, address
```

📸 **Evidence – starting point on all three VMs:**



Network cards before configuration

Card	Meaning
lo	loopback – the machine talking to itself (127.0.0.1). Always present
ens33	1st card = VMware NAT network. gw01 and app01 had internet addresses (192.168.229.x) from VMware's DHCP
ens34	2nd card = lab network (VMnet2). **DOWN with no address** – nothing configured yet

3.2 Find the configuration file and make a backup

💡 **In plain words:** on Ubuntu, network settings live in a small text file in the folder `/etc/netplan/`. Before editing any config file, I make a copy – so I can always go back.

🔧 **Commands:**

```
ls /etc/netplan                                          # which netplan file exists?
sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml # backup copy into my home folder
```

📸 **Evidence:** the file is called `50-cloud-init.yaml`, not `00-installer-config.yaml` as my guide assumed (issue 1):

Netplan filename on app01

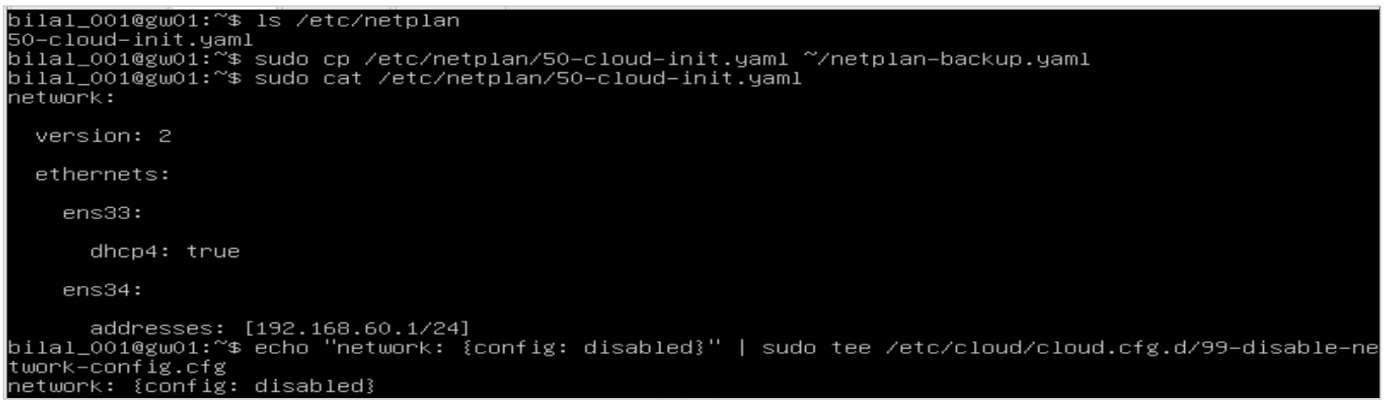
3.3 Stop cloud-init from overwriting my settings

💡 **In plain words:** `cloud-init` is a helper program that sets up the network automatically at first boot – and it may **rewrite my file** on a later boot, silently undoing my work. I leave it a note saying "network: hands off".

🔧 **Command:**

```
# write the line "network: {config: disabled}" into a new cloud-init config file
# (tee writes to the file AND shows the line on screen so I can check it)
echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
```

📸 **Evidence – gw01: backup, current file, cloud-init disabled:**



Backup and cloud-init disable on gw01

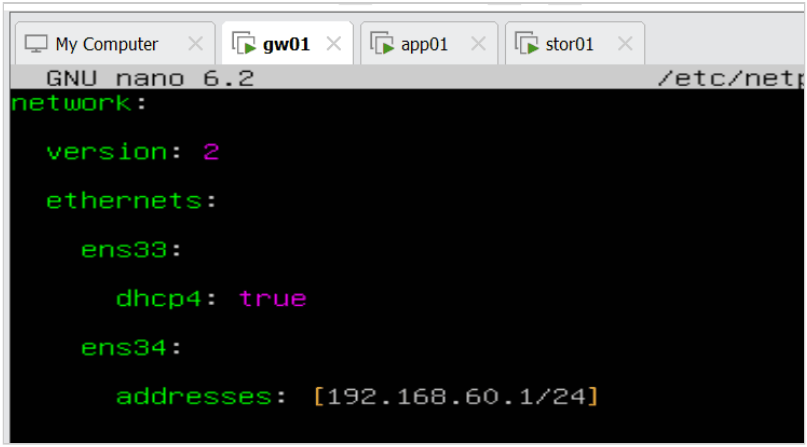
3.4 Write the new network settings

💡 **In plain words:** the netplan file is written in **YAML**, a format where **indentation (spaces) is part of the meaning** – like an outline where sub-points must be indented under their heading. One wrong space and the file is invalid.

🔧 **gw01** (`sudo nano /etc/netplan/50-cloud-init.yaml`):

```
network:
  version: 2
  ethernets:
    ens33:                          # card 1: internet side
      dhcp4: true                   #   get its address automatically from VMware
    ens34:                          # card 2: lab side
      addresses: [192.168.60.1/24] #   fixed address; no gateway — gw01 IS the gateway
```

📸 **Evidence – typed by hand in nano on gw01:**



gw01 netplan file in nano

🔧 **app01** (stor01 identical, with `192.168.60.20/24`):

```
network:
  version: 2
  ethernets:
    ens34:                          # only the lab card is configured
      addresses: [192.168.60.10/24] # fixed address; "/24" = network size (192.168.60.0–255)
      routes:
        - to: default               # for every destination I don't know …
          via: 192.168.60.1         # … send it to gw01 (the "default gateway")
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]  # public DNS servers: translate names like google.com into IPs
```

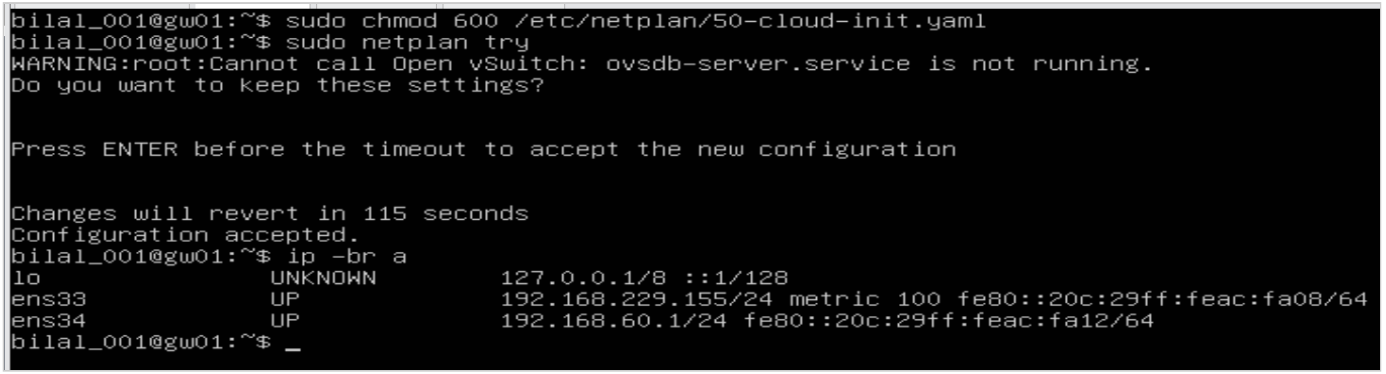
3.5 Apply safely – with an automatic “undo”

💡 **In plain words:** changing network settings on a server is like changing the lock on the only door while you are inside: if it goes wrong, you may lock yourself out. `netplan try` applies the change but **automatically undoes it after 120 seconds** unless I press Enter to confirm. If I can still type, the change works – I press Enter.

🔧 **Commands:**

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml  # only root may read/write the file (netplan warns otherwise)
sudo netplan try                                # apply with auto-rollback; press ENTER to keep
ip -br a                                        # check the new addresses
```

📸 **Evidence – gw01:** ens34 now has `192.168.60.1/24`. The Open vSwitch warning is harmless (issue 4):



netplan try on gw01

📸 **Evidence – app01:** file content and `netplan try`:

```
50-cloud-init.yaml
bilal_002@app01:~$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml
bilal_002@app01:~$ echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-n
etwork-config.cfg
#network: {config: disabled}
bilal_002@app01:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:

  version: 2

  ethernets:

    ens34:

      addresses: [192.168.60.10/24]

      routes:

        - to: default

          via: 192.168.60.1

      nameservers:

        addresses: [1.1.1.1, 8.8.8.8]
bilal_002@app01:~$ sudo chmod 600 /etc/netplan/50-cloud-init.yaml
bilal_002@app01:~$ sudo netplan try
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
Do you want to keep these settings?


Press ENTER before the timeout to accept the new configuration


Changes will revert in 119 seconds
Configuration accepted
```

netplan try on app01

3.6 Problem found: app01 still had its own internet line

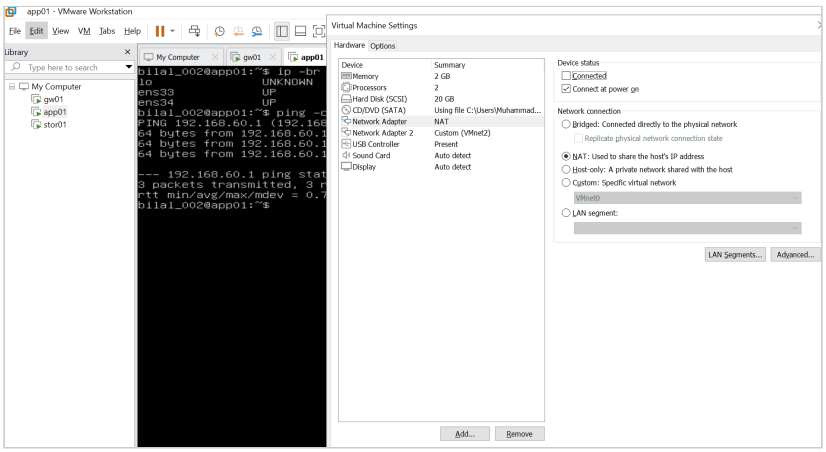
📸 After applying, app01 could reach gw01 – **but ens33 still had an internet address (192.168.229.152)**. That breaks the design: app01 must not have its own way out.

```
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.152/24 metric 100 fe80::20c:29ff:fee7:59c4/64
ens34            UP             192.168.60.10/24 fe80::20c:29ff:fee7:59ce/64
bilal_002@app01:~$ ping -c 3 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=10.8 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=0.755 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=1.33 ms

--- 192.168.60.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 0.755/4.287/10.779/4.596 ms
bilal_002@app01:~$
```

app01 still connected to the NAT network

💡 **In plain words:** turning off the setting inside Linux is not enough – the **virtual cable** was still plugged in. The real fix is to unplug it in VMware itself.

🔧 **Fix – VMware → VM → Settings → Network Adapter (NAT):** untick **Connected** and **Connect at power on**, then reboot.



Disconnecting app01's NAT adapter in VMware

3.7 Proof after reboot

💡 **In plain words:** a setting that disappears after a restart is worthless. I reboot and check that everything is still correct.

🔧 **Commands:**

```
sudo reboot        # restart the VM
ip -br a           # addresses after the restart
ip route           # the routing table: where traffic is sent
```

📸 **Evidence – app01 after reboot:**

```
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            DOWN
ens34            UP             192.168.60.10/24 fe80::20c:29ff:fee7:59ce/64
bilal_002@app01:~$ ip route
default via 192.168.60.1 dev ens34 proto static
192.168.60.0/24 dev ens34 proto kernel scope link src 192.168.60.10
bilal_002@app01:~$ _
```

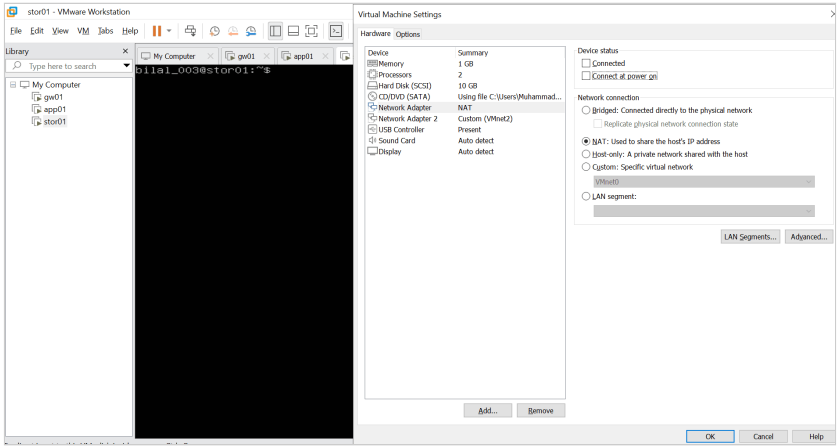
app01 after reboot

Line	Meaning
`ens33 DOWN`	internet card unplugged ✅
`ens34 UP 192.168.60.10/24`	fixed lab address survived the reboot ✅
`default via 192.168.60.1 dev ens34`	“anything not local → send to gw01” ✅
`192.168.60.0/24 dev ens34 … src 192.168.60.10`	lab neighbours are reached directly on ens34 ✅

📸 Evidence – stor01: same method, file written and permissions set, internet adapter unplugged in VMware:

```
bilal_003@stor01:~$ ls /etc/netplan
50-cloud-init.yaml
bilal_003@stor01:~$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml
bilal_003@stor01:~$ echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-
network-config.cfg
network: {config: disabled}
bilal_003@stor01:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:

  version: 2

  ethernets:

    ens34:

      addresses: [192.168.60.20/24]

      routes:

        - to: default

          via: 192.168.60.1

      nameservers:

        addresses: [1.1.1.1, 8.8.8.8]
bilal_003@stor01:~$ sudo chmod 600 /etc/netplan/50-cloud-init.yaml
bilal_003@stor01:~$
```

stor01 netplan file



stor01 NAT adapter disconnected

📸 Evidence – stor01 after the same steps: ens33 DOWN, `192.168.60.20/24`, default via gw01, ping to gw01 3/3 received:

```
bilal_003@stor01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            DOWN
ens34            UP             192.168.60.20/24 fe80::20c:29ff:fe08:26d6/64
bilal_003@stor01:~$ ip route
default via 192.168.60.1 dev ens34 proto static
192.168.60.0/24 dev ens34 proto kernel scope link src 192.168.60.20
bilal_003@stor01:~$ ping -c 3 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=5.36 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=4.76 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=2.04 ms

--- 192.168.60.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.039/4.050/5.357/1.443 ms
bilal_003@stor01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0    74M  1 loop /snap/core22/2955
loop2                       7:2    0  63.8M  1 loop /snap/core20/2922
loop3                       7:3    0    87M  1 loop /snap/lxd/29351
loop4                       7:4    0  38.8M  1 loop /snap/snapd/21759
loop5                       7:5    0 115.2M  1 loop /snap/lxd/40911
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    10G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0   8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   8.2G  0 lvm  /
sr0                        11:0    1  1024M  0 rom
bilal_003@stor01:~$
```

stor01 network and disks

3.8 Problem found: stor01's data disks were missing

💡 **In plain words:** `lsblk` lists the disks of a machine. stor01 should have two extra empty 5 GB disks for the storage phase – but only the system disk `sda` was there. I powered it off, added two 5 GB disks in VMware, and checked again.

🔧 **Command:**

```
lsblk     # list block devices (disks and partitions)
```

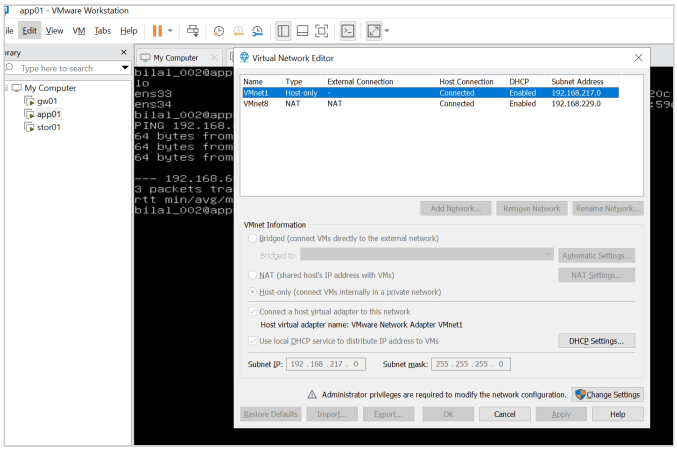
📸 **Evidence – after adding the disks:** `sdb` and `sdc`, 5 GB each, empty (no partitions), ready for Phase 2:

```
bilal_003@stor01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0  63.8M  1 loop /snap/core20/2922
loop2                       7:2    0    87M  1 loop /snap/lxd/29351
loop3                       7:3    0    74M  1 loop /snap/core22/2955
loop4                       7:4    0 115.2M  1 loop /snap/lxd/40911
loop5                       7:5    0  38.8M  1 loop /snap/snapd/21759
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    10G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0   8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   8.2G  0 lvm  /
sdb                         8:16   0     5G  0 disk
sdc                         8:32   0     5G  0 disk
sr0                        11:0    1     2G  0 rom
bilal_003@stor01:~$ _
```

stor01 data disks sdb and sdc

3.9 Note: VMnet2 does not appear in the Virtual Network Editor

📸 The editor lists only VMnet1 and VMnet8:



VMware Virtual Network Editor

💡 **In plain words:** this is correct. VMnet2 is used as a **private switch** with no connection to Windows and no automatic address service. The lab is completely closed – only gw01 connects it to the outside.

---

4. Step 1.2 – Turn gw01 into a router (IP forwarding)

💡 **In plain words:** by default a Linux machine **refuses to pass on** traffic that is not meant for itself – like a receptionist who only takes calls for herself. Switching on **IP forwarding** turns gw01 into a switchboard that passes calls between the lab and the internet. In Phase 0 this was `0` (off).

🔧 **Commands (gw01):**

```
# save the setting in a config file so it survives reboots
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf
sudo sysctl --system            # load all sysctl config files now (no reboot needed)
sysctl net.ipv4.ip_forward      # verify: must say 1
```

📸 **Evidence:** `net.ipv4.ip_forward = 1` (was `0` in the Phase 0 baseline):

```
bilal_001@gw01:~$ echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf
net.ipv4.ip_forward=1
bilal_001@gw01:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
bilal_001@gw01:~$
```

IP forwarding enabled on gw01

---

5. Step 1.3 – NAT: share gw01's internet with the lab

💡 **In plain words:** addresses like 192.168.60.10 are **private** – the internet cannot send replies to them, just like a letter addressed to "Room 10" cannot be delivered without the building's street address. **NAT (Network Address Translation)** works like a company mailroom: when app01 sends something out, gw01 replaces the sender address with **its own** address; when the reply comes back, gw01 remembers who asked and hands it to app01.

The firewall rules decide **who may pass**: the lab may go out, and replies come back in. (Strictly blocking everything else needs a DROP default policy, which follows in Phase 6; today the lab is protected because its private addresses cannot be reached from the internet.)

🔧 **Commands (gw01):**

```
# NAT: traffic from the lab leaving via ens33 gets gw01's address as sender
sudo iptables -t nat -A POSTROUTING -o ens33 -s 192.168.60.0/24 -j MASQUERADE
# allow: lab (ens34) → internet (ens33)
sudo iptables -A FORWARD -i ens34 -o ens33 -j ACCEPT
# allow back in ONLY replies to connections the lab started
sudo iptables -A FORWARD -i ens33 -o ens34 -m state --state ESTABLISHED,RELATED -j ACCEPT
# make the rules survive a reboot
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
```

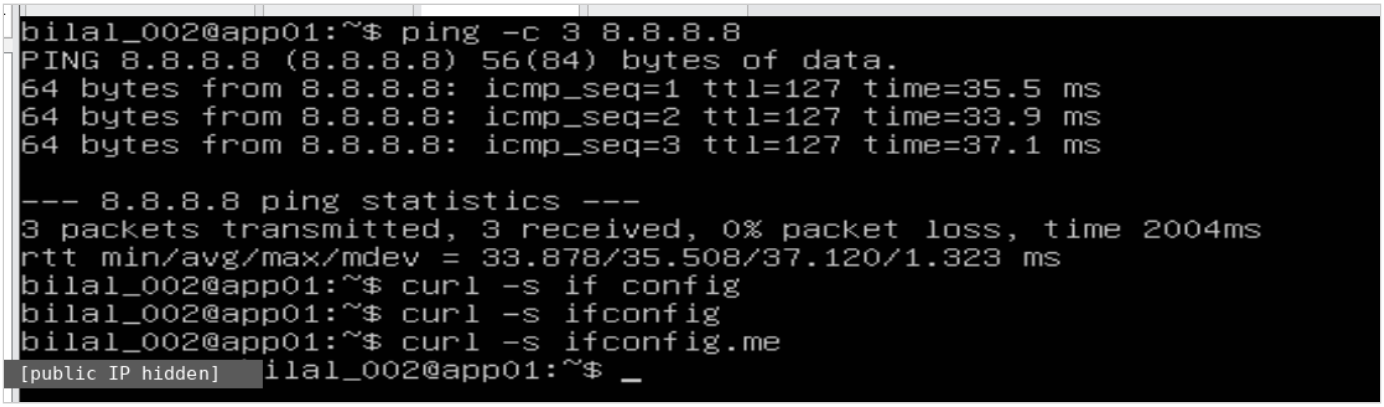
Rule	Plain words	Technical
`MASQUERADE`	"write my return address on outgoing mail"	Source NAT to the current address of ens33; chosen over SNAT because ens33 gets its address via DHCP and may change
`FORWARD ens34 → ens33 ACCEPT`	"staff may make outgoing calls"	Allows new outbound connections from the lab
`FORWARD ens33 → ens34 ESTABLISHED,RELATED`	"only call-backs are put through"	Stateful rule: inbound traffic allowed only if it belongs to a connection the lab started

Test from app01 (which has no internet card of its own)

🔧 **Commands (app01):**

```
ping -c 3 8.8.8.8      # can I reach a server on the internet? (-c 3 = send 3 packets)
curl -s ifconfig.me    # ask a website: "which address do you see me coming from?"
```

📸 **Evidence** (my home public IP is hidden for privacy):



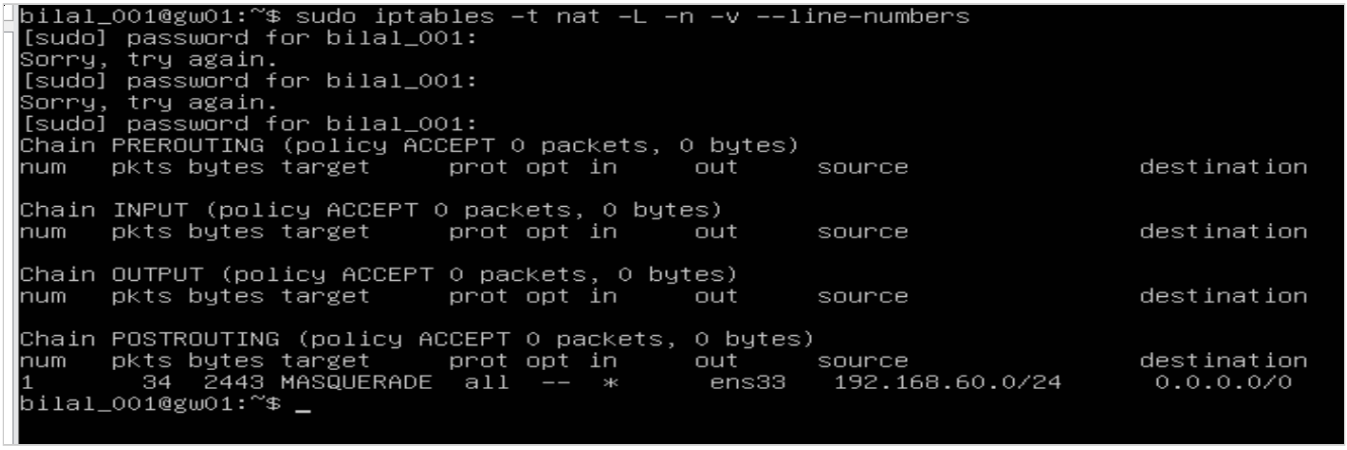
app01 reaches the internet through gw01

Test	Result	What it proves
`ping 8.8.8.8`	✅ 3/3 received, 0 % loss, avg 35.5 ms	app01 reaches the internet – **only possible through gw01**
`curl ifconfig.me`	✅ shows the home network's public address, **not** 192.168.60.10	NAT replaced app01's private address
TTL 127 (instead of 128)	each router lowers TTL by 1	the packets passed through one extra router: gw01

🔧 **Proof on gw01's side** – the NAT rule's packet counter:

```
sudo iptables -t nat -L -n -v --line-numbers   # list NAT rules with packet/byte counters
```

📸 **Evidence:** the MASQUERADE rule translated **34 packets (2443 bytes)** from 192.168.60.0/24 out through ens33 ("Sorry, try again" = mistyped sudo password):



NAT counters on gw01

> Note: `curl` prints the address without a line break, so the prompt appears glued to it (…`bilal_002@app01`). `curl -s ifconfig.me; echo` adds the line break.

---

6. Step 1.4 – Names instead of numbers (`/etc/hosts` and DNS)

💡 **In plain words:** computers only connect to **numbers** (IP addresses), but people – and tools like SSH, Ansible or file shares – prefer **names** like `stor01`. Before connecting, Linux has to look the name up, like a phone looking up a contact before dialing. It looks in two places, in this order:

1. **`/etc/hosts`** – a small **personal contact list** on each machine. Checked **first**.
2. **DNS** – the **public phone book** of the internet. Used only if the name is not in the contact list.

`getent` (= "get entry") asks the system exactly that question: *"what is the address for this name?"*

🔧 **Commands (app01):**

```
        getent hosts stor01                                    # look up "stor01" → nothing yet (unknown name)
        echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts   # add stor01 to app01's contact list (-a = append, don't
overwrite)
        getent hosts stor01                                    # look up again → now 192.168.60.20
        ping -c 1 stor01                                       # use the NAME to reach the server
```

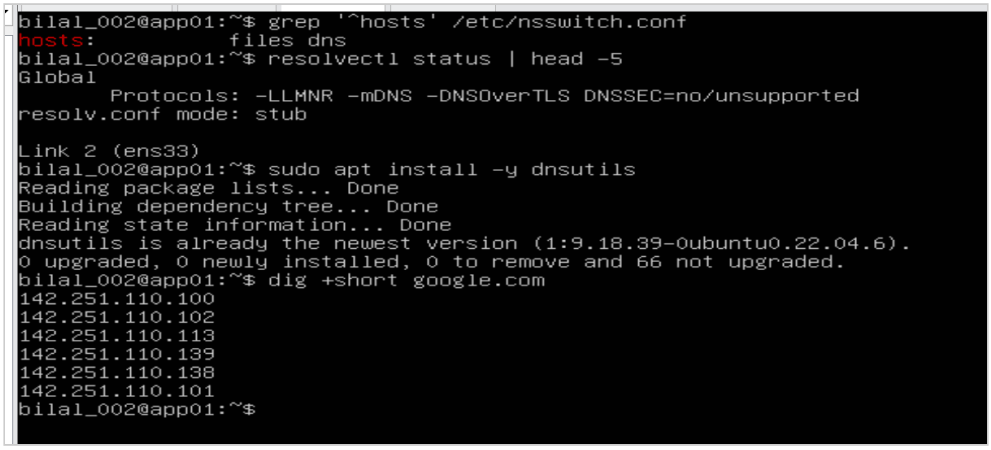
📸 **Evidence:** first lookup empty → entry added → lookup answers → ping by name works (`pinc` was a typo):

getent and ping by name on app01

🔧 **Commands (app01) – lookup order and DNS:**

```
        grep '^hosts' /etc/nsswitch.conf   # show the lookup ORDER for host names
        resolvectl status | head -5        # show the DNS resolver status
        sudo apt install -y dnsutils       # install dig (was already installed)
        dig +short google.com              # ask DNS directly for google.com's addresses
```

📸 **Evidence:**



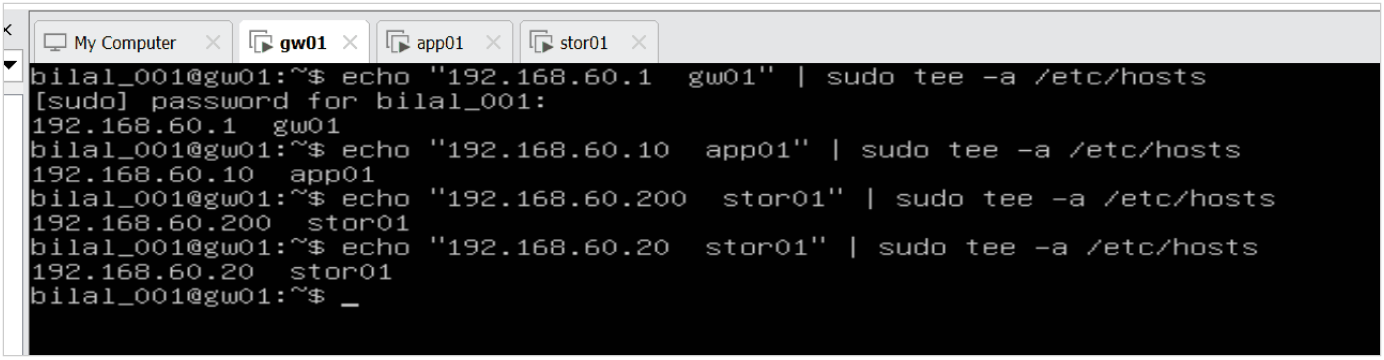
nsswitch order and DNS lookup on app01

Result	Meaning
`hosts: files dns`	`/etc/hosts` (“files”) is checked first, then DNS
`dig` → six `142.251.110.x` addresses	DNS works from app01 **through gw01’s NAT**

All three servers get the contact list

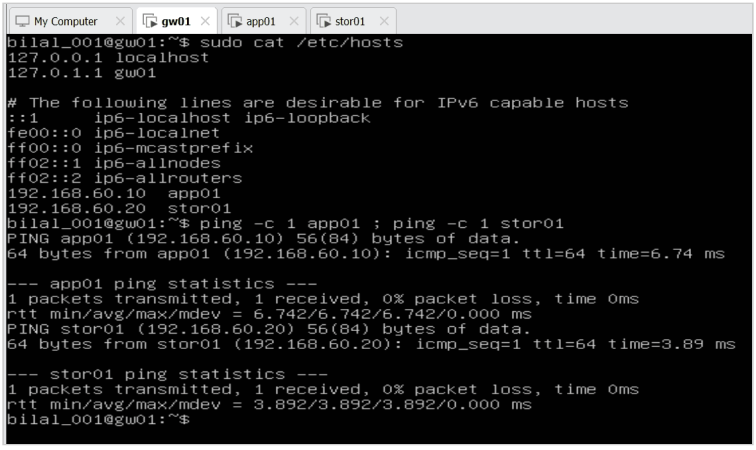
Each machine has its own `/etc/hosts`, so every server received the names of the other two (gw01: app01, stor01 · app01: gw01 · stor01: gw01, app01).

Accidental break test: on gw01 I first typed `192.168.60.200 stor01`, then the correct `.20` line. `/etc/hosts` is read top to bottom and the **first match wins**, so stor01 would have resolved to the wrong address – the exact fault the guide asks you to provoke. The wrong line was removed in nano; a similar slip on stor01 (`192.168.60.20 app01`) was corrected the same way.

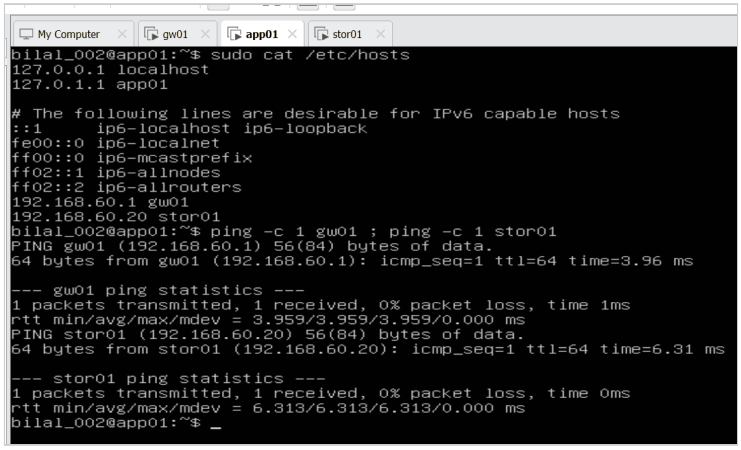


Typo on gw01

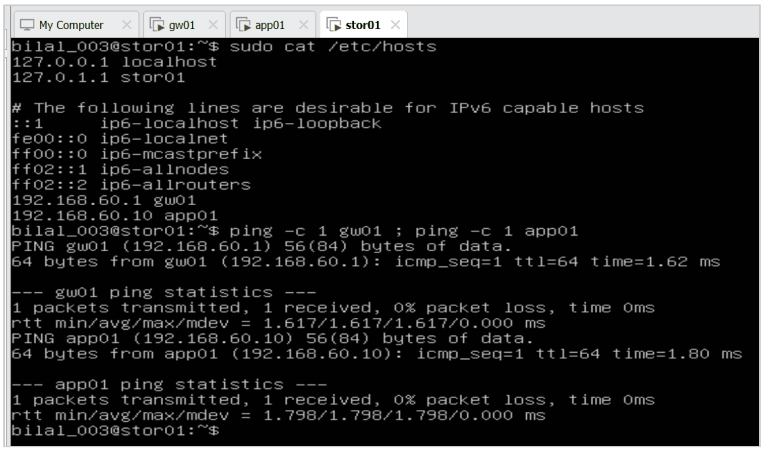
📸 **Evidence – every server reaches the other two by name:**



gw01 pings by name



app01 pings by name



stor01 pings by name

---

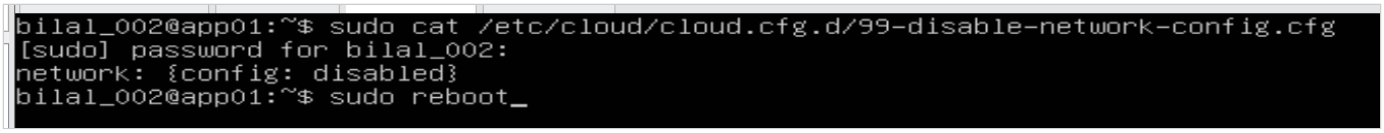
7. Results summary

Requirement	Status	Evidence
Fixed lab IPs on all servers, survive reboot	✅	sections 3.5–3.7
app01 / stor01 have no direct internet adapter	✅	ens33 DOWN on both
gw01 routes between networks	✅	`ip_forward = 1`
Lab reaches internet only via gw01 (NAT)	✅	ping 8.8.8.8 + ifconfig.me from app01
Inbound connections from the internet blocked	🟡	ESTABLISHED,RELATED rule in place; default FORWARD policy still ACCEPT – strict DROP follows in Phase 6. Lab currently unreachable from outside because of double NAT
Servers resolve each other by name	✅	all three, by name (`/etc/hosts`)
stor01 data disks present for Phase 2	✅	`sdb`, `sdc` 5 GB

8. Issues encountered

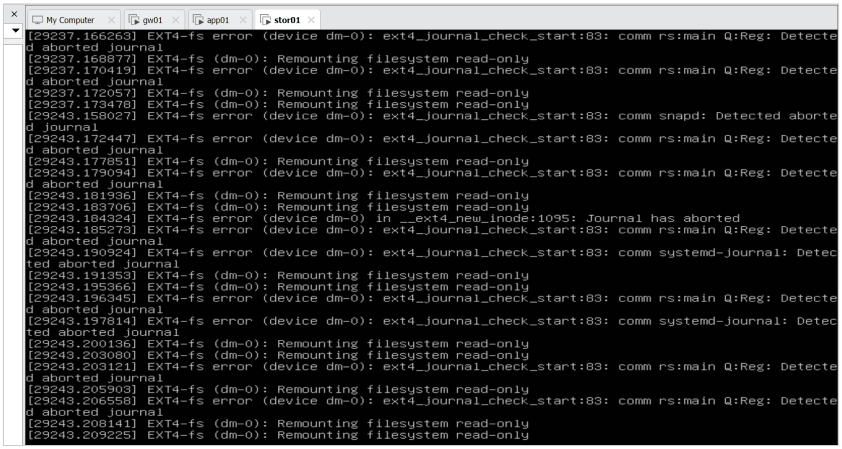
#	Symptom	Cause	Resolution
1	Netplan file named in the guide did not exist	Ubuntu 22.04 installer names it `50-cloud-init.yaml`	Checked with `ls /etc/netplan`, edited the real file, disabled cloud-init network management
2	YAML pasted into the VMware console arrived garbled (Ö instead of `:`, ü instead of `[`, staircase indentation)	VM keyboard layout differs from the host layout; vim auto-indents pasted text	Left vim without saving (`Esc`, `:q!`) and typed the file by hand in nano, using spaces
3	`netplan` warned that file permissions are too open	File readable by all users	`sudo chmod 600 /etc/netplan/50-cloud-init.yaml`
4	`netplan try` printed `Cannot call Open vSwitch`	netplan checks for Open vSwitch, which is not installed	Harmless, ignored – no OVS interfaces are used
5	app01 still had a 192.168.229.x address on ens33	NAT adapter still connected in VMware	Unticked *Connected* and *Connect at power on*, rebooted; ens33 now DOWN
6	VMnet2 not visible in the Virtual Network Editor	It is an isolated custom switch, not a host-only network	Confirmed as correct for the design
7	`lsblk` on stor01 showed only `sda`	The two planned 5 GB data disks had never been added	Powered off, added 2 × 5 GB SCSI disks; `sdb` and `sdc` now present
8	`curl -s if config` / `curl -s ifconfig` printed nothing	Typo / incomplete hostname; `-s` (silent) hides the error	Used the full name `ifconfig.me`; drop `-s` when troubleshooting
9	`ssh bilal_001@192.168.229.155` from Windows PowerShell timed out, although `sshd` was running and listening on port 22	Not investigated further: direct SSH from the Windows host is not part of the design (admin access comes through the WireGuard VPN in Phase 6A)	Deferred; console access used meanwhile
10	On app01 the `tee` output of the cloud-init line appeared as `#network: …`	A `#` makes the line a comment (ignored) – a typing slip	Rewrote the file; `cat` confirms `network: {config: disabled}` (screenshot below)
11	Typos in `/etc/hosts` (`.200` for stor01 on gw01, `.20` for app01 on stor01)	Typing slips; first matching line wins	Verified with `cat`/`getent`, corrected in nano
12	stor01: `EXT4-fs error … Detected aborted journal, Remounting filesystem read-only` (screenshot below)	Virtual disk interrupted on the Windows side (host disk space, sleep or forced stop); Linux protected the data by going read-only	Restarted the VM from VMware; filesystem writable again

📸 Issue 10 – cloud-init file verified on app01 after the fix:



cloud-init file correct on app01

📸 Issue 12 – stor01 filesystem switched to read-only:



stor01 read-only incident

9. Key understanding (in my own words)

These are the conclusions I reached by questioning each step until it made sense. They are written so I can explain them in an interview.

1. The network file and cloud-init

- `/etc/netplan/50-cloud-init.yaml` is the main network settings file: IP address, gateway and DNS. It was written once by **cloud-init** when Ubuntu was installed. At every boot **netplan reads it** and configures the network cards.
- cloud-init is the **writer**, netplan is the **reader**. cloud-init could rewrite the file on a later boot and erase my settings.
- So I created `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` with `network: {config: disabled}`. This only stops **cloud-init from managing the network** – the network itself and netplan keep working.
- Files in a `.d` folder are read in alphabetical order and **the last one wins**, so a `99-` file has the final word. The numbers only matter between files in the **same folder** – 50 (netplan) and 99 (cloud-init) are never compared.
- In Proxmox or the cloud, cloud-init is used the other way round: the admin enters IP, user and SSH key in the hypervisor, and cloud-init writes them into the server at first boot – ideal for cloned templates. In my VMware lab nothing feeds cloud-init, so I configure netplan by hand and keep cloud-init away from the network.

2. What the network file says

- `addresses: [192.168.60.10/24]` – **I am** 192.168.60.10; `/24` means 192.168.60.0–255 are my direct neighbours.
- `routes: - to: default via: 192.168.60.1` – "default" is not a machine, it means **everything I don't know**; `via` names who to hand it to: gw01.
- `nameservers: [1.1.1.1, 8.8.8.8]` – the internet's phone books. A name is first looked up in `/etc/hosts` (my own contact list – that is how `stor01` is found), and only then asked from DNS (that is how `google.com` is found).
- netplan only sets **addresses, gateway and DNS**. Being a router (ip_forward), NAT and firewall (iptables) and VPN (WireGuard) are set elsewhere.

3. The path to the internet

> app01 is 192.168.60.10 and has no internet adapter. For anything outside the lab it goes via gw01 (192.168.60.1), its default gateway. gw01 translates app01's private address into its own (NAT) and passes the traffic on through VMware and the home router to the internet. stor01 works the same way with 192.168.60.20.

4. NAT, forwarding and conntrack

> NAT recognises lab traffic by its IP range (`-s 192.168.60.0/24`) and changes the sender address as it leaves via `-o ens33`. Forwarding decides who may pass using the cards (`-i ens34 -o ens33`). Replies are let back in only if conntrack – gw01's memory of every conversation – confirms they are ESTABLISHED or RELATED to a conversation the lab started. conntrack also remembers which lab machine asked, so NAT can return each reply to the right one.

- **Two jobs, two tables:** the NAT table *changes the address* (`-j MASQUERADE`); the filter table's FORWARD chain *gives permission* (`-j ACCEPT`). A packet is checked by FORWARD first and rewritten in POSTROUTING just before it leaves – whatever order the commands were typed in.
- **`-s`/`-d` take IP addresses, `-i`/`-o` take card names.** `-s ens34` is always wrong. In POSTROUTING only the exit card is known, so `-i` is not allowed there – the lab must be identified by its IP range.
- **States:** NEW = someone starts a conversation; ESTABLISHED = a reply in a conversation the lab started; RELATED = a helper message (e.g. an error) belonging to it. `sudo conntrack -L` on gw01 shows the live list.
- **Honest limit today:** Ubuntu's default FORWARD policy is still ACCEPT, so the "only replies come back" rule is not yet enforced strictly. Right now the lab is protected because its private addresses sit behind two layers of NAT and cannot be reached from the internet. The strict version comes in Phase 6, when the default policy becomes DROP.
- `netfilter-persistent save` must run **last** – it saves only the rules that exist at that moment.

5. Names: /etc/hosts, DNS, getent and ping

> **getent** ("get entries") is like searching the phone book. It looks up a name or IP first in `/etc/hosts`, then in **DNS** if one is configured. It works both ways: name → IP (forward lookup) and IP → name (reverse lookup, e.g. `getent hosts 192.168.60.20` → `stor01`). It only **reads** – it never contacts the machine.
>
> **ping** is like calling. It uses the **same lookup** (`/etc/hosts`, then DNS) to find the number, then really contacts the machine with **ICMP** messages – "are you there?" (echo request) and "yes, I'm here" (echo reply). ICMP has **no ports**. So getent can succeed while ping fails – if the machine is off, the network is broken, or a **firewall blocks ICMP**.
>
> In our lab there is no internal DNS server, so we put the names in `/etc/hosts`. The servers could already reach each other **by IP**; `/etc/hosts` lets them reach each other **by name**.

- **Phone picture:** the contacts app inside the phone = the system resolver (`/etc/hosts` → DNS); getent = opening the contacts and reading a number; ping = pressing *call* – it looks up the same contact, then dials; ICMP = the phone network that carries the call.
- **Troubleshooting rule:** getent fails → a **name** problem (missing/wrong entry or DNS). getent works but ping fails → a **network** problem (machine off, cabling, firewall).

- `/etc/hosts` is a file on **each** machine – fine for a small lab, but 100 servers means 100 files to edit. Companies use an **internal DNS server** for their own names (as the Windows Server does in Project 1) and **public DNS** (1.1.1.1, 8.8.8.8) for internet names.
- **The DNS helper and loopback:** on Ubuntu, programs do not ask 1.1.1.1 directly – they ask a small helper on the same machine, **systemd-resolved**, at **127.0.0.53**. Every **127.x.x.x** address is **loopback** – the machine talking to itself, like putting a note in your own pocket: the traffic never leaves the machine (`lo` in `ip -br a`). The helper answers from its **DNS cache** – short-term memory of name lookups for the whole machine – or asks the real DNS servers through gw01. The cache speeds things up, expires by itself (**TTL**), and can be cleared with `sudo resolvectl flush-caches`. `resolvectl status` shows which DNS servers are in use.
- **DNS troubleshooting in three commands:** `getent hosts <name>` (does the name resolve at all?) → `resolvectl status` (which DNS servers?) → `dig +short <name>` (does DNS itself answer? dig skips `/etc/hosts`).
- `hostname` shows the machine's own name; `getent hosts <name>` asks what address a name has; `tee -a /etc/hosts` appends a new entry.

10. Lessons learned

- **Check, don't assume:** the real file name (`ls /etc/netplan`) differed from the documentation.
- **Back up before editing** any configuration file.
- cloud-init can silently undo manual network changes; disabling its network module makes the admin the owner of the configuration.
- `netplan try` + a reboot test = safe change: automatic undo if wrong, proof of persistence if right.
- Network isolation is enforced at the **hypervisor** (unplugged virtual cable), not only inside the OS.
- Verify hardware against the plan early (`lsblk`) – a missing disk found now costs one minute; found in Phase 2 it blocks the whole storage phase.
- `ip_forward` + MASQUERADE + a stateful FORWARD rule is the minimal recipe for a Linux NAT gateway; the replies-only rule keeps the lab closed to inbound connections.
- Name resolution order (`files` before `dns`) explains why a stale `/etc/hosts` entry can break a healthy service.
- Pasting into a VM console is unreliable across keyboard layouts; typing short config files is faster than debugging garbled input.

11. Next steps

1. **Step 1.5:** ports and sockets – who is listening where (`ss`, `lsof`, `nc`).
2. **Step 1.6:** SSH key login from gw01 to app01 and stor01 (gw01 later becomes the Ansible control node).

Project 2 – Acme Analytics Linux Platform

Day 2 · Phase 1, Steps 1.5–1.9: ports, SSH keys, links and inodes, users and groups, Linux vs cloud NAT

Muhammad Bilal Javed · 15–16 October 2026

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1 – Network foundation: 1.5 ports and sockets · 1.6 SSH keys · 1.7 links and inodes · 1.8 users and groups · 1.9 NAT comparison
Dates	15–16 October 2026
Hosts worked on	app01, stor01, gw01
Status	Step 1.5 ✅ · Step 1.6 ✅ – password-free SSH, remote commands and scp from gw01 to both servers · Step 1.7 ✅ – hard links, soft links, inodes · Step 1.8 ✅ – users, groups, service account · Step 1.9 ✅ – NAT comparison · **Phase 1 complete, snapshot taken**
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the commands, with a comment on every line. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary.

---

1. Objective

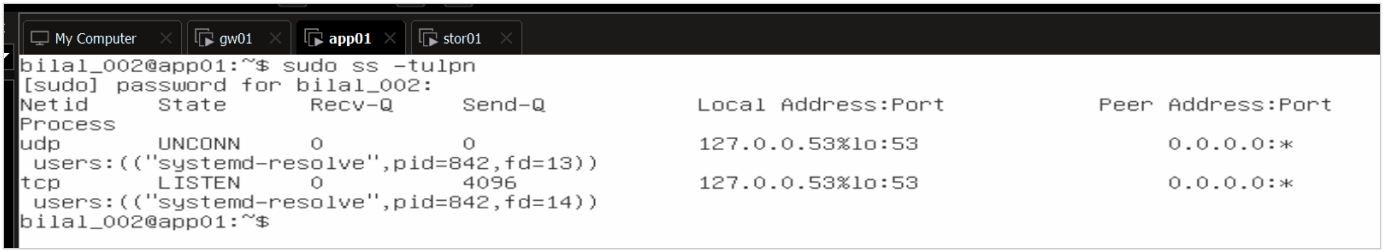
💡 **In plain words:** an IP address is the **address of a building** (which machine). A **port** is a **door number** inside that building (which program). Every service waits behind its own numbered door: SSH behind door 22, DNS behind door 53, a website behind door 80. Before opening new doors, I made an inventory of which doors app01 already has open – and to whom.

🔧 **Technical:** list all listening TCP and UDP sockets on app01 with the owning process, and interpret the result.

2. Procedure

```
sudo ss -tulpn   # ss = socket statistics
#  -t  TCP sockets        -u  UDP sockets
#  -l  only listening      -p  show the owning process
#  -n  numbers, not names  (memory aid: "tulpn" = tulip)
```

📸 **Evidence:**



ss -tulpn on app01

3. Reading the output

Column	Technical meaning	In plain words	In my output
Netid	Protocol: TCP or UDP	TCP = a phone call (confirmed); UDP = a postcard (just sent)	`udp` and `tcp`
State	LISTEN = TCP socket waiting for connections; UNCONN = open UDP socket (UDP never connects); ESTAB = an active TCP connection	LISTEN = a door open and waiting; UNCONN = a letterbox ready for letters	`UNCONN` (udp), `LISTEN` (tcp)
Recv-Q	Bytes received but not yet read by the program	Letters that arrived but are not opened yet	`0` – nothing waiting
Send-Q	On a LISTEN socket: the maximum queue of waiting connections	How many visitors may queue at the door	`4096` on the DNS helper's TCP socket; `0` for UDP (no queue)
Local Address:Port	IP and port on this machine where the socket listens	Building address + door number	`127.0.0.53%lo:53`
Peer Address:Port	The other end of a connection	Who is visiting	`0.0.0.0:*` – nobody yet, anyone may come
Process	Program name, process ID, file descriptor	Which program sits behind the door	`systemd-resolve`, pid 842, fd 13 (UDP) and fd 14 (TCP)

The address in detail: `127.0.0.53 % lo : 53` = the loopback address (127.x = the machine talking to itself), bound to the loopback card `lo`, on port 53 (DNS). Reachable **only from programs inside app01**.

fd (file descriptor): the number a program uses for something it has open – in Linux almost everything is a file, including network sockets. fd 13 and fd 14 are the DNS helper's two sockets.

4. TCP and UDP – the background

	TCP	UDP
Start	**3-way handshake:** SYN → SYN-ACK → ACK ("Can you hear me?" – "Yes, can you hear me?" – "Yes")	No handshake – it just sends
Lost data	Noticed and re-sent automatically – reliable	Lost for good – no check
Speed	Slower, but sure	Faster, no guarantee
In ss	LISTEN → ESTAB	UNCONN
Used for	SSH (22), websites (80/443)	DNS (53), video calls, games

DNS uses UDP for short questions and TCP for answers too large for one packet – that is why the DNS helper listens on both. The firewall rule `ESTABLISHED,RELATED` from Step 1.3 means exactly "packets belonging to a TCP conversation that already completed its handshake".

5. Final verdict

What	Result
Open ports on app01	**1:** port 53 (DNS), UDP + TCP
Program	`systemd-resolved` – the local DNS helper
Reachable from the network?	❌ No – bound to loopback only
SSH (port 22)	❌ Not listening – the package was installed but the service was not enabled
Security view	✅ Minimal attack surface – app01 exposes nothing to the network
Impact on the project	⚠️ No remote administration possible: SSH keys (Step 1.6) and Ansible (Phase 7A) need SSH

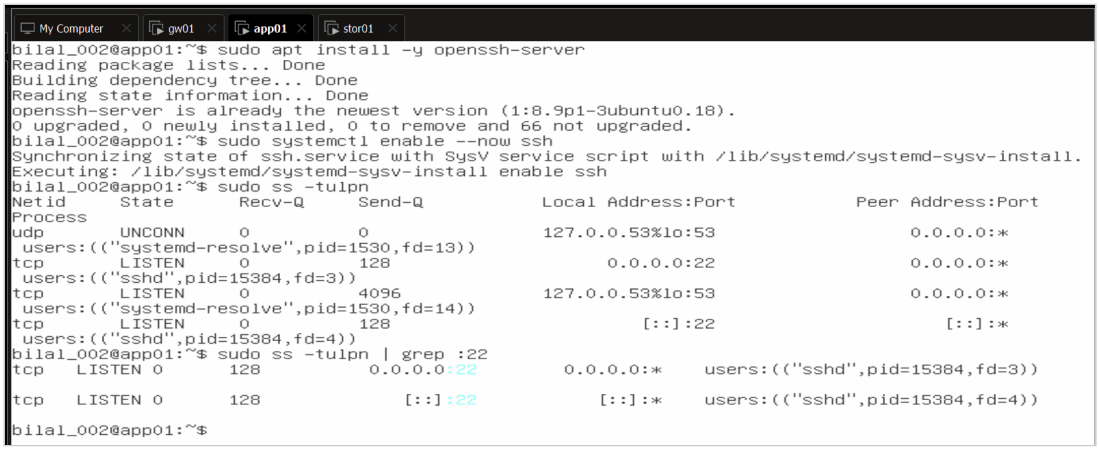
> **Before:** app01 exposed no network services at all – only the local DNS resolver on 127.0.0.53:53, reachable from inside the machine. The SSH package was installed but its service was not running, so it had to be enabled for remote administration.

6. Fix: enable SSH on app01

🔧 **Commands:**

```
sudo apt install -y openssh-server   # make sure the SSH server package is installed
sudo systemctl enable --now ssh      # start it now (--now) AND at every boot (enable)
sudo ss -tulpn | grep :22            # verify: is anything listening on door 22?
```

📸 **Evidence:**



SSH enabled on app01, listening on port 22

Line	Meaning
`openssh-server is already the newest version`	The package was already installed – only the **service was not running**. `enable --now` started it and made it start at every boot
`tcp LISTEN 0.0.0.0:22 … sshd`	SSH waits on door 22 on **all IPv4 addresses** – reachable from gw01 and stor01
`tcp LISTEN [::]:22 … sshd`	The same for IPv6 (`[::]` = all IPv6 addresses)
Send-Q 128	sshd's waiting queue – up to 128 connections may queue
`systemd-resolve` pid 1530 (before: 842)	New process number after the VM restart – normal

Before → after: app01 went from **0 doors open to the network** to **1** – SSH on port 22.

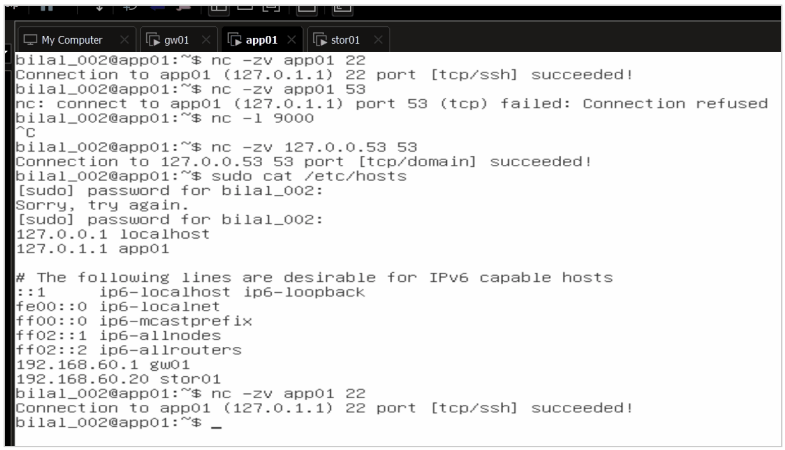
7. Testing a port: `nc -zv` and the binding address

💡 **In plain words:** ping only proves a machine is alive. To prove a **service** answers, you knock on its **door** (port). `nc` (netcat) does exactly that – the admin's standard check when "ping works but the application doesn't".

🔧 **Commands (on app01 – the machine calling itself):**

```
nc -zv app01 22        # -z: only check the door, send no data · -v: print the result
nc -zv app01 53        # same for door 53
nc -zv 127.0.0.53 53   # door 53 at the DNS helper's own address
cat /etc/hosts         # where the name "app01" points to
```

📸 **Evidence:**



Port tests on app01 with nc -zv

Test	Result	Why
`app01:22` (= 127.0.1.1)	✅ succeeded	sshd listens on `0.0.0.0` = **every** address of the machine
`app01:53` (= 127.0.1.1)	❌ Connection refused	The DNS helper listens **only on 127.0.0.53** – a different address
`127.0.0.53:53`	✅ succeeded	The right address – the DNS service **is** running

Three loopback addresses (the whole range 127.x.x.x means "this machine itself", all on the card `lo`):

Address	Created by	Seen in	Purpose
`127.0.0.1`	Worldwide standard	`/etc/hosts` → `localhost`	"Myself" for any program
`127.0.1.1`	Ubuntu installer (hostname)	`/etc/hosts` → `app01`	The machine finds its own name without a network
`127.0.0.53`	systemd-resolved	`/etc/resolv.conf` → `nameserver`	The DNS helper's private address

Refused vs. timeout – two different failures:

Result	What happened	Usual cause
Connection refused	The machine answered "nobody here"	Service not running, or listening on another address
Timeout	No answer at all	Firewall silently dropping, network broken, machine off

Refused points to the **service**; *timeout* points to the **network or firewall**.

8. Key understanding (in my own words)

> **Netid** is the protocol, TCP or UDP. **TCP** starts every conversation with a 3-way handshake (SYN, SYN-ACK, ACK); after that the connection is **ESTAB**. **UDP** has no handshake – it just sends.

Every port leads directly to its own program. The port number decides which program receives the traffic: port **22** → **sshd** (SSH), port **53** → the **DNS helper**, port **80** → a web server. Traffic does **not** pass through the DNS helper first. A port with no program behind it is closed – nobody answers.

LISTEN = a TCP door is open and waiting. **UNCONN** = a UDP door is open. **Recv-Q** = data received but not read yet (0 = nothing waiting). **Send-Q** on a LISTEN socket = how many visitors may wait in the queue (4096 for the DNS helper's TCP socket). **Local address** = this machine's address and door, e.g. `127.0.0.53%lo:53` = the DNS helper on the loopback card, port 53, reachable only from inside app01. **Peer address** = who is visiting – `0.0.0.0:*` = nobody yet; in a real connection it shows the visitor's IP and its random port, e.g. `192.168.60.1:51234` = gw01.

A server can test itself by calling its own address – loopback (127.x.x.x). `localhost` (127.0.0.1) and `app01` (127.0.1.1) both mean "myself". SSH listens on **all** addresses (`0.0.0.0`), so `localhost:22` and `app01:22` both work. The DNS helper listens on **one fixed address only, 127.0.0.53**, so only `127.0.0.53:53` works – any other address gets "connection refused". **The binding address matters:** `0.0.0.0` = the door is open at every entrance of the building; `127.0.0.53` = the door exists at one entrance only.

A port is a door number; a socket is a program standing at that door on a specific machine (IP + port + protocol); a connection is two sockets talking to each other.

DNS has one job: translate a name into an IP address – the phone book, not the phone line. Lookup order on app01: **`/etc/hosts`** (own contact list) → **local DNS helper** `127.0.0.53` (answers from its cache) → **outside DNS server** `1.1.1.1` / `8.8.8.8` through gw01. Once the IP is known, the real traffic goes directly to that IP and port. In a company, an **internal DNS server** answers internal names; in our lab `/etc/hosts` does that job.

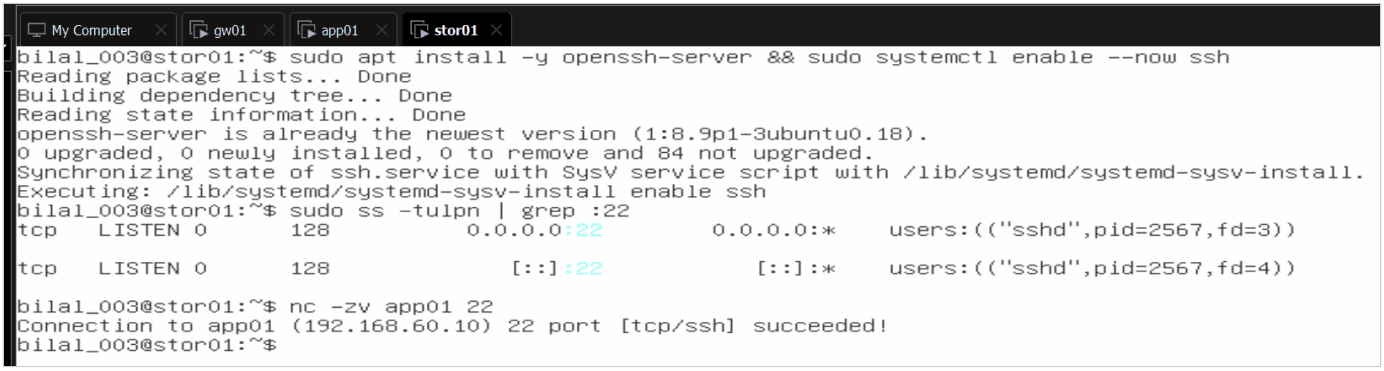
Before the fix, the only open port on app01 was 53, used by the local DNS helper, and only from inside app01 – it accepted **no connections from the network**. Next: enable SSH so port 22 opens.

9. Step 1.5 finished – SSH reachable across the lab network

🔧 **On stor01:** enable SSH and test app01 **from another machine** (on gw01 SSH was already running; only checked):

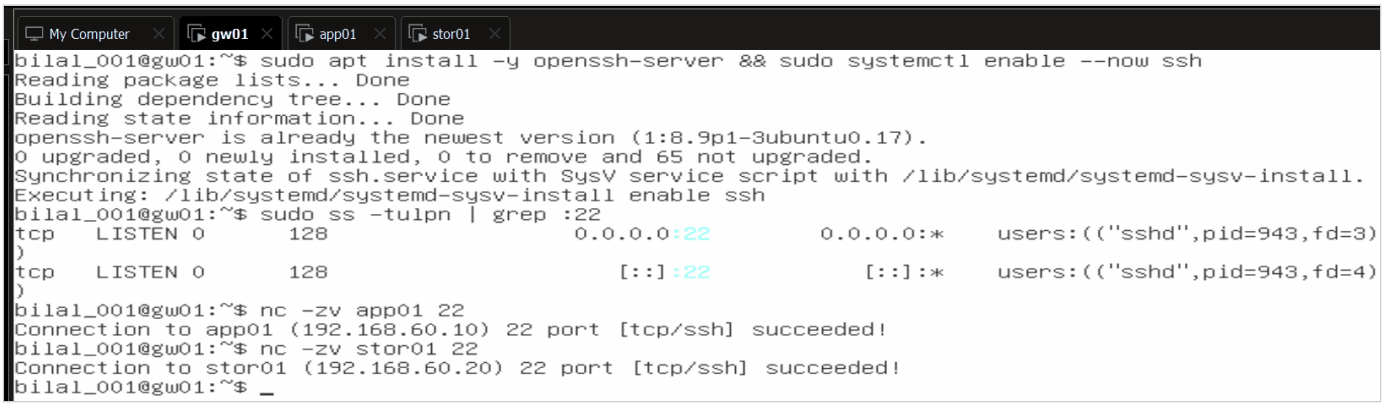
```
sudo apt install -y openssh-server && sudo systemctl enable --now ssh
sudo ss -tulpn | grep :22     # LISTEN 0.0.0.0:22 = door 22 open to the network
nc -zv app01 22               # knock on app01's SSH door over the lab network
```

📸 **Evidence – stor01:** SSH listening, and app01 reached at **192.168.60.10** (not loopback):



stor01: SSH enabled, nc to app01 succeeded

📸 **Evidence – gw01:** SSH listening, both servers reachable on port 22:



gw01: SSH listening, nc to app01 and stor01 succeeded

From	To	Result
stor01	app01 (192.168.60.10):22	✅ succeeded
gw01	app01 (192.168.60.10):22	✅ succeeded
gw01	stor01 (192.168.60.20):22	✅ succeeded

Step 1.5 complete: every server runs SSH on port 22 (`0.0.0.0` and `[::]`), reachable across the lab network.

10. Step 1.6 – SSH keys: log in without a password

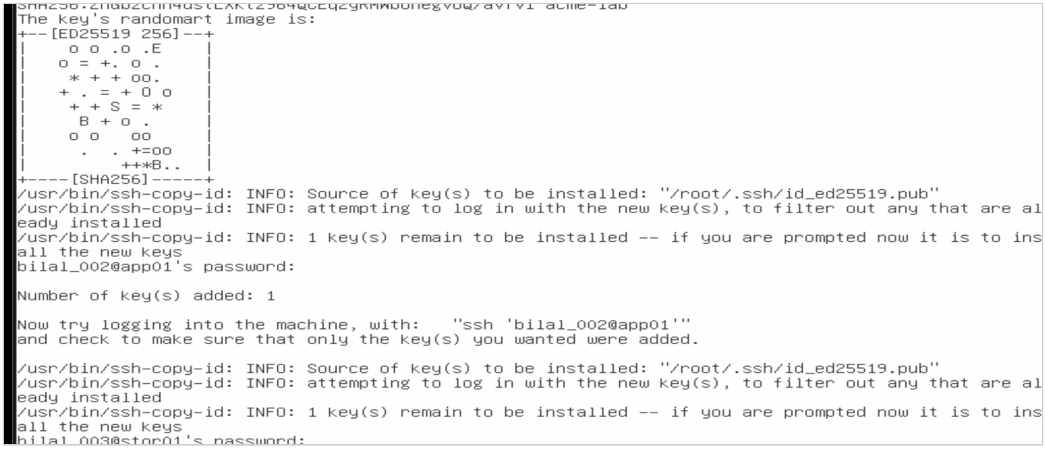
💡 **In plain words:** a password is something you **type**; an SSH key is a **pair of files** – a private key that never leaves gw01 (like a house key in your pocket) and a public key that is copied to each server (like a lock fitted to the door). If the key fits the lock, the door opens – no password. gw01 needs this because it becomes the **Ansible control node** in Phase 7A and must reach every server automatically.

🔧 **On gw01, as the normal user bilal_001 (not root):**

```
ssh-keygen -t ed25519 -C "acme-lab"   # create the key pair (Enter 3×: default path, no passphrase for the lab)
ssh-copy-id bilal_002@app01           # copy the PUBLIC key to app01 (asks the password once)
ssh-copy-id bilal_003@stor01          # same for stor01
```

Issue: key first created for root

📸 The first attempt ran as root – the key landed in `/root/.ssh/`, so password-free login would only work with `sudo ssh …`:



ssh-copy-id using /root/.ssh/id_ed25519.pub

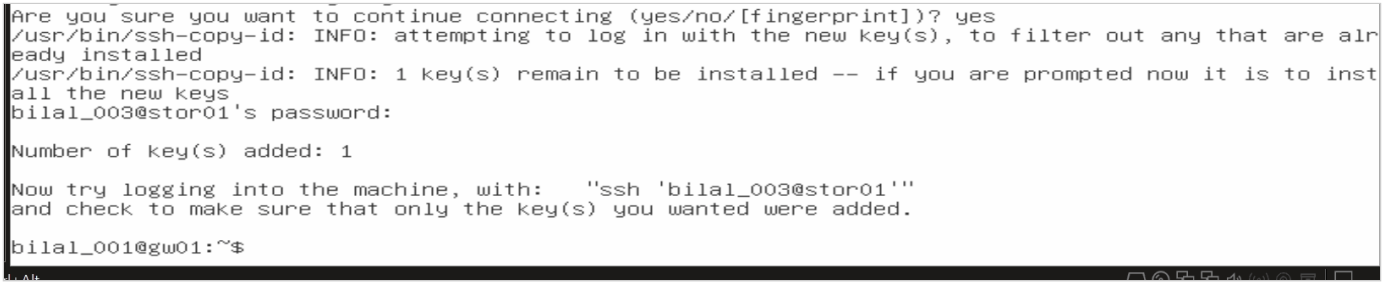
Fix: repeated as **bilal_001**, without `sudo`. The key now lives in `/home/bilal_001/.ssh/`. (The root key copied earlier will be removed during hardening in Phase 6 – servers should not trust gw01's root user.)

📸 **Evidence – correct key for bilal_001:**

ssh-keygen as bilal_001, key in /home/bilal_001/.ssh

"The authenticity of host … can't be established" – first visit to a server: gw01 asks whether to trust the server's ID (fingerprint). After `yes` it is stored in `~/.ssh/known_hosts`; if a server's ID ever changes, SSH warns – protection against impostors.

📸 **Evidence – key added to stor01 ("Number of key(s) added: 1"):**

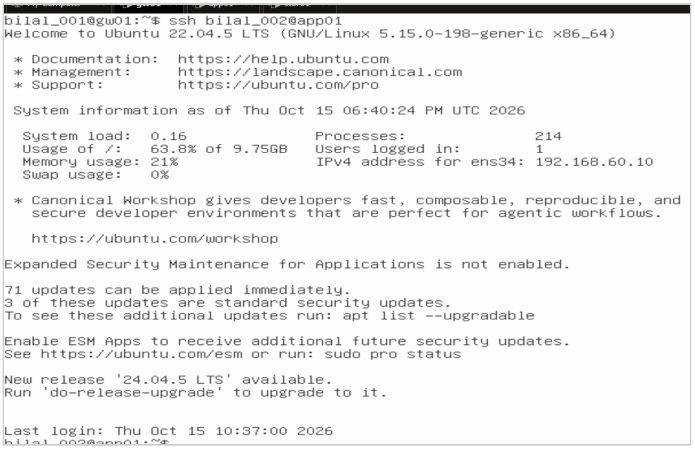


ssh-copy-id to stor01

The proof: login without a password

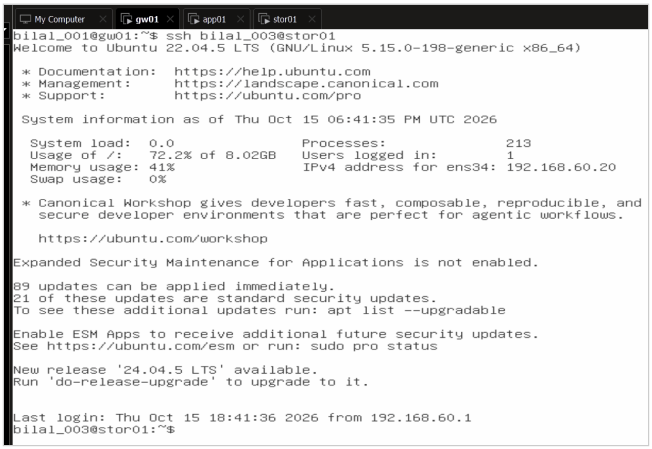
```
ssh bilal_002@app01     # log in to app01 — no password prompt
exit                    # close the session, back to gw01
ssh bilal_003@stor01    # log in to stor01 — no password prompt
exit                    # back to gw01
```

📸 **app01** – logged in with no password; the welcome screen confirms `IPv4 address for ens34: 192.168.60.10`:



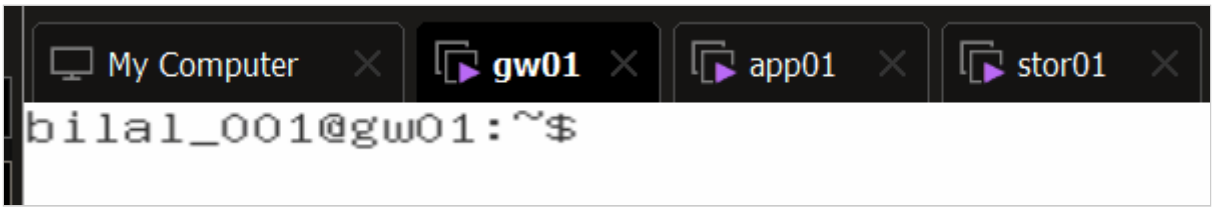
SSH login to app01 without password

📸 **stor01** – same; note `Last login … from 192.168.60.1` – Linux records who logged in from where (the start of an audit trail):



SSH login to stor01 without password

📸 **`exit`** closes the remote session – the prompt `bilal_001@gw01:~$` shows I am back on gw01:



Back on gw01 after exit

Habit: always read the prompt before typing – `user@host` tells you which server the next command runs on.

Observed on the welcome screens	Meaning
app01 / 63.8 % of 9.75 GB · stor01 / 72.2 % of 8.02 GB	stor01's system disk is filling – data moves to the separate LVM disks in Phase 2
71 (app01) / 89 (stor01) updates, 3 / 21 security	to be applied during hardening (Phase 6)
"New release 24.04.5 available"	deliberately **not** upgraded – the lab stays on 22.04

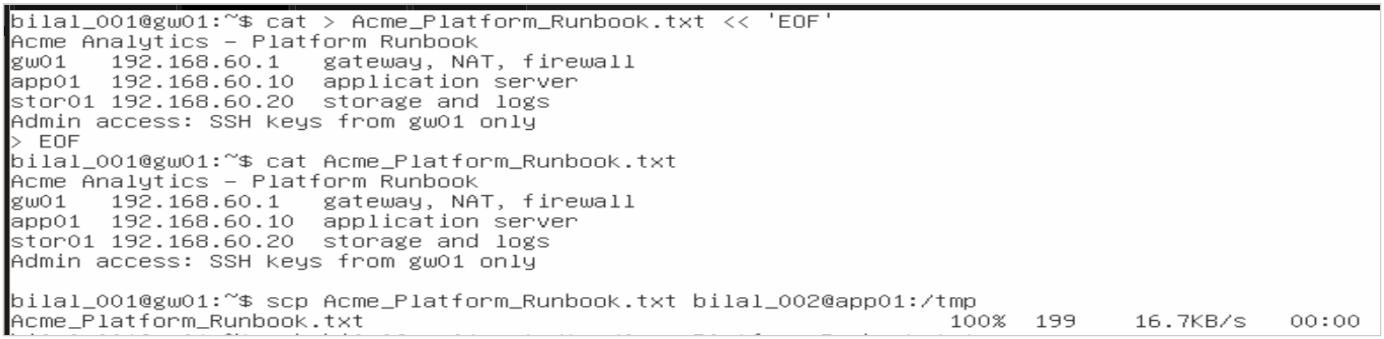
Copying a file over SSH: `scp`

💡 **In plain words:** `scp` (secure copy) sends a file to another server through the same encrypted SSH door – and because the key is installed, no password is needed. Instead of a throw-away test file I created a short **platform runbook** – the kind of handover note admins keep.

```
cat > Acme_Platform_Runbook.txt << 'EOF'      # write the lines below into the file …
Acme Analytics - Platform Runbook
gw01   192.168.60.1   gateway, NAT, firewall
app01  192.168.60.10  application server
stor01 192.168.60.20  storage and logs
Admin access: SSH keys from gw01 only
EOF                                           # … until the end marker EOF ("End Of File")
scp Acme_Platform_Runbook.txt bilal_002@app01:/tmp/        # copy it to app01's /tmp folder
ssh bilal_002@app01 cat /tmp/Acme_Platform_Runbook.txt     # run ONE command on app01 and show the result
```

Heredoc (`<< 'EOF'`): "write everything I type until you see EOF". The closing `EOF` stands alone on its line; the quotes keep the text exactly as typed. File names use underscores – spaces in Linux file names need quoting and break scripts.

📸 **Evidence – file created and copied (`100% 199 bytes`, no password):**



Runbook created with a heredoc and copied with scp

📸 **Evidence – read back on app01 with a one-shot SSH command:**

Runbook read on app01 via ssh

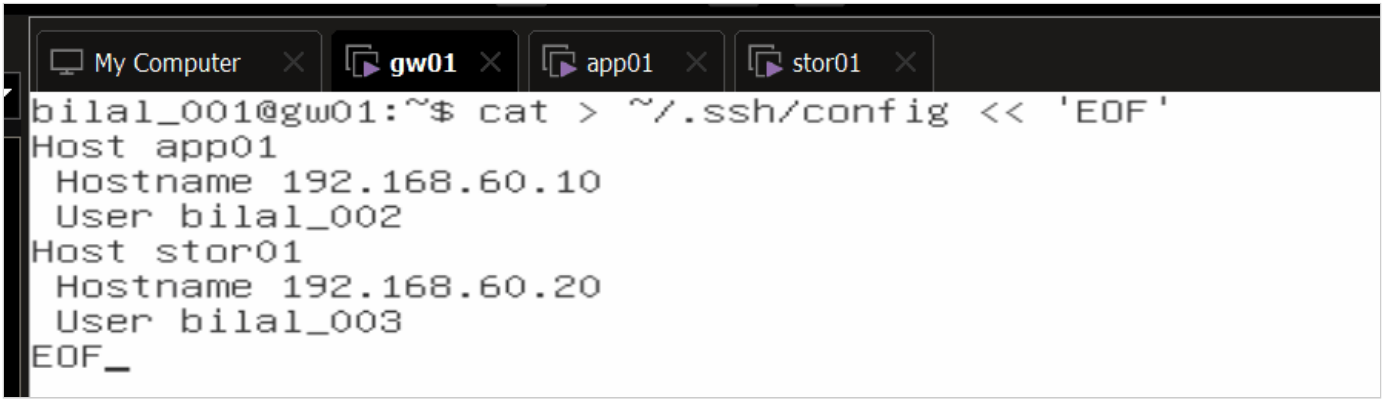
`ssh user@host command` runs a single command on the remote server and returns – the same pattern Ansible uses later.

SSH shortcuts: `~/.ssh/config`

💡 **In plain words:** a personal address book for SSH – type `ssh app01` instead of `ssh bilal_002@192.168.60.10`.

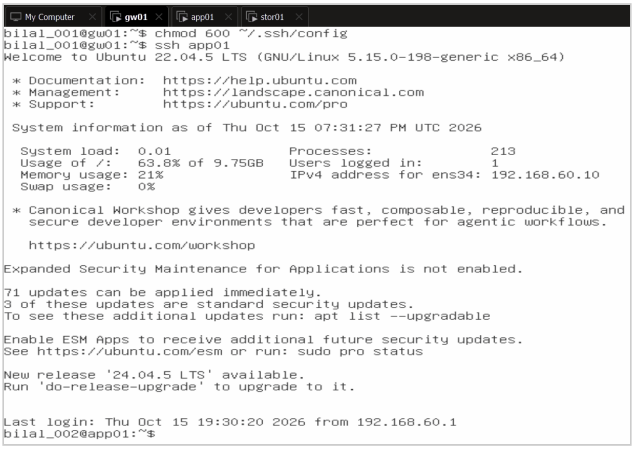
```
cat > ~/.ssh/config << 'EOF'
Host app01
    HostName 192.168.60.10
    User bilal_002
Host stor01
    HostName 192.168.60.20
    User bilal_003
EOF
chmod 600 ~/.ssh/config   # only the owner may read it – SSH refuses a config others can read
ssh app01                 # log in by short name – no user, no password
exit
ssh stor01
exit
```

📸 **Evidence – the shortcut file:**



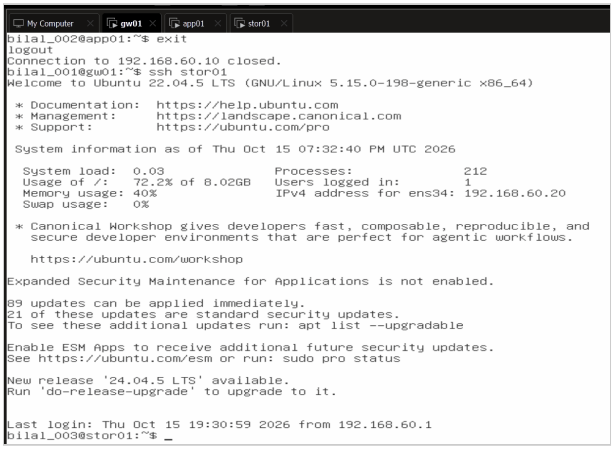
~/.ssh/config with Host entries for app01 and stor01

📸 **Evidence – `ssh app01` logs in as bilal_002 without a password:**



ssh app01 by alias

📸 Evidence – `exit` ("Connection to 192.168.60.10 closed"), then `ssh stor01`:



exit and ssh stor01 by alias

Interactive transfer: `sftp`

```
sftp app01                          # open a file session over SSH
sftp> ls                            # list files in the home folder on app01
sftp> cd /tmp                       # change folder on app01
sftp> get Acme_Platform_Runbook.txt # download to gw01
sftp> exit
```

📸 Evidence – runbook downloaded from app01 to gw01 (100 %):

sftp session: ls, cd, get

Why not FTP?

> **FTP sends usernames, passwords and files in clear text** – anyone listening on the network can read them. **SCP and SFTP run inside SSH**, so everything is **encrypted**, and with keys no password crosses the network at all. Companies do not use plain FTP for administration.

Tool	Use	Encrypted
`scp`	copy one file quickly	✅ (SSH)
`sftp`	browse and transfer interactively	✅ (SSH)
`ftp`	legacy file transfer	❌ clear text

Step 1.6 complete: gw01 reaches app01 and stor01 by key and by short name, runs remote commands and copies files with `scp` and `sftp` – all encrypted, all without a password.

11. Step 1.7 – Hard links, soft links and inodes

💡 **In plain words:** every file has an **ID card** (the *inode*) holding its owner, permissions, size, dates and where the data sits on disk. The **file name is only a label** in a folder pointing to that ID card. - A **hard link** = a **second label on the same ID card**. Delete one label, the file lives on. - A **soft link** = a **signpost** with its own ID card that only says “go to that name”. Delete the original, the signpost points to nothing.

🔧 **Technical:** a directory is a list of *name → inode number*. `ln` adds another directory entry for the same inode (link count +1). `ln -s` creates a new inode of type symlink that stores a path. The inode and its data blocks are freed only when the link count reaches 0 **and** no process still has the file open.

Why in this project: Phase 2 backs up app data to stor01 with `rsync --link-dest`. Every day’s backup folder hard-links the files that did not change to yesterday’s copy, so 30 daily backups look full-size but take little extra space. A soft link `/backups/latest` points to the newest day and is moved each night. The same signpost idea is used by nginx (`sites-enabled → sites-available`) in Phase 7. This lab proves the mechanism on real files first.

Procedure (on app01, as bilal_002)

```
mkdir -p ~/acme && cd ~/acme
echo "APP_PORT=8080" > app.conf          # an Acme config file
ln app.conf app.conf.backup              # hard link: second name, same inode
ln -s app.conf current.conf              # soft link: signpost holding the path "app.conf"
ls -li                                   # first column = inode number, third = link count
echo "LOG_LEVEL=info" >> app.conf.backup # write through the hard link
cat app.conf                             # the change appears here too — same file
stat app.conf | head -3                  # Inode and Links: 2
```

📸 **Evidence – same inode 276722 and link count 2 for both names; the soft link has its own inode 276727; an edit via the hard link shows in the original:**

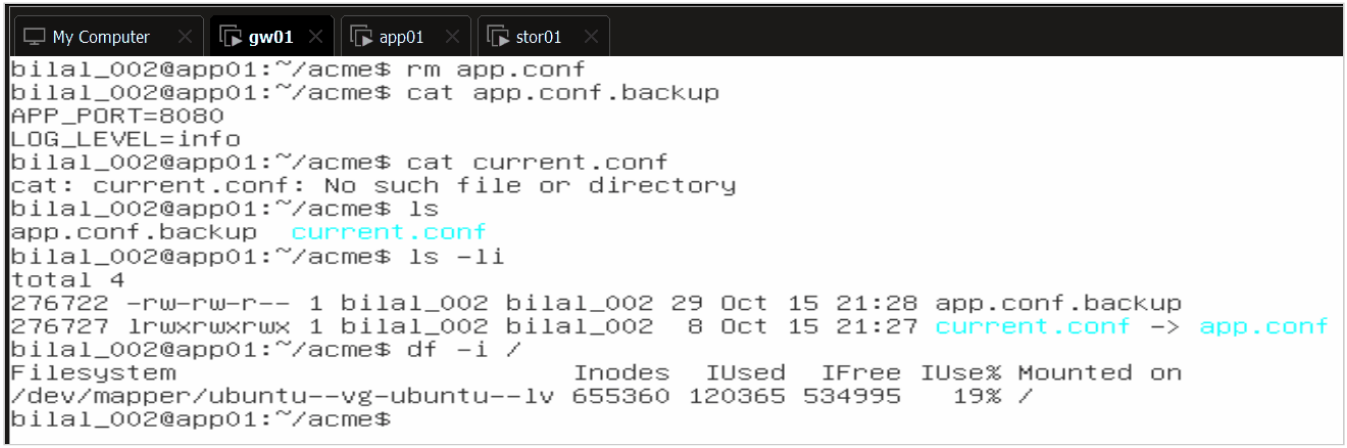
```
bilal_002@app01:~/acme$ ls -li
total 8
276722 -rw-rw-r-- 2 bilal_002 bilal_002 14 Oct 15 21:26 app.conf
276722 -rw-rw-r-- 2 bilal_002 bilal_002 14 Oct 15 21:26 app.conf.backup
276727 lrwxrwxrwx 1 bilal_002 bilal_002  8 Oct 15 21:27 current.conf -> app.conf
bilal_002@app01:~/acme$ echo "LOG_LEVEL=info" >> app.conf.backup
bilal_002@app01:~/acme$ cat app.conf
APP_PORT=8080
LOG_LEVEL=info
bilal_002@app01:~/acme$ stat app.conf | head -3
  File: app.conf
  Size: 29              Blocks: 8          IO Block: 4096   regular file
Device: fd00h/64768d    Inode: 276722      Links: 2
bilal_002@app01:~/acme$
```

ls -li, write through the hard link, stat

The “accidental delete” test

```
rm app.conf           # simulate a colleague deleting the file
cat app.conf.backup   # data is still there
cat current.conf      # "No such file or directory" — the signpost is broken
ls -li                # link count of the survivor is now 1; current.conf shown in red
df -i /               # inode capacity of the root file system
```

📸 **Evidence – hard link survives, soft link broken, inode usage 19 %:**



rm original, hard link survives, soft link broken, df -i

Reading the results

Observation	Meaning
`276722 … 2 … app.conf` and `276722 … 2 … app.conf.backup`	One file, two names (link count 2)
`276727 lrwxrwxrwx … current.conf -> app.conf`	Separate inode; type `l` = symlink; size 8 = the length of the text "app.conf"
`LOG_LEVEL=info` appears in `app.conf`	A hard link is **not a copy** – an edit through either name changes the one file
After `rm`: `app.conf.backup` still readable, link count **1**	Data is freed only when the last name is gone
`current.conf` shown in red, `cat` fails	Dangling soft link – it points to a name that no longer exists
`df -i /`: 655,360 inodes, 120,365 used, **19 %**	Each file uses one inode; when inodes run out you get "No space left on device" even with free GB – worth monitoring (Phase 5A)

Key understanding (in my own words)

> Linux works with inode numbers; the name I type is only a label in a folder. A **hard link** is a second label on the same inode – deleting one name loses nothing, but it does **not** protect against bad edits, because it is the same file. A **soft link** has its own inode and only holds a path – deleting it never harms the original, but deleting the original breaks it. Hard links protect important files against accidental deletion; real protection against bad changes still needs backups (Phase 2).

Step 1.7 complete. `~/acme` stays on app01 as sample data.

12. Step 1.8 – Users, groups and a service account (continued 2026-10-16)

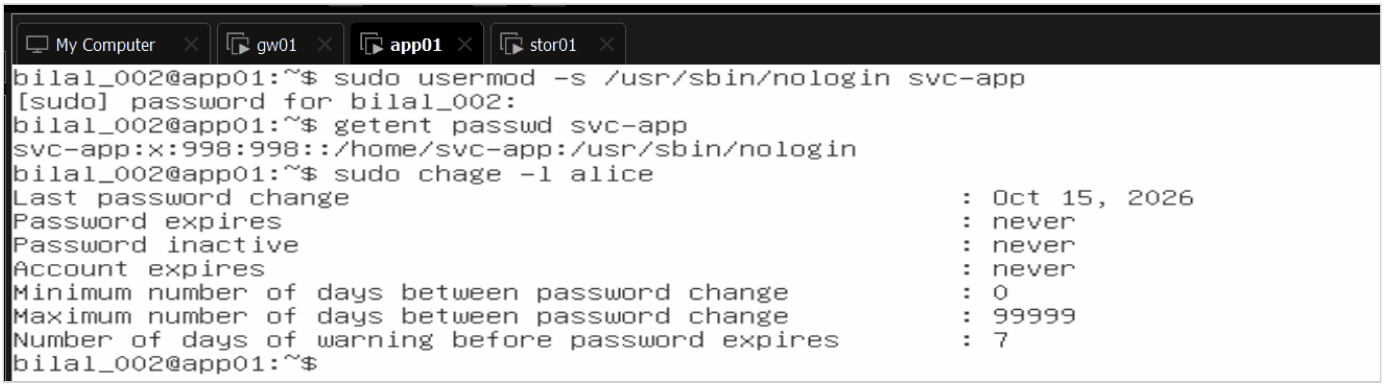
💡 **In plain words:** every person gets their own account, people who do the same job share a **group**, and every **program** gets its own low-rights account that nobody can log in to. When someone leaves, their account is **locked**, not shared or forgotten.

Why in this project: the security questionnaire in the brief asks "who can log in, and with which rights?". This step answers it: named users, a team group, and a no-login service account. In Phase 3 the Acme app's systemd service runs as `svc-app`; in Phase 6 groups decide who may use `sudo` and Docker; in Phase 7A Ansible automates exactly this joiner/leaver process.

Procedure (on app01)

```
sudo groupadd devs                                  # team group (it already existed)
sudo useradd -m -s /bin/bash -G devs alice          # -m home folder, -s shell, -G extra group
sudo useradd -m -s /bin/bash -G devs bob
sudo useradd -r -s /usr/sbin/nologin svc-app        # -r system account, nobody can log in
sudo passwd alice                                   # give alice a password
id alice                                            # uid, primary group, secondary groups
getent group devs                                   # members of devs
sudo chage -l alice                                 # password aging ("change age")
sudo usermod -L bob                                 # lock (leaver)
sudo passwd -S bob                                  # status: L = locked, P = password, NP = none
sudo usermod -U bob                                 # unlock (returner)
```

📸 **Evidence – svc-app has the shell `/usr/sbin/nologin` and system uid 998; alice's password aging:**

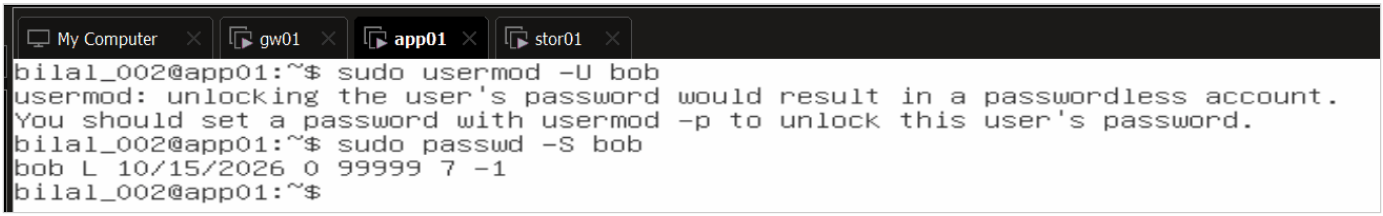


getent passwd svc-app and chage -l alice

📸 **Evidence – devs = alice, bob; svc-app in its own group only; bob locked (L):**

getent group devs, id svc-app, lock bob

📸 **Evidence – unlock refused, because bob has no password yet:**



usermod -U refused: passwordless account

Reading the results

Output	Meaning
`uid=1001(alice) gid=1002(alice) groups=1002(alice), 1001(devs)`	Primary group = her own private group; secondary group = devs
`devs:x:1001:alice,bob`	Both developers are in the team group
`svc-app:x:998:998::/home/svc-app:/usr/sbin/nologin`	uid **below 1000** = system/service account; shell `nologin` = no human login
`chage -l alice`: expires never, max 99999	No password expiry yet – to be set by policy in Phase 6 (e.g. `chage -M 90`)
`bob L 10/15/2026 0 99999 7 -1`	Status **L** = locked; then min/max/warn days and inactive (-1 = not set)
`usermod -U bob` refused	Safety feature: unlocking an account without a password would leave it open with **no password at all**. Set a password first (`sudo passwd bob`), then lock/unlock.

Option cheat sheet

Option	Meaning
`useradd -m`	make the home folder
`useradd -s` / `usermod -s` (small s)	set the shell
`useradd -G`	add to extra groups
`useradd -r`	system account (uid < 1000, no home folder) – low rights come from no extra groups and not being root
`usermod -L` / `-U`	lock / unlock
`passwd -S` (capital S)	show the password status
`chage -l` / `-M 90`	show aging / force a new password every 90 days

Key understanding – final summary

> On app01, the server where the Acme application runs, we created named accounts for each developer – alice and bob – each with their own home folder (`-m`) and login shell (`-s`), and put both in a shared team group `devs`, so rights are given to the team instead of to each person; each user has a primary group (their own) and a secondary group (devs). We then created `svc-app` as a **system account** (`-r`, uid 998, below 1000) with the shell `nologin` and no password: no human can log in as it, it exists only so a program can run under it. Every service runs as some user – the main sshd as root with its pre-login part as `sshd`, nginx as `www-data` – and giving each service its own low-rights account means a hacked service reaches only that account's own files, not root. `chage` shows and sets the password-aging rules, and `usermod -L` / `-U` lock and unlock an account – the leaver/returner process. Linux refused to unlock bob while he had no password, because a human account must have a password or key before it is opened. This step answers the Acme customer's security question "who can log in, and with which rights?", and it is the base for later phases: in Phase 3 the app runs as `svc-app`, in Phase 6 groups control `sudo` and Docker access, and in Phase 7A Ansible automates joiners and leavers.

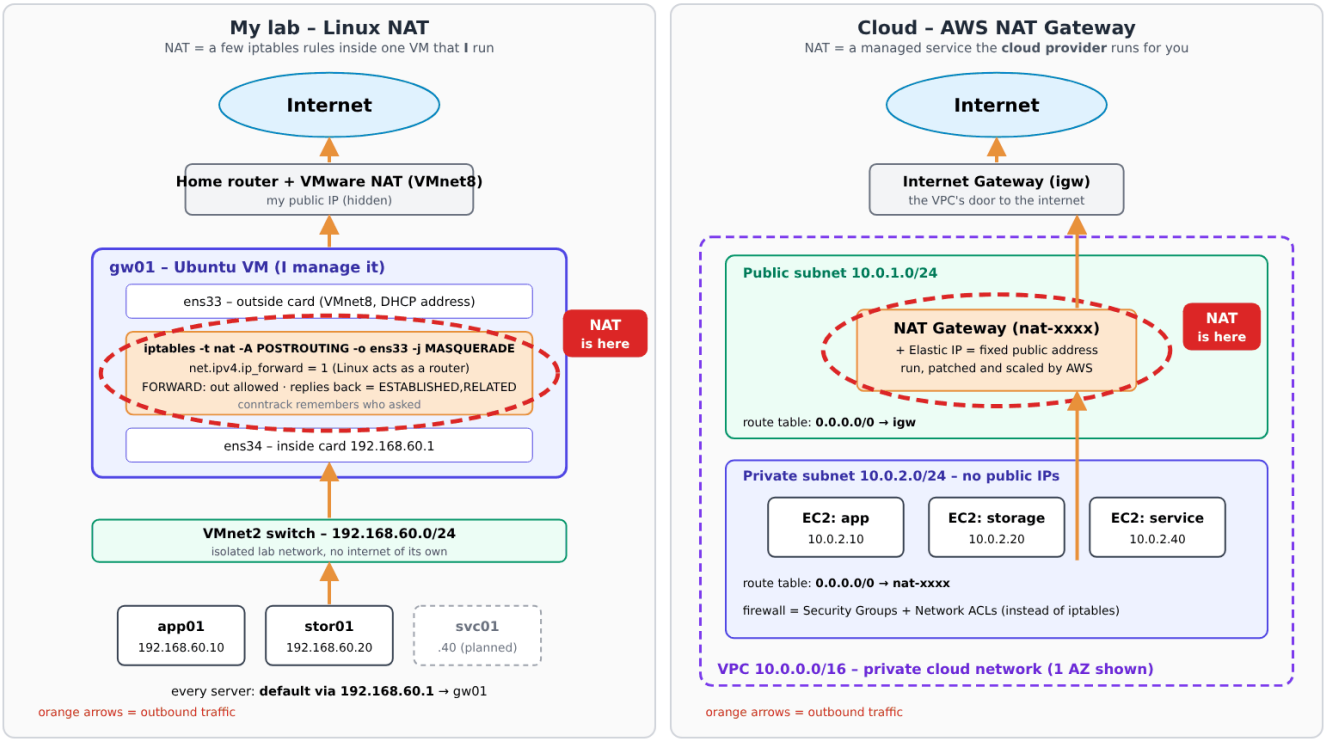
Step 1.8 complete.

13. Step 1.9 – Linux NAT vs Cloud NAT

💡 **In plain words:** NAT is the mailroom that swaps the private sender address for the company's public address on the way out and hands the replies back to the right person. In my lab I **build** that mailroom myself on gw01. In the cloud you **rent** it as a ready-made service.

Where does NAT live? Linux NAT (my lab) vs Cloud NAT (AWS)

Same job in both: private servers reach the internet, but the internet cannot start a connection to them. The red circle marks the NAT.



Same job: swap the private sender address for a public one on the way out, and let only the replies back in. **Difference:** in the lab I build, patch and keep the NAT alive; in the cloud the provider does, and charges per hour and per GB.

My lab piece	Cloud equivalent (AWS)	What it does
gw01 + iptables MASQUERADE	NAT Gateway	Rewrites the source address of outgoing traffic
ens33 / VMware NAT address	Elastic IP	The public address the internet sees
Home router / VMnet8	Internet Gateway	The door between the network and the internet
VMnet2 192.168.60.0/24	Private subnet	Servers with private addresses only
default via 192.168.60.1	Route table 0.0.0.0/0 → nat-xxxx	Sends unknown destinations to the NAT
iptables FORWARD rules	Security Groups + NACLs	Decides which traffic may pass

Muhammad Bilal Javed · Acme Analytics Linux Platform · Phase 1, Step 1.9

Where does NAT live? Linux NAT in my lab vs AWS NAT Gateway

Where the NAT sits

	My lab (gw01)	Cloud (AWS)
The NAT itself	3 kernel settings on one VM: `ip_forward = 1`, iptables `MASQUERADE`, `FORWARD` rules	A **NAT Gateway** placed in a **public subnet**
How servers find it	`default via 192.168.60.1` in netplan on each server	Private subnet route table `0.0.0.0/0 → nat-xxxx`
Public address	The address of gw01's outside card (VMware NAT / home router)	An **Elastic IP** attached to the NAT Gateway
Way out to the internet	Home router / VMnet8	**Internet Gateway** (public subnet route `0.0.0.0/0 → igw`)
Firewall	iptables `FORWARD` chain	Security Groups + Network ACLs

Comparison across providers

	gw01 (mine)	AWS NAT Gateway	Azure NAT Gateway	GCP Cloud NAT
Who runs it	Me	AWS (managed)	Azure (managed)	Google (managed)
Where	One VM	Public subnet; private route table `0.0.0.0/0 → nat-xxxx`	Attached to a subnet	Configured on a Cloud Router; no NAT VM at all (software-defined)
High availability	None – one VM is a single point of failure	Redundant inside one Availability Zone; one gateway per AZ for full resilience	Managed, zonal	Managed, distributed
Public IP	Address of the NAT adapter	Elastic IP	Public IP(s) or prefix	Auto-allocated or reserved IPs
Cost model	The VM I already run	Per hour + per GB processed	Per hour + per GB processed	Per hour + per GB processed

In three sentences

1. **What stays the same:** the job – outbound source translation, so private servers reach the internet while nobody on the internet can open a connection to them.
2. **What the cloud adds:** the provider runs, patches, scales and keeps the NAT available; in my lab, if gw01 is off, app01 and stor01 lose the internet.
3. **What I would watch:** cost – managed NAT is charged per hour **and per GB**, so heavy traffic (updates, backups, container image pulls) through it gets expensive; options such as VPC endpoints for cloud storage keep that traffic off the NAT.

Key understanding (in my own words)

> In my lab the NAT is not a box – it is three settings inside gw01: forwarding on, a MASQUERADE rule, and FORWARD rules that let replies back. Servers reach it because their default route points to gw01. In the cloud the same job is a managed NAT Gateway in a public subnet, reached through a route table entry and given a fixed Elastic IP; the provider keeps it running and bills per hour and per GB. Same job, different owner.

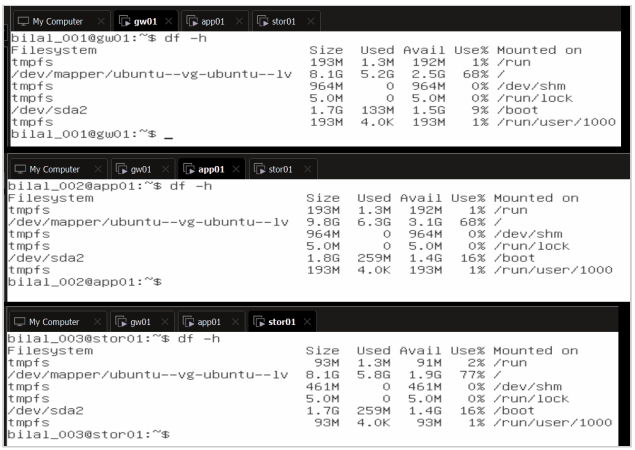
14. End of Phase 1 – updates and snapshot

All three VMs were updated (`sudo apt update && sudo apt -y upgrade`), shut down cleanly and given a VMware snapshot – the **restore point** for everything built in Phase 1.

VM	Snapshot name	Root disk	Used
gw01	`20-Network-gw01`	8.1 G	68 %
app01	`20-Network-app01`	9.8 G	68 %
stor01	`20-Network-stor01`	8.1 G	77 %

Description on every snapshot: *Phase 1 done: static IPs, NAT via gw01, hosts, SSH keys, users/groups, updates installed.*

📸 **Evidence – disk usage on gw01, app01, stor01 before the snapshot:**



df -h on all three VMs

💡 **Why:** a snapshot taken while the VM is **off** is clean and small (no memory saved). If Phase 2 (storage) breaks something, I return to this point in one click instead of rebuilding the network. stor01's root disk is the fullest, which is fine: backups will go to its separate data disks (`sdb`, `sdc`) with LVM in Phase 2.

Phase 1 complete ✅

15. Next steps

1. **Phase 0 leftover:** switch app01 and stor01 to server mode (`multi-user.target`).
2. **Phase 1A:** GLPI service desk – tickets, knowledge base and inventory on svc01.
3. **Phase 2:** storage – LVM on stor01's data disks, NFS share, backups with hard links, LVM snapshots.

Project 2 – Acme Analytics Linux Platform

Day 3 · Phase 1A: the service desk – svc01, storage, network story, GLPI running, asset inventory

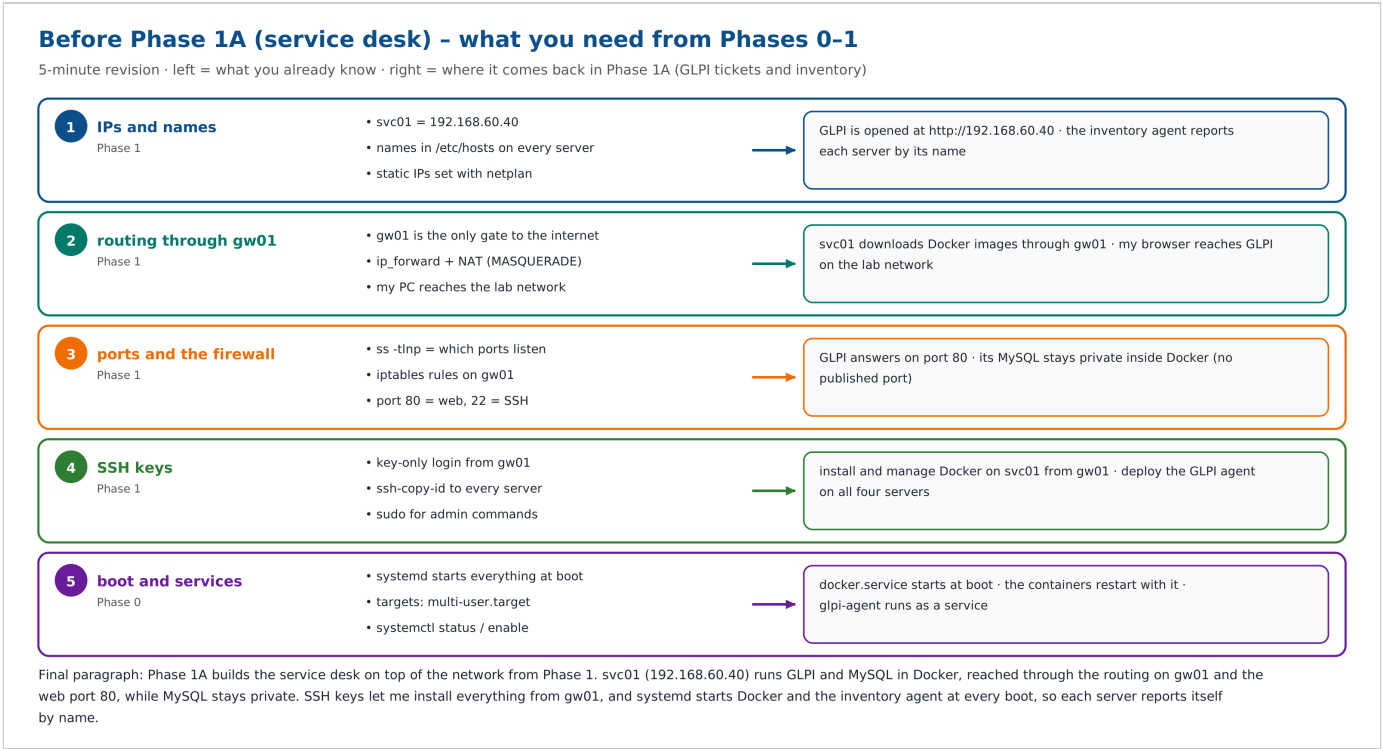
Muhammad Bilal Javed · 16–17 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1A – Service desk: GLPI tickets, knowledge base and asset inventory
Host worked on	svc01 (new VM)
Status	Step 1A.1 🟡 – svc01 installed, networked, names and SSH key working, disk grown to 23 GB, fully patched · Step 1A.2 ✅ – Docker + GLPI running, opened in the browser · Step 1A.4 ✅ – GLPI agent on all 4 servers, automatic inventory, snapshot `25-service-desk`
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary.

Before Phase 1A – quick revision (Phases 0–1)

💡 **5 minutes:** what I already know from Phases 0–1 and where it comes back in Phase 1A.



Before Phase 1A – quick revision

---

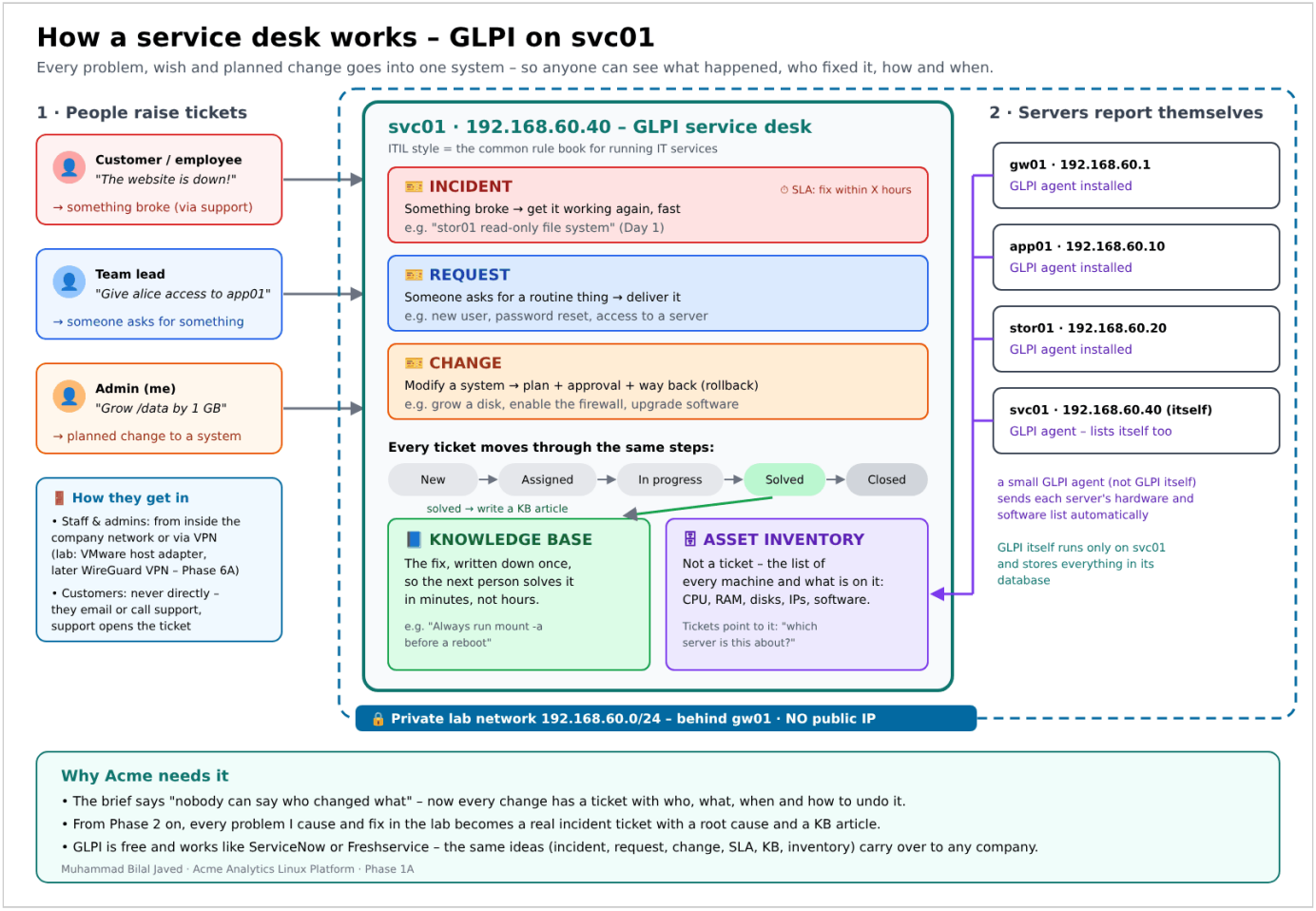
1. The story – why a service desk, and why now

💡 **In plain words:** until now I fixed problems and wrote them down in these reports. A real IT team does not work like that: every problem, every change and every “please give me access” goes into a **ticket system**, so anyone can see *what happened, who fixed it, how, and when*. Phase 1A gives Acme that system.

Where it fits in the Acme story. The brief says: *“nobody can say who changed what”* and *“the first enterprise customer’s security questionnaire asks questions the team cannot answer.”* The network (Phase 1) now works, so this is the right moment: from Phase 2 on, **every incident I cause and fix becomes a real ticket**, every planned change (e.g. “grow /data by 1 GB”) becomes a **change** ticket with a rollback plan, and every server appears in an **asset inventory** automatically.

Ticket type	Meaning	Example in this project
Incident	Something broke	“stor01 read-only file system” (happened on Day 1)
Change	A planned change with a plan and a rollback	“Grow /data by 1 GB”, “Enable UFW on stor01”
Request	Someone asks for a routine thing	“Give alice access to app01”, a password reset
Knowledge base (KB)	The written fix, so the next person solves it faster	“Always run `mount -a` before a reboot”
Asset inventory	Every server with CPU, RAM, disks, IPs and installed software	gw01, app01, stor01, svc01

📊 **The picture – how a service desk works:**



How a service desk works – GLPI on svc01

Key understanding (in my own words).

> In the IT industry a problem is an **incident**, and handling them is **incident management**. When someone asks for a routine thing – a new user, a password reset, access to a server – it is a **request**. When I modify a system itself – grow a disk, enable a firewall, upgrade software – it is a **change**, which needs a plan, approval and a way back. All three are **tickets** in GLPI, and every ticket moves through the same steps: new → assigned → in progress → solved → closed; a solved fix is written down in the **knowledge base**. The **asset inventory** is not a ticket: it is the list of every machine and what is on it, filled automatically by the GLPI agent on each server, so a ticket can say which server it is about. GLPI follows **ITIL**, the common rule book for running IT services, and is used by many European companies and public bodies.

Who can reach svc01? svc01 has **no public IP** – it sits on the private lab network behind gw01, like app01. Staff and admins reach it from inside the company network (in the lab: my PC through VMware’s host adapter; from Phase 6A through the WireGuard VPN). **Customers never reach it directly**: they email or call support, and support opens the ticket for them. A service desk holds the details of every server, so exposing it to the internet would be a security risk.

Why svc01 appears twice in the picture: in the middle it is the **GLPI application** (the help desk); on the right it is the **machine** svc01, which also runs a GLPI agent and lists itself in the inventory. GLPI itself runs only on svc01; the other servers get only the small agent.

Incident or change? “The server is broken, please update it” is an **incident** (something is broken). “The server runs an old version, upgrade it next Tuesday” is a **change** (planned). The SLA – how fast a ticket must be handled – is the IT team’s promise, not something the customer sets.

Why GLPI. GLPI is a free, ITIL-style service desk used by many European companies and public bodies. Freshservice or ServiceNow need a paid licence; GLPI teaches the same concepts – incident, change, request, SLA, KB, CMDB – and they carry over to any tool.

Why it matters for my job search. Many of the junior IT / DevOps postings I reviewed ask for ticket systems, SLAs, asset management or knowledge articles. In an interview I can say: *"I ran my lab through GLPI. Every incident I caused has a ticket, a root cause and a KB article, and every server is in the inventory."*

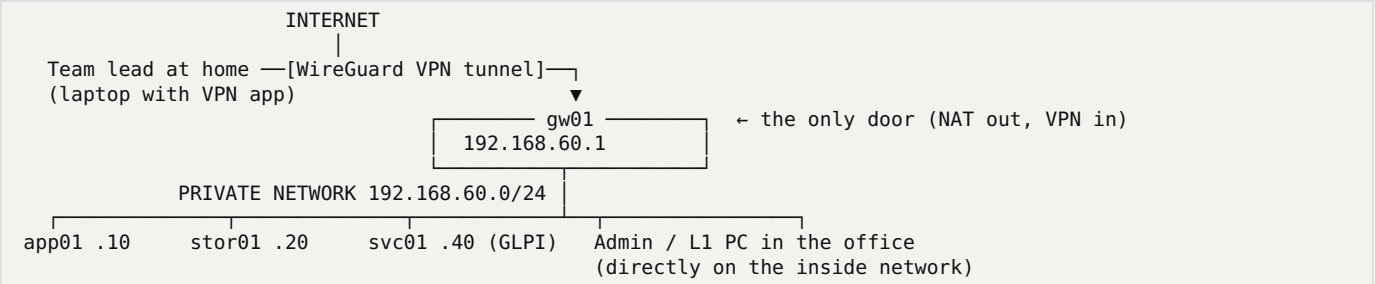
Why a separate server. The service desk must keep working when the servers it tracks are broken – if app01 is down, I still need to open the incident ticket. So it gets its own VM, `svc01`. Later, Project 1 reuses the same svc01 as a second "customer" – one service desk serving two customers is how IT system houses and managed service providers work.

2. svc01 in the lab

VM	Role (plain words)	Role (technical)	IP
gw01	The gatekeeper	NAT, firewall	192.168.60.1
app01	The shop window	App server	192.168.60.10
stor01	The archive room	Storage, backups, logs	192.168.60.20
svc01	**The help desk**	**GLPI tickets, KB, inventory (Docker)**	**192.168.60.40**

2a. How people reach svc01 – and where gw01 comes in

💡 **In plain words:** gw01 is the **only door** between the private network and the internet. People already **inside** (in the office) go straight to svc01. People **outside** (at home) must come in through the door with a **VPN** – like a company badge for remote workers. Strangers and customers never get in.



Who	Where	Path to svc01	Through gw01?
Employee / team lead in the office	On the company network	Browser → `http://192.168.60.40`	❌ already inside
Team lead / admin from home	On the internet	VPN app → encrypted tunnel to gw01 → private network → svc01	✅ via the VPN
Stranger on the internet	Outside, no VPN	Blocked – svc01 has no public IP, gw01 lets in VPN users only	🚫
Customer	Outside	Never connects – emails or calls support; L1 opens the ticket	—
Me in the lab, now	Windows PC	VMware host adapter on VMnet2 → browser → `http://192.168.60.40`	❌
Me in the lab, from Phase 6A	Like the home worker	WireGuard VPN to gw01 → svc01	✅

Lab note: VMware may give the Windows host adapter on VMnet2 the address 192.168.60.1 – gw01's address. Before opening GLPI, check it in the Virtual Network Editor / Windows network settings and use a free address such as 192.168.60.250.

2b. Three examples, start to finish

How the GLPI agent fits in: a small program on each server looks at its own machine once a day (CPU, RAM, disks, IP, software) and sends the report to `http://svc01` – finding svc01 through `/etc/hosts` or DNS. GLPI stores it as an **asset**. The agent creates no tickets and nobody logs in through it. Tickets come from **people**; in each ticket the person picks the affected server ("Item: app01") from the inventory, which links the ticket to the machine.

Thing	Question it answers	Used by
`/etc/hosts` / DNS	“What is stor01’s IP?”	Machines, to connect
Asset inventory	“What is app01, what does it have?”	People, to know and plan
Ticket	“What is wrong or needed, and who did what?”	People, to work

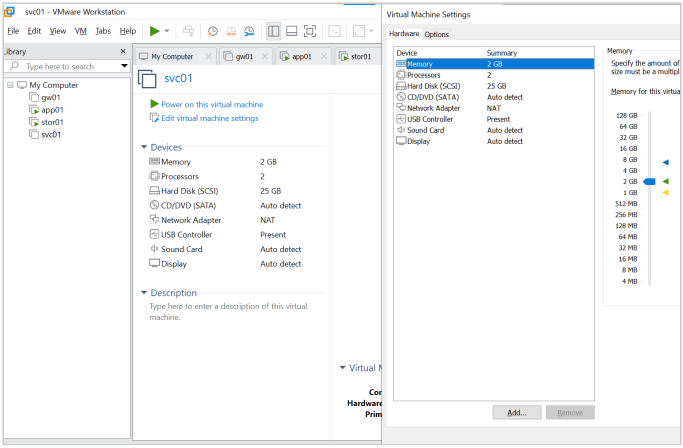
1. **Alice (employee) → Incident.** “I can’t open the shared folder on app01.” Item: app01. L1 checks (app01 pings, folder hangs) → escalates to **Linux Admins (L2)** with notes → L2 finds the firewall blocking NFS on stor01, fixes it, writes the root cause → Solved → KB article → alice confirms → Closed.
2. **Team lead → Request.** “New developer carol starts tomorrow; account on app01 in group devs.” L1 runs `useradd -m -s /bin/bash -G devs carol` and `passwd carol` → Solved → Closed. When carol leaves: a new request → `usermod -L carol`.
3. **Admin → Change.** “/data on stor01 is 90 % full.” The inventory shows 4 GB free in the storage pool → change ticket “Grow /data by 1 GB” with plan (`lvextend -r -L +1G`), risk, rollback (LVM snapshot first) and approval → done → `df -h` checked → Closed. The next agent run shows the new size automatically.

In the lab I play every role with separate GLPI accounts: `alice`/`bob` as requesters, `tech.bilal` in the groups **Service Desk (L1)** and **Linux Admins (L2)**. Escalation = reassigning the ticket to the higher group with notes; the history stays with the ticket.

3. VM settings

Setting	Value	Why
OS	Ubuntu Server **22.04.5** LTS (planned 24.04; the 22.04 ISO was used)	Same version as the other servers; GLPI runs in Docker either way
RAM	2 GB	GLPI + its database in Docker
CPU	2 vCPU	Smooth web interface; 1 would also work
Disk	25 GB	Database, ticket attachments, Docker images
Network	**One adapter, Custom → VMnet2**	Lab network only, like app01 and stor01 – internet only through gw01’s NAT
CD/DVD	Ubuntu Server ISO, connect at power on	Installation medium
Sound card, USB	Optional – can be removed	A server does not need them
Hostname / user	`svc01` / `bilal_004`	Same naming pattern as the other servers
IP (set in the installer)	`192.168.60.40/24`, gateway `192.168.60.1`, DNS `8.8.8.8`	VMnet2 has no DHCP, so the address is set by hand
Installer option	✅ Install OpenSSH server	So gw01 can manage it over SSH with keys

📸 **Evidence – VMware’s default settings: 2 GB, 2 CPUs, 25 GB, but the network adapter is on NAT:**

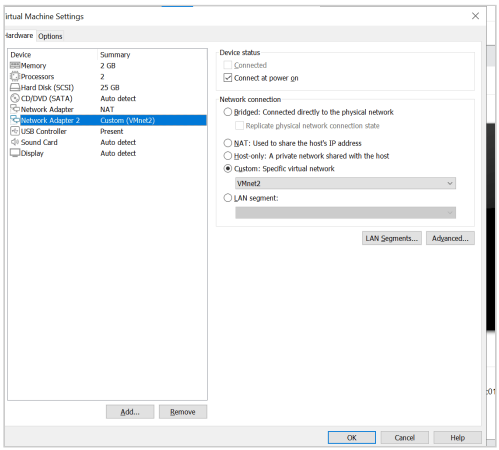


svc01 default settings with NAT

Issue: network adapter on NAT

Seen	VMware creates a new VM with its adapter on **NAT** (direct internet). Then a second adapter was added on VMnet2, so the VM had two adapters.
Why it matters	With NAT, svc01 would bypass gw01 – the gatekeeper design from Phase 1 would be broken, and the NAT adapter would have to be unplugged and netplan fixed after the install (the same trouble app01 had on Day 1).
Fix	Keep one adapter on **Custom → VMnet2**; **remove** the NAT adapter.
Lesson	Check every new VM’s network before the first boot – the default is not the design.

📸 **Evidence – Network Adapter 2 on Custom (VMnet2), connect at power on:**

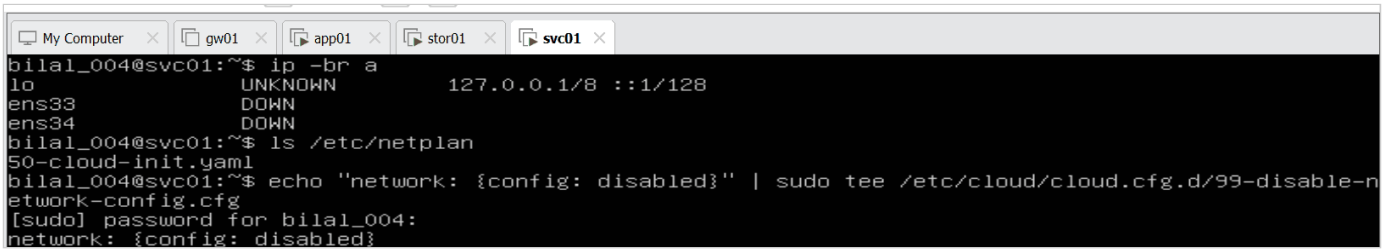


adapter on VMnet2

4. Installation and first boot

4.1 No network after the install

📸 **Evidence – two cards, both DOWN, no IP address; only one netplan file:**

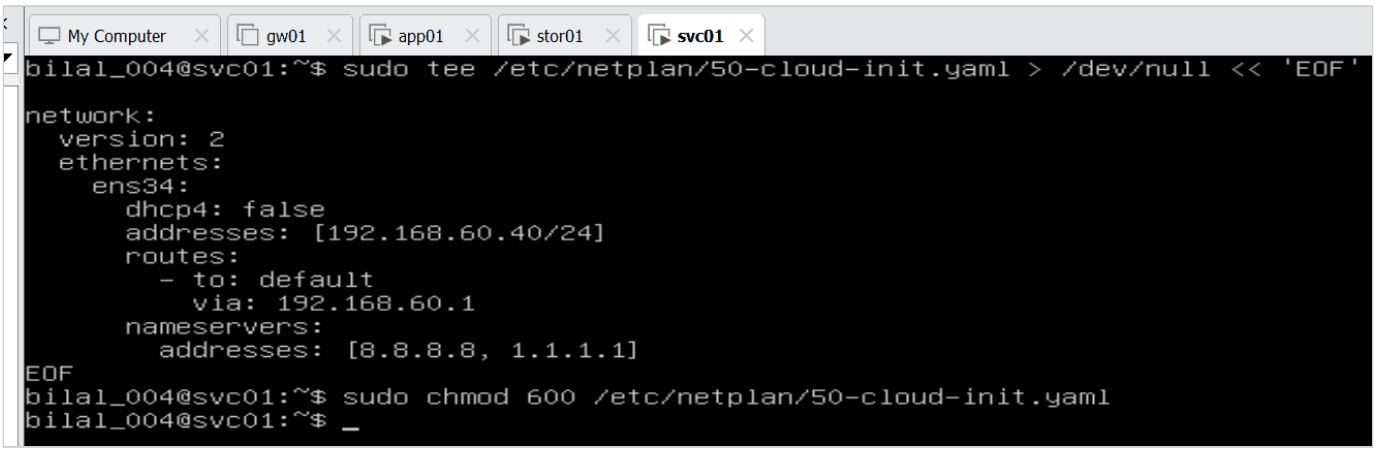


ens33 and ens34 down

Seen	`ip -br a` shows ens33 and ens34, both **DOWN** and without an address; `ip route` is empty; `ping` says *Network is unreachable*.
Cause	The NAT adapter was still attached during the install (two cards), and the manual network settings from the installer were not applied.
Fix	Stop cloud-init from managing the network (as on app01 and stor01, Day 1) and write the netplan file for ens34 (the VMnet2 card) by hand.

```
        echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
        sudo tee /etc/netplan/50-cloud-init.yaml > /dev/null << 'EOF'
        network:
          version: 2
          ethernets:
            ens34:
              dhcp4: false
              addresses: [192.168.60.40/24]
              routes:
                - to: default
                  via: 192.168.60.1
              nameservers:
                addresses: [8.8.8.8, 1.1.1.1]
        EOF
        sudo chmod 600 /etc/netplan/50-cloud-init.yaml   # only root may read it
        sudo netplan try                                 # applies it; reverts after 120 s unless I press Enter
```

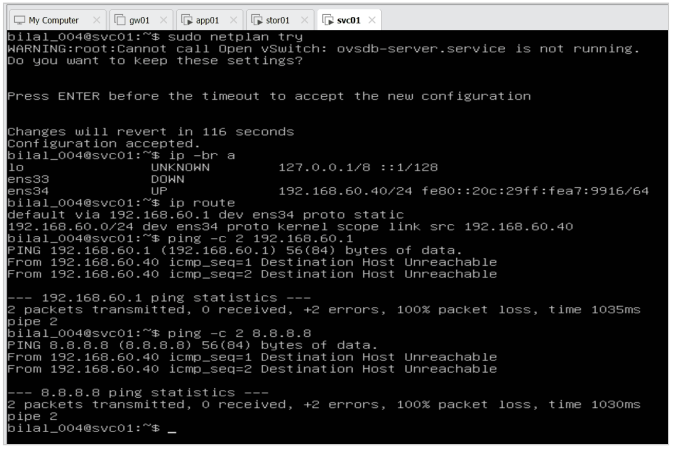
📸 **Evidence – the netplan file:**



netplan file for ens34

4.2 Address works – but gw01 was off

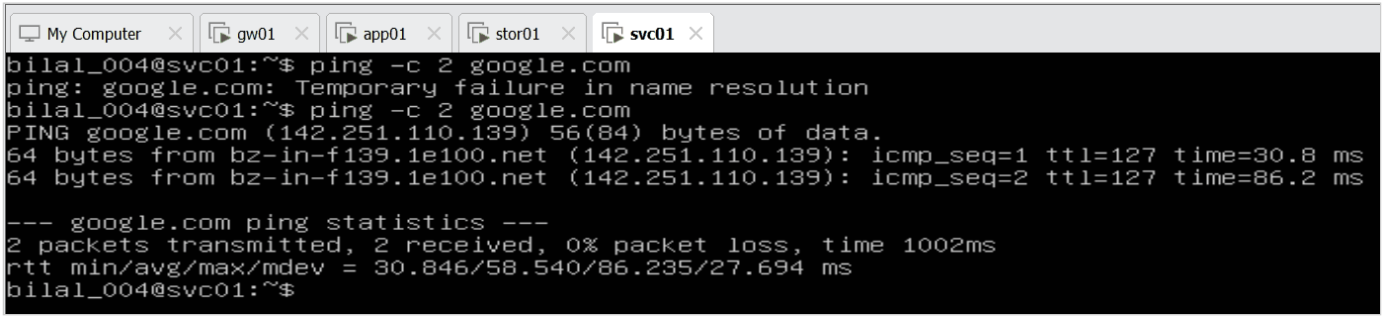
📸 **Evidence – `ens34 UP 192.168.60.40/24`, default route via .1, but "Destination Host Unreachable":**



netplan try, ping unreachable

Seen	svc01 has its address and route, yet `ping 192.168.60.1` and `ping 8.8.8.8` return *Destination Host Unreachable*, and `ping google.com` *Temporary failure in name resolution*.
Cause	**gw01 was powered off.** svc01 asked "who is 192.168.60.1?" and nobody answered. Without the gateway there is no way to the internet – and the DNS server 8.8.8.8 is on the internet.
Fix	Power on gw01.
Lesson	**Always start gw01 first.** The servers can talk to each other without it, but not to the internet or DNS.

📸 **Evidence – after starting gw01: google.com resolves and answers:**



DNS and internet through gw01

4.3 Names: `/etc/hosts` on all four servers

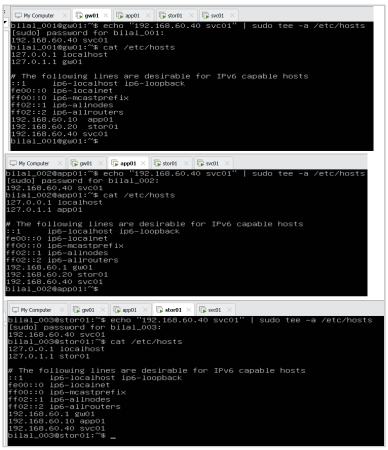
Each machine has its **own** contact list, so the new server must be added to the others, and svc01 must learn the others. A machine never lists itself – it already knows itself as `127.0.1.1`.

Machine	Lists
gw01	app01, stor01, **svc01**
app01	gw01, stor01, **svc01**
stor01	gw01, app01, **svc01**
svc01	gw01, app01, stor01

```
echo "192.168.60.40 svc01" | sudo tee -a /etc/hosts      # on gw01, app01, stor01
echo "192.168.60.1 gw01"    | sudo tee -a /etc/hosts     # on svc01 (one line per echo)
echo "192.168.60.10 app01"  | sudo tee -a /etc/hosts
echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts
```

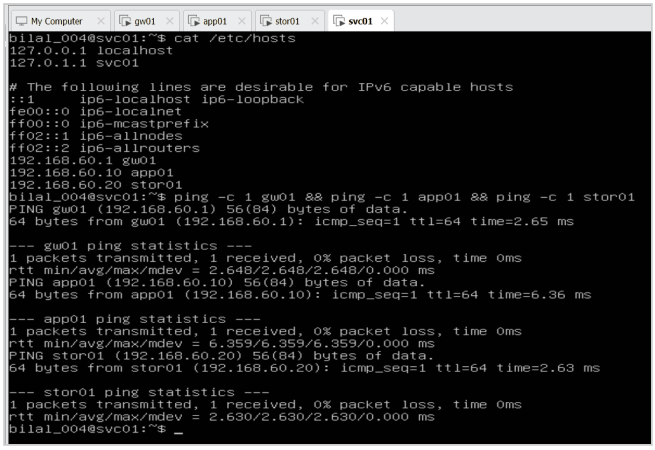
`printf "a\nb\n"` can write several lines in one command because it understands `\n` (new line); plain `echo` does not.

📸 Evidence – svc01 added on gw01, app01 and stor01:



hosts on the other three servers

📸 Evidence – svc01's list and ping by name to all three (`ttl=64` = same network, no router in between; `&&` = run the next command only if this one worked):

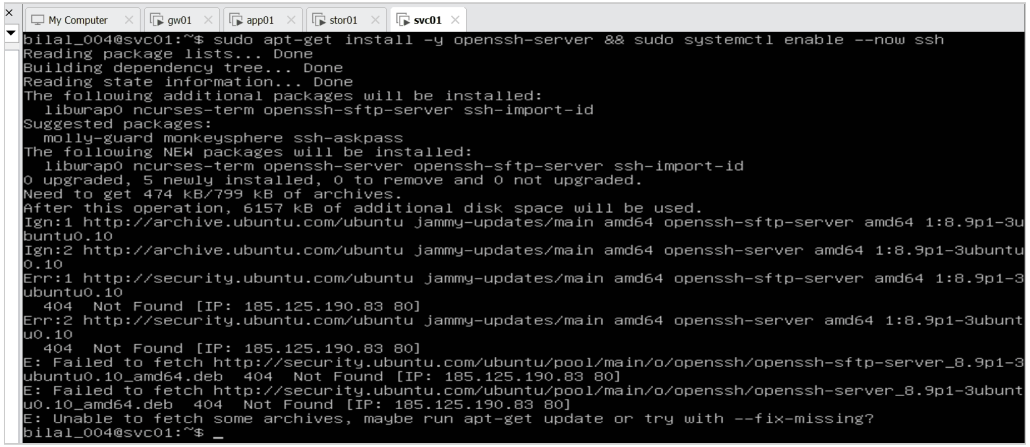


svc01 hosts and ping by name

4.4 SSH: package list out of date

Seen	`apt install openssh-server` fails with **404 Not Found**.
Cause	Not a network problem – the server was reached. The **package list was old**: it asked for a version that has since been replaced, so the file no longer exists.
Fix	`sudo apt update` first, then install.
Lesson	On a fresh server, always run `apt update` before `apt install`.

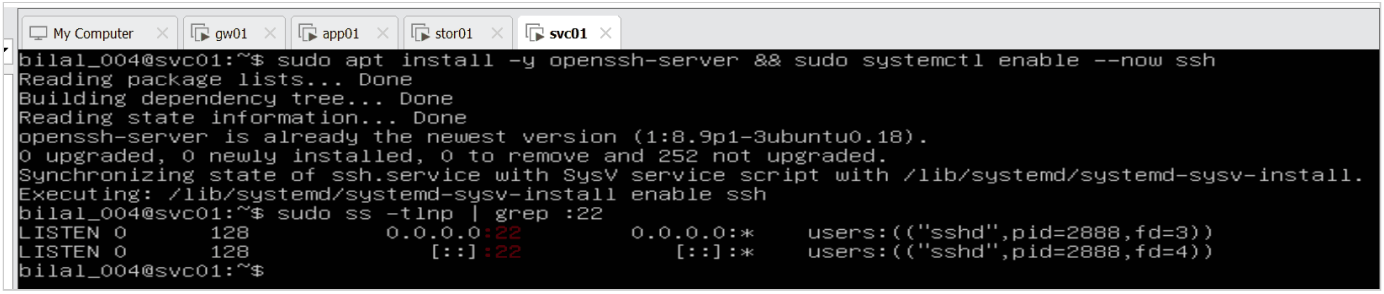
📸 Evidence – the 404 (note `jammy` = Ubuntu 22.04):



apt 404

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo ss -tlnp | grep :22
```

📸 **Evidence – sshd listening on 0.0.0.0:22 and [::]:22:**



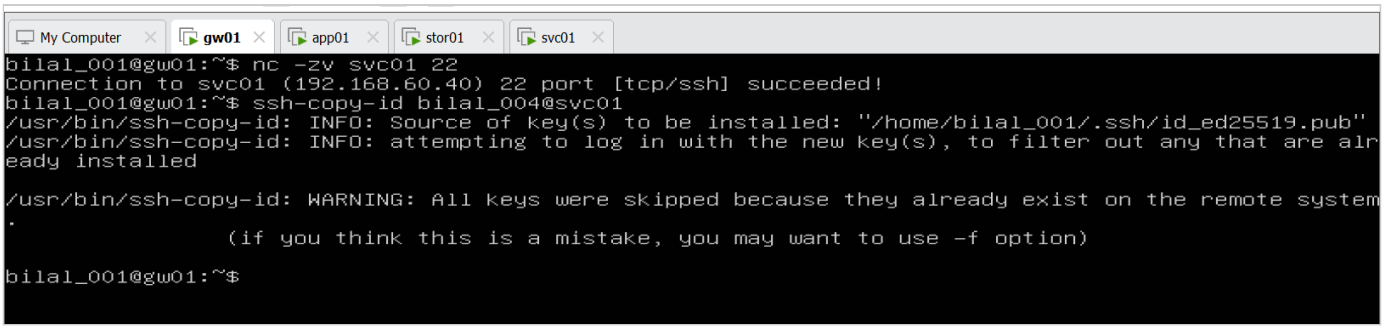
sshd listening

4.5 Key login and shortcut from gw01

```
nc -zv svc01 22                 # door open?
ssh-copy-id bilal_004@svc01     # copy bilal_001's public key
cat >> ~/.ssh/config << 'EOF'
Host svc01
    HostName 192.168.60.40
    User bilal_004
EOF
ssh svc01
```

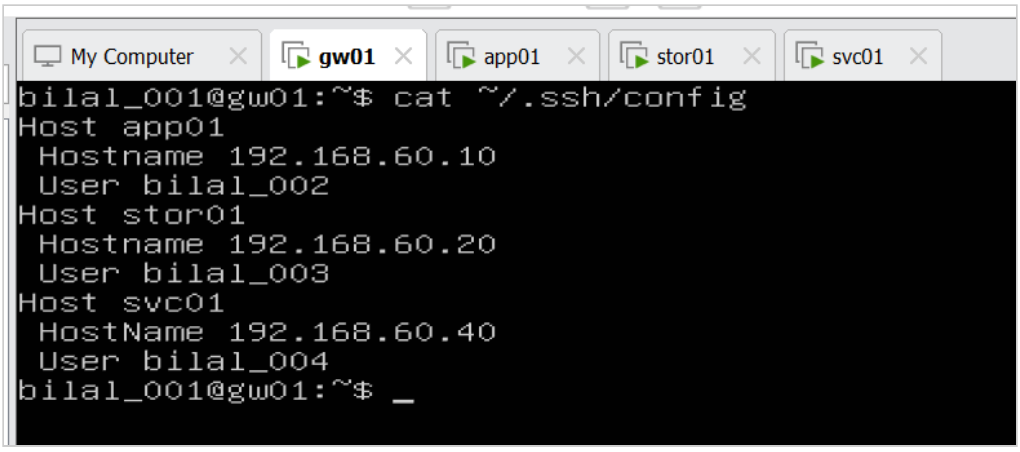
`ssh-copy-id` reported *"All keys were skipped because they already exist on the remote system"* – not an error: it first tests whether the key already works, and it did, so it avoided a duplicate.

📸 **Evidence – port 22 open, key already present:**



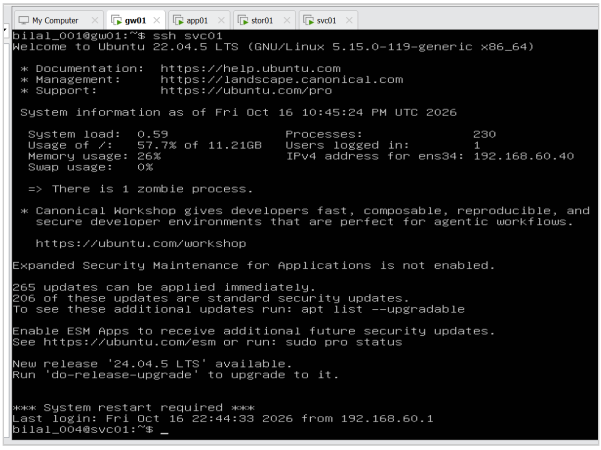
nc and ssh-copy-id

📸 **Evidence – gw01's SSH address book now has app01, stor01 and svc01 (keywords are not case-sensitive: `Hostname` = `HostName`):**



ssh config

📸 **Evidence – `ssh svc01` logs in as bilal_004 without a password:**



ssh svc01

4.6 Read from the welcome screen

Line	Meaning / action
Ubuntu 22.04.5 LTS	Same release as the other servers
Usage of /: 57.7 % of 11.21 GB	The installer used only about 11 GB of the 25 GB disk for the system; the rest is free in the LVM pool → **grow it before Docker** (`lvextend -r -l +100%FREE`)
265 updates, 206 security	Run `sudo apt upgrade -y`
New release 24.04 available	**Not** upgrading – all servers stay on 22.04
1 zombie process	Harmless; gone after reboot (processes are Phase 4)
Times in UTC	Optional: `timedatectl set-timezone Europe/Berlin`, so ticket times match local time
System restart required	Covered by the reboot after the upgrade

Key understanding (in my own words)

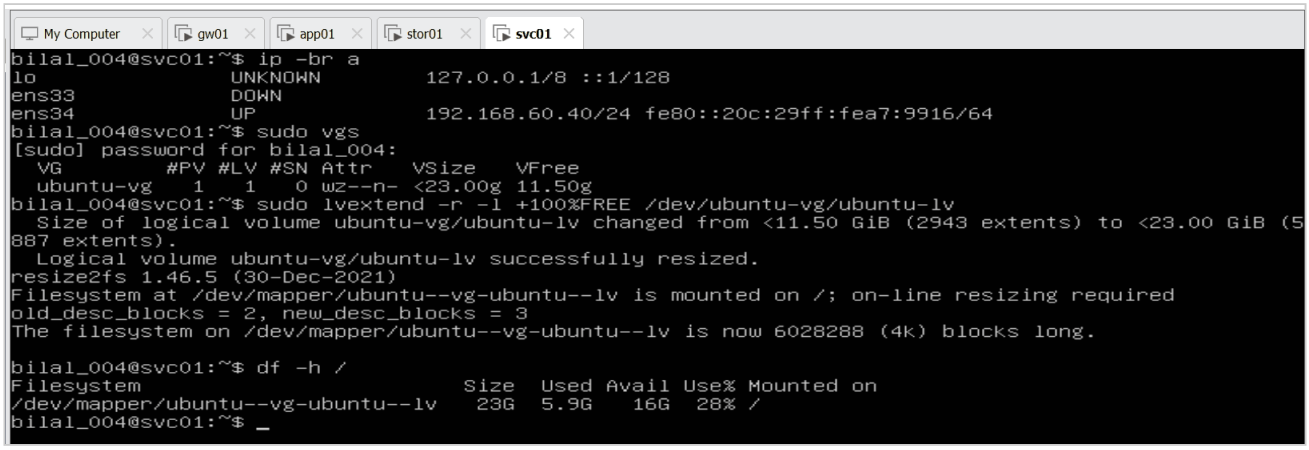
> svc01 joined the lab like app01 and stor01: one card on VMnet2, a fixed address set in netplan, and the default route pointing to gw01. Because gw01 is the only door, svc01 has no internet and no DNS while gw01 is off. Every machine has its own `/etc/hosts`, so a new server must be added to the others' lists and learn theirs. On a fresh server, `apt update` comes before `apt install`. With the key and the SSH shortcut, gw01 now manages all three servers by name, without passwords.

5. Growing the disk before Docker (LVM) – 2026-10-17

💡 **In plain words:** the welcome screen showed the system using only **11.5 GB of the 25 GB disk**. GLPI is about to move in – Docker images, a MySQL database with every ticket, user and inventory record, and ticket attachments – and all of it grows every day. A full disk makes MySQL crash, which would take the service desk itself down: exactly the Acme problem *"disks fill up without warning"*. So the disk was grown **before** installing anything. The system was also fully patched first, because GLPI will open a web service on port 80.

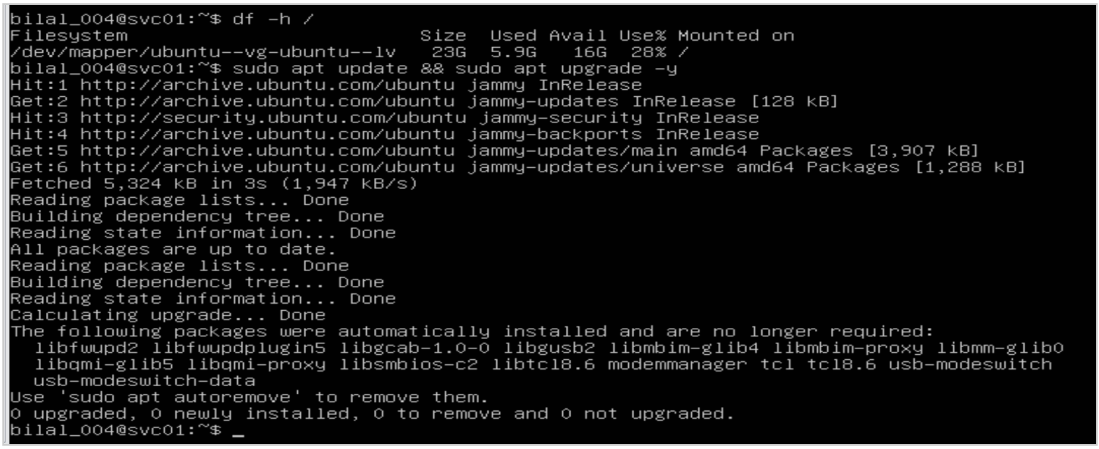
```
        ip -br a                                              # ens34 192.168.60.40 (ens33 = old NAT adapter, to be
removed)
        sudo vgs                                              # the pool: VSize 23 GB, VFree 11.5 GB unused
        sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv  # give the volume all free space; -r grows the file
system too
        df -h /                                               # 23 GB, 28 % used
        sudo apt update && sudo apt upgrade -y                # patch before opening a service
```

📸 **Evidence – `vgs` shows 11.5 GB free in the pool; `lvextend` grows the volume from 11.5 to 23 GB while running; `df` shows 23 GB:**



vgs, lvextend, df

📸 Evidence – all packages up to date:

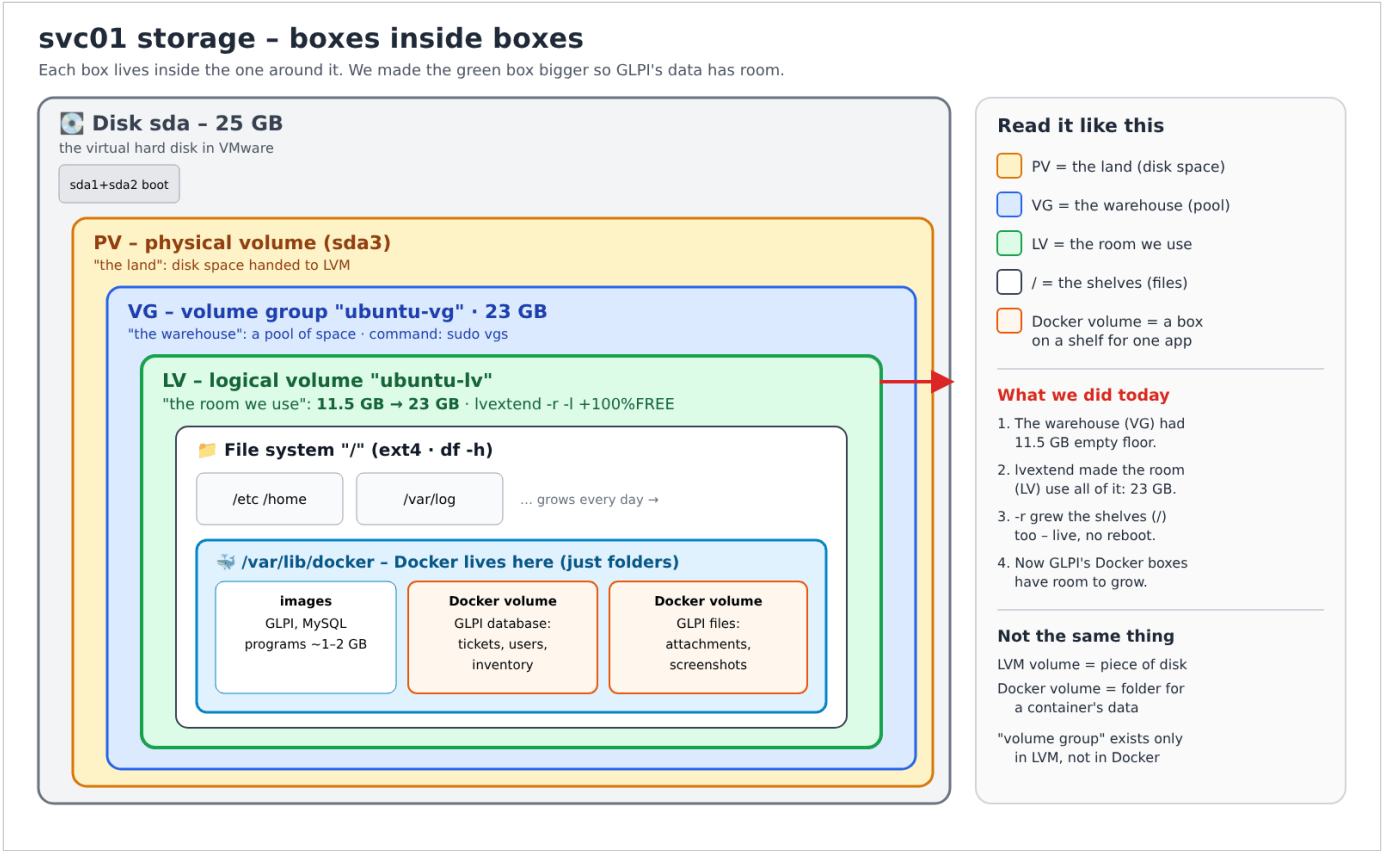


apt upgrade

Reading vgs

Column	Value	Meaning
VG	ubuntu-vg	Name of the pool
#PV	1	Built from 1 physical volume
#LV	1	1 logical volume cut from it
#SN	0	No snapshots
VSize / VFree	23 GB / 11.5 GB	Pool size / part not given to any volume yet

The picture – boxes inside boxes



svc01 storage layers

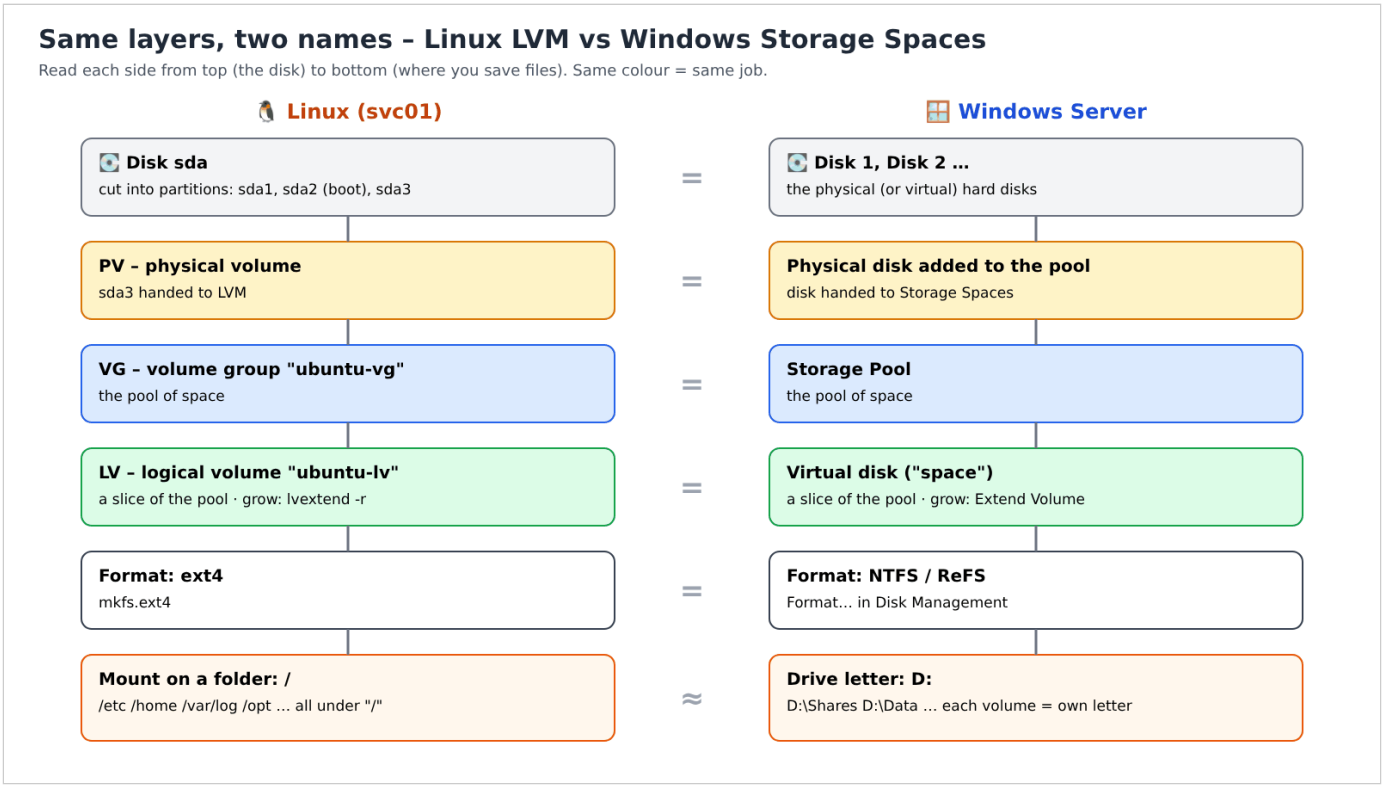
Part of the command	Meaning
`lvextend`	Make a logical volume bigger
`-l +100%FREE`	Small **l**: add all free space of the pool (capital `-L +5G` would add an exact size)
`-r`	Resize the **file system** too – otherwise the volume grows but the space is not usable
`/dev/ubuntu-vg/ubuntu-lv`	Path pattern `/dev/<VG>/<LV>`

Docker: image, container, mounted folder

Word	Simple meaning	Here
Image	The installer – a ready-made program package that never changes	`glpi/glpi`, `mysql:8.0` (kept in `/var/lib/docker`)
Container	The running program made from the image	GLPI and MySQL running
Mounted folder (volume)	A real svc01 folder shown inside the container, where it saves its data	`/opt/helpdesk/storage/mysql` → `/var/lib/mysql` inside MySQL

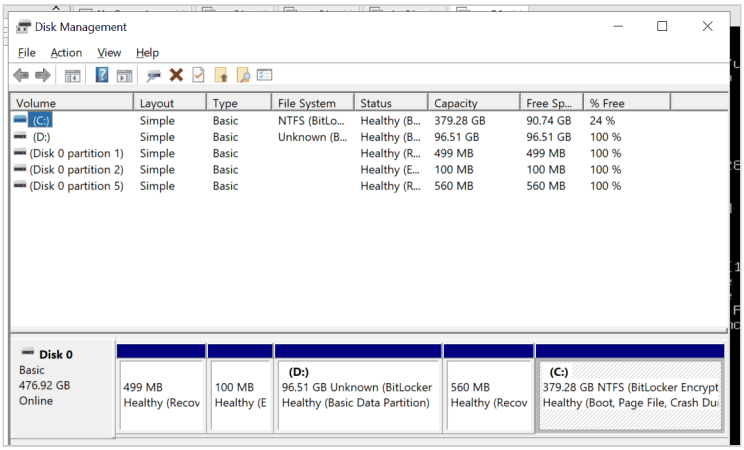
A container is thrown away when it is updated or recreated; data saved in the mounted folder on svc01 survives.

Same layers in Windows



Linux LVM vs Windows Storage Spaces

📸 On my own PC (Disk Management, `diskmgmt.msc`): a “Basic” disk – Disk 0 cut into partitions with drive letters (C:, D:), no pool layer:



Windows Disk Management

Where to look in Windows: **Disk Management** (Win + X) for disks, partitions and letters; **Storage Spaces** (Control Panel, or Server Manager → File and Storage Services → Storage Pools on Windows Server) for pools; PowerShell `Get-Disk`, `Get-Partition`, `Get-Volume`, `Get-StoragePool`.

Key understanding (in my own words)

> When I created svc01, I gave it one 25 GB virtual disk, **sda**. A disk can be divided into **partitions**, like C: and D: in Windows. Here, sda1 and sda2 are for booting, and **sda3** (23 GB) is used as a **physical volume (PV)**. On its own, a PV is just raw space. It is never divided: once it is added to a **volume group (VG)**, a pool of space, it belongs to that VG only. One VG can be built from one or several PVs. Here there is one VG, `ubuntu-vg` (23 GB), made from one PV. For a second VG, I would first create another partition on the disk, which becomes its own PV.
>
> Inside a VG I can create many **logical volumes (LVs)**. Here there was one, `ubuntu-lv`, with 11.5 GB, and the other 11.5 GB were free. I could have used the free space for more LVs; separate LVs give **isolation of space**: if one fills up (for example `/var/log` with logs), the others are not affected, and each LV can be grown, backed up or snapshotted on its own. This is about disk space, not security; data is protected by permissions, firewalls and encryption. On svc01 I chose to grow the single LV to the full 23 GB with `lvextend -r -l +100%FREE` while the system was running, and `-r` grew the file system too.
>
> An LV is only usable once it is **formatted** (ext4) and **mounted**; this one is mounted as `/`, the umbrella for every folder (`/etc`, `/home`, `/var/log`, `/var/lib/docker`, `/opt`).
>
> The same idea in Windows: physical disks (≈ PVs) are added to a **Storage Pool** (≈ VG) in Storage Spaces. The pool is cut into **virtual disks** (≈ LVs), which are formatted with NTFS or ReFS (≈ ext4) and given a

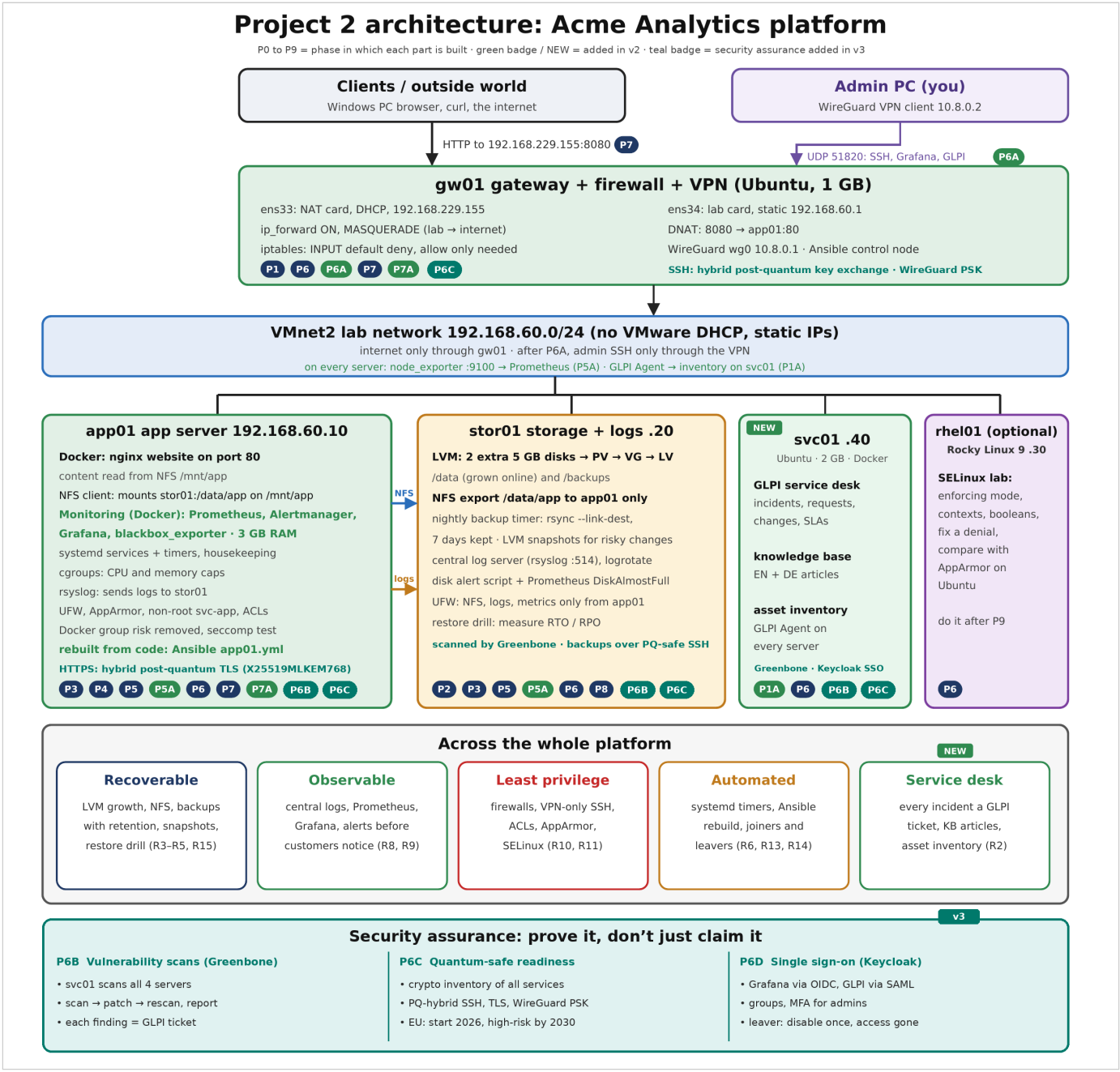
drive letter such as D: (≈ mounting). Extend Volume in Disk Management grows a volume live, like `lvextend -r`. Linux uses one folder tree under `/`, while Windows uses drive letters, but the layers are the same: disk → pool → volume → file system → mount point. On a normal PC like mine the disk is “Basic”: only partitions with drive letters, no pool – storage pools are used on servers with several disks.

For the service desk, Docker takes **images** (the installers for GLPI and MySQL) and runs them as **containers**. Their data is saved in **mounted folders** on svc01, for example `/opt/helpdesk/storage/mysql`, which appears inside MySQL as `/var/lib/mysql`. So tickets, users and inventory stay safe on `/` even if a container is deleted or upgraded, and that is why I grew the disk before installing GLPI.

6. The network story – from the first picture to today

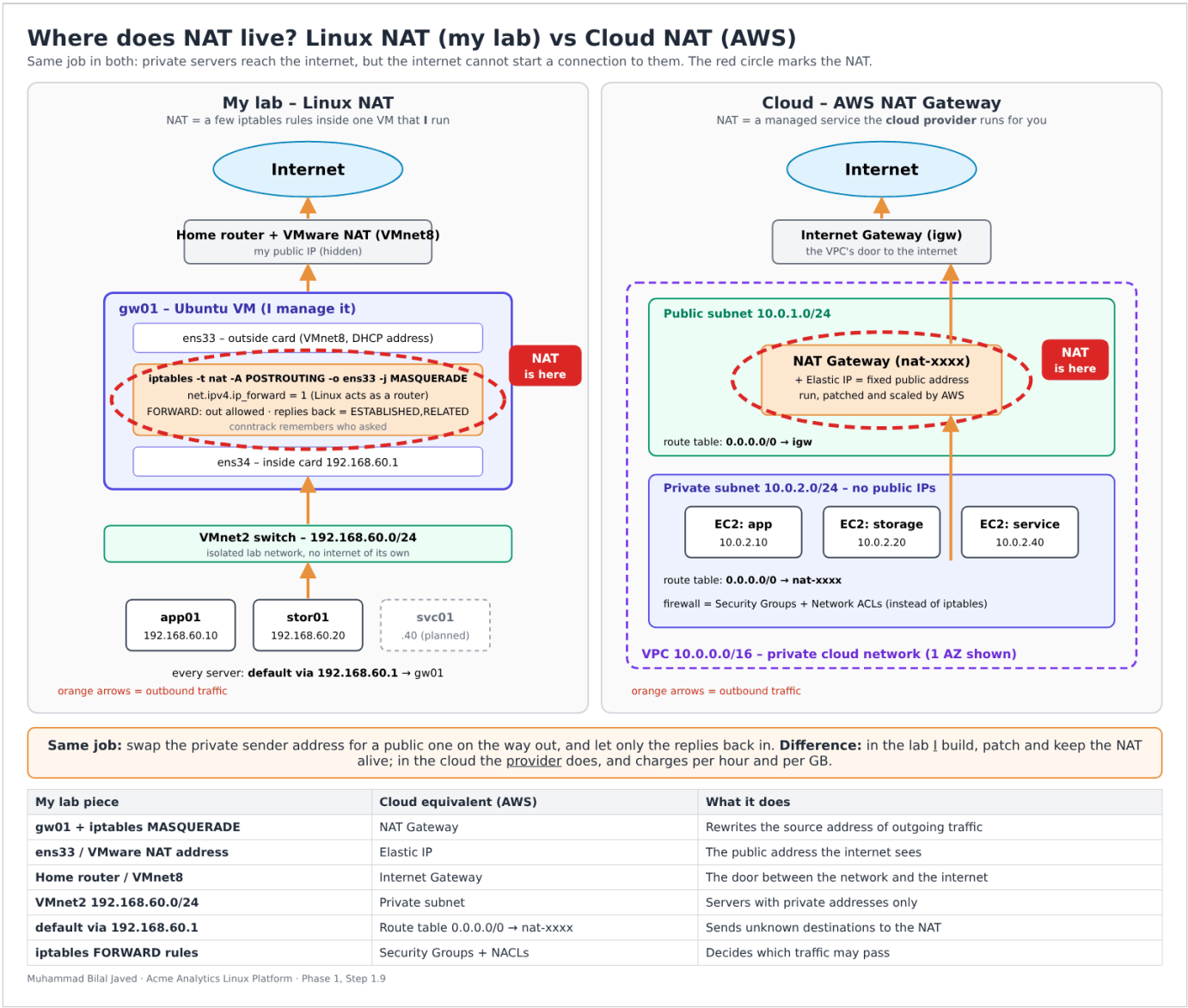
💡 **Read this section alone and you have the whole network in your head – no need to go back to Day 1.**

Picture 1 – the plan. Acme gets four servers on one private lab network, and one gatekeeper (gw01) between that network and the internet.



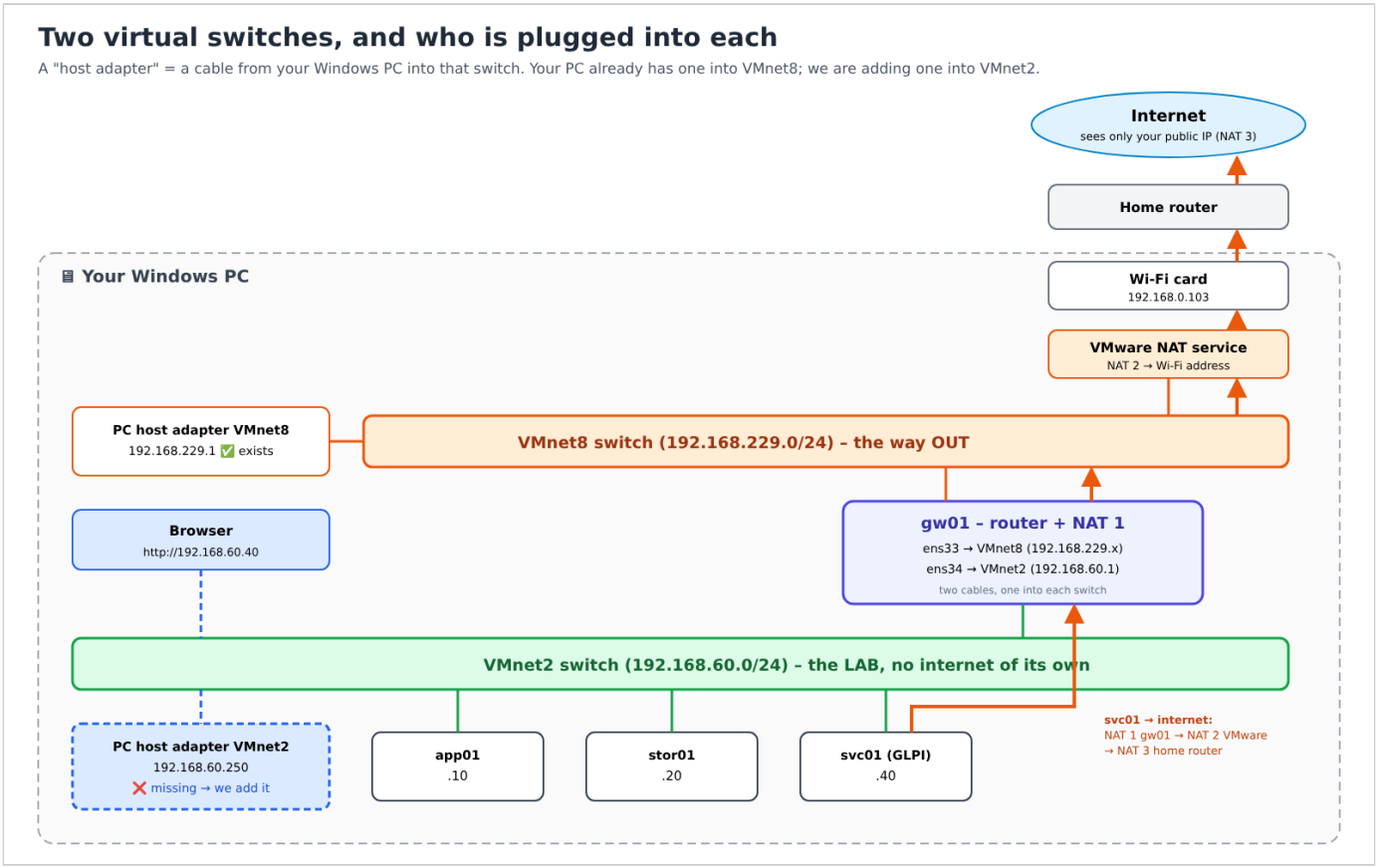
Project architecture

Picture 2 – the gatekeeper (Phase 1). In the beginning every VM had its own NAT card straight to the internet. That is easy but unsafe: nobody controls the traffic. So the servers’ outside cards were removed. Only **gw01** keeps two cards: **ens33** towards VMware’s **VMnet8** (the way out) and **ens34** = `192.168.60.1` on **VMnet2** (the lab). With IP forwarding and an iptables MASQUERADE rule, gw01 became a router with NAT – the same job a cloud NAT gateway does.



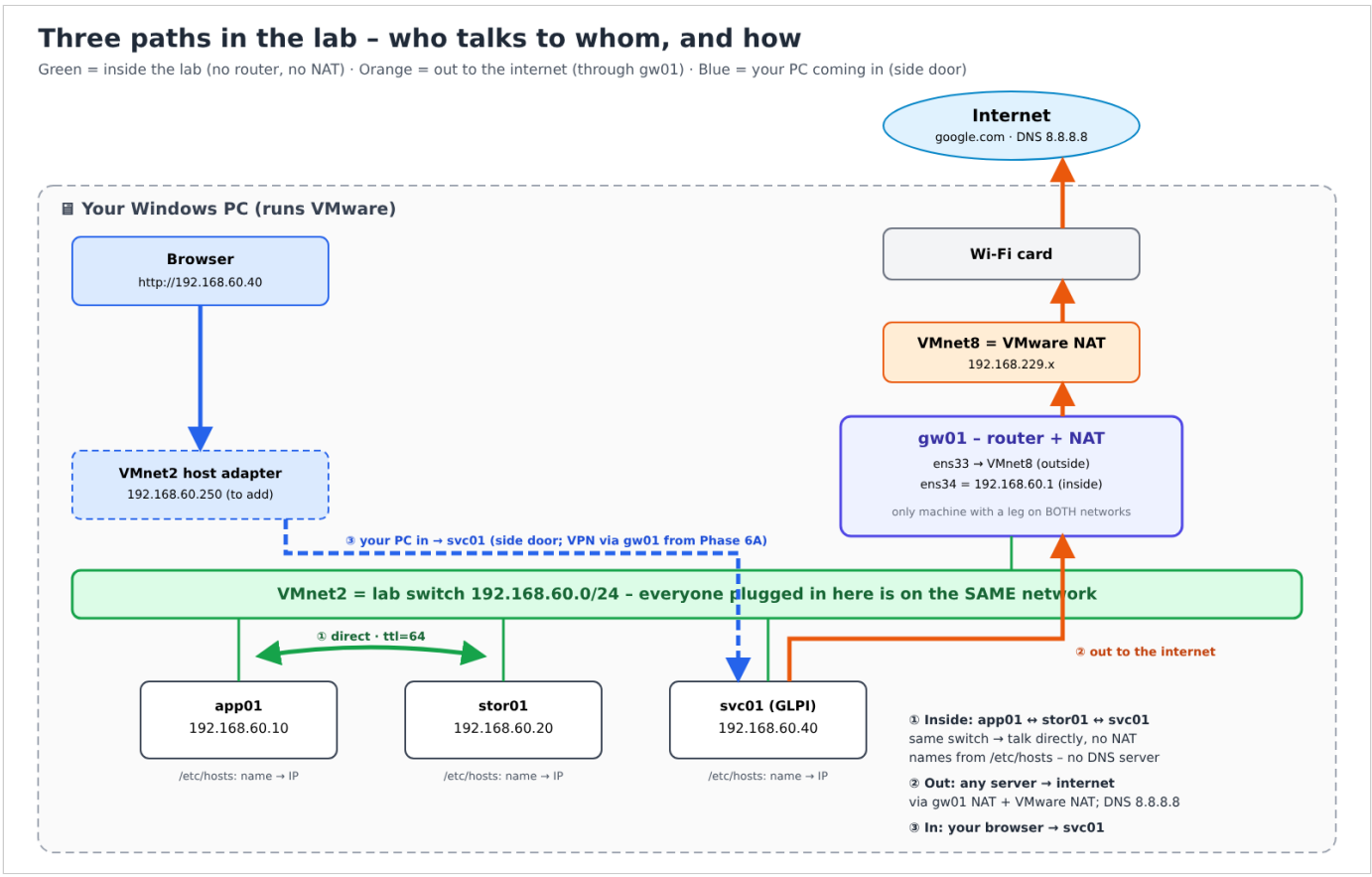
Linux NAT vs cloud NAT

Picture 3 – two virtual switches. VMware builds switches in software. **VMnet2** is the lab switch: app01, stor01, svc01 and gw01's inside card are plugged into it, so they talk to each other **directly** (no router, no NAT – `ttl=64`); `/etc/hosts` only turns their names into IPs. **VMnet8** is the way out: only gw01's outside card is plugged into it. A *host adapter* is a cable from my Windows PC into one switch – it already had one into VMnet8, but **none into VMnet2**.



Two switches and the host adapters

Picture 4 – three paths. ① **Inside:** server ↔ server over VMnet2, directly. ② **Out:** a server → gw01 (**NAT 1**, to gw01's VMnet8 address) → VMware NAT (**NAT 2**, to my Wi-Fi card's address) → home router (**NAT 3**, to my public IP) → internet. DNS (8.8.8.8) is on the internet too, so name lookups for google.com take the same path – that is why nothing worked while gw01 was off. ③ **In:** my browser → svc01. This needs my PC on the lab switch, so the **VMnet2 host adapter** is added with the free address `192.168.60.250` (not .1 – that is gw01). It is a local side door for this one PC, not a VPN; in **Phase 6A** it is replaced by a WireGuard VPN through gw01 – the way a real remote worker comes in.



Three paths in the lab

Why today: GLPI now runs on svc01 and answers `200 OK` from svc01 itself and from gw01. To use the service desk in a browser, my PC needs path ③.

How names are found (two places, in order – `hosts: files dns` in `/etc/nsswitch.conf`):

Order	Where Linux looks	Lab names (stor01)	Internet names (google.com)
1	`/etc/hosts` (local list)	✅ found here	❌ not there
2	DNS via the helper 127.0.0.53 → 8.8.8.8	❌ no DNS server knows the lab	✅ found here (and cached)

Switch, router, bridge:

Device	Like	In the lab
Switch	an extension board – devices on the same network plug in and talk directly	VMnet2 (lab board), VMnet8 (way-out board)
Router	a person between two boards passing messages across	gw01 – one card on each board
Bridge (VMware “Bridged”)	plugging a VM straight into the home network	not used – the lab stays separate

Network adapter (VMware) = `ens34` / `ens33` (Linux) = network card (NIC) – one thing, three names.

> **Key understanding (in my own words).** Every server has a network card. VMware calls it a “Network Adapter”. Inside Linux the card is called `ens34` or `ens33`.
>
> A virtual switch is like an extension board. VMnet2 is the lab board. VMnet8 is the board for the way out.
>
> app01, stor01 and svc01 are plugged into VMnet2. gw01 is also plugged into VMnet2 with its inside card. All of them have an address in the same network. So they talk to each other directly by IP. This worked from the first day.
>
> To use names, Linux looks in two places. First it checks `/etc/hosts`. Then it asks DNS. Our lab names are written in `/etc/hosts`, so they are found in the first place. Internet names like google.com are not in `/etc/hosts`. So Linux asks DNS. The helper 127.0.0.53 forwards the question to 8.8.8.8 and keeps the answer in a cache.
>
> For the internet, the servers send their traffic to gw01. gw01 has a second card on VMnet8. The address is changed three times on the way out. First gw01 changes it to its own address on VMnet8. Then VMware’s NAT changes it to my Wi-Fi card’s address. Then my home router changes it to my public IP. The DNS question to 8.8.8.8 also takes this way. So nothing on the internet works while gw01 is off.
>
> My Windows PC had a card on VMnet8 but no card on VMnet2. So my browser could not reach svc01. Now I give my PC a card on VMnet2 with the tick “Connect a host virtual adapter”. The card gets the address 192.168.60.250. My browser can now open GLPI on svc01 directly. Nothing changes for the servers.

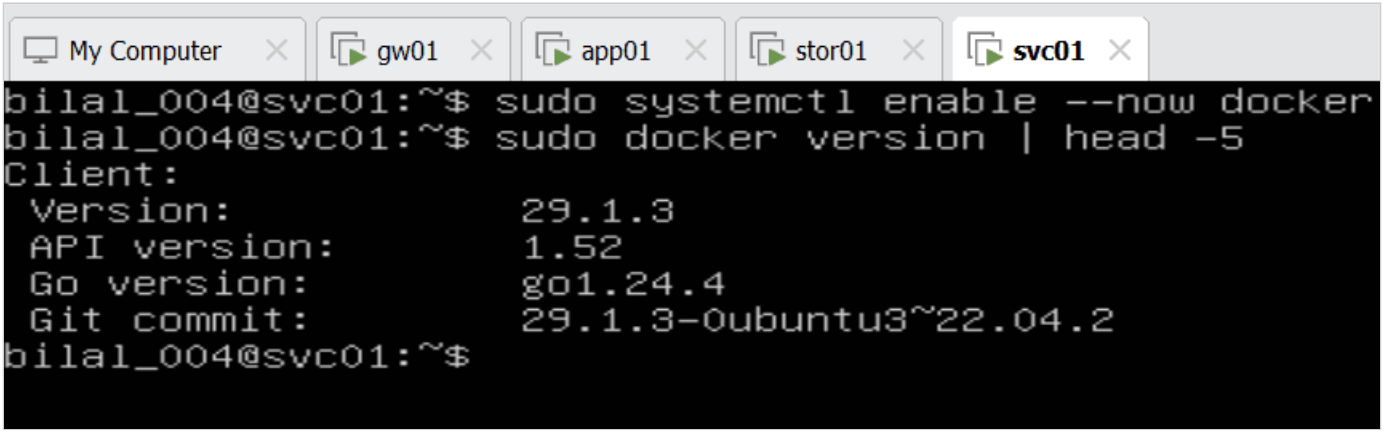
7. Step 1A.2 – Docker and GLPI running, opened in the browser

💡 **In plain words:** GLPI and its MySQL database run as two **containers** on svc01, described in one small file (`compose.yaml`). Their data lives in folders on svc01, so it survives updates. Then my Windows PC got a plug on the lab switch so the browser can open GLPI.

7.1 Install Docker

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2   # Docker engine + "docker compose"
sudo systemctl enable --now docker                                   # start now and at every boot
sudo docker version | head -5
```

📸 **Evidence – Docker 29.1.3 installed:**



docker version

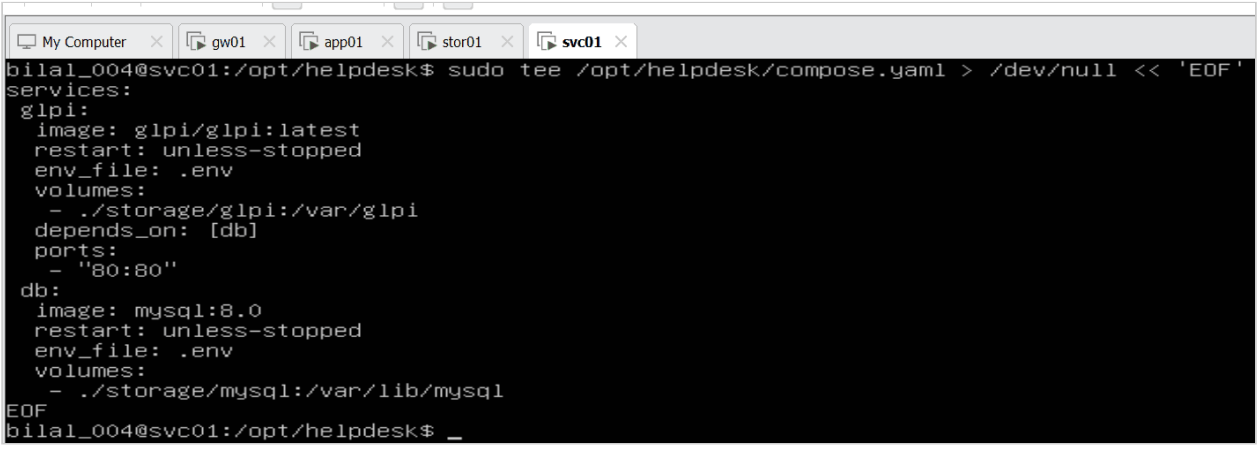
7.2 Settings file and compose file

```
sudo mkdir -p /opt/helpdesk && cd /opt/helpdesk
sudo tee /opt/helpdesk/.env > /dev/null << 'EOF'
GLPI_DB_HOST=db
GLPI_DB_PORT=3306
GLPI_DB_NAME=glpi
GLPI_DB_USER=glpi
GLPI_DB_PASSWORD=<lab-db-password>
MYSQL_RANDOM_ROOT_PASSWORD=yes
MYSQL_DATABASE=glpi
MYSQL_USER=glpi
MYSQL_PASSWORD=<lab-db-password>
EOF
sudo chmod 600 /opt/helpdesk/.env       # passwords inside — only root may read it
```

The real password is not published here; the `.env` file stays on svc01 only.

```
# /opt/helpdesk/compose.yaml — which containers to run
services:
  glpi:
    image: glpi/glpi:latest          # the GLPI installer (image)
    restart: unless-stopped          # start again after a crash or reboot
    env_file: .env
    volumes:
      - ./storage/glpi:/var/glpi     # svc01 folder shown inside the container
    depends_on: [db]                 # start the database first
    ports:
      - "80:80"                      # svc01 port 80 → container port 80
  db:
    image: mysql:8.0
    restart: unless-stopped
    env_file: .env
    volumes:
      - ./storage/mysql:/var/lib/mysql
```

📸 **Evidence – the compose file:**



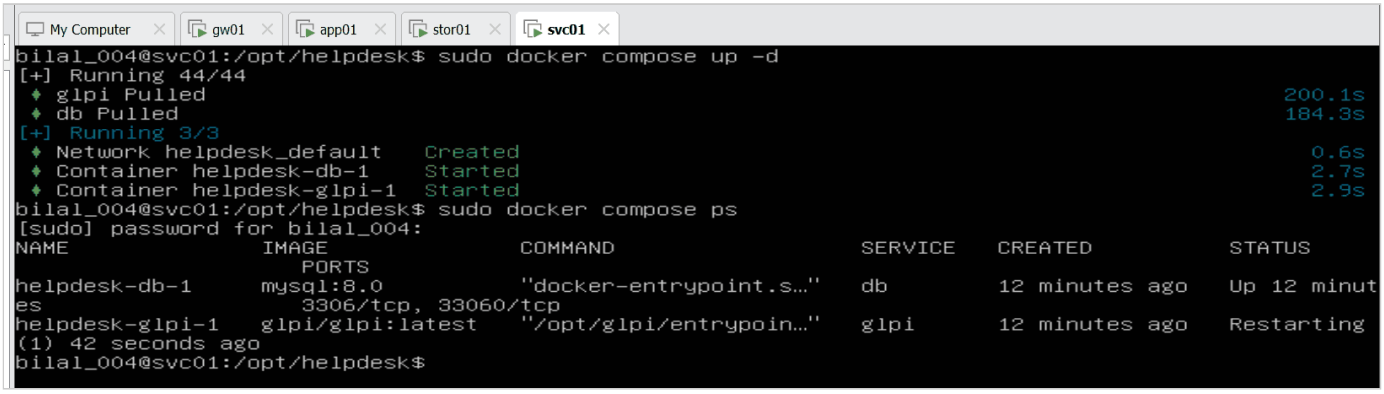
compose.yaml

Lesson – YAML is strict. My first attempts failed: `Services:` instead of `services:` (keywords are lowercase), `glpi:Latest` instead of `latest` (the tag is case-sensitive), `mysql: 8.0` with a space (YAML reads it as a new key → *"mapping values are not allowed"*), and `var/glpi` without the leading `/`. Check a file before starting it: `sudo docker compose config --quiet && echo "YAML OK"`.

7.3 Start – and the permission problem

```
sudo docker compose up -d      # download images, create network, start both containers
sudo docker compose ps
```

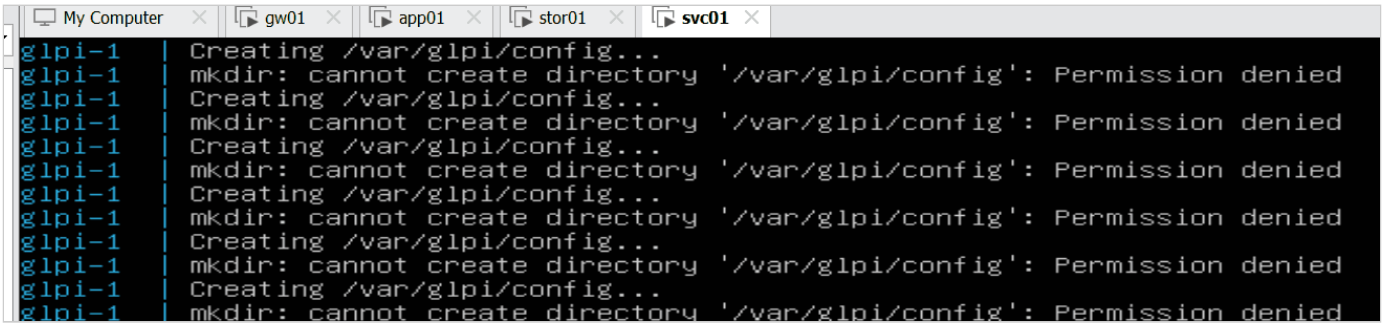
📸 **Evidence – images pulled (~3 min), db "Up", but glpi "Restarting (1)":**



compose up, glpi restarting

Seen	`glpi` keeps restarting with exit code 1.
Read the log	`sudo docker compose logs glpi --tail 40` → *mkdir: cannot create directory '/var/glpi/config': Permission denied*
Cause	Docker created the folder `storage/glpi` on svc01 as **root**. Inside the container GLPI runs as **www-data (user ID 33)**, which may not write there.
Fix	Give the folder to user 33.
Lesson	When a container restarts in a loop, read its logs first – they name the cause. File permissions (Step 1.8) also apply across the container boundary.

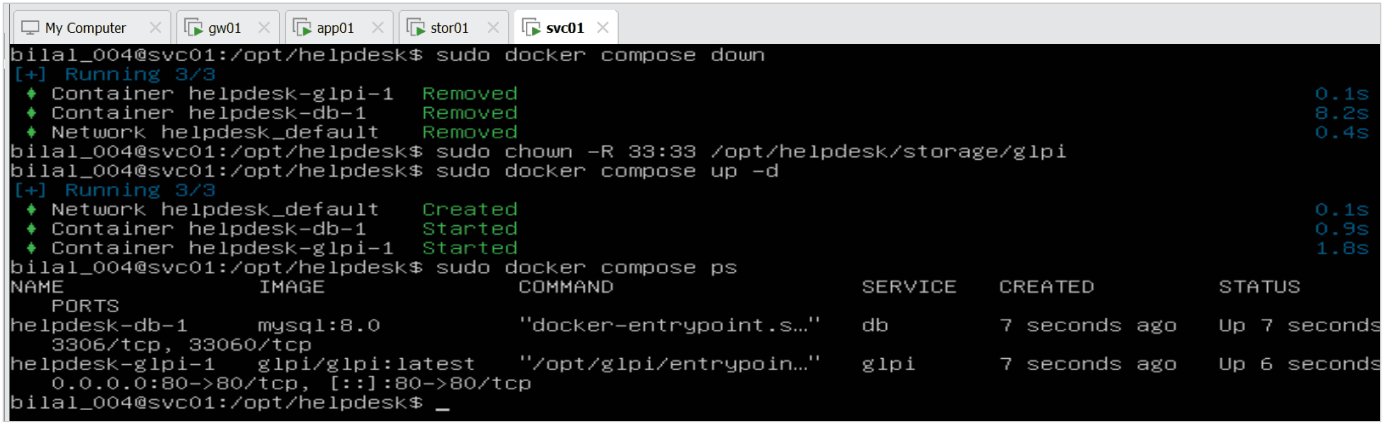
📸 Evidence – the log:



permission denied

```
sudo docker compose down
sudo chown -R 33:33 /opt/helpdesk/storage/glpi    # 33 = www-data inside the container
sudo docker compose up -d
sudo docker compose ps
```

📸 Evidence – both containers "Up"; port 0.0.0.0:80 → 80:

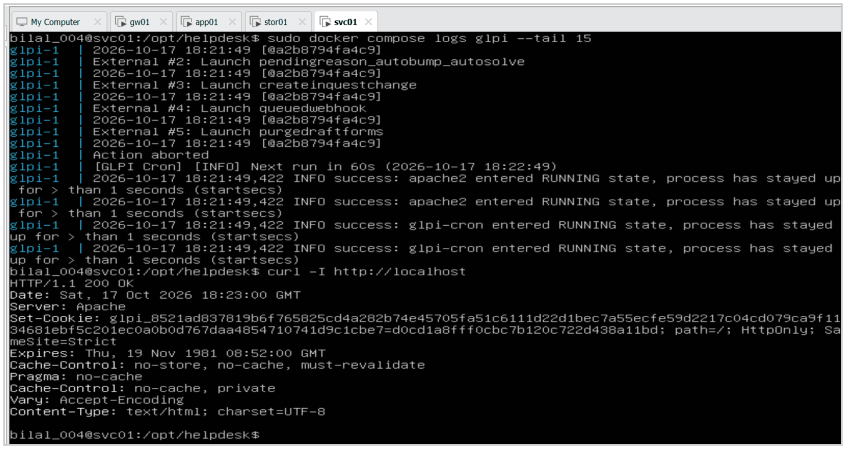


glpi up

7.4 Test from svc01 and from gw01

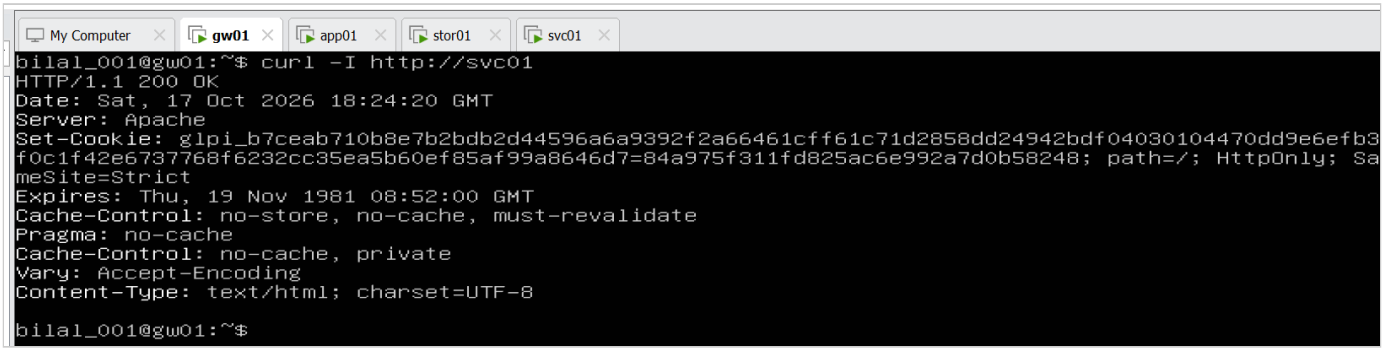
```
sudo docker compose logs glpi --tail 15   # apache2 and glpi-cron "RUNNING"
curl -I http://localhost                   # on svc01
curl -I http://svc01                       # on gw01 — name from /etc/hosts
```

📸 Evidence – Apache and GLPI cron running; `HTTP/1.1 200 OK` on svc01:



glpi running, 200

📸 Evidence – `200 OK` from gw01 over the lab network:



gw01 curl 200

HTTP status codes – the server's short answer:

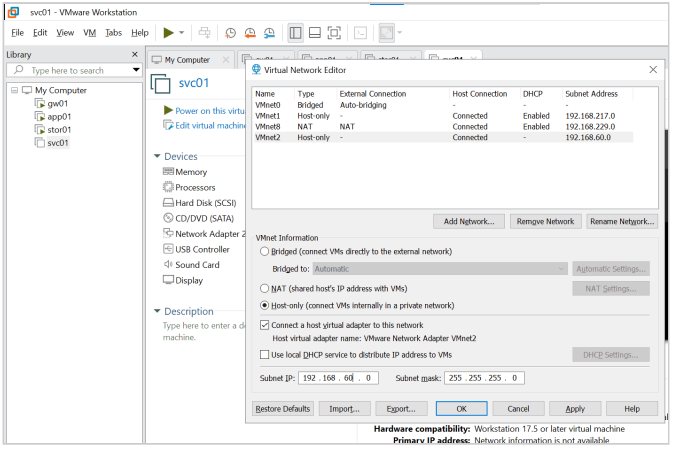
Family	Meaning	Examples
2xx	Success	**200 OK** – page delivered
3xx	Redirect	301 moved for good · 302 go to login first
4xx	Client error – look at the request or permissions	401 log in first · 403 forbidden · 404 not found
5xx	Server error – look at the logs, database, disk	500 app crashed · 502 bad gateway · 503 unavailable

7.5 My PC on the lab switch – the VMnet2 host adapter

`ipconfig` on Windows showed adapters for VMnet1 and VMnet8 but **none for VMnet2**, so the browser could not reach 192.168.60.40 (`ping` timed out). The Virtual Network Editor needs administrator rights (**Change Settings**); VMnet2 was not even listed, so it was added.

Setting (VMnet2)	Value	Why
Type	Host-only	Private lab network, no NAT of its own
Connect a host virtual adapter	✅	Gives the Windows PC a plug on VMnet2
Use local DHCP	☐ off	Servers have fixed addresses
Subnet	192.168.60.0 / 255.255.255.0	The lab network

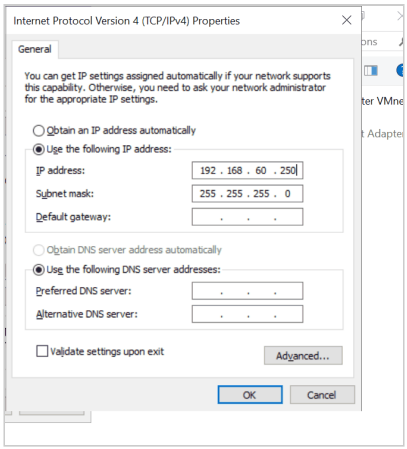
📸 Evidence – VMnet2 in the Virtual Network Editor:



VMnet2 host adapter

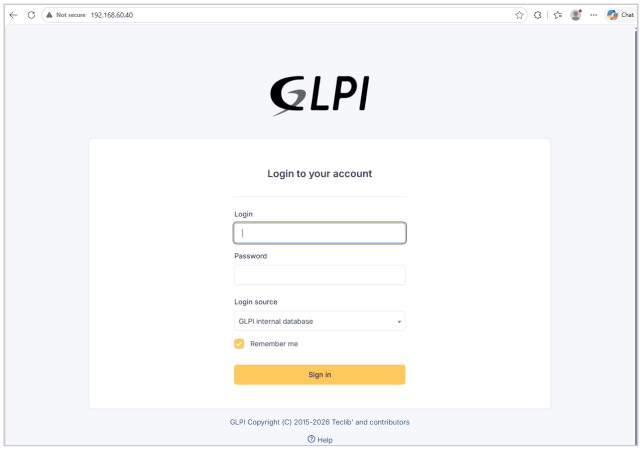
VMware gives the new adapter `192.168.60.1` – gw01's address – so it was set by hand to **192.168.60.250**, mask 255.255.255.0, **no gateway and no DNS** (the internet must keep using Wi-Fi; this is only a side door into the lab).

📸 **Evidence – Windows adapter VMnet2 set to 192.168.60.250:**



ncpa VMnet2 IPv4

📸 **Evidence – GLPI login page in the browser at http://192.168.60.40 ("Not secure" = plain HTTP, fine inside the lab):**

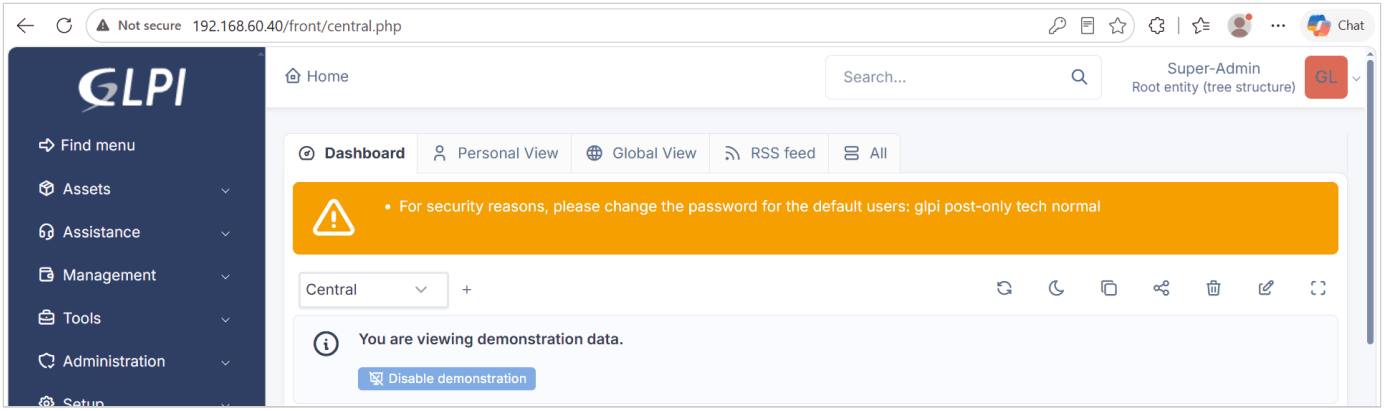


GLPI login

7.6 First login and default passwords

GLPI ships with four accounts whose passwords everybody knows: `glpi` (super-admin), `tech`, `normal`, `post-only`. GLPI shows a warning until they are changed.

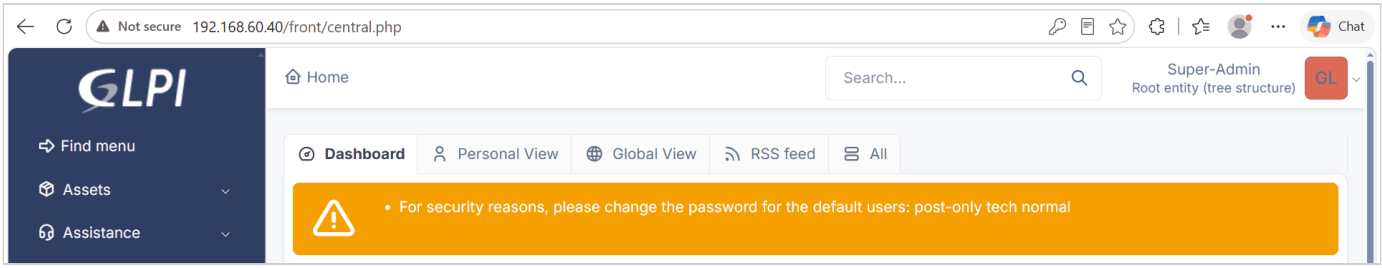
📸 **Evidence – first login as Super-Admin with the warning (the numbers are GLPI demonstration data, not mine – "Disable demonstration" removes them):**



first login

Administration → Users → `glpi` → **Change password** → **Save** (the form shows "Password change pending" until Save is clicked).

📸 **Evidence – `glpi` no longer in the warning; `tech`, `normal`, `post-only` to be disabled next:**



glpi password changed

Key understanding (in my own words)

GLPI runs on svc01 as two containers – the GLPI web application and its MySQL database – described in one `compose.yaml`. Their data is kept in folders on svc01, so it survives updates. The first start failed because GLPI runs inside the container as user 33 and could not write to a folder owned by root; the container log showed it, and `chown 33:33` fixed it. GLPI then answered `200 OK` on svc01 and from gw01. My Windows PC had no plug on the lab switch, so I added a VMnet2 host adapter with the free address 192.168.60.250 and opened GLPI in the browser. The default passwords are being changed first, because everybody knows them.

8. Step 1A.4 – GLPI agent: automatic asset inventory – 2026-10-17

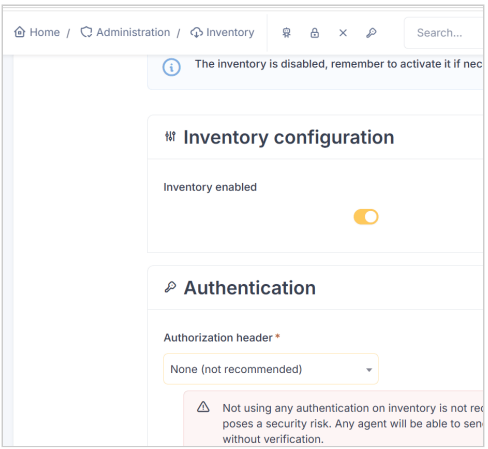
💡 **In plain words:** a small program, the **GLPI agent**, now runs on every server. Once a day it collects what the machine is made of – hardware, operating system, serial number, installed software, network cards – and sends it to GLPI on svc01. So the asset list fills itself and stays up to date; nobody types it in. The agent is not a person and creates no tickets. (The guide puts Step 1A.4 after 1A.3; I did the inventory first because it only needs GLPI running.)

8.1 Switch the inventory on in GLPI

Administration → Inventory → **Inventory enabled** → Authorization header → **Save**.

Setting	Value	Why
Inventory enabled	On	GLPI accepts reports from agents
Authorization header	**None (not recommended)**	svc01 is only on the private lab network – only the 4 servers and my PC can reach it. Keeps the first test simple.
Risk	Any machine on the network could send a fake inventory	Acceptable inside the lab
To do – Phase 6 (hardening)	Switch to **Basic Authentication**; re-run the agents with `--user` and `--password`	Before → after for the security report

📸 **Evidence – inventory enabled, no authentication (GLPI shows the warning itself):**



inventory enabled

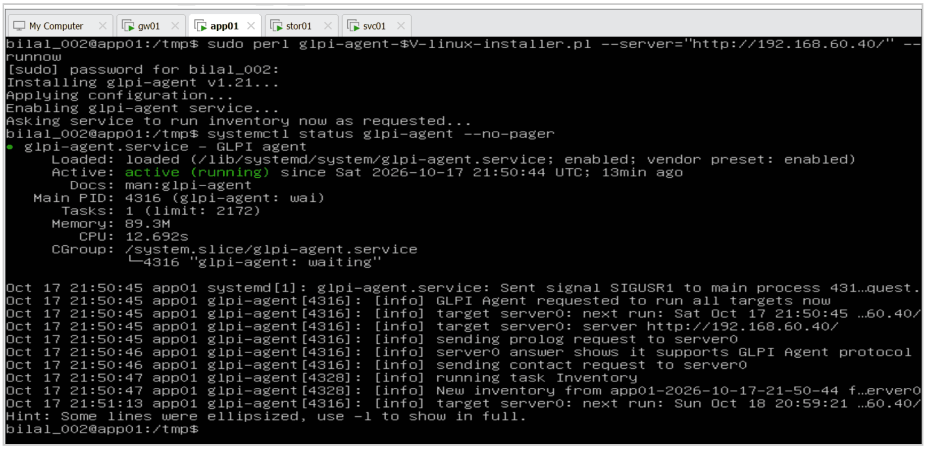
8.2 Install the agent on every server

The same commands on app01, stor01, svc01 and gw01:

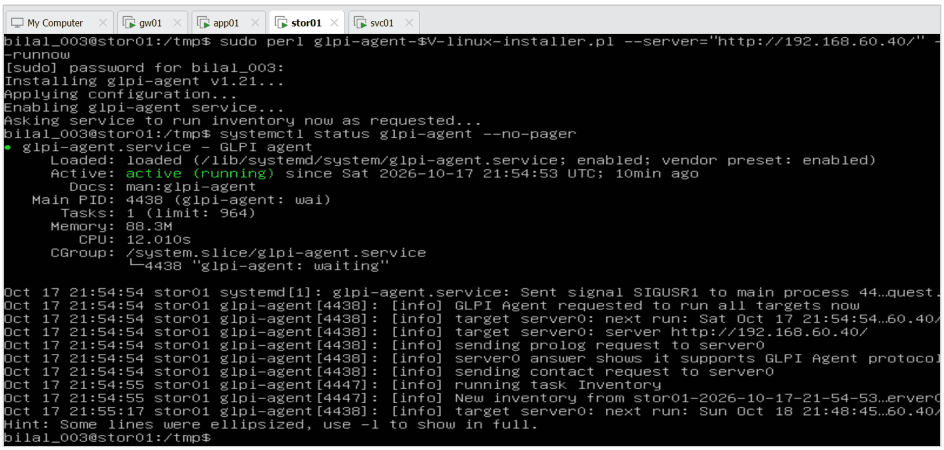
```
cd /tmp && V=1.21                                          # agent version
wget https://github.com/glpi-project/glpi-agent/releases/download/$V/glpi-agent-$V-linux-installer.pl
sudo perl glpi-agent-$V-linux-installer.pl --server="http://192.168.60.40/" --runnow
#   --server = where GLPI lives (svc01)   --runnow = send the first inventory immediately
systemctl status glpi-agent --no-pager                     # service running, enabled at boot
```

What a good run looks like in the log: *prolog request → server answer shows it supports GLPI Agent protocol → running task Inventory* → **New inventory from <server>**. The agent then waits and reports again about once a day (*next run: …*).

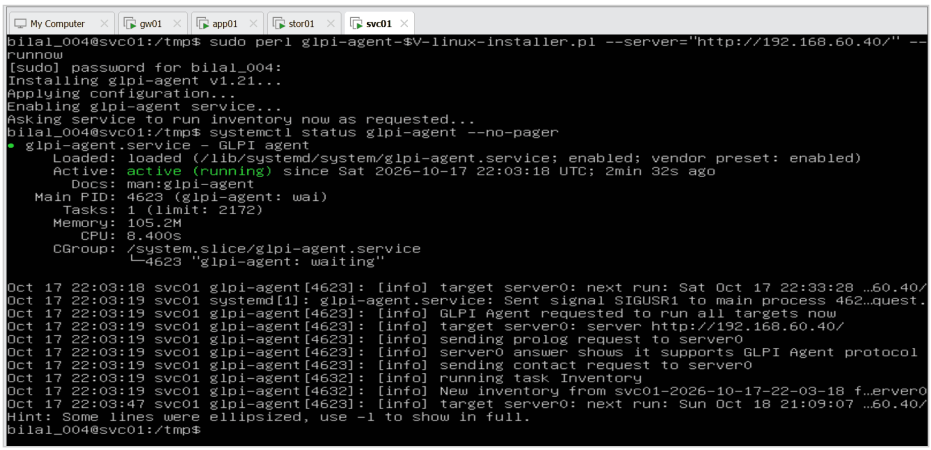
📸 **Evidence – app01, stor01 and svc01 sent their first inventory:**



app01 agent



stor01 agent

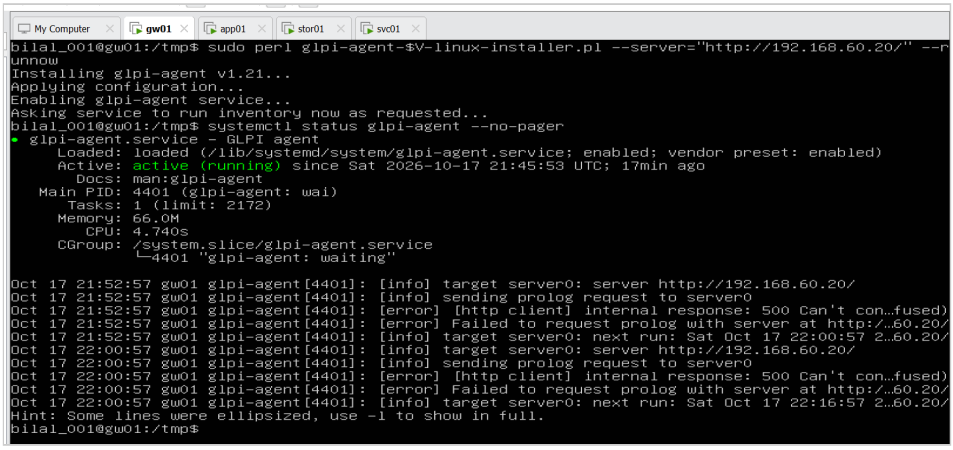


svc01 agent

8.3 Issue – gw01 sent to the wrong address

Seen	On gw01: `[error] [http client] internal response: 500 Can't connect … refused` – *Failed to request prolog with server at http://192.168.60.20/*
Cause	I typed `--server="http://192.168.60.20/"`. **.20 is stor01**, which has no web server; GLPI is on **.40** (svc01).
How the log helped	*Connection refused* = the agent reached a machine, but nothing listened on that port. So the first thing to check is the address.
Fix	Correct the address in the agent's settings file, restart, send the inventory now.

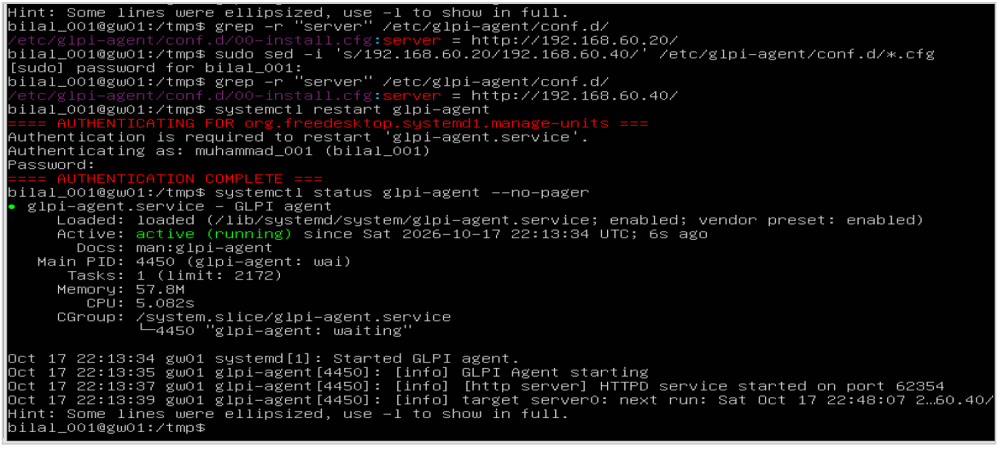
📸 **Evidence – the error:**



gw01 wrong server

```
        grep -r "server" /etc/glpi-agent/conf.d/                                    # shows server = http://
192.168.60.20/
        sudo sed -i 's/192.168.60.20/192.168.60.40/' /etc/glpi-agent/conf.d/*.cfg   # .20 → .40
        grep -r "server" /etc/glpi-agent/conf.d/                                    # check: now .40
        sudo systemctl restart glpi-agent                                           # load the new setting
        sudo glpi-agent --force                                                     # send the inventory now, don't wait
```

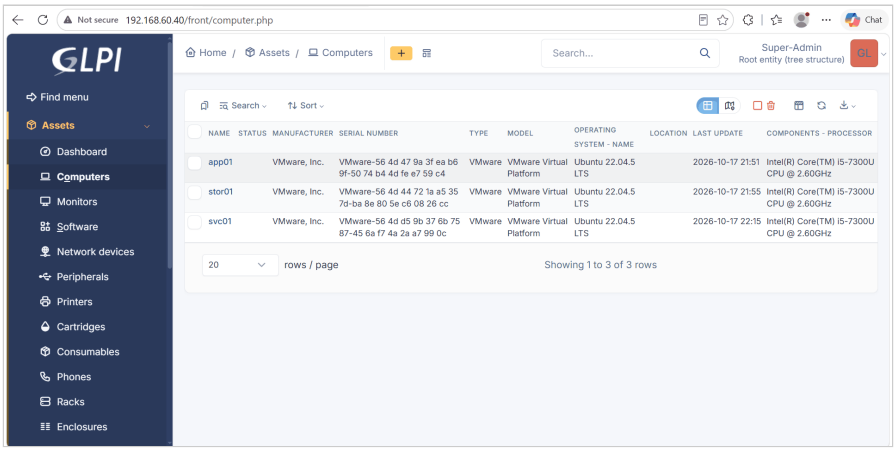
📸 Evidence – setting changed to .40, agent restarted:



gw01 fixed

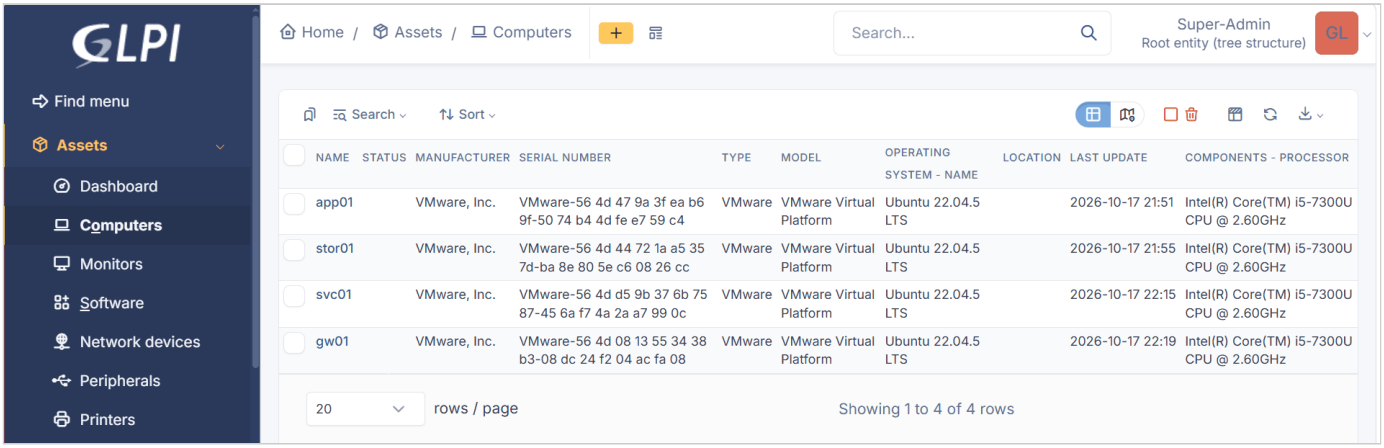
8.4 The result – Assets → Computers

📸 Evidence – first 3 of 4 (gw01 still missing):



3 computers

📸 Evidence – all 4 servers, after `glpi-agent --force` on gw01:



4 computers

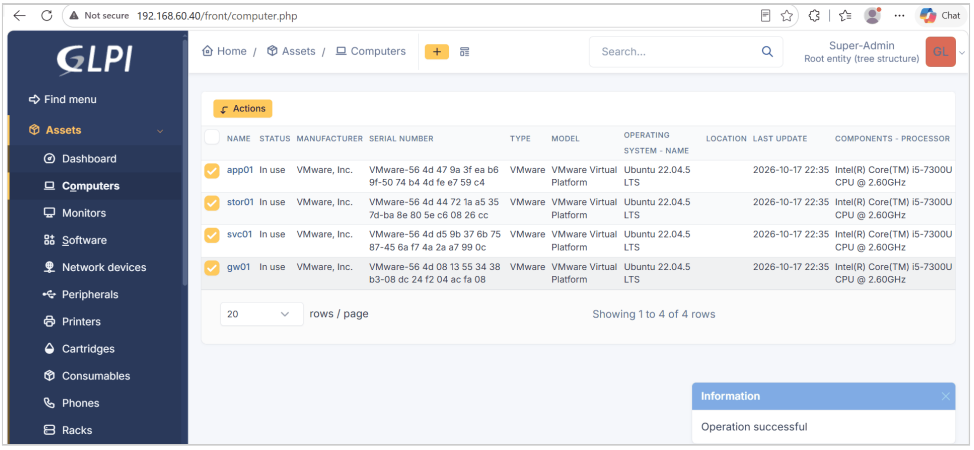
Filled by the agent (automatic)	Filled by a person
Manufacturer / model – *VMware Virtual Platform* (so GLPI knows they are VMs)	**Status** – is the machine in use, in stock, broken?
Serial number – each VM's unique ID	**Location** – where it is
Operating system – Ubuntu 22.04.5 LTS	(later: owner, group, contract)
Processor – my PC's i5-7300U, shared by the VMs	
Last update – when the agent last reported	

The agent can only know facts it can read from the machine. Business facts are added by people – and the agent's daily report does not overwrite them.

8.5 Status and location for all 4 at once

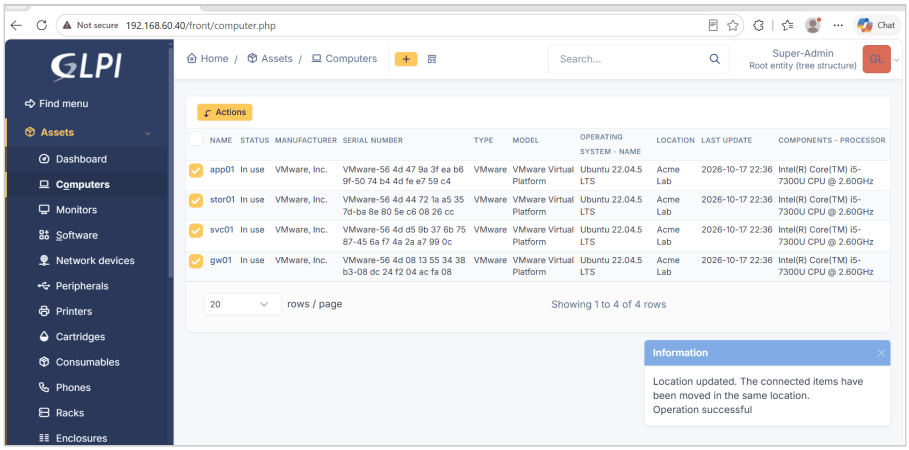
1. Setup → Dropdowns → **Common** → *Statuses of items* → **+** → `In use`; *Locations* → **+** → `Acme Lab`.
2. Assets → Computers → tick all 4 → **Actions → Update** → *Status* → In use → **Post**; again for *Location* → Acme Lab.

📸 **Evidence – Status "In use" on all 4 ("Operation successful"):**



status in use

📸 **Evidence – Location "Acme Lab" on all 4:**



location acme lab

8.6 Snapshots

All four VMs shut down with `sudo poweroff` (gw01 last – it is the router), then:

VM	Snapshot	Description
gw01	`25-service-desk-gw01`	GLPI agent installed, reports to svc01 (.40)
app01	`25-service-desk-app01`	GLPI agent installed, reports to svc01
stor01	`25-service-desk-stor01`	GLPI agent installed, reports to svc01
svc01	`25-service-desk-svc01`	GLPI + MySQL in Docker, inventory on, 4 computers In use / Acme Lab; default accounts still active

No risk overnight: the VMs are off, and GLPI is only on the private lab network. If this snapshot is ever restored, the default accounts must be disabled again.

Key understanding (in my own words)

> The GLPI agent is a small program on each server. It reads the machine's hardware, operating system, serial number and software and sends this to GLPI on svc01 at 192.168.60.40, about once a day. So the asset list fills itself and stays correct. I switched the inventory on in GLPI without authentication, which is fine inside the private lab; in Phase 6 I will add Basic Authentication. Three servers reported at once. gw01 failed with "connection refused" because I had typed .20 – that is stor01, which has no web server. The log showed the address, I corrected it in the agent's settings file, and gw01 reported too. Now all four servers are listed. The agent fills the technical facts; I added the business facts – Status "In use" and Location "Acme Lab" – for all four at once with Actions → Update.

9. Next steps

1. Start gw01 first, then the others.
2. Disable `tech`, `normal`, `post-only` (Active: No) and click **Disable demonstration** – screenshot both.
3. First ticket: a **Change** – *Introduce GLPI service desk and asset inventory* – with plan, rollback (`docker compose down`) and time spent.
4. Step 1A.3: set GLPI up like a real service desk – entity *Acme Analytics*, categories, SLAs, users, knowledge-base categories.

Project 2 – Acme Analytics Linux Platform

Day 4 · Phase 1A wrap-up and Phase 2: LVM, NFS, backups and restore

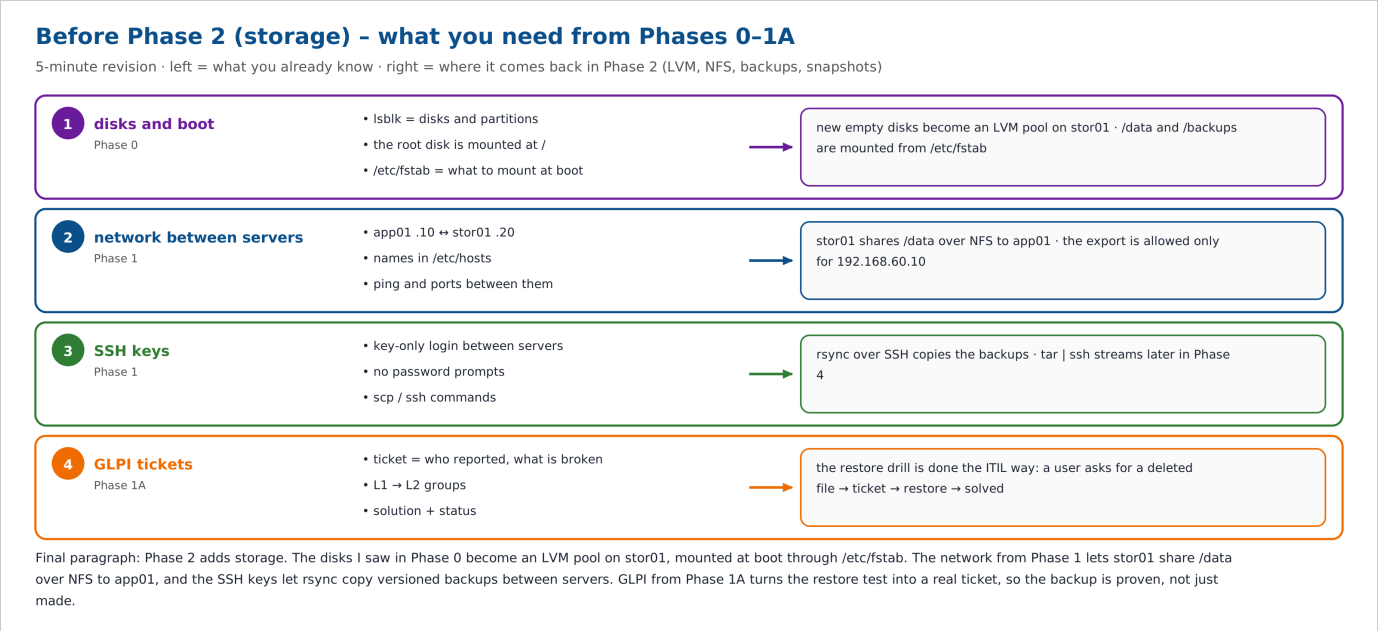
Muhammad Bilal Javed · 18 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1A – Service desk: GLPI tickets, knowledge base and asset inventory
Hosts worked on	svc01 (GLPI), stor01 (LVM, NFS server), app01 (NFS client)
Status	Phase 1A ✅ for now – default accounts disabled, demo data off, Change #1 recorded. Entity, SLAs and users (Step 1A.3) added later, when the first tickets need them · Phase 2 🟡 – Steps 2.1 (LVM), 2.2 (NFS) and 2.3 (backups + restore through a ticket) ✅; LVM snapshot next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary.

Before Phase 2 – quick revision (Phases 0–1A)

💡 **5 minutes:** what I already know from Phases 0–1A and where it comes back in Phase 2.



Before Phase 2 – quick revision

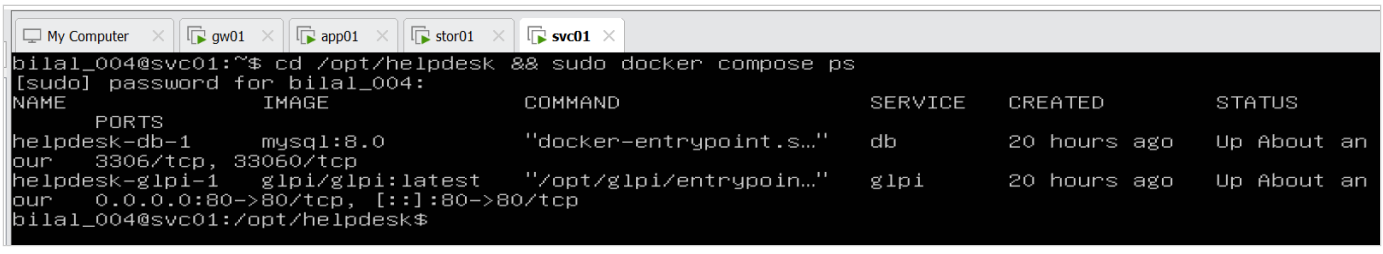
---

1. Start of the day – does GLPI survive a full power-off?

💡 **In plain words:** last night all four VMs were shut down. Today they were started again, gw01 first (it is the router). GLPI came back by itself – nobody had to start it.

```
cd /opt/helpdesk && sudo docker compose ps     # both containers "Up"
```

📸 **Evidence – containers created 20 hours ago, "Up about an hour" (since svc01 booted):**



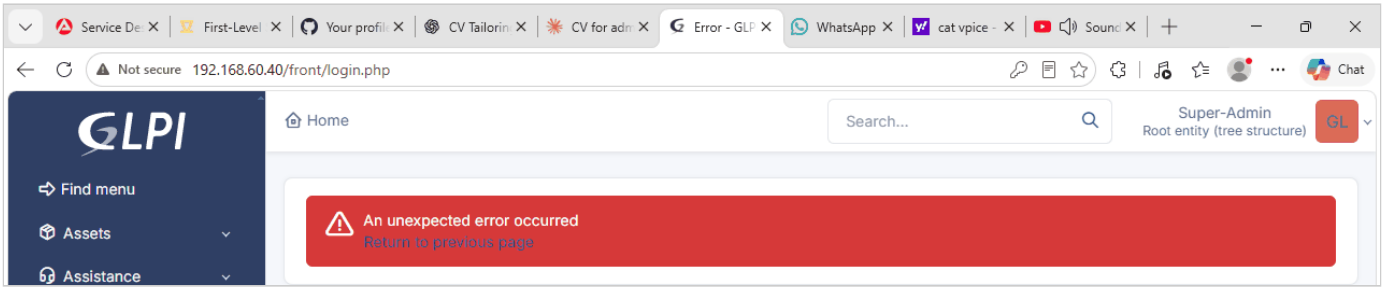
containers back after reboot

Why: `restart: unless-stopped` in `compose.yaml` tells Docker to start the containers again whenever the server boots. A service that needs a person to start it after every reboot is an outage waiting to happen.

Issue – “An unexpected error occurred”

Seen	The browser tab left open from last night, on `login.php`, showed *“An unexpected error occurred”* – while the top right already showed *Super-Admin*.
Cause	The login form carries a one-time security code (CSRF token). After the restart it was no longer valid. GLPI itself was fine.
Fix	Open a fresh address: `http://192.168.60.40/` (or click **Home**).
If it comes back	`sudo docker compose logs glpi --tail 30` and `/opt/helpdesk/storage/glpi/log/php-errors.log`

📸 **Evidence:**

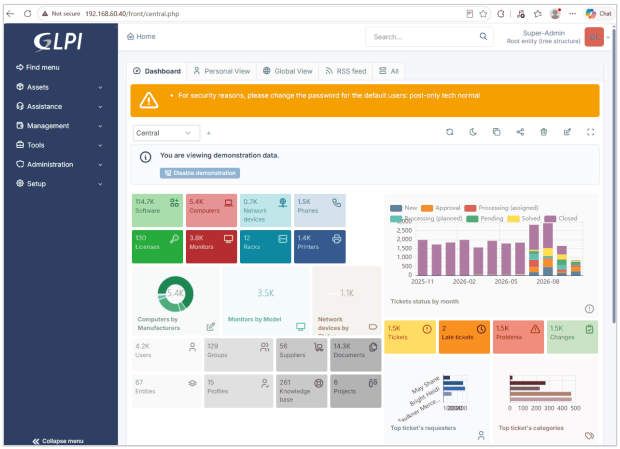


stale login tab

2. Clean-up – default accounts and demo data

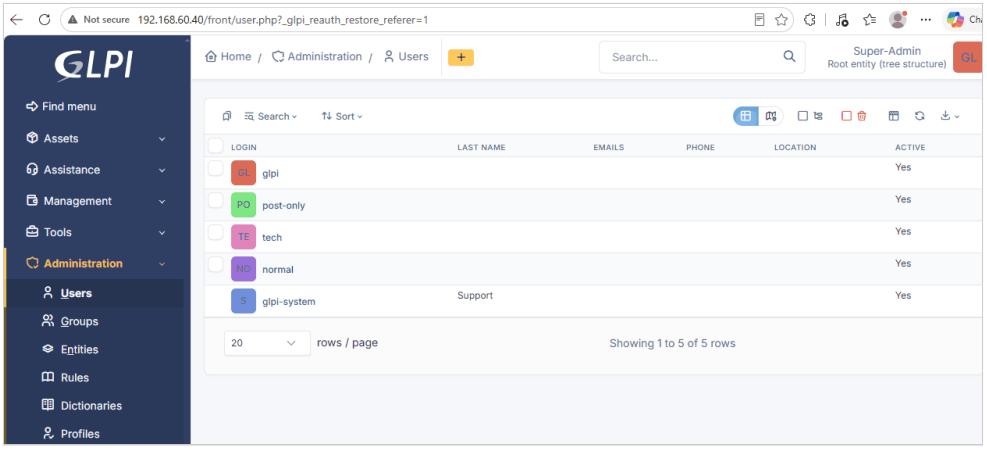
💡 **In plain words:** GLPI ships with four accounts whose passwords are printed in every guide on the internet. The `glpi` password was changed on Day 3; the other three are not needed, so they are switched off. The dashboard was also full of invented sample numbers.

📸 **Evidence – before: GLPI warns about the default passwords; “You are viewing demonstration data” (5.4K computers, 1.5K tickets – none of it real):**



dashboard before

📸 **Evidence – before: all five accounts Active:**

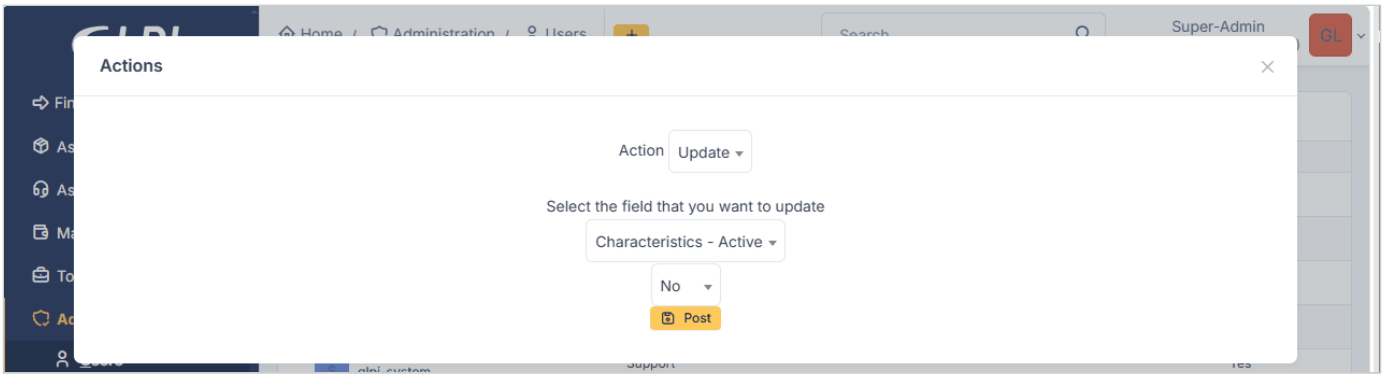


users before

Account	Action	Why
`glpi`	keep (password already changed)	my Super-Admin account
`post-only`, `tech`, `normal`	**Active: No**	default passwords known to everyone; not used
`glpi-system`	leave	GLPI's own account for automatic jobs; cannot be ticked

Administration → Users → tick the three → **Actions → Update → Characteristics – Active → No → Post**. Then Home → **Disable demonstration**.

📸 **Evidence – the mass update:**



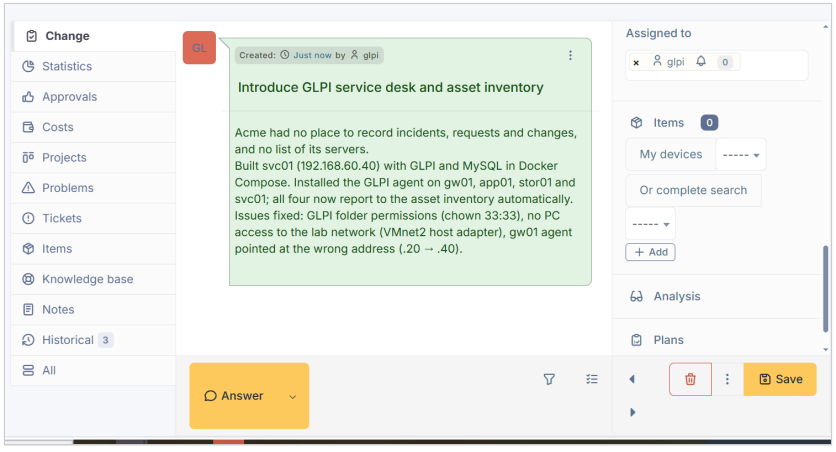
active no

3. The first ticket – Change #1

💡 **In plain words:** from now on, every piece of work is recorded in GLPI. The first record describes the service desk itself, written as a **Change**, because it was planned work with a way back.

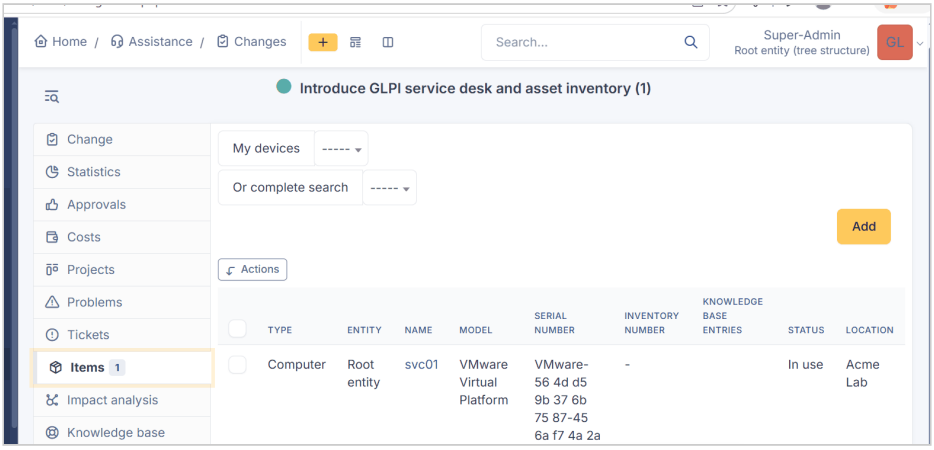
Field	Value
Title	Introduce GLPI service desk and asset inventory
Description	Acme had no place to record incidents, requests and changes, and no list of its servers. Built svc01 (192.168.60.40) with GLPI and MySQL in Docker Compose; GLPI agent on all four servers. Issues fixed: folder permissions (`chown 33:33`), no PC access to the lab network (VMnet2 host adapter), gw01 agent pointed at the wrong address (.20 → .40).
Item	svc01
Impacts	No downtime – a new server only; the existing servers only received a small agent
Control list	Containers Up; HTTP 200; 4 computers in Assets
Deployment plan	Docker + `compose up` → `chown 33:33` → VMnet2 host adapter → agent on 4 servers
Backout plan	`docker compose down`; restore snapshot `24-svc01-installed`; remove `glpi-agent` from the servers
Status	Closed

📸 **Evidence – Change #1 created:**



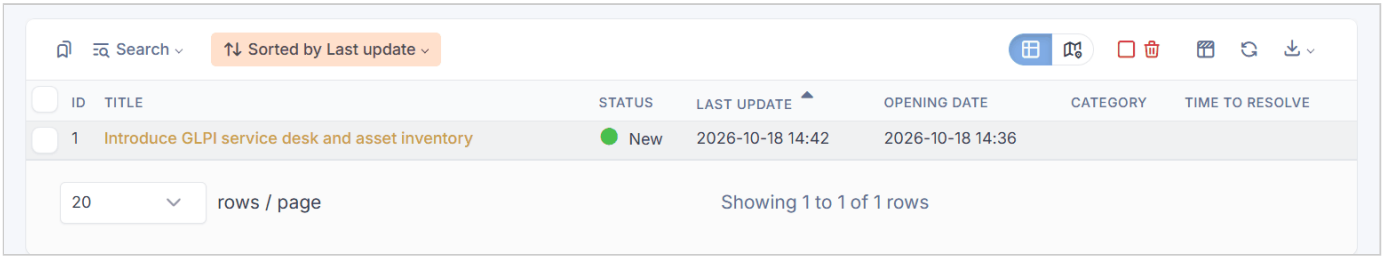
change created

📸 **Evidence – svc01 linked to the change (status In use, location Acme Lab – from the inventory):**



change item

📸 Evidence – Changes list, ID 1:



changes list

A change in GLPI can pass through New → Evaluation → Approval → Accepted → Test → Applied → Review → Closed. This one was already finished, so it goes straight to Closed.

4. ITIL basics – three ticket types and the SLA

Type	Use it for	Example in this project
Incident (Assistance → Tickets)	Something is broken	NFS mount hangs after a firewall change
Request (Assistance → Tickets, type Request)	Someone wants something	“Please give carol read access to /srv/shared”
Change (Assistance → Changes)	Planned work with a plan and a rollback	“Grow /data by 1 GB”

SLA (Service Level Agreement) – the promise of how fast IT reacts and fixes, by priority:

Priority	Example	Time to own	Time to resolve
High	App down, nobody can work	1 hour	8 hours
Medium	One user cannot reach a shared folder	4 hours	3 days
Low	“Please install a tool for me”	1 day	1 week

Time to own = how fast a technician must take the ticket (“I’m on it”). *Time to resolve* = how fast it must be fixed. GLPI starts both clocks when a ticket is created and marks it **late** when a promise is missed – the “2 Late tickets” on the demo dashboard were exactly that.

Key understanding (in my own words)

> In GLPI every piece of work is recorded as one of three types: an **Incident** when something is broken, a **Request** when someone asks for something, and a **Change** for planned work. A change carries extra boxes – impact, deployment plan and backout plan – so another engineer can see what was changed and how to undo it. My first change records the service desk itself, linked to svc01.
>
> An **SLA** (Service Level Agreement) is a written promise of how fast IT will react to a problem and fix it, depending on how serious it is. Every ticket gets a priority. For a high-priority problem, like the app being down, Acme promises that a technician takes the ticket within 1 hour (time to own) and fixes it within 8 hours (time to resolve); a medium problem gets 4 hours and 3 days. GLPI starts both clocks when the ticket is created and shows it as late when a promise is missed. So technicians know what to work on first and when to escalate, and managers can report how many tickets were solved on time. “Work according to SLAs” in a job ad means picking up tickets on time, handling the most urgent ones first, and escalating early before a deadline is missed.

5. Phase 2 – storage on stor01: LVM from empty disks

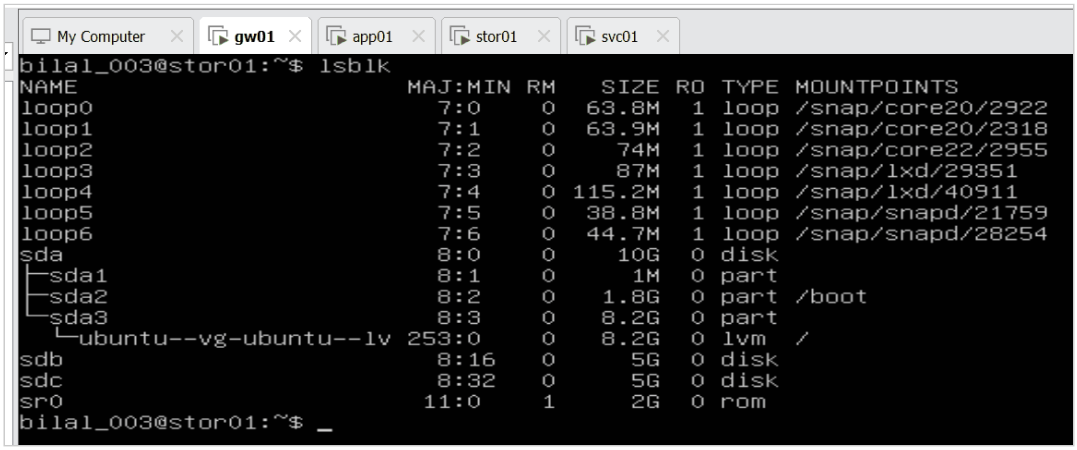
💡 **In plain words:** Acme’s application runs on **app01**, its data and backups live on **stor01**. stor01 needs storage that can **grow** when it fills up – without stopping anything. LVM gives exactly that: disks go into a pool, and volumes are cut from the pool and grown later.

```
sdb + sdc  →  pvcreate  →  PV, PV          (disks handed to LVM)
           →  vgcreate  →  VG "vgdata"     (one 10 GB pool)
           →  lvcreate  →  LV, LV          (slices: lvdata, lvbackups)
           →  mkfs.ext4 →  file system     (format)
           →  mount     →  /data, /backups (usable folders)
```

5.1 The disks

```
lsblk
```

📸 Evidence – sda = system disk (sda1/sda2 boot, sda3 = the system's own LVM); sdb and sdc = two empty 5 GB disks; loop0–6 are snap images, sr0 the CD drive:

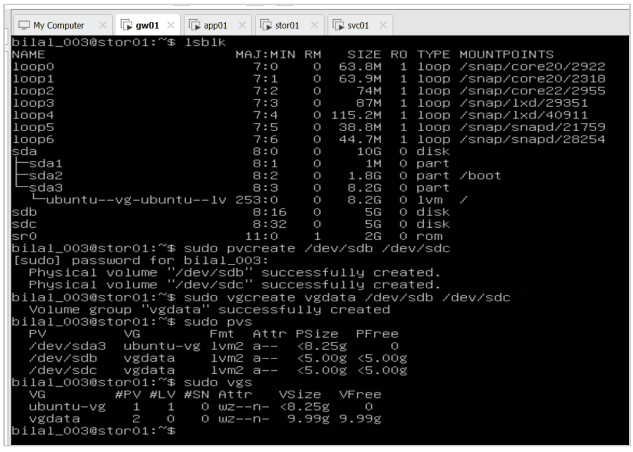


lsblk

5.2 Disks → physical volumes → volume group (the pool)

```
sudo pvcreate /dev/sdb /dev/sdc          # hand both disks to LVM (wipes them — check lsblk first)
sudo vgcreate vgdata /dev/sdb /dev/sdc   # one pool from both
sudo pvs                                 # the PVs
sudo vgs                                 # vgdata: 2 PVs, 0 LVs, 9.99 GB, all free
```

📸 Evidence:



pvcreate vgcreate

On svc01 the PV was a **partition** (sda3, because sda also holds the boot partitions); here the PVs are **whole disks** – they only store data. Both are normal.

5.3 Cut, format, mount

```
sudo lvcreate -L 3G -n lvdata vgdata       # 3 GB for app data
sudo lvcreate -L 3G -n lvbackup vgdata     # 3 GB for backups — about 4 GB stay free
sudo mkfs.ext4 /dev/vgdata/lvdata          # format
sudo mkfs.ext4 /dev/vgdata/lvbackup
```

📸 Evidence – 786,432 blocks × 4 KB = 3 GB; 196,608 inodes = the most files this volume can hold; the UUID is its permanent ID:

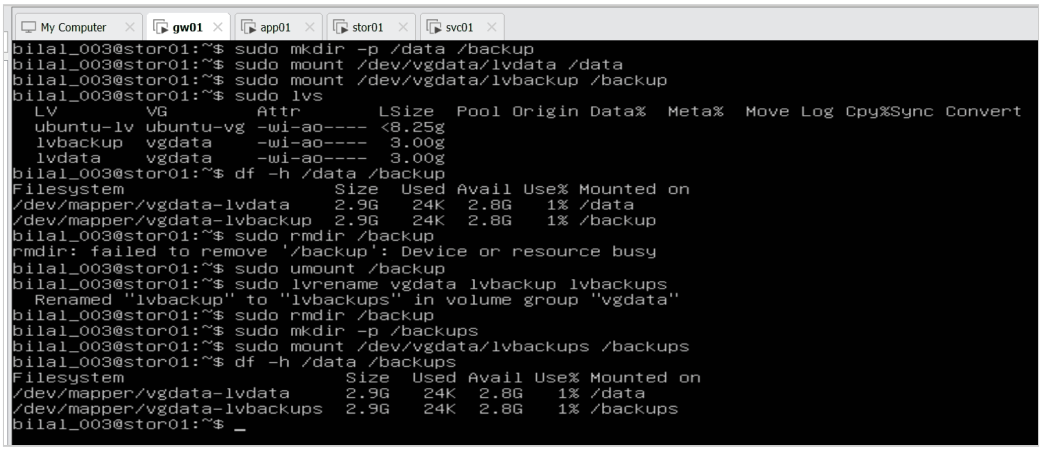
```
bilal_003@stor01:~$ sudo lvcreate -L 3G -n lvdata vgdata
  Logical volume "lvdata" created.
bilal_003@stor01:~$ sudo lvcreate -L 3G -n lvbackup vgdata
  Logical volume "lvbackup" created.
bilal_003@stor01:~$ sudo mkfs.ext4 /dev/vgdata/lvdata
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 786432 4k blocks and 196608 inodes
Filesystem UUID: a973da7a-5455-42c7-8608-58b604d1317c
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

bilal_003@stor01:~$ sudo mkfs.ext4 /dev/vgdata/lvbackup
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 786432 4k blocks and 196608 inodes
Filesystem UUID: cf760933-4488-4253-b6d3-59a6555d2732
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

lvcreate mkfs

The volume was first mounted on `/backup`; to keep the names consistent it became `/backups`, and the LV was renamed to match:

```
        sudo umount /backup                          # rmdir before this failed: "Device or resource busy"
        sudo lvrename vgdata lvbackup lvbackups      # renaming does not change the UUID
        sudo rmdir /backup && sudo mkdir -p /backups
        sudo mount /dev/vgdata/lvbackups /backups
        df -h /data /backups
# 2.9 GB each (ext4 keeps room for its journal, inode tables and 5 % for root)
```

📸 **Evidence:**



rename and mount

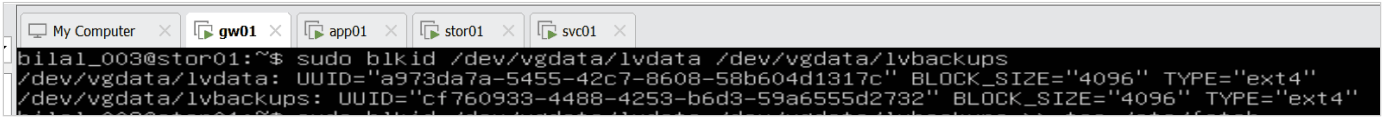
Lesson: a folder with a volume mounted on it is “busy” – `umount` first, then remove it. Separate volumes give isolation of space: full backups cannot fill the app’s `/data`.

5.4 Make the mounts permanent – `/etc/fstab`

`mount` lasts only until the next reboot. `/etc/fstab` is the list of what to mount at every boot.

```
        sudo blkid /dev/vgdata/lvdata /dev/vgdata/lvbackups      # the UUIDs
        sudo cp /etc/fstab /etc/fstab.bak                        # backup = way back
        echo 'UUID=a973da7a-5455-42c7-8608-58b604d1317c  /data     ext4  defaults  0 2' | sudo tee -a /etc/fstab
        echo 'UUID=cf760933-4488-4253-b6d3-59a6555d2732  /backups  ext4  defaults  0 2' | sudo tee -a /etc/fstab
```

📸 **Evidence – the UUIDs:**



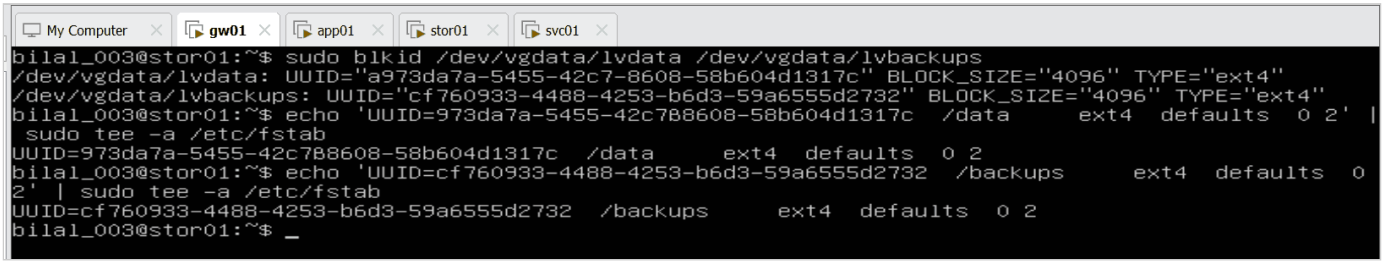
blkid

Column	Meaning
`UUID=…`	Which volume – never changes, even if disk names (sdb, sdc) do
`/data`	Where to mount it
`ext4`	File system type
`defaults`	Normal options (read/write …)
`0`	Old “dump” backup tool: off
`2`	Check the disk at boot, after / (which is 1)

Issue – one wrong character in a UUID

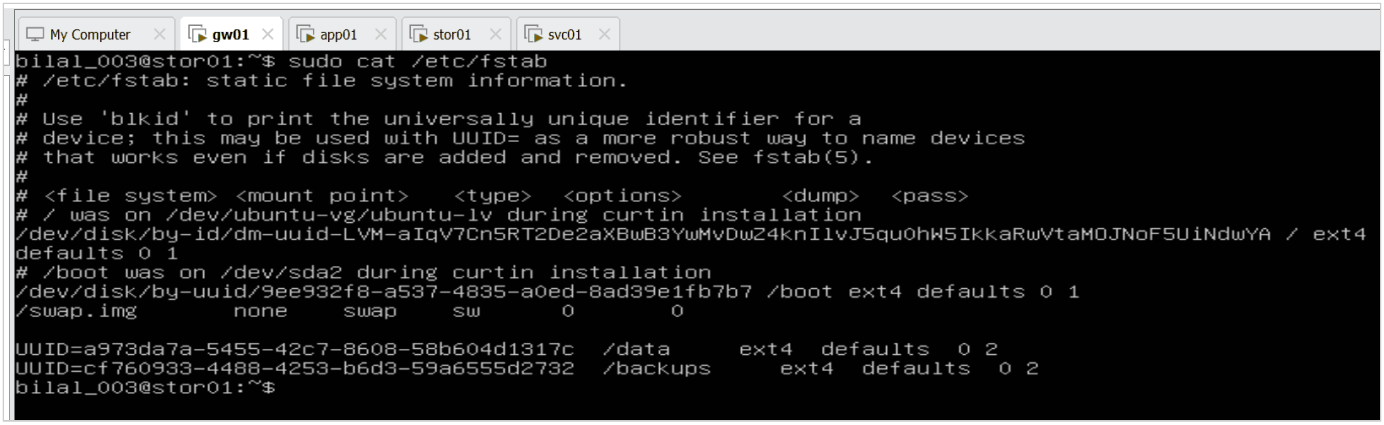
Seen	The `/data` line read `UUID=973da7a-…-42c7B8608-…` – the first a missing and a `-` typed as `B`.
Risk	At boot Linux would not find that volume and could stop in **emergency mode** – the server down because of one character.
Fix	Corrected in `nano /etc/fstab`, compared character by character with `blkid`.
Lesson	After every fstab change: `sudo mount -a` **before** a reboot. Better still, never type a UUID: `echo "UUID=$(sudo blkid -s UUID -o value /dev/vgdata/lvdata) /data ext4 defaults 0 2"`

📸 **Evidence – the wrong line:**



uuid typo

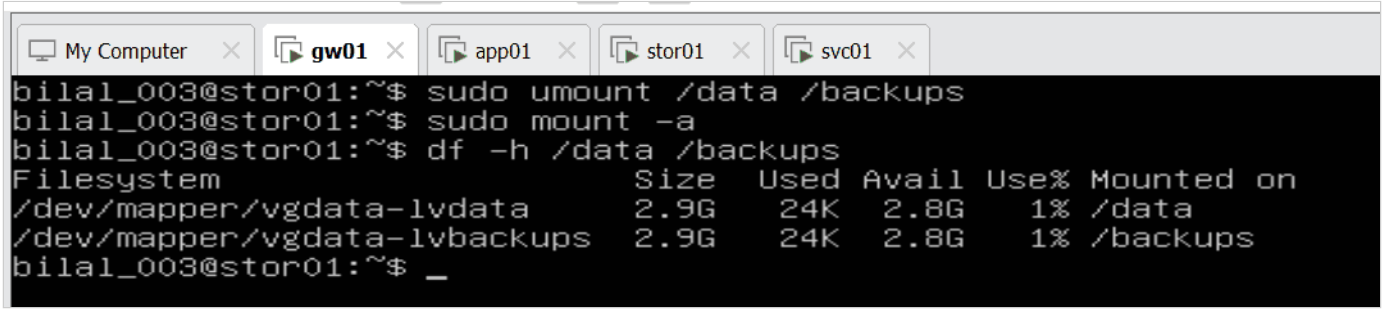
📸 **Evidence – fixed; both UUIDs match `blkid`:**



fstab fixed

```
sudo umount /data /backups
sudo mount -a              # no output = fstab is correct
df -h /data /backups
```

📸 **Evidence – mounted again from fstab alone:**



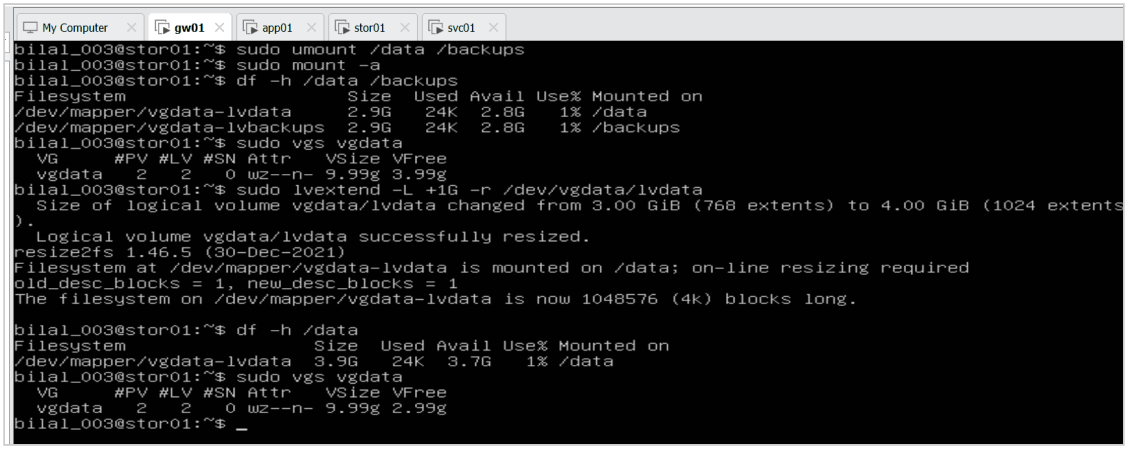
mount -a

5.5 Grow /data while it is running

💡 **The story:** a month later Acme's data has grown and `/data` is almost full. Without LVM: stop the app, add a disk, copy data. With LVM: take free space from the pool, live.

```
sudo vgs vgdata                                 # VFree 3.99 GB
sudo lvextend -L +1G -r /dev/vgdata/lvdata      # +1 GB; -r grows ext4 too
df -h /data                                     # 3.9 GB, no unmount, no downtime
sudo vgs vgdata                                 # VFree 2.99 GB
```

📸 **Evidence – 3.00 GiB (768 extents) → 4.00 GiB (1024 extents); "on-line resizing"; the pool's free space drops by 1 GB:**



lvextend

`-L +1G` (capital L) adds an exact size; on svc01 `-l +100%FREE` (small l) took all free space. LVM hands out space in 4 MB blocks called **extents**.

6. Phase 2 – NFS: app01 uses a folder on stor01

💡 **In plain words:** the app runs on app01, but its files should live on stor01, where the storage and backups are. **NFS** (Network File System) lets app01 use a stor01 folder over the network as if it were its own disk.

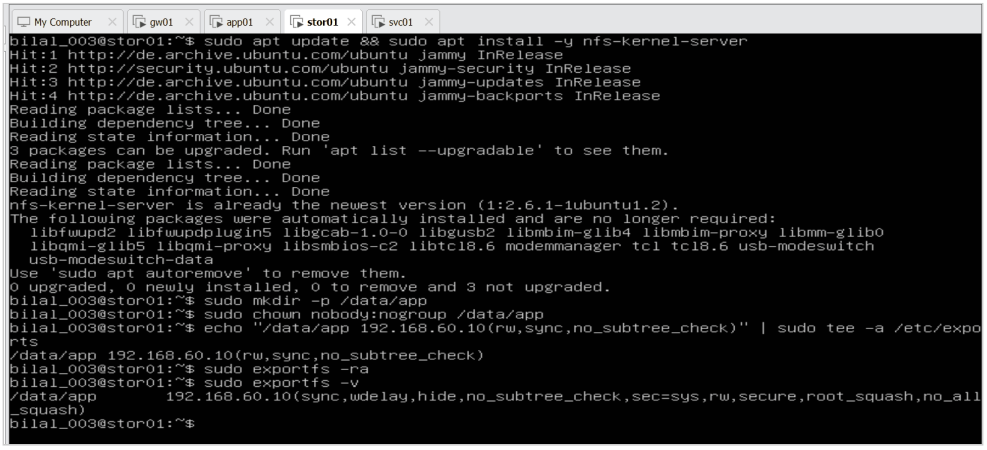
```
stor01 /data/app  ──NFS over the lab network──►  app01 /mnt/app
   (server: shares the folder)                  (client: mounts it)
```

6.1 stor01 – the server

```
sudo apt install -y nfs-kernel-server                          # already installed – nothing to download
sudo mkdir -p /data/app
sudo chown nobody:nogroup /data/app                            # neutral owner (see root_squash)
echo "/data/app 192.168.60.10(rw,sync,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra                                              # apply the share list
sudo exportfs -v                                               # what is shared, to whom
```

Part of the `/etc/exports` line	Meaning
`/data/app`	The folder to share
`192.168.60.10`	**Only app01** may mount it
`rw`	Read and write
`sync`	Write to disk before confirming – safer
`no_subtree_check`	Skip an old, slow check – standard choice
`root_squash` (default)	root on app01 is treated as `nobody` on stor01 – root on one machine is not root on the other

📸 **Evidence – shared with app01 only:**

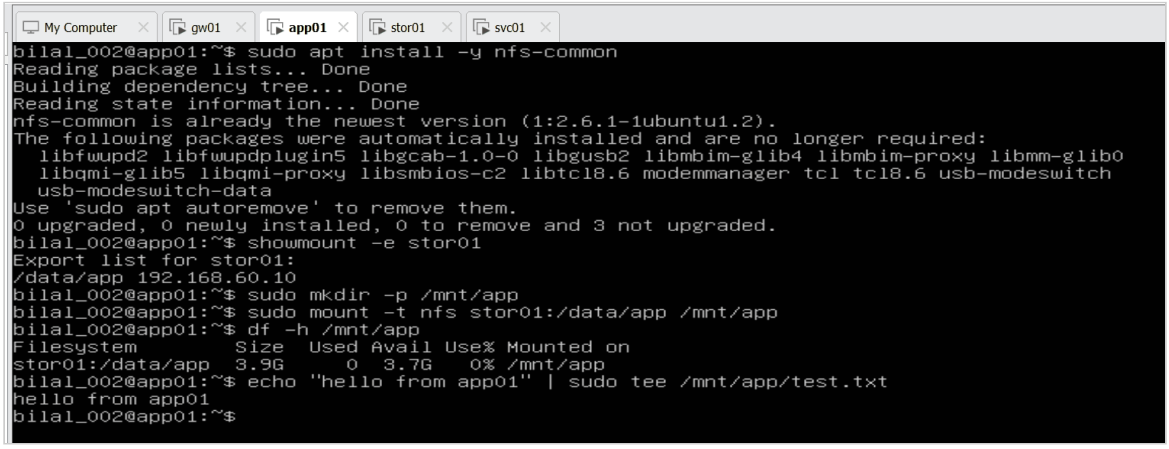


nfs server

6.2 app01 – the client, and the proof

```
sudo apt install -y nfs-common
showmount -e stor01                                # what does stor01 share?
sudo mkdir -p /mnt/app
sudo mount -t nfs stor01:/data/app /mnt/app
df -h /mnt/app                                     # stor01:/data/app, 3.9 GB
echo "hello from app01" | sudo tee /mnt/app/test.txt   # write on app01
```

📸 **Evidence – app01:**



nfs client

```
cat /data/app/test.txt      # on stor01 — the same text
ls -l /data/app             # owner nobody:nogroup — root_squash at work
```

📸 **Evidence – stor01 sees the file at once:**

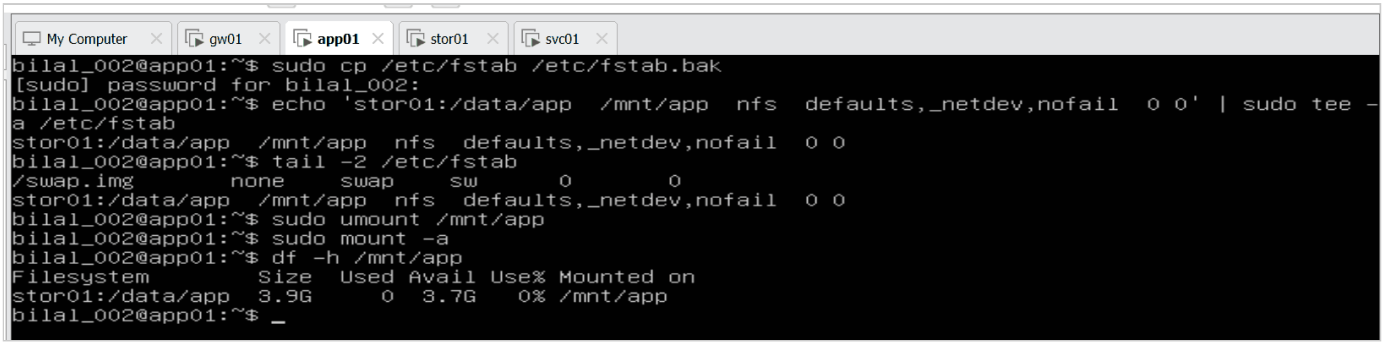
stor01 sees file

6.3 Permanent on app01

```
sudo cp /etc/fstab /etc/fstab.bak
echo 'stor01:/data/app  /mnt/app  nfs  defaults,_netdev,nofail  0 0' | sudo tee -a /etc/fstab
sudo umount /mnt/app
sudo mount -a
df -h /mnt/app
```

Option	Meaning
_netdev	A **network** disk – wait for the network before mounting
nofail	If stor01 is off, **app01 still boots**, just without /mnt/app
0 0	No dump, no disk check – stor01 checks its own disks

📸 **Evidence – mounted from fstab:**



app01 fstab

Lesson: a server must never fail to boot because **another** server is down.

Key understanding (in my own words)

Acme's application runs on app01, and its data and backups live on stor01. stor01 needed storage that can **grow** and an easy way to **share** files with app01.

LVM. `lsblk` showed two empty 5 GB disks, sdb and sdc. A raw disk cannot be used directly, so I built LVM layer by layer: `pvcreate` made both disks **physical volumes**; `vgcreate` put them into one **volume group**, `vgdata` – a 10 GB **pool** (a VG can be built from one or many PVs); `lvcreate` cut two 3 GB **logical volumes** from the pool, `lvdata` and `lvbackups`, so full backups cannot fill the app's space; `mkfs.ext4` formatted them and `mount` attached them to `/data` and `/backups`. Mounts disappear after a reboot, so I backed up `/etc/fstab` and added each volume's **UUID** from `blkid`. One wrong character in a UUID taught me to always test with `mount -a` before a reboot. Finally `lvextend -L +1G -r` grew `/data` from 3 to 4 GB while it was mounted – the main reason LVM exists.

NFS. On stor01 I installed the NFS server, created `/data/app` owned by `nobody:nogroup`, and shared it in `/etc/exports` with **only app01**, read/write. `root_squash` treats root from app01 as "nobody" on stor01. On app01 I installed the NFS client, checked the share with `showmount -e stor01` and mounted it on `/mnt/app`; a file written on app01 appeared at once on stor01. In app01's fstab I used `_netdev` (wait for the network) and `nofail` (boot even if stor01 is down).

In one line: stor01 now has storage that grows without downtime (LVM) and shares it with app01 over the network (NFS).

7. Phase 2 – backups: archive, mirror, versions, restore

💡 **In plain words:** a backup is a copy kept somewhere safe, so a lost file can come back. I made three kinds, all in `/backups` on stor01 (the `lvbackups` volume), and then proved they work by restoring a deleted file – through a real ticket.

```
stor01
/data/app/                      ← the original (shared with app01 over NFS)
/backups/
├── data-2026-10-18.tar.gz      ← ARCHIVE: one compressed file per day          (file)
├── mirror/                     ← MIRROR: exact copy of /data/app, only "now"   (real folder)
├── v1/  v2/  v3/  2026-10-19/  ← VERSIONS: one folder per run, history        (real folders)
└── latest -> 2026-10-19        ← shortcut to the newest version               (soft link)
```

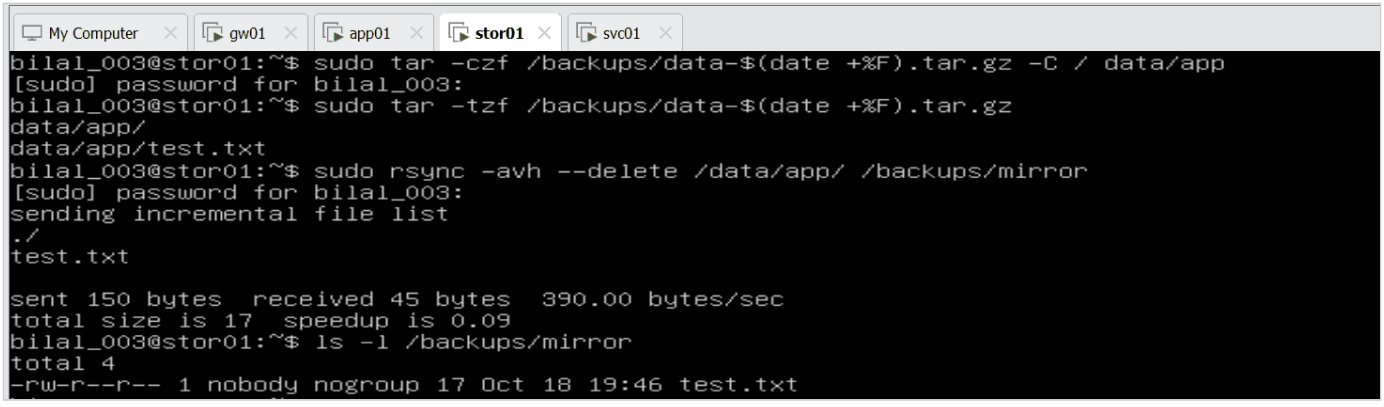
`/data`, `/backups` and `/data/app` are folders I created; `/mnt` is the standard empty folder for temporary mounts.

7.1 Archive (tar) and mirror (rsync)

```
sudo tar -czf /backups/data-$(date +%F).tar.gz -C / data/app   # pack /data/app into ONE dated, compressed file
tar -tzf /backups/data-$(date +%F).tar.gz                      # LIST what is inside – nothing is unpacked
sudo mkdir -p /backups/mirror                                   # a new, empty real folder
sudo rsync -avh --delete /data/app/ /backups/mirror/            # make it an exact copy of /data/app
```

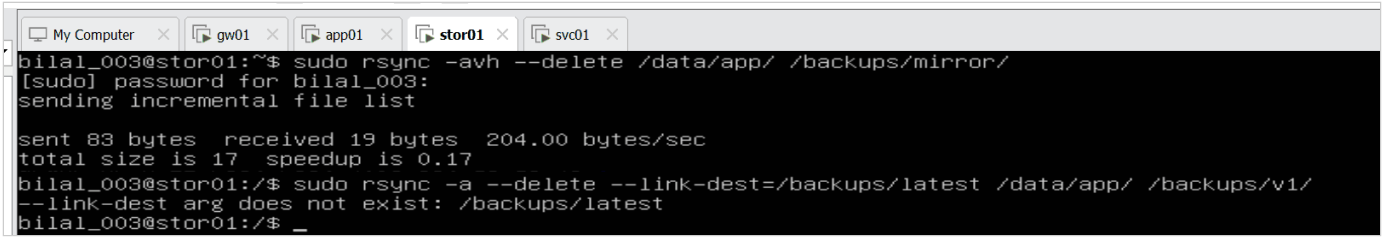
Part	Meaning
`tar -c -z -f`	**c**reate an archive, compress with g**z**ip, **f**ile name follows
`$(date +%F)`	today's date in the name, e.g. `2026-10-18`
`-C /`	go to `/` first, so the archive stores the path `data/app/…`; restore with `tar -xzf … -C /` puts files back in place
`rsync -a`	**a**rchive mode: subfolders, owners, permissions, dates kept
`-v` / `-h`	show each file / human-readable sizes
`--delete`	a file deleted in the source is removed from the **mirror** too – the source is never touched
`/data/app/`	trailing `/` = the *contents* of app

📸 **Evidence – the archive lists `data/app/test.txt`; rsync copies `test.txt`, keeping owner `nobody` and the original time:**



tar and mirror

📸 **Evidence – run again with nothing changed: rsync copies nothing (only "sent 83 bytes"). Then the first versioned run warns `--link-dest arg does not exist` – normal, there is no previous version yet:**



mirror rerun

	Archive (tar)	Mirror (rsync)
What it is	One compressed file per day	A normal folder with independent copies
History	Yes – one file per day	**No – only the state of the last run**
Restore one file	Unpack first	Copy it back

Hard link vs rsync mirror: a hard link is a second name for the **same** file (same inode) – a change appears instantly under both names, and damage hits both, so it is **not a backup**. The mirror holds **independent copies** (own inodes) and changes only when rsync runs – it **is** a backup, but `--delete` also copies mistakes at the next run.

7.2 Versions with hard links – history without wasting space

```
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v1/   # FROM /data/app  TO v1  COMPARE
WITH latest
        sudo ln -sfn /backups/v1 /backups/latest                                     # latest → v1
        echo "new file" | sudo tee /data/app/new.txt                                 # a change
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v2/
        sudo ln -sfn /backups/v2 /backups/latest                                     # latest → v2
        sudo rm /data/app/test.txt                                                   # the "accident"
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v3/
        sudo ln -sfn /backups/v3 /backups/latest                                     # latest → v3
        ls -l /backups
        ls -li /backups/v1 /backups/v2 /backups/v3
```

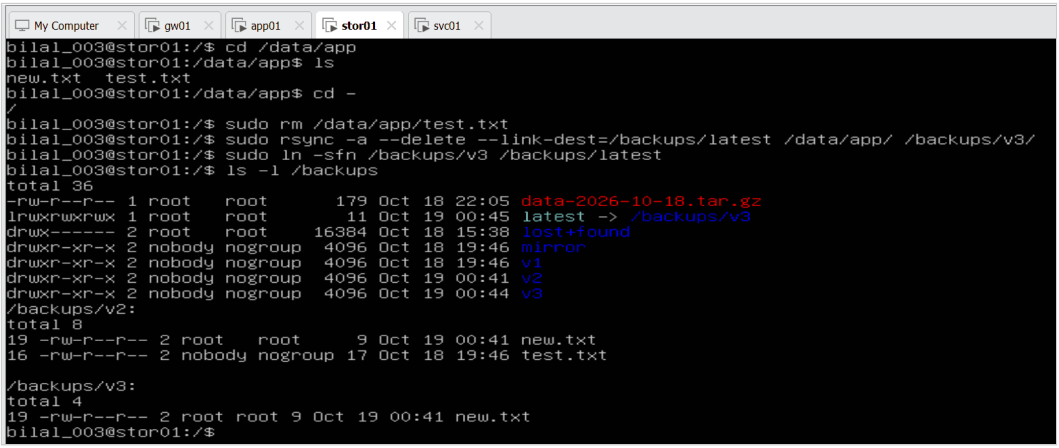
Piece	Job
`/data/app/`	**FROM** – the original
`/backups/v2/`	**TO** – the new version folder (rsync creates it)
`--link-dest=/backups/latest`	**COMPARE WITH** – the previous version: unchanged files become **hard links**, changed or new files are copied
`ln -sfn`	**s**oft link, **f**orce-replace, **n**o-follow – move the `latest` shortcut to the new version

📸 **Evidence – v1, then a new file, then v2:**

```
bilal_003@stor01:/$ sudo ln -sfn /backups/v1 /backups/latest
bilal_003@stor01:/$ echo "new file" | sudo tee /data/app/new.txt
new file
bilal_003@stor01:/$ sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v2/
bilal_003@stor01:/$ sudo ln -sfn /backups/v2 /backups/latest
```

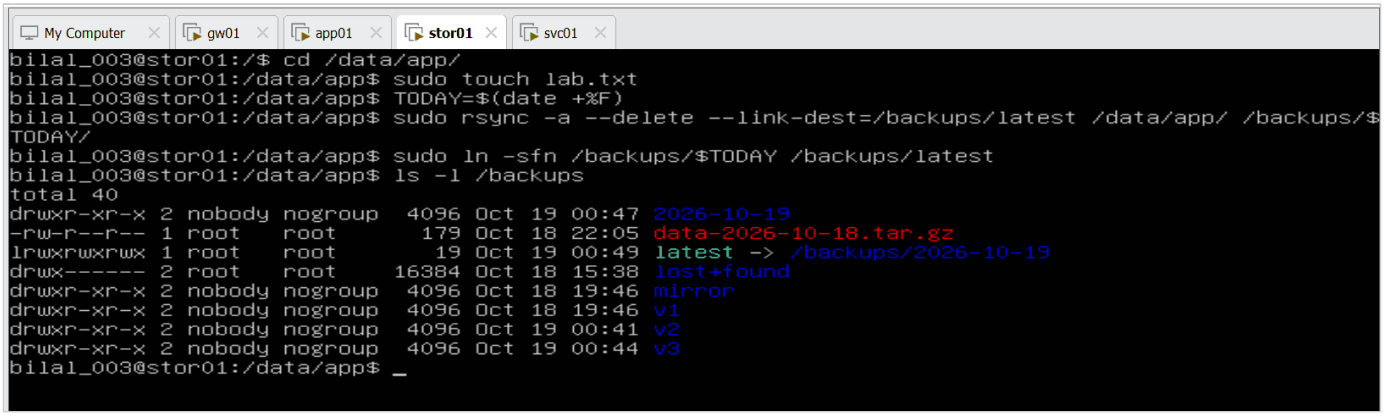
v1 v2

📸 **Evidence – after deleting test.txt, v3; `latest -> /backups/v3`; in v2 `test.txt` has inode 16 with link count 2 (shared with v1 – one copy on disk), `new.txt` inode 19 count 2 (shared with v3); v3 has no test.txt, but** v1 and v2 still do**:**



versions

📸 **Evidence – a real dated version, `/backups/2026-10-19`, and `latest` moved to it:**



dated backup

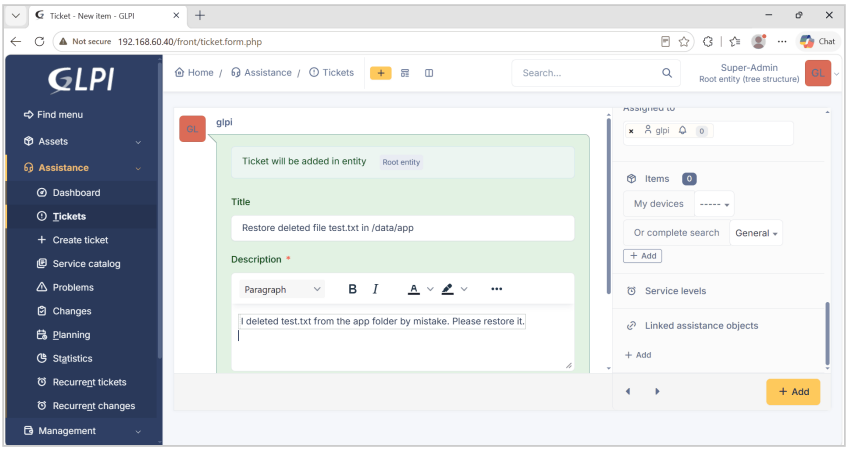
Run	latest before	What rsync does	latest after
1	does not exist (warning)	copies everything into v1	→ v1
2	→ v1	compares with v1: only changes copied, the rest hard-linked	→ v2
3	→ v2	compares with v2	→ v3

Old versions are **never changed** – that is the history. Hard links are safe here because they join backup to backup, not the original, and backup files are never edited.

7.3 Restore drill – through a ticket

💡 **The story:** Alice deletes `test.txt` by mistake and opens a ticket. Normal employees cannot touch `/backups` – it belongs to root and is handled by IT. The technician picks the version taken **before** the deletion and restores it.

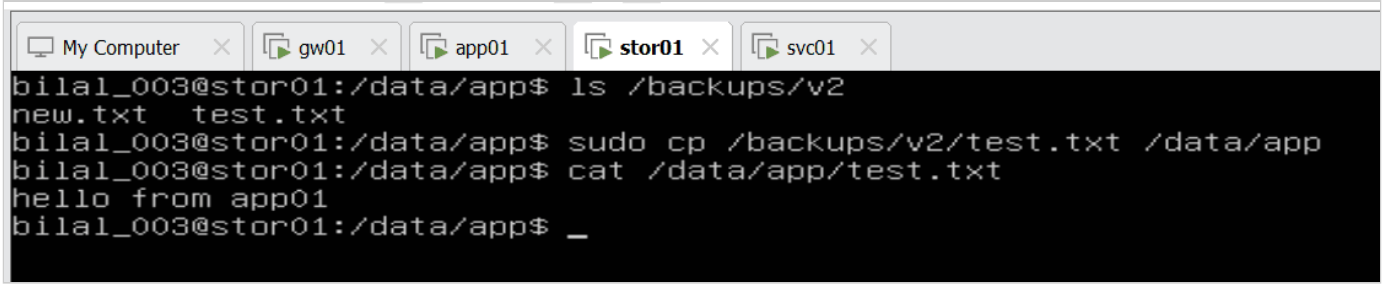
📸 **Evidence – Request ticket created (category *Storage & Backup*):**



ticket

```
ls /backups/v2                              # v3 no longer has the file – v2 does
sudo cp /backups/v2/test.txt /data/app      # restore: OUT of the backup, BACK to /data/app
cat /data/app/test.txt                      # "hello from app01"
```

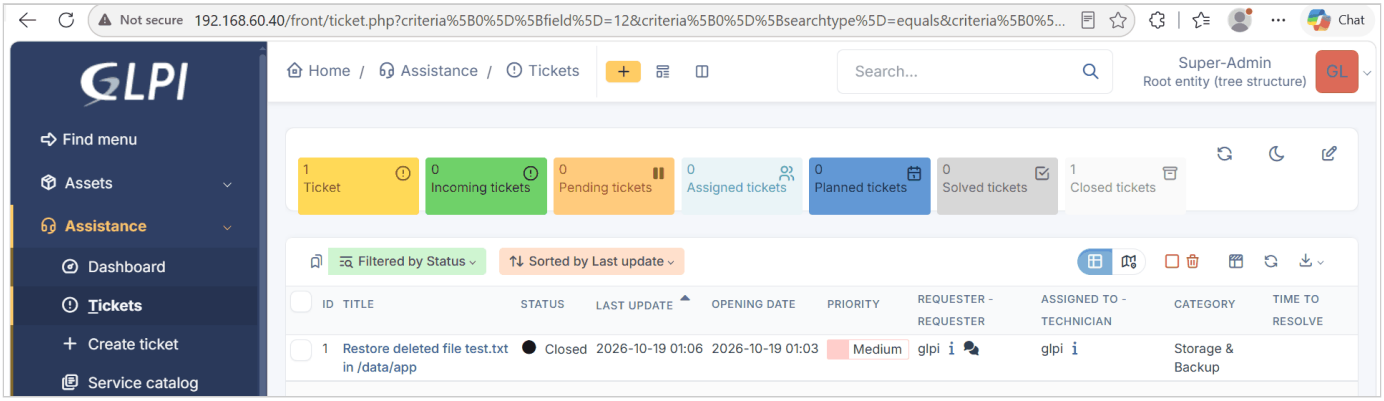
📸 **Evidence – restored and readable:**



restore

Solution written in the ticket: *Restored test.txt from backup version v2 (taken before the deletion) with `cp /backups/v2/test.txt /data/app/`. Content checked. Root cause: accidental deletion. No data lost.*

📸 **Evidence – ticket #1, Request, Storage & Backup,** Closed**:**



ticket closed

Key understanding (in my own words)

In `/backups` I keep three kinds of backup. The **archive** (`tar`) is one compressed **file** per day, for example `data-2026-10-18.tar.gz`; `tar -tzf` only lists what is inside. The **mirror**, `/backups/mirror`, is a **real folder**: `rsync -avh --delete` fills it with independent copies of `/data/app`. It is not live – it is updated only when rsync runs, and then copies only what is new or changed, like a patch; with `--delete`, files removed from the original are also removed from the mirror, so it only knows "now".

For **history**, each rsync run creates a **new version folder** (v1, v2, v3, or one per date). It takes the files **from** `/data/app`, writes them **to** the new folder, and **compares with** `/backups/latest`, the previous version. Changed or new files are really copied; unchanged files become **hard links** to the previous version, so every version looks complete but only the changes use space. Old versions are never changed. `/backups/latest` is the only **shortcut** (soft link); after each run `ln -sfn` moves it to the newest version, so the next run always compares with the most recent one.

A backup only counts once it has been restored. Alice deleted `test.txt` by mistake and opened a ticket. Normal employees cannot touch `/backups` – IT handles it at the administrative level. As the technician I found that v3, taken after the deletion, no longer had the file, but v2 did. I restored it with `sudo cp /backups/v2/test.txt /data/app/`, checked it with `cat`, wrote the solution in the ticket and closed it. The drill proves that the backups hold real files, that I can pick the right version, and that a lost file comes back working.

🗒️ All the files that make changes survive a reboot in this project: persistence cheat sheet.

8. Next steps

1. **Break test (Step 2.2):** from gw01, `sudo mount -t nfs stor01:/data/app /mnt` – must be refused: only app01 is in `/etc/exports`. Being reachable is not being allowed.
2. **Step 2.4 – LVM snapshot:** an undo button before risky changes – and why a snapshot is **not** a backup.
3. Remaining Phase 2 tickets (LVM build, grow /data, fstab incident, NFS share), then VMware snapshots `30-storage`.

Project 2 – Acme Analytics Linux Platform

Day 5 · Phase 2 complete, Phase 3 started: snapshot rollback, job control, systemd timer and socket

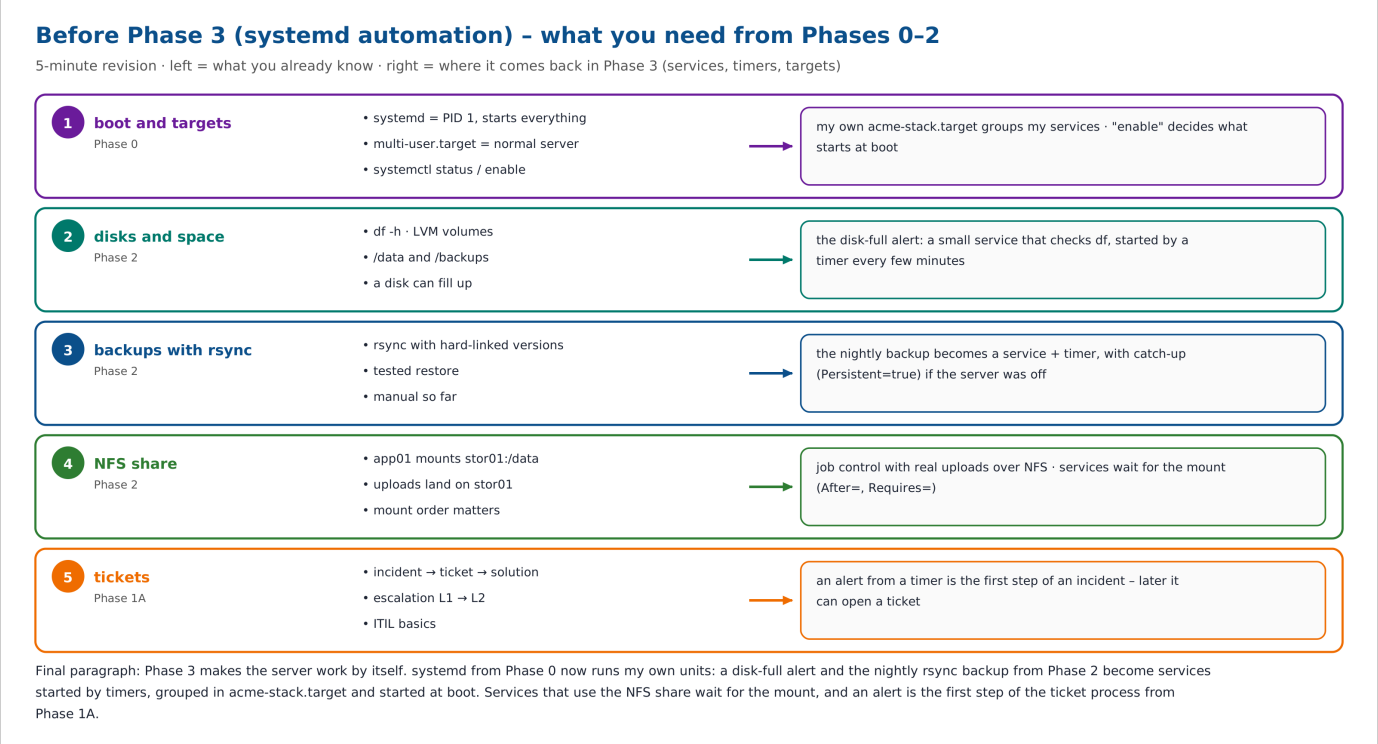
Muhammad Bilal Javed · 20 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	2 – Storage, sharing and recovery · 3 – Automation
Hosts worked on	gw01 (access test), stor01 (LVM snapshot, NFS server), app01 (NFS client)
Status	Phase 2 ✅ – Steps 2.1–2.4 done; VMware snapshots `30-storage-*` · Phase 3 🟡 – Step 3.1 job control ✅ · Step 3.2 disk-full alert as a systemd service + timer ✅ · Step 3.3 socket activation ✅
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 4.

Before Phase 3 – quick revision (Phases 0–2)

💡 **5 minutes:** what I already know from Phases 0–2 and where it comes back in Phase 3.



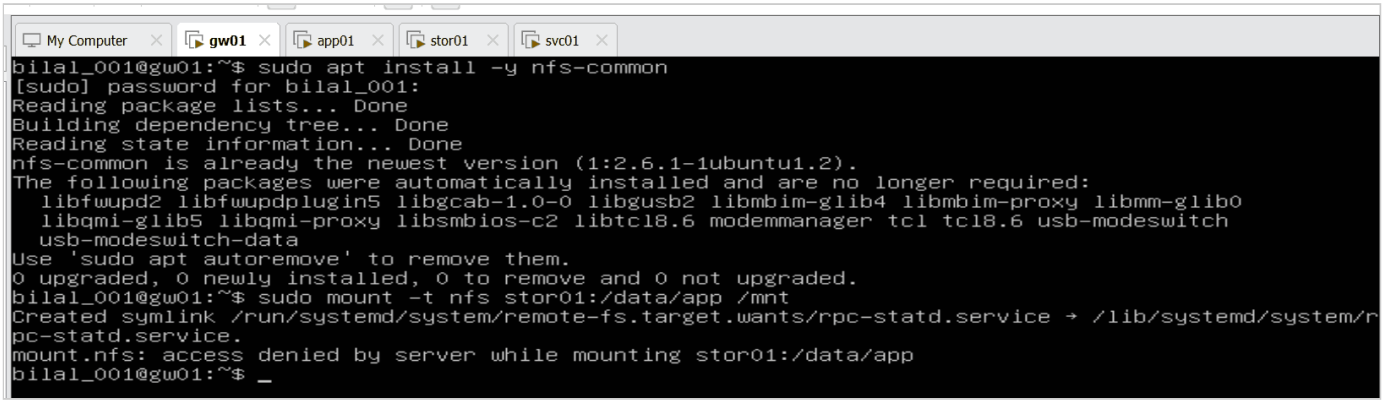
Before Phase 3 – quick revision

1. Break test – reachable is not allowed (Step 2.2)

💡 **In plain words:** gw01 has stor01 in `/etc/hosts` and can ping it – but it is not listed in stor01's `/etc/exports`. Can it use the share anyway?

```
# on gw01
sudo apt install -y nfs-common                  # NFS client tools (already installed)
sudo mount -t nfs stor01:/data/app /mnt         # /mnt = standard empty folder for temporary mounts
```

📸 **Evidence – `access denied by server`:**



access denied

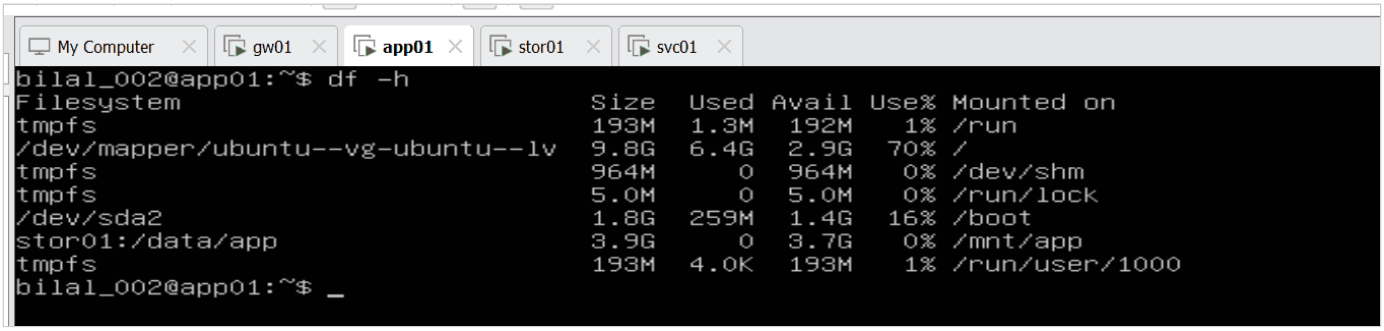
Lesson: being able to reach a server is not permission to use its shares. Only the IPs listed in `/etc/exports` (here only app01, 192.168.60.10) may mount `/data/app`.

2. Where the app's data really lives

```
app01: /mnt/app  ═ NFS ═▶  stor01: /data/app  ─inside─▶  /data  ─mounted from─▶  lvdata (logical volume)
                                                  /backups ─mounted from─▶  lvbackups
```

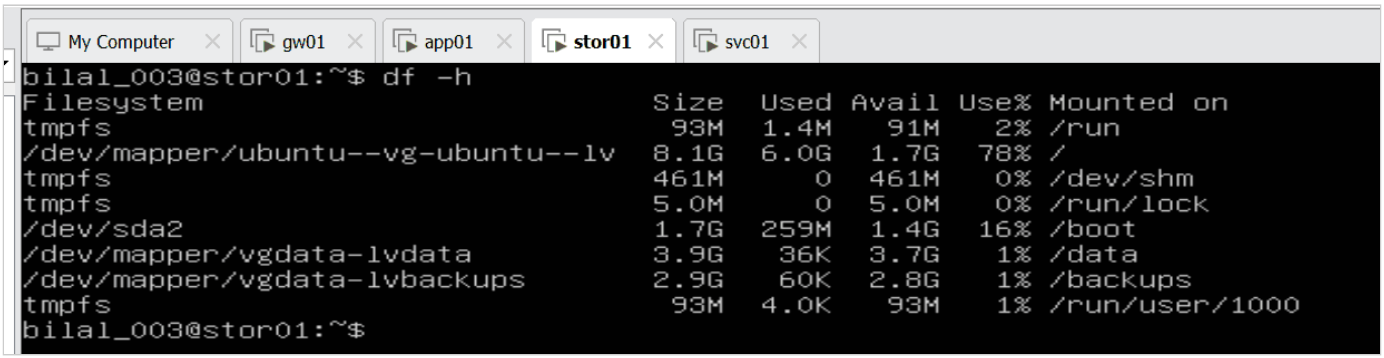
- `/mnt/app` is a folder on **app01**; `/data/app` is a folder on **stor01**. Over NFS, everything done in `/mnt/app` really happens in `/data/app`.
- On stor01 the logical volume **`lvdata`** is mounted on `/data` – so every file in `/data/app` is stored on `lvdata`. **The work happens on `lvdata`; the backups are on another volume, `lvbackups`.**

📸 **Evidence – app01: `stor01:/data/app` on `/mnt/app`:**



app01 df

📸 **Evidence – stor01: `lvdata` on `/data`, `lvbackups` on `/backups`:**



stor01 df

3. LVM snapshot – the undo button (Step 2.4)

💡 **The story:** a risky change is coming to `/data` (an update, a cleanup). A mistake made from app01 lands directly on `lvdata`, and the rsync mirror – which keeps only the current state – would copy it at its next run. The dated versions still hold good files, but restoring file by file is slow. So **before** the change: take a snapshot – a frozen state of the whole volume – and roll back in seconds if it goes wrong.

3.1 Take the snapshot, make the “mistakes”

```
# on app01 — release the share first (see the issue below)
sudo umount /mnt/app

# on stor01
ls /data/app                                          # lab.txt new.txt test.txt
sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata    # -s snapshot of lvdata; 1 GB room for changes
sudo lvs
echo "bad change" | sudo tee /data/app/oops.txt       # mistake 1
sudo rm /data/app/test.txt                            # mistake 2
ls /data/app                                          # lab.txt new.txt oops.txt
```

lvs shows	Meaning
`lvdata  owi-aos---`	**o** = origin: a snapshot is watching this volume
`snap1  swi-a-s---  1.00g  Origin lvdata`	**s** = snapshot of lvdata, 1 GB room for changes
`Data% 0.01`	How full the snapshot’s room is – at 100 % it becomes invalid

The mistakes were made on stor01 for simplicity; from app01 through `/mnt/app` the effect is identical – it is the same file on `lvdata`, and the snapshot protects the volume, whoever made the change.

📸 **Evidence:**

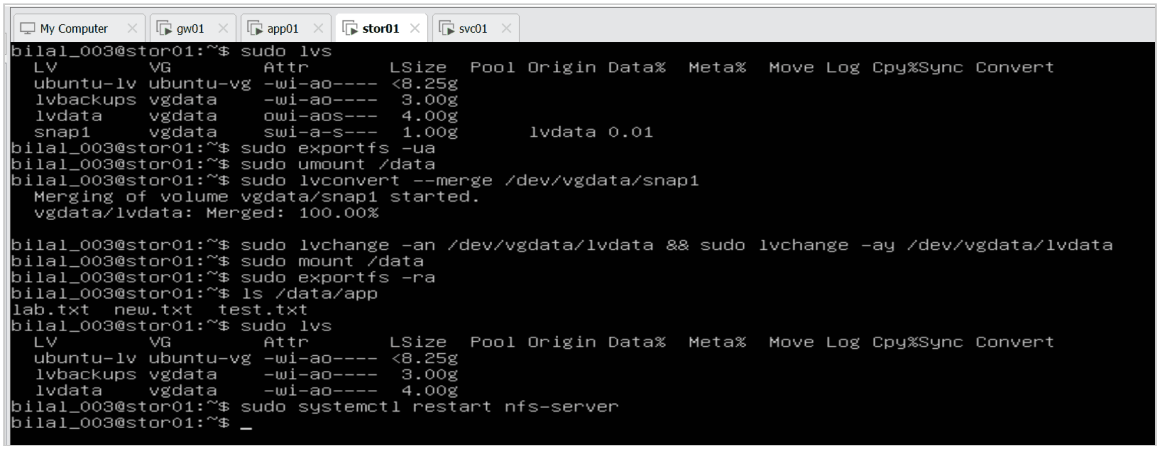
```
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  test.txt
bilal_003@stor01:~$ sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata
[sudo] password for bilal_003:
  Logical volume "snap1" created.
bilal_003@stor01:~$ sudo lvs
  LV        VG         Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  ubuntu-lv ubuntu-vg  -wi-ao---- <8.25g
  lvbackups vgdata     -wi-ao----  3.00g
  lvdata    vgdata     owi-aos---  4.00g
  snap1     vgdata     swi-a-s---  1.00g      lvdata 0.01
bilal_003@stor01:~$ echo "bad change" | sudo tee /data/app/oops.txt
bad change
bilal_003@stor01:~$ sudo rm /data/app/test.txt
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  oops.txt
bilal_003@stor01:~$ _
```

snapshot taken

3.2 Roll back

```
# on stor01
sudo exportfs -ua                                     # stop sharing /data/app
sudo umount /data                                     # nobody may use the volume during the merge
sudo lvconvert --merge /dev/vgdata/snap1              # back to the frozen state
sudo lvchange -an /dev/vgdata/lvdata && sudo lvchange -ay /dev/vgdata/lvdata   # deactivate / activate
sudo mount /data
sudo exportfs -ra                                     # share again
ls /data/app                                          # lab.txt new.txt test.txt — oops.txt gone
sudo lvs                                              # snap1 gone — merged into lvdata
```

📸 **Evidence – `Merged: 100.00%`; test.txt back, oops.txt gone; snap1 gone:**

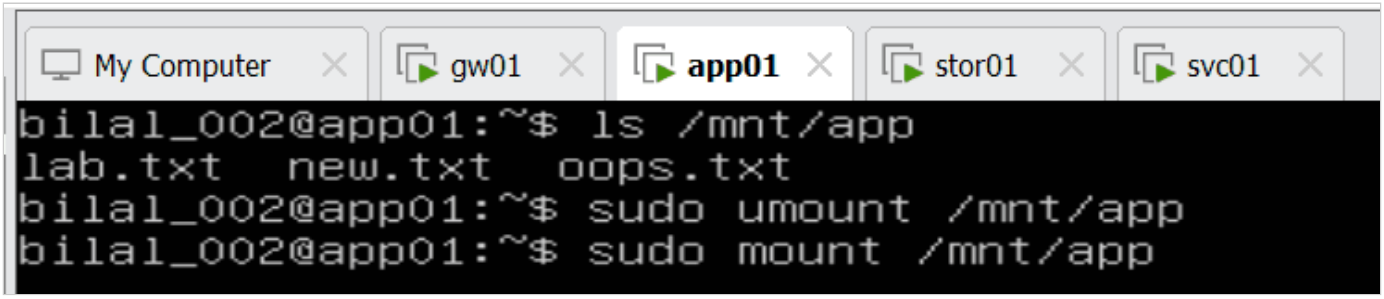


rollback

3.3 Issue – app01 hangs after the rollback (stale NFS mount)

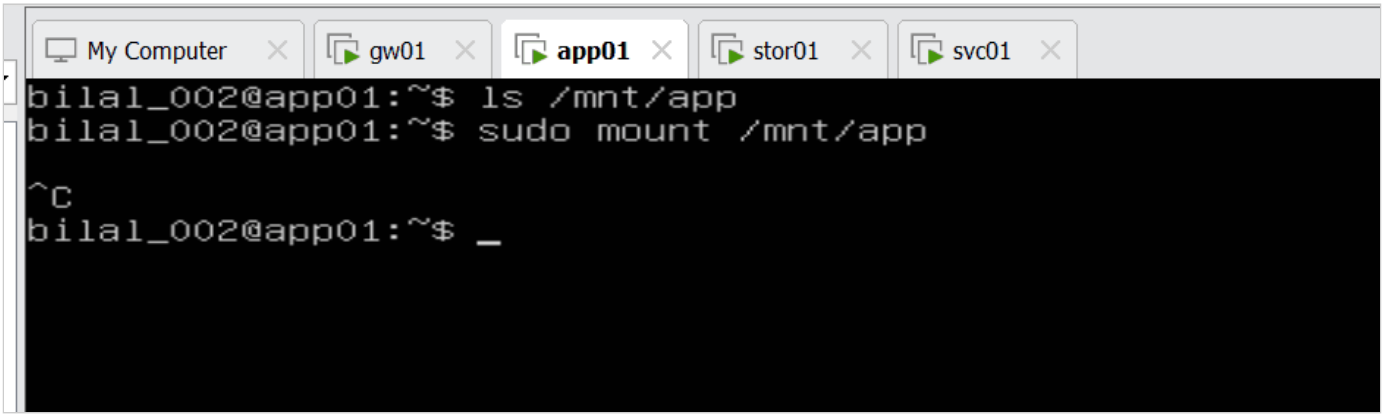
Seen	After the rollback, `mount /mnt/app` on app01 hung and had to be stopped with Ctrl+C.
Cause	app01 still had the share mounted **during** the rollback – `ls /mnt/app` still showed `oops.txt`; it was unmounted only afterwards. stor01 pulled the volume away under a connected client, which was left with an old, invalid connection (a **stale** mount).
Fix	stor01: `sudo systemctl restart nfs-server` · app01: `sudo umount -f -l /mnt/app` (force, lazy), `showmount -e stor01`, `sudo mount -a`
Lesson	Before rolling back a volume shared over NFS, **unmount every client first** – the client lets go, then the server.

📸 **Evidence – app01 still mounted during the rollback (oops.txt visible):**



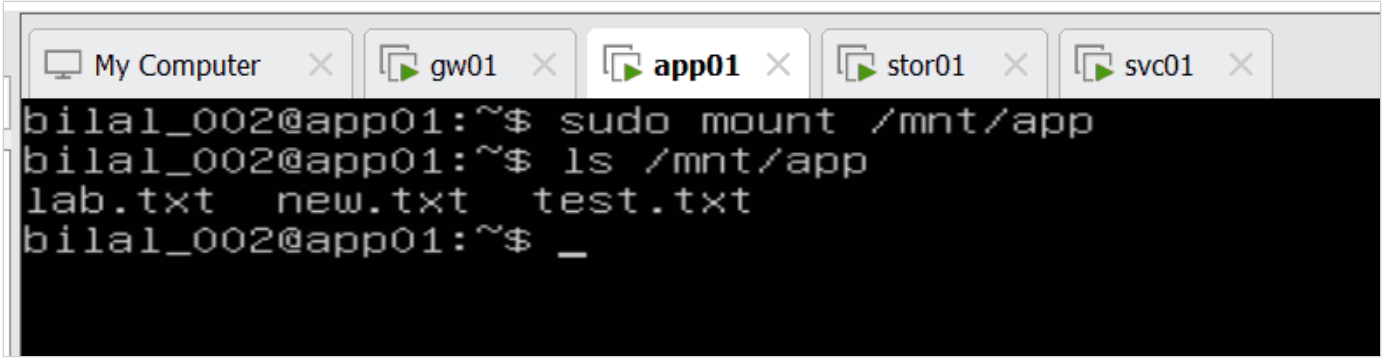
still mounted

📸 **Evidence – the mount hangs:**



hang

📸 **Evidence – after the fix, app01 sees the rolled-back files:**



fixed

4. Snapshot vs backup

	LVM snapshot	Backup (versions in `/backups`)
What	A **frozen state of the whole volume** at the moment it was taken	**Separate copies** of the files
How	Keeps only the old version of blocks that change afterwards (copy-on-write), in its small room	Full files (unchanged ones hard-linked)
Where	**Same disks** as `lvdata`	Separate volume `lvbackups` – ideally another server
Restore	The **whole volume at once**, in seconds	Files **one by one**, from any day
Survives a reboot	Yes	Yes
Lost when	Disks die, its room fills to 100 %, merged or removed	Deleted, or the disks die – hence copies off-site

Snapshot = quick undo, short-term, same disks. Backup = recovery, long-term, separate volume.

5. VMware snapshots

VM	Snapshot	Description
gw01	`30-storage-gw01`	End of Phase 2; NFS client tools for the access test
app01	`30-storage-app01`	NFS client: stor01:/data/app on /mnt/app (fstab, `_netdev,nofail`)
stor01	`30-storage-stor01`	LVM vgdata (lvdata /data 4 GB, lvbackups / backups 3 GB), NFS server, tar/rsync backups v1–v3
svc01	`30-storage-svc01`	GLPI; restore ticket closed

Key understanding (in my own words)

> On app01, `/mnt/app` is a folder. On stor01, `/data/app` is a folder. They are connected over NFS, so everything done in `/mnt/app` on app01 really happens in `/data/app` on stor01. On stor01, the logical volume `lvdata` is mounted on `/data`, so every file in `/data`, including `/data/app`, is stored on `lvdata`. The work happens on `lvdata`; the backups are stored on another logical volume, `lvbackups`, mounted on `/backups`.
>
> If someone on app01 makes a wrong or risky change in `/mnt/app`, the mistake lands directly on `lvdata`. The rsync mirror only keeps the current state, so at its next run it would copy the mistake too. A backup is a set of separate copies of the files on `lvbackups`: from it I can restore files one by one, from any day, but that is slow for a big mistake.
>
> So before a risky change I take an **LVM snapshot**: a frozen state of the whole `lvdata` volume, taken at the moment I create it. If the change goes wrong, I roll back the whole volume at once. To roll back, nobody may be using the volume: first app01 unmounts `/mnt/app`, then stor01 stops sharing (`exportfs -ua`) and unmounts `/data`. I learned this the hard way – app01 was still mounted during my rollback, was left with a stale mount and hung, until I restarted the NFS server and force-unmounted the client.
>
> Both survive a reboot, but a snapshot lives on the same disks and becomes invalid if its room fills up: **snapshot = quick undo, short-term, same disks; backup = recovery, long-term, separate volume.** And gw01 showed that being able to reach a server is not permission to use its shares.

6. Phase 3 starts – automation (Step 3.1: job control)

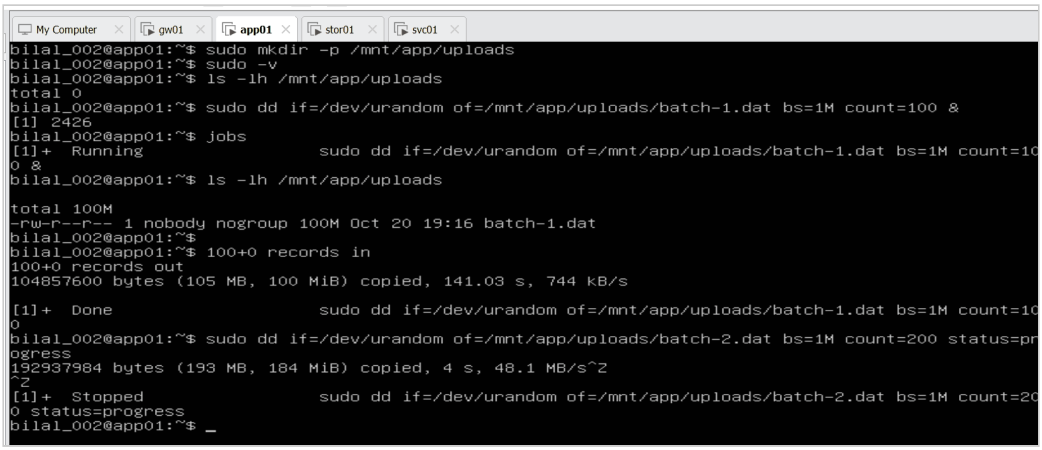
💡 **The story:** so far every command ran by hand, and long commands blocked the terminal. Acme's customers upload large files from **app01**; through NFS they land in `/data/app/uploads` on **stor01**, where they must be backed up. Long jobs like these should **not block my terminal**, **not slow down the app**, and **not die when my connection drops**.

```
app01: /mnt/app/uploads  ═ NFS ═▶  stor01: /data/app/uploads (lvdata)  ─backup─▶  /backups (lvbackups)
       "customers upload"                                                  nohup tar from an SSH session
```

6.1 Customer uploads on app01 – foreground, background, pause, priority, kill

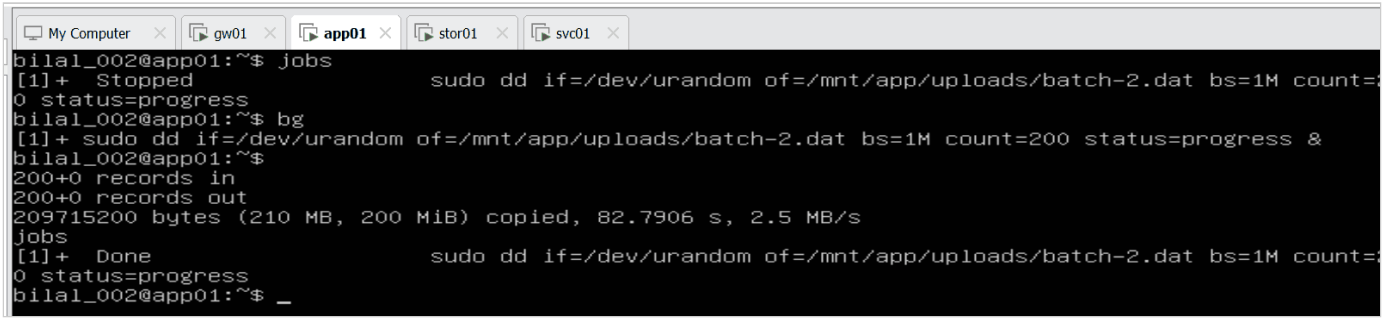
```
        # app01
        sudo rm -rf /mnt/app/uploads && sudo mkdir -p /mnt/app/uploads   # clean start (really /data/app/uploads on
stor01)
        sudo -v                                                          # cache the password — background sudo jobs
cannot ask for it

        # batch-1: in the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100 &
        jobs                                                             # [1]+ Running — the prompt is free
        ls -lh /mnt/app/uploads                                          # keep working while it uploads

        # batch-2: forgot the & — pause and send to the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-2.dat bs=1M count=200 status=progress
        #   Ctrl+Z → [1]+ Stopped
        jobs; bg; jobs                                                   # Stopped → Running → Done
```

📸 **Evidence – batch-1 in the background (`Running` → `Done`), batch-2 paused with Ctrl+Z (`Stopped`):**



bg and ctrl+z

📸 **Evidence – `bg`: it continues in the background and finishes:**



bg done

Issue – Ctrl+Z and Ctrl+C seemed to do nothing

Seen	During an upload to `/mnt/app`, `^Z ^C ^Z …` appeared on the screen but the job did not react – until, after a while, `[1]+ Stopped`.
Cause	dd was writing to a **network disk**. While a process waits for NFS to confirm a write it is in state **D – uninterruptible sleep** and cannot react to signals until that write finishes.
Check	`ps -o pid,stat,cmd -C dd` – STAT `D` = waiting on the disk
Lesson	A slow or hanging NFS server can freeze programs on the client – the same root as yesterday's stale mount. dd's speed (575 kB/s) also counts the paused minutes; the real speed was about 9 MB/s.

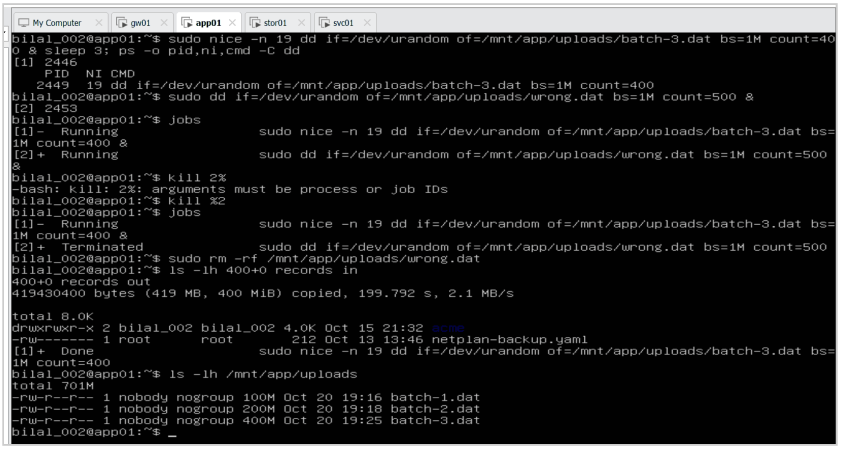
📸 **Evidence:**

```
bilal_002@app01:~$ sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100
^Z


^C



^C^Z^Z^Z^Z^Z^Z



[1]+  Stopped                 sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=10
0
bilal_002@app01:~$ jobs
[1]+  Stopped                 sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=10
0
bilal_002@app01:~$ _
```

ctrl+z waits

```
        # batch-3: low priority — start dd and check its nice value in the same line
        sudo nice -n 19 dd if=/dev/urandom of=/mnt/app/uploads/batch-3.dat bs=1M count=400 & sleep 3; ps -o pid,ni,cmd -
C dd

        # wrong.dat: started by mistake — stop only that job
        sudo dd if=/dev/urandom of=/mnt/app/uploads/wrong.dat bs=1M count=500 &
        jobs                                # [1] batch-3 Running, [2] wrong.dat Running
        kill %2                             # [2] Terminated — batch-3 keeps running
        sudo rm -f /mnt/app/uploads/wrong.dat
        ls -lh /mnt/app/uploads             # batch-1 100M, batch-2 200M, batch-3 400M
```

📸 **Evidence – `NI 19`; two jobs at once, only `%2` killed; the three batches, owned by `nobody` (root_squash):**



nice kill result

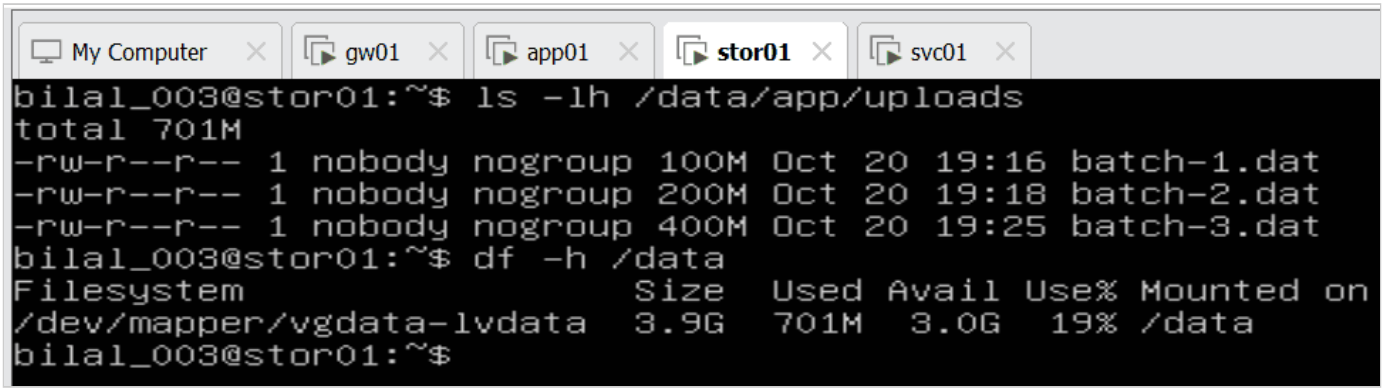
Tool	Meaning	Real use
&	Start in the background	A long upload or backup without blocking the terminal
Ctrl+Z → bg / fg	Pause, continue in background / foreground	Rescue a job started without &
jobs	List my jobs with numbers	
kill %2 / kill PID	Stop by job number (same terminal) / by process ID (anywhere)	A job started by mistake
nice -n 19 / renice -n 19 -p PID	Priority -20 (most important) … 19 (politest), at start / while running	A daytime backup that does not slow the app
nohup … &	Keep running after logout	A long job over an SSH connection that may drop

&/fg/bg decide **where** a job runs (does it block the terminal); nice/renice decide **how important** it is when the CPU is busy – all jobs still run at the same time. For disk-heavy jobs `ionice -c3` is the disk equivalent.

6.2 On stor01 – the uploads arrived; a backup that survives a logout

```
        # stor01
        ls -lh /data/app/uploads          # the three batches, 701 MB
        df -h /data                       # 19 % used
```

📸 **Evidence:**

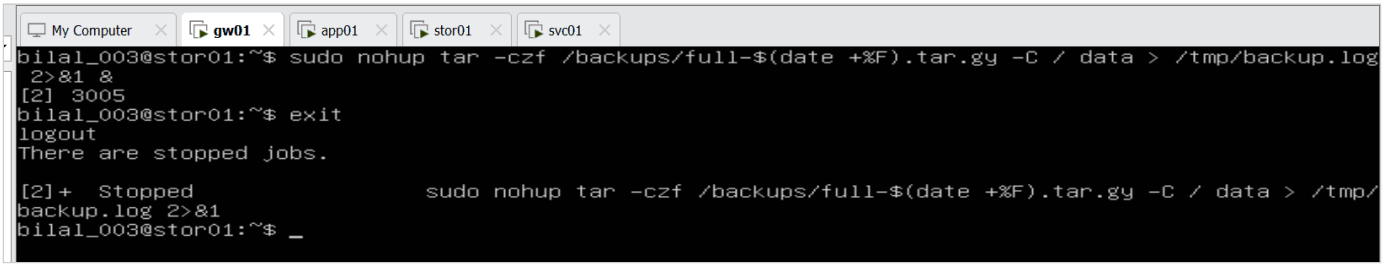


uploads arrived

Issue – the background backup went “Stopped”

Seen	The second `sudo nohup tar … &` showed `[2]+ Stopped`, and `exit` answered *There are stopped jobs* instead of logging out.
Cause	sudo wanted a password. A background job **cannot read from the keyboard**, so Linux paused it. Bash refuses the first `exit` while a job is stopped.
Lesson	Run `sudo -v` in the foreground **before** a background sudo job, and check `jobs` before `exit`. (The archive name also had a typo, `.tar.gy` – removed.)

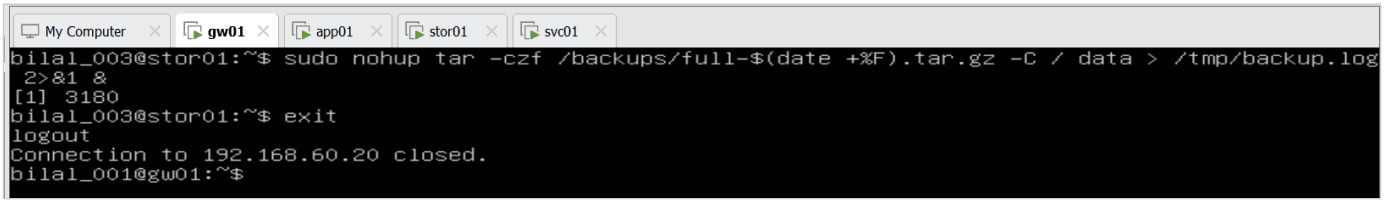
📸 **Evidence:**



stopped

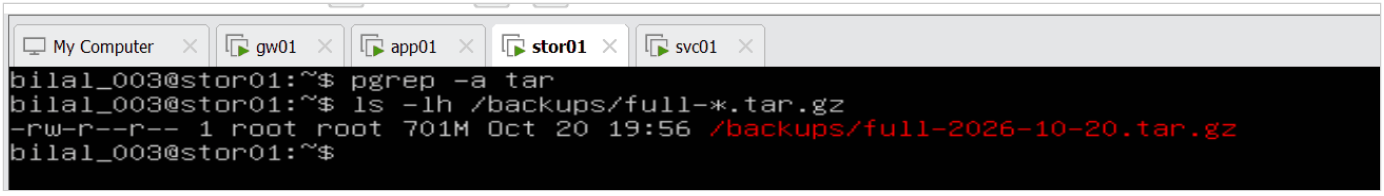
```
# gw01 — plays "my laptop"
ssh stor01
sudo -v
sudo nohup tar -czf /backups/full-$(date +%F).tar.gz -C / data > /tmp/backup.log 2>&1 &
exit                                    # really logged out: "Connection to 192.168.60.20 closed."
ssh stor01
pgrep -a tar                            # nothing — finished
ls -lh /backups/full-*.tar.gz           # 701M, finished at 19:56 — after I had left
sudo rm -f /backups/full-*.tar.gz       # free the space again
```

📸 **Evidence – started, logged out:**



nohup logout

📸 **Evidence – the archive was finished while I was gone:**



finished after logout

The archive is as big as the data (701 MB): random files – like real photos, videos and zipped uploads – hardly compress. Text and logs shrink a lot. That matters when planning backup space.

Key understanding (in my own words)

Acme's customers upload files from app01 through `/mnt/app`, which NFS stores in `/data/app/uploads` on stor01. Long jobs like these uploads and backups should not block my terminal or slow down the app.

Foreground and background: any command can run in the foreground or the background. In the foreground, the terminal is busy until the command finishes. With `&` it runs in the background, and I get the prompt back at once. If I forget the `&`, `Ctrl+Z` pauses the job and `bg` lets it continue in the background; `fg` brings it back. `jobs` lists my jobs with their numbers. A wrong job is stopped with `kill %number` (job number, same terminal) or `kill PID` (process ID, from anywhere). On a network disk (NFS), a job waiting for a write reacts to Ctrl+Z or kill only after that write finishes.

Priority: `nice` and `renice` set a job's priority from -20 (most important) to 19 (politest). All jobs run at the same time, but when the CPU is busy, lower numbers get more CPU time. `nice` sets it at the start; `renice` changes it while the job runs. That is how a backup can run during the day without slowing down the app.

Surviving a logout: a job started from an SSH session is normally killed when the connection closes. With `nohup … &` it keeps running on the server after I log out. I started a 700 MB backup on stor01 from gw01, logged out, came back, and the archive had been finished while I was gone. Two lessons: run `sudo -v` first, because a background job cannot answer a password prompt (it becomes "Stopped"); and check `jobs` before `exit`, because Bash will not log out while a job is stopped. The archive was as big as the data (701 MB), because random files like customer uploads hardly compress.

7. Step 3.2 – disk-full alert: a systemd service and timer

💡 **The story:** files uploaded on app01 – today by me as a simulation, later by customers once the app is published through gw01 (Phase 7) – land in `/mnt/app`, which is really `/data` on stor01 (`lvdata`). If `/data` fills up, uploads fail and the app breaks, often before anyone in IT notices. In Phase 2 I can grow `/data` with LVM – but only if someone knows it is filling up. So, like a phone warning "storage almost full", stor01 must **watch its own disk and warn by itself** – no human, no manual command.

```
disk-alert.timer ──every 5 min──► disk-alert.service ──runs──► disk-alert.sh ──if too full──► journal (system
log)
     WHEN                              HOW                          WHAT
```

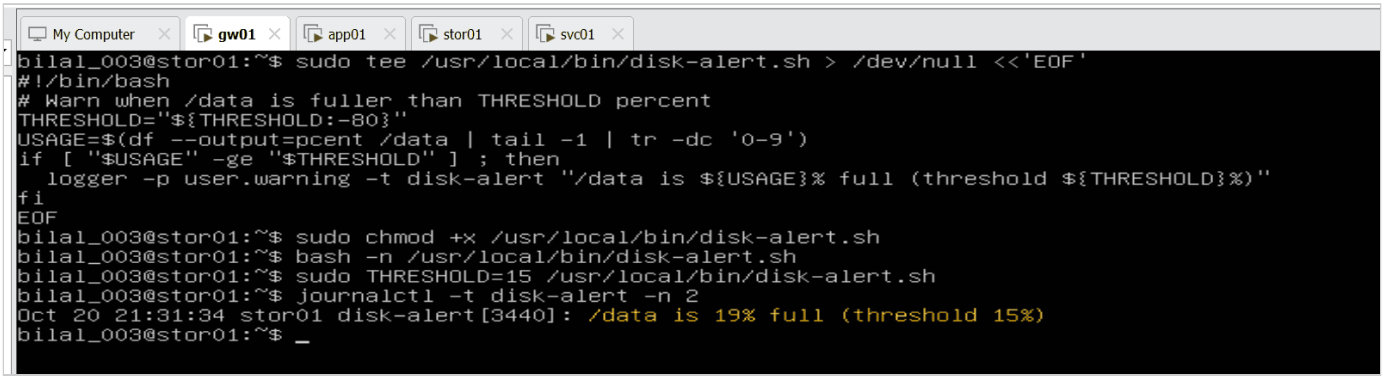
All on **stor01** – the disk to watch lives there.

7.1 The script – WHAT to check

```
sudo tee /usr/local/bin/disk-alert.sh > /dev/null <<'EOF'
#!/bin/bash
# Warn when /data is fuller than THRESHOLD percent
THRESHOLD="${THRESHOLD:-80}"
USAGE=$(df --output=pcent /data | tail -1 | tr -dc '0-9')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
  logger -p user.warning -t disk-alert "/data is ${USAGE}% full (threshold ${THRESHOLD}%)"
fi
EOF
sudo chmod +x /usr/local/bin/disk-alert.sh
bash -n /usr/local/bin/disk-alert.sh              # syntax check without running – no output = OK
sudo THRESHOLD=15 /usr/local/bin/disk-alert.sh    # one manual run with a low test limit
journalctl -t disk-alert -n 2
```

Line	Meaning
`#!/bin/bash`	Run this file with Bash
`THRESHOLD="${THRESHOLD:-80}"`	Use the given limit – or 80 % if none is given
`df --output=pcent /data | tail -1 | tr -dc '0-9'`	How full `/data` is, as a plain number: `tr -dc '0-9'` deletes everything except digits, so 19% → 19
`if [ "$USAGE" -ge "$THRESHOLD" ]`	If usage is **g**reater or **e**qual to the limit … (`[` is a command – spaces inside `[ ]` are required)
`logger -p user.warning -t disk-alert "…"`	… write a **warning** to the system log, tagged `disk-alert`

📸 **Evidence – syntax OK; manual run: `/data is 19% full (threshold 15%)`:**



script manual run

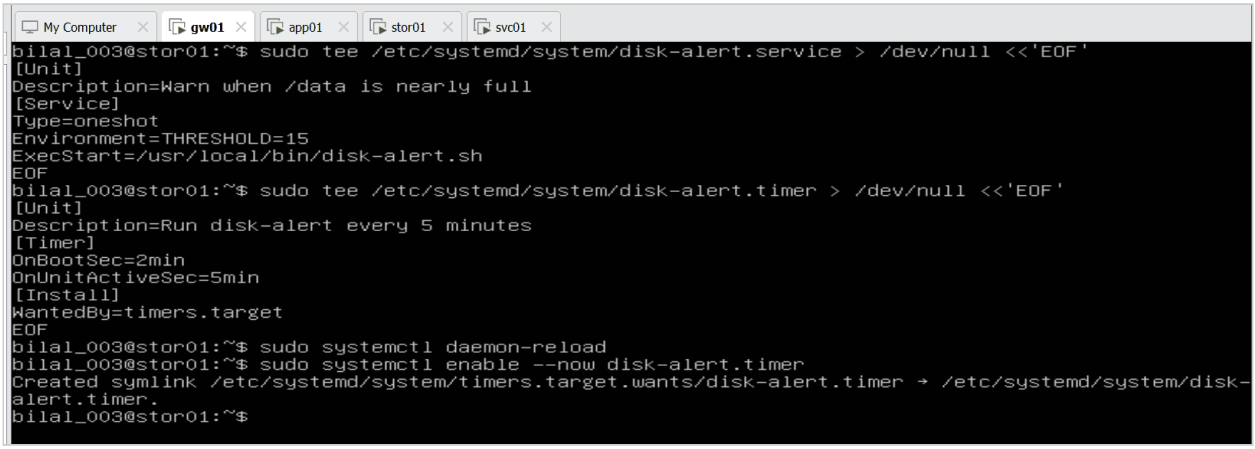
Lesson: my first version had two typing mistakes (a) inside the quotes, no spaces inside `[ ]`). `bash -n` checks a script **before** it runs – use it every time.

7.2 The service (HOW) and the timer (WHEN)

```
sudo tee /etc/systemd/system/disk-alert.service > /dev/null <<'EOF'
[Unit]
Description=Warn when /data is nearly full
[Service]
Type=oneshot
Environment=THRESHOLD=15
ExecStart=/usr/local/bin/disk-alert.sh
EOF

sudo tee /etc/systemd/system/disk-alert.timer > /dev/null <<'EOF'
[Unit]
Description=Run disk-alert every 5 minutes
[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload                    # systemd reads the new files
sudo systemctl enable --now disk-alert.timer    # start now + at every boot
systemctl list-timers | grep disk-alert
systemctl status disk-alert.service --no-pager
```

Setting	Meaning
`Type=oneshot`	Run once and finish – the service does not stay running
`Environment=THRESHOLD=15`	Test limit – my uploads put `/data` at 19 %, so the alert fires now
`OnBootSec=2min` · `OnUnitActiveSec=5min`	First run 2 min after boot, then every 5 min
`WantedBy=timers.target` + `enable`	Start the timer automatically at every boot

📸 **Evidence – units created, timer enabled:**



service timer

📸 **Evidence – `list-timers`: NEXT in 4 min, LAST 48 s ago; `status`:** TriggeredBy: disk-alert.timer**, `status=0/SUCCESS`, then `inactive (dead)` until the next run:**

```
bilal_003@stor01:~$ systemctl list-timers | grep disk-alert
Tue 2026-10-20 22:24:36 UTC 4min 11s left Tue 2026-10-20 22:19:36 UTC 48s ago       disk-alert.timer
                disk-alert.service
bilal_003@stor01:~$ systemctl status disk-alert.service --no-pager
● disk-alert.service - Warn when /data is nearly full
     Loaded: loaded (/etc/systemd/system/disk-alert.service; static)
     Active: inactive (dead) since Tue 2026-10-20 22:19:36 UTC; 1min 30s ago
TriggeredBy: ● disk-alert.timer
    Process: 3571 ExecStart=/usr/local/bin/disk-alert.sh (code=exited, status=0/SUCCESS)
   Main PID: 3571 (code=exited, status=0/SUCCESS)
        CPU: 85ms

Oct 20 22:19:36 stor01 systemd[1]: Starting Warn when /data is nearly full...
Oct 20 22:19:36 stor01 disk-alert[3576]: /data is 19% full (threshold 15%)
Oct 20 22:19:36 stor01 systemd[1]: disk-alert.service: Deactivated successfully.
Oct 20 22:19:36 stor01 systemd[1]: Finished Warn when /data is nearly full.
bilal_003@stor01:~$ _
```

triggered by timer

7.3 The real limit – silent when fine, loud when full

```
sudo sed -i 's/THRESHOLD=15/THRESHOLD=80/' /etc/systemd/system/disk-alert.service
sudo systemctl daemon-reload                        # always after changing a unit file
sudo systemctl start disk-alert.service             # 19 % < 80 % → no new line: no news is good news
journalctl -t disk-alert -n 3                       # 22:51, 22:56, 23:07 — runs by the timer, by themselves
sudo fallocate -l 2500M /data/app/uploads/huge-upload.dat    # simulate a huge upload — reserves 2.5 GB instantly
df -h /data                                         # 86 %
sudo systemctl start disk-alert.service
journalctl -t disk-alert -n 3                       # "/data is 86% full (threshold 80%)"
sudo rm /data/app/uploads/huge-upload.dat
df -h /data                                         # back to 19 %
```

📸 **Evidence – limit 80 %, silent at 19 %; the timer's own runs; the "huge upload" takes `/data` to 86 %:**

My Computer | **gw01** | app01 | stor01 | svc01

```
bilal_003@stor01:~$ journalctl -t disk-alert -n 5
Oct 20 21:31:34 stor01 disk-alert[3440]: /data is 19% full (threshold 15%)
Oct 20 22:19:36 stor01 disk-alert[3576]: /data is 19% full (threshold 15%)
bilal_003@stor01:~$ sudo sed -i 's/THRESHOLD=15/THRESHOLD=80/' /etc/systemd/system/disk-alert.servic
e
[sudo] password for bilal_003:
bilal_003@stor01:~$ grep THRESHOLD /etc/systemd/system/disk-alert.service
Environment=THRESHOLD=80
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl start disk-alert.service
bilal_003@stor01:~$ journalctl -t disk-alert -n 5
Oct 20 22:51:24 stor01 disk-alert[3652]: /data is 19% full (threshold 15%)
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
bilal_003@stor01:~$ sudo fallocate -l 2500M /data/app/uploads/huge-upload.dat
bilal_003@stor01:~$ df -h /data
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvdata  3.9G  3.2G  546M  86% /data
bilal_003@stor01:~$
```

silent then 86

📸 **Evidence – the alert fires: `/data is 86% full (threshold 80%)`; after clean-up 19 %:**

My Computer | **gw01** | app01 | stor01 | svc01

```
bilal_003@stor01:~$ sudo systemctl start disk-alert.service
bilal_003@stor01:~$ journalctl -t disk-alert -n 3
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
bilal_003@stor01:~$ journalctl -t disk-alert -n 4
Oct 20 22:51:24 stor01 disk-alert[3652]: /data is 19% full (threshold 15%)
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
bilal_003@stor01:~$ journalctl -t disk-alert -n 5
Oct 20 22:46:06 stor01 disk-alert[3637]: /data is 19% full (threshold 15%)
Oct 20 22:51:24 stor01 disk-alert[3652]: /data is 19% full (threshold 15%)
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
bilal_003@stor01:~$ df -h /data
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvdata  3.9G  701M  3.0G  19% /data
bilal_003@stor01:~$
```

alert 86

Time (UTC)	Journal	Meaning
21:31	19 % (threshold 15 %)	Manual run of the script
22:19 → 23:07	19 % (threshold 15 %)	**The timer**, every ~5 min, by itself (gaps = VM paused)
~23:08	–	Limit 80 %: the disk is fine, the script stays silent
23:13	**86 % (threshold 80 %)**	The simulated huge upload crossed the real limit

Key understanding (in my own words)

Files uploaded on app01 (today by me as a simulation, later by customers once the app is published through gw01) go into `/mnt/app`, which is really `/data` on stor01, the `lvdata` volume. If `/data` fills up, new uploads fail and the app breaks, often before anyone in IT notices. In Phase 2 I could grow `/data` with LVM, but only if someone knows it is filling up. So, like a phone that warns "storage almost full", stor01 should **watch its own disk and warn us by itself**, without a human running a command. The script `/usr/local/bin/disk-alert.sh` reads how full `/data` is with `df`, keeps only the number with `tr -dc '0-9'` (so `19%` becomes `19`), and compares it with a threshold: 80 % normally, 15 % for my test. If the usage is greater than or equal to the threshold, `logger` writes a **warning to the system log**, tagged `disk-alert`, which I read with `journalctl -t disk-alert`. I checked the script with `bash -n` and ran it once by hand.

Then I turned it into automation: `disk-alert.service` says *how* to run it (`Type=oneshot` – run once and finish), and `disk-alert.timer` says *when* – 2 minutes after boot and then every 5 minutes. `systemctl enable --now` started the timer and made it survive reboots; `status` showed *TriggeredBy: disk-alert.timer* and *SUCCESS*. The timer wakes the service, the service runs the script, and the script writes a warning only if `/data` is too full. With the test limit, warnings appeared every few minutes by themselves. With the real limit of 80 % (`sed` + `daemon-reload`) the script stayed silent at 19 %; a simulated 2.5 GB upload (`fallocate`) pushed `/data` to 86 %, and the alert *"/data is 86% full (threshold 80%)"* appeared. stor01 now watches its own disk, and the warning comes early enough to grow the volume or clean up.

8. Step 3.3 – socket activation: start a service only when someone connects

💡 **The story:** a rarely used service does not need to run all day and use memory. Instead, **systemd itself listens on the port** and starts the program only when a client connects – then it ends again. Real systems use this: `ssh.socket` (newer Ubuntu), `docker.socket`, `cups.socket` (printing).

What a unit is. Everything systemd manages is a **unit** – a small text file in `/etc/systemd/system/`. The ending says what kind:

Unit type	Says	In this project
`.service`	**how** to run a program	`disk-alert.service`, `hello@.service`
`.timer`	**when** to start a service	`disk-alert.timer`
`.socket`	**which port** to listen on – start the service on connect	`hello.socket`
`.target`	a **group** of units	`multi-user.target` (Day 1), `acme-stack.target` (next)

```
client connects to :9999 ──► hello.socket (systemd listens)
                                │ same name "hello" → template hello@.service
                                ▼
                     one copy starts → echo → output back to the client → ends
```

All on **stor01**.

```
sudo tee /etc/systemd/system/hello.socket > /dev/null <<'EOF'
[Socket]
ListenStream=9999
Accept=yes
[Install]
WantedBy=sockets.target
EOF
sudo tee /etc/systemd/system/hello@.service > /dev/null <<'EOF'
[Service]
ExecStart=/bin/sh -c 'echo "hello from stor01 — started on demand"'
StandardOutput=socket
EOF
sudo systemd-analyze verify /etc/systemd/system/hello@.service   # check the unit before testing
sudo systemctl daemon-reload
sudo systemctl enable --now hello.socket
ss -tlnp | grep 9999                                             # LISTEN *:9999 — systemd holds the port
nc localhost 9999                                                # → hello from stor01 — started on demand
```

Setting	Meaning
`ListenStream=9999`	Listen on TCP port 9999
`Accept=yes`	Each connection gets its own short-lived copy of the template `hello@.service`
`@` in `hello@.service`	**Template** – one instance per connection
`StandardOutput=socket`	The program's output goes back through the connection to the client

📸 **Evidence – the socket unit:**

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/hello.socket > /dev/null <<'EOF'
[Socket]
ListenStream=9999
Accept=yes
[Install]
WantedBy=sockets.target
EOF
```

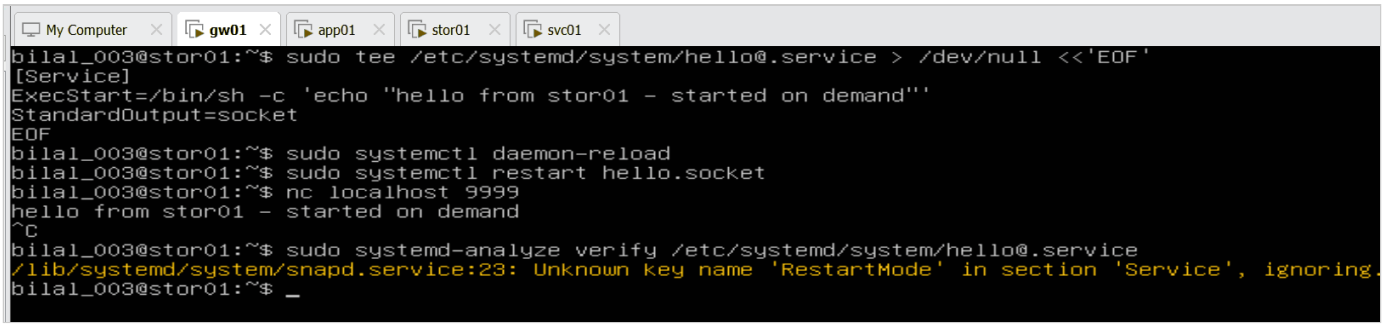
socket unit

📸 **Evidence – enabled; `LISTEN *:9999`:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now hello.socket
Created symlink /etc/systemd/system/sockets.target.wants/hello.socket → /etc/systemd/system/hello.so
cket.
bilal_003@stor01:~$ ss -tlpn | grep 9999
LISTEN 0      4096               *:9999             *:*
bilal_003@stor01:~$ nc localhost 9999
```

listening

📸 **Evidence – the greeting on connect; `systemd-analyze verify` shows no error for my file (the yellow line is about Ubuntu's own `snapd.service`):**



on demand

Lessons from my mistakes: the service was first named `hello.service` (with `Accept=yes` it must be the template `hello@.service`), the section was misspelled (`[Servuce]`), and `ExecStart` lacked the leading `/` – each time `nc` just waited. `ExecStart` needs a full path, `cat` the unit once, and `systemd-analyze verify` catches such mistakes before testing.

Key understanding (in my own words)

> In systemd, everything it manages is a **unit**: a small text file in `/etc/systemd/system/` that describes one thing, and the ending of the name says what kind. A **`.service`** says *how* to run a program, a **`.timer`** says *when* to start a service, a **`.socket`** says *which network port* to listen on, and a **`.target`** is a *group* of units, like `multi-user.target` (server mode). After creating or changing a unit, `systemctl daemon-reload` makes systemd read it; `enable` starts it at every boot and `--now` also starts it immediately. In Step 3.2 I used a service plus a timer for the disk alert. In Step 3.3 I used a **socket**: a rarely used service does not need to run all day, so systemd itself listens on the port (`hello.socket`, port 9999) and starts the program **only when someone connects**. The socket and the service are linked by their name (`hello`). With `Accept=yes`, each connection gets its own short-lived copy of the template `hello@.service` (the `@` means template), and `StandardOutput=socket` sends the program's output straight back through the connection to the client. `nc localhost 9999` returned *"hello from stor01 – started on demand"*; before and after, nothing was running. Real systems use the same idea, for example `ssh.socket`, `docker.socket` and `cups.socket`. My mistakes taught me three checks: the template needs the `@`, `ExecStart` needs a full path starting with `/`, and `systemd-analyze verify` checks a unit file before testing – like `bash -n` for scripts.

9. Next steps

Continued in Day 6.

1. **A real-world socket-activation example** (instead of a greeting) – then Step 3.4: the `acme-stack.target` grouping the Acme units.
2. **Step 3.5:** cron vs systemd timer, and the **nightly backup** of `/data/app` running by itself at 02:00 (`Persistent=true` catches up after downtime).
3. Phase 2 tickets in GLPI: LVM build, grow /data, fstab UUID typo, NFS share, stale NFS mount after rollback.

Project 2 – Acme Analytics Linux Platform

Day 6 · Phase 3 complete: custom target, cron vs timer, nightly backup by timer

Muhammad Bilal Javed · 21 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	3 – Automation
Hosts worked on	stor01
Status	Phase 3 ✅ – Steps 3.1–3.3 (Day 5) · Step 3.4 custom target ✅ · Step 3.5 cron vs timer and nightly backup by timer ✅
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 5.

---

1. Revision – what a systemd unit really is

💡 **In plain words:** before going on, I explained Steps 3.2 and 3.3 again in my own words and corrected what I had misunderstood.

I thought	Correct
A unit is a script	A unit is a **settings file**; it **points** to the program or script with `ExecStart=`
Everything in Linux is a unit	Only what **systemd manages**: services, timers, sockets, targets, mounts … – `/etc/hosts` is not a unit
All units are in `/etc/systemd/system/`	Linux's own units: `/lib/systemd/system/` · **my** units: `/etc/systemd/system/`
The `.timer` file must name its service	The link is the **same first name**: `disk-alert.timer` → `disk-alert.service`, `hello.socket` → `hello@.service`
`WantedBy=timers.target` links timer and service	It only makes the timer start **at every boot** (after `enable`)
`multi-user.target` = things related to users	The normal **server state**: networking and services, text login, no desktop; `graphical.target` = that + desktop
`nc` = network card	**netcat** – a small client tool that connects to a port

```
TIMER:   disk-alert.timer ──same name──► disk-alert.service ──ExecStart──► disk-alert.sh ──logger──► journal
           WHEN                              HOW                                WHAT
SOCKET:  client connects :9999 ──► hello.socket ──same name──► one copy of hello@.service ──► answer to client
──► copy ends
```

`Accept=yes` vs `Accept=no`: like a **call centre** – every caller gets their own new agent (one copy of `hello@.service` per connection) – versus a **receptionist** – the first caller wakes up **one** manager (`ssh.service`) who then handles everyone.

Key understanding (in my own words)

systemd is the program that starts and manages services on a Linux server. Everything it manages (SSH, Docker, my disk alert) is a **unit**: a small settings file that says how something should run. Linux's own units are in `/lib/systemd/system/` (for example `ssh.service` and `ssh.socket`), and the units I create for automation go in `/etc/systemd/system/`. The ending of the name says the type: **`.service`** = **how** to run a program, **`.timer`** = **when** to run it, **`.socket`** = **which port** to listen on, and **`.target`** = a **group** of units that start together. After creating or changing a unit I run `systemctl daemon-reload`, and `enable --now` starts it now and at every boot.

Timer (Step 3.2): the script `/usr/local/bin/disk-alert.sh` checks how full `/data` is and writes a warning to the journal only if usage reaches the threshold (80 %). `disk-alert.service` says **how** to run it (`Type=oneshot`, `ExecStart=` the script), and `disk-alert.timer` says **when** – 2 minutes after boot, then every 5 minutes. The timer finds its service **by the same first name** (`disk-alert`); `WantedBy=timers.target` only makes the timer start at every boot. The chain is **timer → service → script → journal**.

Socket (Step 3.3): here the trigger is a **connection**, not a time. A rarely used service does not need to run all day, so **systemd itself listens on the port** and starts the program only when someone connects. `hello.socket` says `ListenStream=9999` and finds its service by the same first name, `hello`. With **`Accept=yes`**, every connection gets **its own short-lived copy** of the service – that is why the service is a **template**, `hello@.service`: the `@` means one file, many copies, one per connection. `ExecStart=/bin/sh -c 'echo …'` is the program, and **`StandardOutput=socket`** sends its output straight back through the connection to the client. I tested it with `nc localhost 9999` and got *"hello from stor01 – started on demand"*; the copy ended, so nothing was running before or after. The chain is **client connects → socket → one copy of the service → answer → copy ends**. Real servers use this too: with `ssh.socket`, systemd holds port 22 and starts `ssh.service` on the first connection (with `Accept=no`, one sshd handles all connections itself – no template). On stor01, `ssh.socket` is disabled and `ssh.service` runs all the time – the classic choice for a server admins use daily. My three lessons: a per-connection service needs the `@`, `ExecStart` needs a full path starting with `/`, and `systemd-analyze verify` checks a unit before I use it.

2. Step 3.4 – a custom target: one command for the whole Acme stack

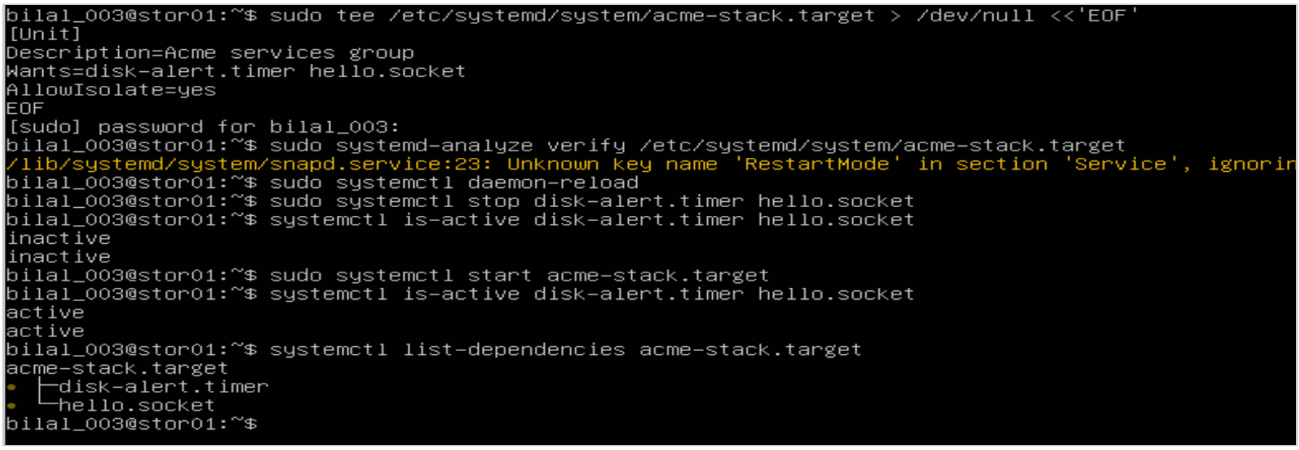
💡 **The story:** stor01 now has several Acme units of my own – the disk alert, the hello socket, later the nightly backup. During maintenance an admin should not have to remember every name. One group, **`acme-stack.target`**, starts them all with one command – the same idea as `multi-user.target`, but for Acme.

All on **stor01**.

```
        sudo tee /etc/systemd/system/acme-stack.target > /dev/null <<'EOF'
        [Unit]
        Description=Acme services group
        Wants=disk-alert.timer hello.socket
        AllowIsolate=yes
        EOF
        sudo systemd-analyze verify /etc/systemd/system/acme-stack.target
        sudo systemctl daemon-reload

        # test: stop both members, then start the whole group with ONE command
        sudo systemctl stop disk-alert.timer hello.socket
        systemctl is-active disk-alert.timer hello.socket      # inactive inactive
        sudo systemctl start acme-stack.target
        systemctl is-active disk-alert.timer hello.socket      # active active
        systemctl list-dependencies acme-stack.target          # the group as a tree
```

Setting	Meaning
`Wants=`	The members of the group – started when the target starts
`AllowIsolate=yes`	The server may later be switched into this target like a “mode”

📸 **Evidence – both members stopped, one `start` of the target, both active again; the group tree (the yellow line is Ubuntu’s own `snapd.service`, not my file):**



acme-stack target

Key understanding (in my own words)

A **target** is a unit that **groups other units** – services, timers or sockets – so that starting the target starts all its members with one command. Linux’s own targets are in `/lib/systemd/system/`: `timers.target` holds all the timers (that is why each timer has `WantedBy=timers.target`), and `multi-user.target` is the normal **server state** – networking, SSH and all other services, with text login and no desktop. `graphical.target` is `multi-user.target` plus the graphical desktop, so targets can contain other targets; my servers boot to `multi-user.target`. On stor01 I created my own group, `acme-stack.target`, and `Wants=disk-alert.timer hello.socket` made both units its members. To test it, I stopped both (`inactive inactive`), started only `acme-stack.target`, and both became `active` again; `systemctl list-dependencies acme-stack.target` showed the group as a tree. In a real company this is how an admin brings a whole application stack back after maintenance with one command – for example `company-app.target` – instead of remembering every service name.

3. Reference – looking inside the real server

💡 **In plain words:** to see that these ideas are not only lab examples, I looked at how Ubuntu itself uses sockets, targets and timers on stor01.

```
        # stor01
        systemctl list-sockets                                  # 29 sockets held by systemd — hello.socket on
[::]:9999
        sudo systemctl status ssh.socket ssh.service            # socket mode or always-on?
        systemctl list-dependencies multi-user.target --no-pager    # what "server mode" contains
        ls /etc/systemd/system/multi-user.target.wants/         # members I or packages enabled
        ls /etc/systemd/system/timers.target.wants/
        ls /etc/systemd/system/sockets.target.wants/
```

📸 **Evidence – `ssh.socket` disabled and inactive, `ssh.service` enabled and running since boot (5.2 MB); `ExecStartPre=/usr/sbin/sshd -t` tests the configuration before starting:**

```
bilal_003@stor01:~$ sudo systemctl status ssh.socket ssh.service
[sudo] password for bilal_003:
● ssh.socket - OpenBSD Secure Shell server socket
     Loaded: loaded (/lib/systemd/system/ssh.socket; disabled; vendor preset: enabled)
     Active: inactive (dead)
     Listen: [::]:22 (Stream)
   Accepted: 0; Connected: 0;

● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-10-21 16:40:15 UTC; 36min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1031 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1067 (sshd)
      Tasks: 1 (limit: 964)
     Memory: 5.2M
        CPU: 1.939s
     CGroup: /system.slice/ssh.service
             └─1067 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

ssh socket vs service

📸 **Evidence – `multi-user.target` contains other targets: `timers.target` with my `disk-alert.timer` next to Ubuntu's own timers (`apt-daily`, `logrotate`, `fstrim`), `getty.target` with the console login `getty@tty1.service` (a template again), and `nfs-client.target` / `remote-fs.target`:**

```
bilal_003@stor01:~$ systemctl list-dependencies multi-user.target --no-pager | tail -35
●   ├─apport-autoreport.timer
●   ├─apt-daily-upgrade.timer
●   ├─apt-daily.timer
●   ├─disk-alert.timer
●   ├─dpkg-db-backup.timer
●   ├─e2scrub_all.timer
●   ├─fstrim.timer
●   ├─fwupd-refresh.timer
●   ├─logrotate.timer
●   ├─man-db.timer
●   ├─motd-news.timer
●   ├─snapd.snap-repair.timer
●   ├─systemd-tmpfiles-clean.timer
●   ├─ua-timer.timer
●   ├─update-notifier-download.timer
●   └─update-notifier-motd.timer
●   cloud-init.target
●   ├─cloud-config.service
●   ├─cloud-final.service
●   ├─cloud-init-local.service
●   └─cloud-init.service
●   getty.target
●   ├─getty-static.service
●   └─getty@tty1.service
●   nfs-client.target
●   ├─auth-rpcgss-module.service
●   ├─nfs-blkmap.service
●   ├─rpc-statd-notify.service
●   └─remote-fs-pre.target
●   remote-fs.target
●   └─nfs-client.target
●     ├─auth-rpcgss-module.service
●     ├─nfs-blkmap.service
●     ├─rpc-statd-notify.service
●     └─remote-fs-pre.target
bilal_003@stor01:~$
```

multi-user tree

📸 **Evidence – `enable` = a link in a `….wants/` folder: `ssh.service`, `nfs-server.service`, `glpi-agent.service` in `multi-user.target.wants/`; `disk-alert.timer` in `timers.target.wants/`; `hello.socket` in `sockets.target.wants/`:**

```
bilal_003@stor01:~$ ls /etc/systemd/system/multi-user.target.wants/
console-setup.service         pollinate.service                       snap-lxd-29351.mount
cron.service                  remote-fs.target                        snap-lxd-40911.mount
dmesg.service                 rpcbind.service                         snap.lxd.activate.service
e2scrub_reap.service          rsyslog.service                         snap-snapd-21759.mount
glpi-agent.service            secureboot-db.service                   snap-snapd-28254.mount
grub-common.service           snap-core20-2318.mount                  ssh.service
grub-initrd-fallback.service  snap-core20-2922.mount                  systemd-networkd.service
irqbalance.service            snap-core22-2955.mount                  systemd-resolved.service
lxd-agent.service             snapd.apparmor.service                  thermald.service
ModemManager.service          snapd.autoimport.service                ua-reboot-cmds.service
networkd-dispatcher.service   snapd.core-fixup.service               ubuntu-advantage.service
nfs-client.target             snapd.recovery-chooser-trigger.service  ufw.service
nfs-server.service            snapd.seeded.service                    unattended-upgrades.service
open-vm-tools.service         snapd.service
bilal_003@stor01:~$ ls /etc/systemd/system/timers.target.wants/
apport-autoreport.timer  e2scrub_all.timer    motd-news.timer
apt-daily.timer          fstrim.timer         snapd.snap-repair.timer
apt-daily-upgrade.timer  fwupd-refresh.timer  ua-timer.timer
disk-alert.timer         logrotate.timer      update-notifier-download.timer
dpkg-db-backup.timer     man-db.timer         update-notifier-motd.timer
bilal_003@stor01:~$ ls /etc/systemd/system/sockets.target.wants/
apport-forward.socket  iscsid.socket      snapd.socket                       systemd-networkd.socket
dm-event.socket        multipathd.socket  snap.lxd.daemon.unix.socket        uuidd.socket
hello.socket           rpcbind.socket     snap.lxd.user-daemon.unix.socket
bilal_003@stor01:~$
```

wants folders

What I saw	Meaning
`list-sockets`: 29 sockets, `hello.socket` → ACTIVATES –	systemd holds many ports and paths for Linux itself; – because a new copy of the template starts per connection
`ssh.socket` disabled, `ssh.service` running	stor01 uses classic always-on SSH – normal for a server admins use daily
`ExecStartPre=/usr/sbin/sshd -t`	Test the config **before** start – like `bash -n` and `systemd-analyze verify`
`Docs: man:sshd(8)`	The manual pages, not the configuration (that is `/etc/ssh/sshd_config`)
`.mount` units in the tree	Mounts are units too – `/data` and `/backups` from fstab become `data.mount`, `backups.mount`
● / other symbol	Active now / not running now – often a one-time job already finished at boot
`….wants/` folders	`enable` creates a link there – that link is what makes a unit a member of a target

Key understanding (in my own words)

> `systemctl list-dependencies multi-user.target` showed me what "server mode" really is: a tree of units. Its direct members are services like `cron`, `rsyslog`, `nfs-server`, `rpcbind`, `ssh`, `glpi-agent` and `open-vm-tools`, plus `.mount` units – mounts are units too. It also contains **other targets**: `timers.target`, where my `disk-alert.timer` sits next to Ubuntu's own timers (`apt-daily`, `logrotate`, `fstrim`); `sockets.target` with `hello.socket`; `getty.target`, whose `getty@tty1.service` is the console login and also a template with `@`; and `remote-fs.target` for NFS. The `….wants/` folders in `/etc/systemd/system/` show how membership works: `enable` simply creates a link in the folder of the target named in `WantedBy=`. On stor01, `ssh.socket` is disabled and `ssh.service` always runs, and its `ExecStartPre=sshd -t` tests the configuration before every start. So a target is a group, groups can contain groups, `enable` adds a member, and booting the server simply means systemd starting this whole tree.

4. Step 3.5 part A – cron, the old way

💡 **The story:** before systemd timers, Linux ran scheduled jobs with **cron**. Many companies still have hundreds of cron jobs – backups, reports, clean-ups – so an admin must be able to read and write them. I ran the disk check with cron for a few minutes, compared it with the timer, then removed it.

All on **stor01**.

```
crontab -e                          # my job list – first time: choose 1 (nano)
# added at the bottom:
*/2 * * * * THRESHOLD=15 /usr/local/bin/disk-alert.sh

crontab -l                                  # the line is there
grep CRON /var/log/syslog | tail -3         # cron started the job
journalctl -t disk-alert -n 3               # the warnings the script wrote
crontab -e                                  # afterwards: line deleted again
```

```
*/2    *     *     *      *        command
min    hour   day   month  weekday          * = every · */2 = every 2 · 0 2 * * * = daily at 02:00
```

📸 **Evidence – my crontab line; `syslog`: cron started it as `bilal_003` at 20:08, next to Ubuntu's own hourly cron jobs run as root:**

```
bilal_003@stor01:~$ crontab -l
# m h  dom mon dow   command
*/2 * * * * THRESHOLD=15 /usr/local/bin/disk-alert.sh

bilal_003@stor01:~$ grep CRON /var/log/syslog | tail -3
Oct 21 18:17:01 stor01 CRON[9116]: (root) CMD (   cd / && run-parts --report /etc/cron.hourly)
Oct 21 19:17:01 stor01 CRON[9215]: (root) CMD (   cd / && run-parts --report /etc/cron.hourly)
Oct 21 20:08:01 stor01 CRON[9479]: (bilal_003) CMD (THRESHOLD=15 /usr/local/bin/disk-alert.sh)
bilal_003@stor01:~$
```

crontab and syslog

📸 **Evidence – the script's warnings at 20:08 and 20:10 – exactly every 2 minutes (the 86 % line is yesterday's test, before a reboot):**

```
bilal_003@stor01:~$ journalctl -t disk-alert -n 3
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
-- Boot 6e4ad773005e4624b8f2adda0aba8d00 --
Oct 21 20:08:01 stor01 disk-alert[9485]: /data is 19% full (threshold 15%)
Oct 21 20:10:02 stor01 disk-alert[9498]: /data is 19% full (threshold 15%)
bilal_003@stor01:~$
```

cron every 2 min

```
cron (every 2 min) ──runs──► disk-alert.sh ──THRESHOLD=15 given──► 19 ≥ 15? yes ──logger──► journal
```

Lesson: the crontab line only says **when** to run the script – not where warnings go. The **script** decides that: `THRESHOLD="${THRESHOLD:-80}"` takes the value I pass (15), and only if usage ≥ threshold does `logger` write to the journal.

That is why the same script works unchanged with cron and with a systemd timer. The server clock is UTC, so 20:08 is 22:08 Berlin time.

Key understanding (in my own words)

> Before systemd timers, Linux ran scheduled jobs with **cron**, and many companies still use it, so an admin must be able to read it. Each user has their own job list, the **crontab**. `crontab -e` opens it in an editor (nano), and every job is **one line**: five time fields (minute, hour, day of month, month, weekday) followed by the command. On stor01 I added `*/2 * * * * THRESHOLD=15 /usr/local/bin/disk-alert.sh`. `*/2` means every 2 minutes, and each `*` means every hour, day, month and weekday. Cron, like a timer, only **runs** the script; the crontab line does not say where warnings go. The **script** decides that. `THRESHOLD="${THRESHOLD:-80}"` takes the value I pass (15), or 80 if none is given. `df` reads how full `/data` is (19 %), and only if 19 ≥ 15 does `logger` write the warning to the journal. The chain is **cron → script → logger → journal**. `crontab -l` showed my line. `grep CRON /var/log/syslog` proved that cron started the job as `bilal_003` at 20:08, next to Ubuntu's own hourly cron jobs run as root. `journalctl -t disk-alert` showed *"/data is 19% full (threshold 15%)"* at 20:08 and 20:10, exactly every 2 minutes. So the same script works unchanged with cron and with a systemd timer. Cron is simple, one line, but it has two weaknesses. First, **the logs are split**: the start of the job is in `/var/log/syslog`, and the script's warning is in the journal. Second, **missed runs are lost**: if the server is off at the scheduled time, cron simply skips that run and nothing tells me. A systemd timer solves both. `journalctl -u <name>.service` shows the start, the output and the result in one place, and `Persistent=true` runs a missed job at the next boot. Then I removed the cron line again with `crontab -e`.

5. Step 3.5 part B – the nightly backup runs by itself

💡 **The story:** in Phase 2 I made versioned backups by hand. A backup that depends on someone remembering it will one day be forgotten. Now stor01 backs up `/data/app` **every night at 02:00**, while nobody uploads – and if the server was off at 02:00, it **catches up at the next boot**.

```
nightly-backup.timer ──02:00──► nightly-backup.service ──runs──► nightly-backup.sh ──► /backups/<date> + latest + "finished" in journal
      WHEN                          HOW                           WHAT
```

All on **stor01**.

```
# 1. The script — WHAT
sudo tee /usr/local/bin/nightly-backup.sh > /dev/null <<'EOF'
#!/bin/bash
# Nightly versioned backup of /data/app, keep 7 days
TODAY=$(date +%F)
rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/$TODAY/
ln -sfn /backups/$TODAY /backups/latest
find /backups -maxdepth 1 -type d -name '20*' -mtime +7 -exec rm -rf {} +
logger -t nightly-backup "backup of /data/app to /backups/$TODAY finished"
EOF
sudo chmod +x /usr/local/bin/nightly-backup.sh      # sudo — the file belongs to root
bash -n /usr/local/bin/nightly-backup.sh            # syntax check — for scripts only

# 2. The service — HOW
sudo tee /etc/systemd/system/nightly-backup.service > /dev/null <<'EOF'
[Unit]
Description=Nightly backup of /data/app
[Service]
Type=oneshot
ExecStart=/usr/local/bin/nightly-backup.sh
EOF

# 3. The timer — WHEN
sudo tee /etc/systemd/system/nightly-backup.timer > /dev/null <<'EOF'
[Unit]
Description=Nightly backup at 02:00
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
[Install]
WantedBy=timers.target
EOF

sudo systemd-analyze verify /etc/systemd/system/nightly-backup.*   # check unit files
sudo systemctl daemon-reload
sudo systemctl enable --now nightly-backup.timer
systemctl list-timers | grep nightly

# 4. Test now — don't wait until 02:00
sudo systemctl start nightly-backup.service
ls -l /backups
journalctl -u nightly-backup.service -n 5

# 5. Add the trigger to the Acme group
sudo systemctl add-wants acme-stack.target nightly-backup.timer
systemctl list-dependencies acme-stack.target
```

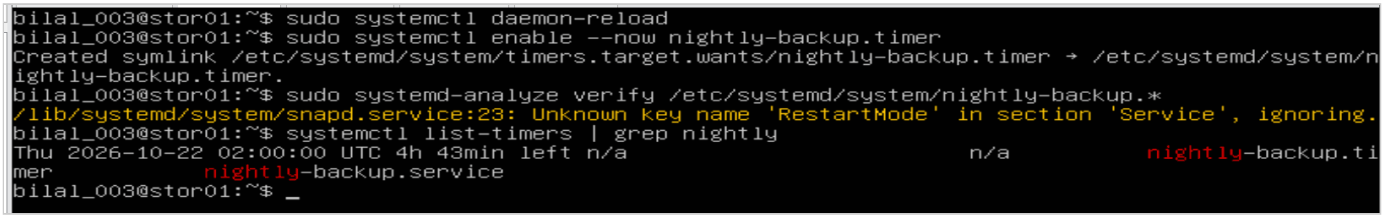
Line	Meaning
`rsync -a --delete --link-dest=/backups/latest`	Copy `/data/app` to today's folder; unchanged files become **hard links** to the last version – no extra space
`ln -sfn /backups/$TODAY /backups/latest`	Move the `latest` shortcut to the new version
`find … -name '20*' -mtime +7 -exec rm -rf {} +`	Delete dated versions older than 7 days (`v1`–`v3`, `mirror` and the `.tar.gz` are not touched)
`OnCalendar=*-*-* 02:00:00`	Every year-month-day at 02:00 (server time = UTC → 04:00 Berlin)
`Persistent=true`	If the server was off at 02:00, run the missed backup at the next boot – cron cannot do this
`add-wants`	Creates a link in `acme-stack.target.wants/` – the timer becomes a member of the group

📸 Evidence – the timer unit:

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/nightly-backup.timer > /dev/null <<'EOF'
[Unit]
Description=Nightly backup at 02:00:00
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
[Install]
WantedBy=timers.target
EOF
```

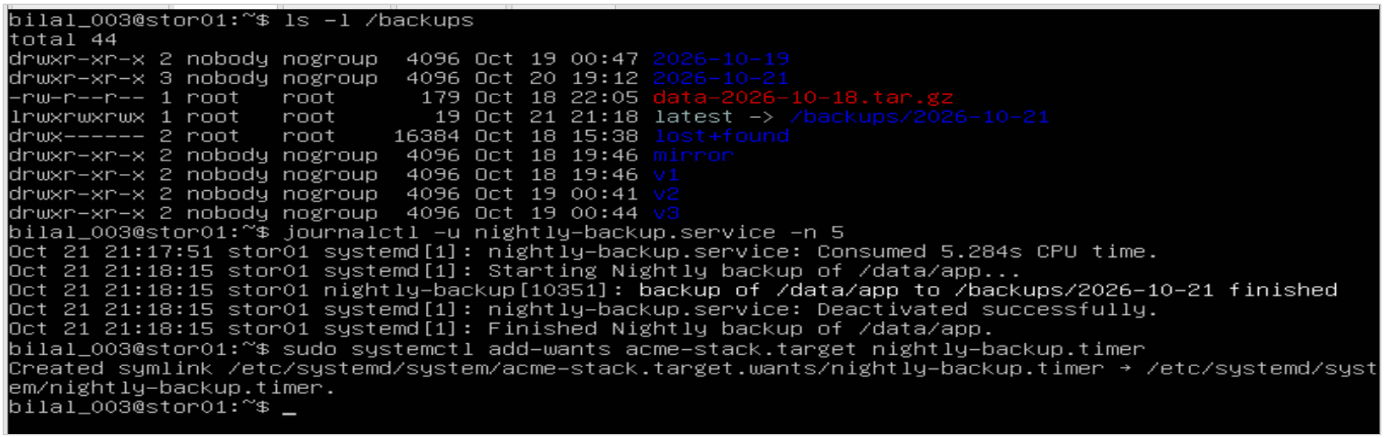
timer unit

📸 Evidence – `enable --now` creates the link in `timers.target.wants/`; `verify` shows no error for my files; next run Thu 02:00 UTC:



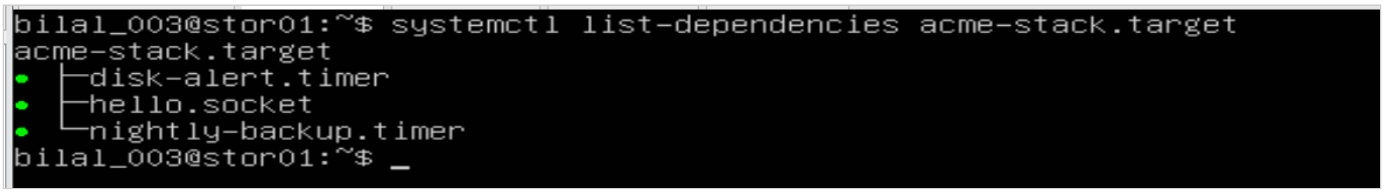
enable verify list-timers

📸 **Evidence – `/backups/2026-10-21` created, `latest → /backups/2026-10-21`; the journal shows start, "finished" and *Deactivated successfully* in one place; the timer added to `acme-stack.target`:**



backup folder journal add-wants

📸 **Evidence – the Acme group now has three members:**



three members

Lessons: `bash -n` is for **scripts** only – a `.service` or `.timer` file is checked with `systemd-analyze verify`. A heredoc must end with exactly the word it started with (`<<'EOF'` … `EOF`); a mistyped start word leaves the shell waiting at `>` – press Ctrl+C and type it again. The first run copied the 700 MB of new uploads (5.3 s CPU); a second run was instant because nothing had changed – the hard-link idea at work.

Why the timer and not the service in the target? A target can hold **any kind of unit**. What I add depends on how the job should run:

Kind of job	What goes into the target	Example
Runs **all the time**	the **service** itself	`nfs-server.service`, `ssh.service`, a web server
Runs **at a time**	the **timer**	`nightly-backup.timer`, `disk-alert.timer`
Runs **on a connection**	the **socket**	`hello.socket`

Key understanding (in my own words)

The nightly backup now runs by itself on stor01, with the pattern **timer → service → script → journal**. The script `/usr/local/bin/nightly-backup.sh` does what I did by hand in Phase 2. `rsync -a --delete --link-dest=/backups/latest` copies `/data/app` into a folder named with today's date, and unchanged files become hard links that take no extra space. `ln -sfn` moves the `latest` link to the new folder, `find … -mtime +7` deletes versions older than 7 days, and `logger` writes "finished" to the journal. `nightly-backup.service` says **how** (`Type=oneshot`), and `nightly-backup.timer` says **when**: `OnCalendar=*-*-* 02:00:00`, every night, plus `Persistent=true`, which runs a missed backup at the next boot if the server was off. `systemd-analyze verify` checked the unit files (`bash -n` is only for scripts), and `enable --now` created the link in `timers.target.wants/`. `list-timers` shows the next run at 02:00 UTC (04:00 in Berlin, because the server runs on UTC). I didn't wait for the night: `systemctl start nightly-backup.service` created `/backups/2026-10-21`, moved `latest` to it, and `journalctl -u nightly-backup.service` showed the start, the "finished" message and *Deactivated successfully* all in one place. That is the advantage over cron, together with the catch-up. Finally, `systemctl add-wants acme-stack.target nightly-backup.timer` made the backup the third member of the Acme group, next to `disk-alert.timer` and `hello.socket`. A target can hold **any kind of unit**: services, timers, sockets, mounts or other targets. What I add depends on how the job should run. A service that must run **all the time**, like the NFS server or a web server, goes into the target **as a service**; `multi-user.target` holds `ssh.service` and `nfs-server.service` this way. A job that runs **at a time** or **on a connection** goes in through its **trigger**, the timer or socket, which starts the service with the same name

only when needed. That is why I added `nightly-backup.timer` and not `nightly-backup.service`. If the service were in the target, every start of the target would run a backup immediately, for example in the middle of the day after maintenance.

6. Phase 3 complete – what stor01 now does by itself

Unit	Kind	What it does
`disk-alert.timer` → `disk-alert.service`	timer + service	Every 5 min: warn in the journal if `/data` ≥ 80 %
`hello.socket` → `hello@.service`	socket + template	Port 9999: start a short-lived service per connection
`nightly-backup.timer` → `nightly-backup.service`	timer + service	Every night 02:00: versioned backup of `/data/app`, keep 7 days, catch up after downtime
`acme-stack.target`	target	Groups the three triggers – one command for the Acme stack

	cron	systemd timer
Logging	Start in `/var/log/syslog`, output elsewhere	Everything in `journalctl -u <name>`
Missed run (server off)	Skipped	`Persistent=true` runs it at the next boot
Grouping, dependencies	None	Targets, can wait for mounts or network
Simplicity	One line	Two files

Rule of thumb: cron for quick one-line jobs; systemd timers for anything that needs logging, catch-up or grouping.

7. Next steps

1. Check that the cron line is gone (`crontab -l` shows only comments); shut down and take the VMware snapshot **`40-automation`** of all four servers.
2. **Phase 4:** processes, signals and cgroups.
3. Phase 2 tickets in GLPI: LVM build, grow /data, fstab UUID typo, NFS share, stale NFS mount after rollback.
4. At the end of the project: how companies run servers as VMs on **VMware ESXi / vCenter**, and what Workstation's *Connect to Server* does.

Project 2 – Acme Analytics Linux Platform

Day 7 · Phase 4 starts: where a slow application comes from, stopping a runaway process, pipes

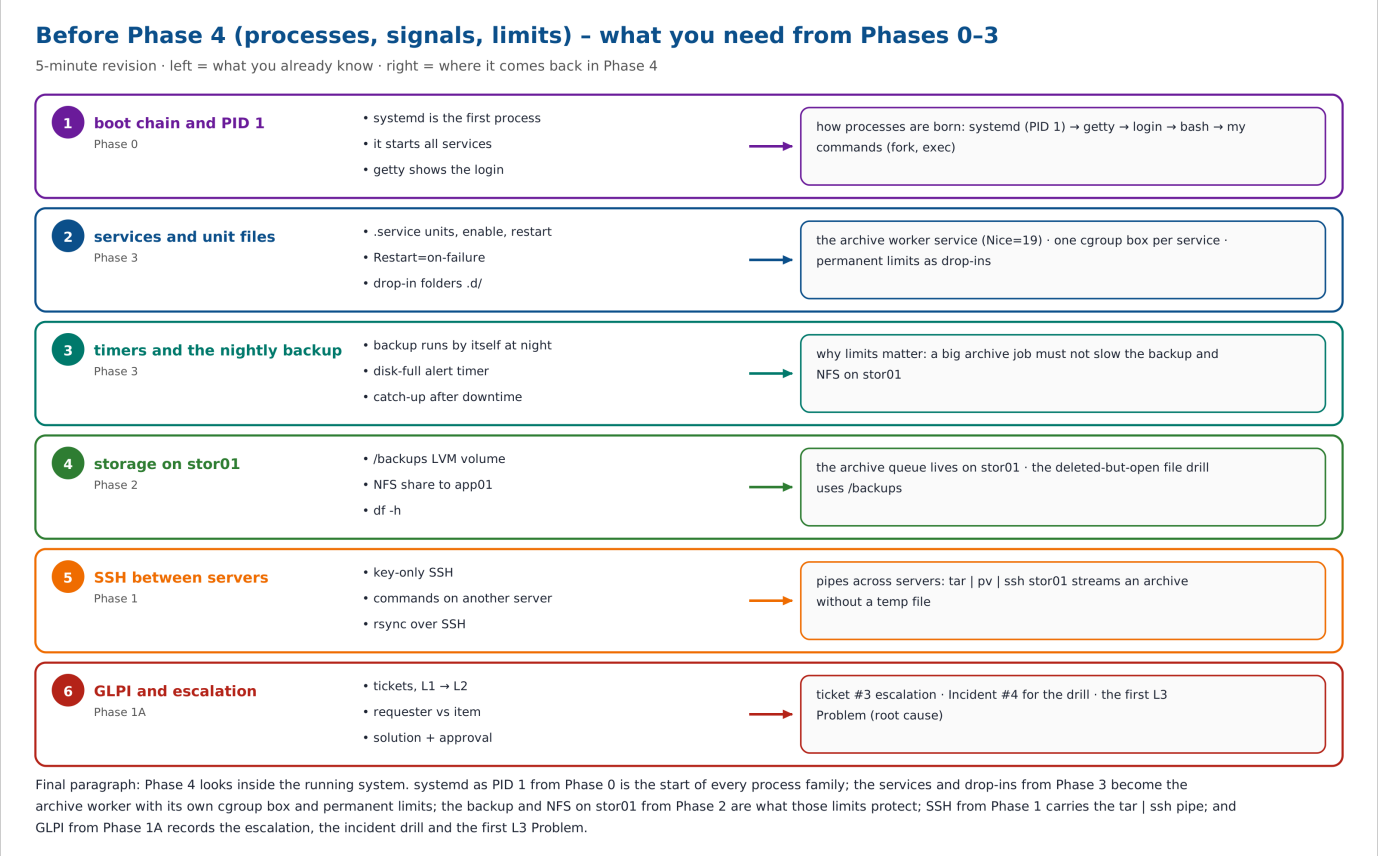
Muhammad Bilal Javed · 24 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01, gw01 (powered on), svc01 (GLPI)
Status	Phase 3 ✅ (Day 6) · Phase 4 🟡 – Step 4.1 runaway process ✅ (concept, hands-on on app01, GLPI ticket #2) · Step 4.2 pipes ✅ (FIFO as reference)
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 6.

Before Phase 4 – quick revision (Phases 0–3)

💡 **5 minutes:** what I already know from Phases 0–3 and where it comes back in Phase 4.



Before Phase 4 – quick revision

1. A question from industry – too many backups, and backup logs that grow too big

💡 **In plain words:** a friend who works as a Linux administrator asked: in a real company backups are taken all the time – what happens when the backups, and the backup **logs**, become huge? These are two different problems with two different fixes.

Problem	Industry fix	Already in this project
The backups grow	Retention (e.g. 7 daily, 4 weekly, 12 monthly) · incremental copies · deduplication (restic, borg) · move old copies to cheaper storage (AWS S3 → Glacier with a lifecycle rule)	`find … -mtime +7` keeps 7 days · `rsync --link-dest` hard links (Day 6)
The backup log grows	Log a summary, not every file (`rsync -v` writes one line per file) · rotate and compress logs with `logrotate` · journal size limit (`SystemMaxUse=`) · watch `/backups` and `/var/log` too	One `logger` line per run · logrotate comes in Phase 5

Interview line: *"I keep backups small with incremental copies, retention and cheaper storage for old copies, and I keep logs small by logging summaries and rotating them with logrotate."*

2. Real-world troubleshooting practice – how the incident drills fit in

💡 **In plain words:** to train like an on-call administrator, each phase will end with an **incident drill on my own Acme servers**: something realistic breaks, and I fix it like a real ticket – triage → hypotheses → commands → fix → verify → write-up in GLPI. The scenarios come from a hands-on troubleshooting lab (SadLab) and an L2/L3 incident workbook; only the cases that match this platform are used.

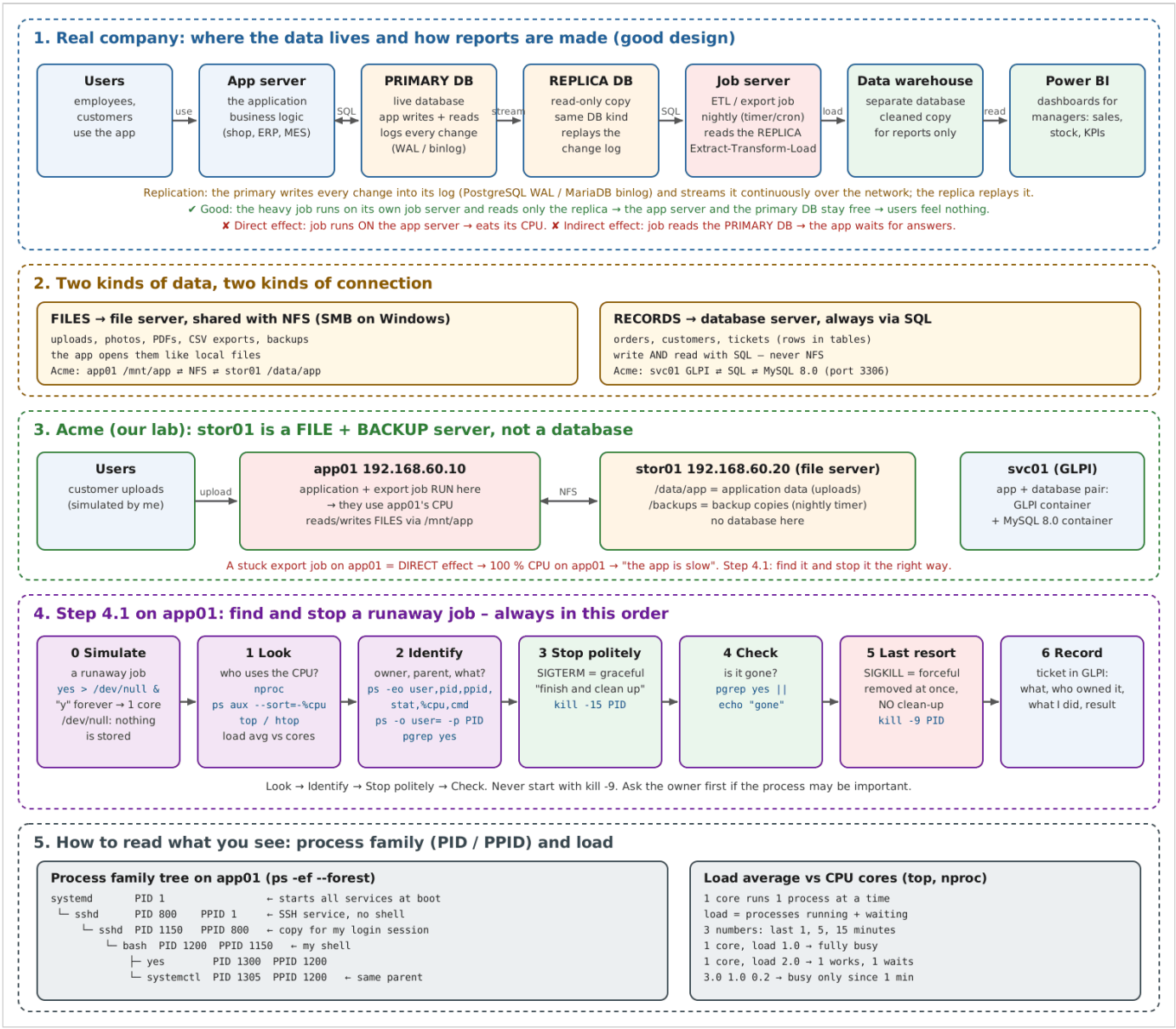
Phase	Drill
4 – Processes, signals	Runaway log writer · deleted file still filling the disk (`lsof +L1`) · "the server is slow"
5 – Logging	Backup log too big → logrotate · log rotation stalls a service
Backups	Backup "succeeds" but the restore fails → add a verify step to the nightly backup
6 – Security	SSH key login rejected · 403 Forbidden · firewall rule blocks a service
7 – Web service	Nginx won't start · works locally, not from outside
8 – Disaster drill	Read-only root after a power cut · wrong device mounted · inodes full

Several "advanced" workbook incidents were already solved for real in this project: a process hung on NFS after a storage failover (Day 5 stale mount and **D state**), an LVM snapshot during maintenance (Day 5), the wrong device mounted (why fstab uses **UUIDs**, Day 4), a backup that must be proven by a restore (Day 4 restore drill), and a cron job versus a timer (Day 6).

3. Phase 4 starts – Step 4.1: where a slow application comes from

💡 **The story:** app01 is Acme's application server. In real companies the most common daily problems are a **process that eats all the CPU**, a **job that hangs**, or a program that **won't stop properly**. Phase 4 teaches how to **find** the guilty process, **stop it the right way**, and finally **cap** what a service may use, so one bad program cannot freeze the whole server. Before touching commands, I worked out where such a process comes from in a real company and in Acme.

📸 **Architecture – where the data lives, how reports are made (primary → replica → job server → data warehouse → Power BI), files vs records, Acme, the order of work on app01, the process family and the load average:**



Step 4.1 architecture

Remember it in four lines:

```
Real company:  Users → App server ⇄ PRIMARY DB → (WAL/binlog stream) → REPLICA → Job server (ETL) → Data warehouse → Power BI
Replication:   the app writes SQL only to the primary; the primary logs every change and streams the log to the replica, which replays it
Acme:          Users → app01 (runs the app) ⇄ NFS (files) ⇄ stor01 (stores files + backups)  ·  svc01: GLPI ⇄ SQL ⇄ MySQL 8.0
Step 4.1:      Simulate → Look → Identify → Stop politely → Check → (kill -9 last) → Record
```

All on **app01**:

```
sudo apt install -y htop stress-ng
nproc                                                    # how many CPU cores
yes > /dev/null &                                        # simulate a runaway job
ps aux --sort=-%cpu | head -5                            # who uses most CPU
top                                                      # load average, %CPU, STAT – q to quit
htop                                                     # same, clearer – q to quit
ps -eo user,pid,ppid,stat,%cpu,cmd --sort=-%cpu | head -5  # owner, parent, state, command
ps -o user= -p $(pgrep yes)                              # who owns it
ps -ef --forest | less                                   # the process family tree
kill -15 $(pgrep yes)                                    # SIGTERM – graceful stop
pgrep yes || echo "gone"                                 # check
```

Term	Meaning
Runaway job	Busy and out of control – 100 % CPU, never finishes
Hung job	Stuck waiting – almost 0 % CPU, never finishes
Load average	Processes running + waiting for the CPU; 1-, 5- and 15-minute averages, compared with `nproc`
PID / PPID	The process's own number / the number of the process that started it
SIGTERM (`kill -15`) / SIGKILL (`kill -9`)	Graceful stop with clean-up / forceful stop, no clean-up – last resort
Primary / replica	Live database / read-only copy of the same kind, kept up to date by streaming the change log
WAL / binlog	The change log of PostgreSQL / MariaDB – the source of replication

3.1 The hands-on run on app01

Issue – `apt install` hangs at `0% [Working]`

Seen	On app01, `sudo apt install -y htop stress-ng` printed `Ign:` lines again and again and stayed at `0% [Working]`.
Cause	**gw01 was powered off.** gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT) – a **single point of failure**.
Fix	Powered on gw01; the install worked at once.
Lesson	When a server "cannot reach the internet", check the path hop by hop: `ip route` → ping the gateway → ping 8.8.8.8 → ping a name (DNS).

📸 **Evidence:**

```
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
0% [Working]_
```

apt hangs

📸 **Evidence – 2 CPU cores; the runaway job started as job [1], PID 1918:**

```
bilal_002@app01:~$ sudo apt install -y htop stress-ng
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
htop is already the newest version (3.0.5-7build2).
stress-ng is already the newest version (0.13.12-2ubuntu1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0
  libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager tcl tcl8.6 usb-modeswitch
  usb-modeswitch-data
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
bilal_002@app01:~$ nproc
2
bilal_002@app01:~$ yes > /dev/null &
[1] 1918
bilal_002@app01:~$ jobs
[1]+  Running                 yes > /dev/null &
```

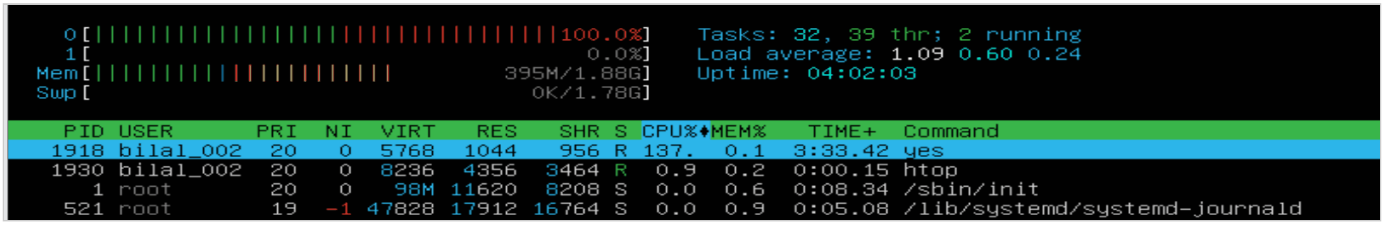
nproc yes

📸 **Evidence – Look: `yes` at the top with 99.9 % CPU, state R, owner `bilal_002` (shown cut as `bilal_0+`); without the minus in `--sort=-%cpu`, ps sorts lowest first:**

```
bilal_002@app01:~$ ps aux --sort=-%cpu | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
bilal_0+    1918 99.9  0.0   5768  1044 tty1     R    18:57   2:40 yes
root         711  0.5  0.4 314816  9312 ?        Ssl  14:59   1:15 /usr/bin/vmtoolsd
root        1557  0.3  0.0      0     0 ?        I    16:27   0:32 [kworker/0:0-events]
root        1550  0.2  0.0      0     0 ?        I    16:27   0:25 [kworker/1:1-events]
bilal_002@app01:~$
```

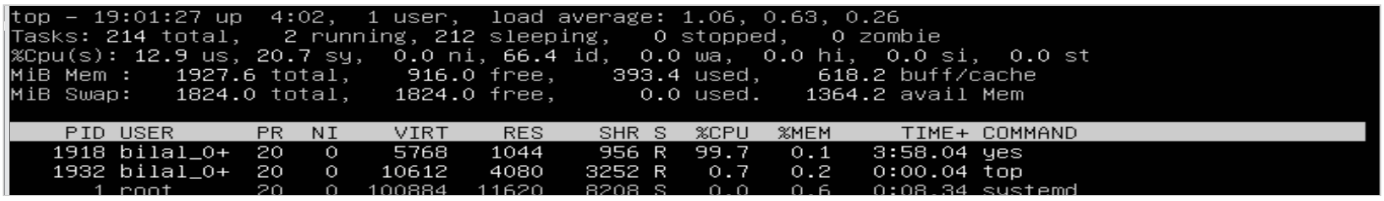
ps aux

📸 **Evidence – `htop`: core 0 at 100 %, core 1 at 0 % – `yes` fills exactly one of the two cores; load average 1.09:**



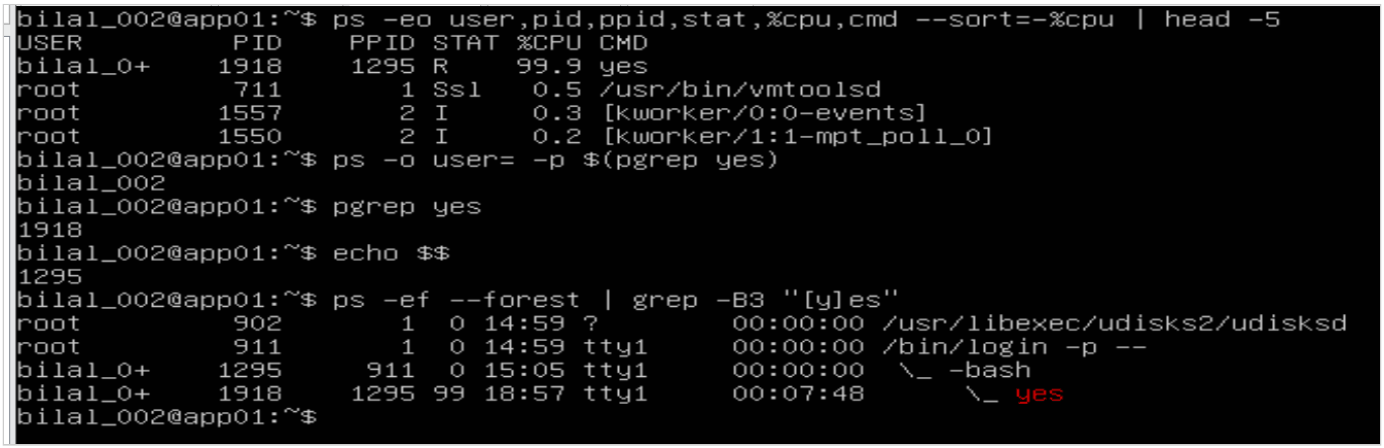
htop

📸 **Evidence – `top`: load average `1.06, 0.63, 0.26` – about 1.0 over the last minute and still rising over 5 and 15 minutes; `2 running, 0 zombie`:**



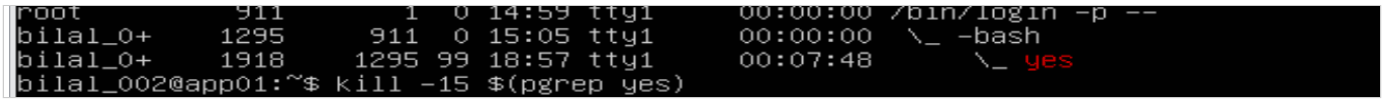
top

📸 **Evidence – Identify: PID 1918, PPID 1295; `echo $$` = 1295, so the parent is my bash; the family tree: `/bin/login` (console tty1) → `-bash` → `yes`. Over SSH, the parent of bash would be `sshd`:**



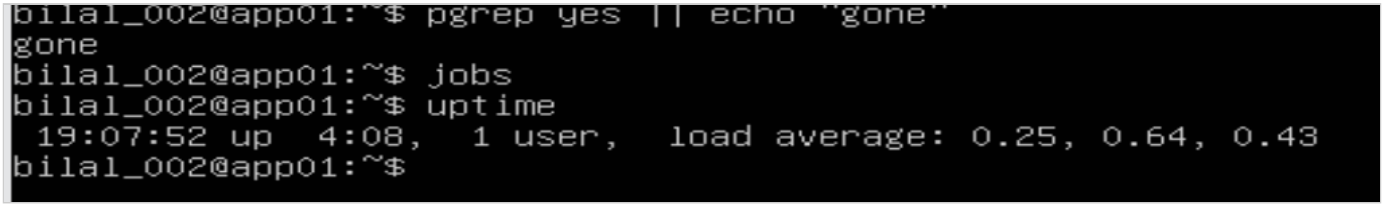
identify

📸 **Evidence – Stop politely with SIGTERM:**



kill 15

📸 **Evidence – Check: "gone", no jobs left, the load average falling (`0.25`, 0.64, 0.43 – the 1-minute value drops first):**

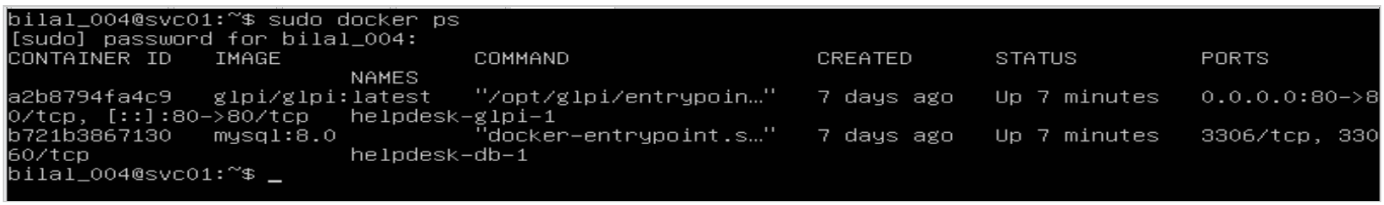


gone

Lesson: read the output **above** a success message. When `pgrep` was typed without a name it failed with an error – and `||` printed "gone" anyway. Only `pgrep yes || echo "gone"` is real proof.

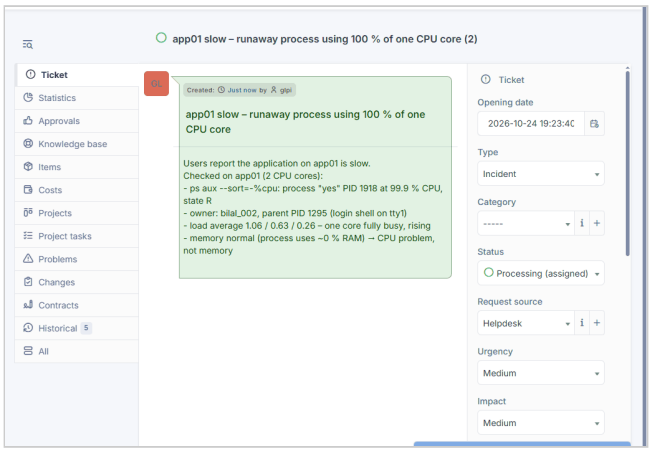
3.2 Record – GLPI ticket #2

📸 **Evidence – GLPI runs as an app + database pair on svc01: `glpi/glpi` publishes port 80 to the network; the database `mysql:8.0` shows only `3306/tcp` – not published, reachable only by GLPI inside Docker:**



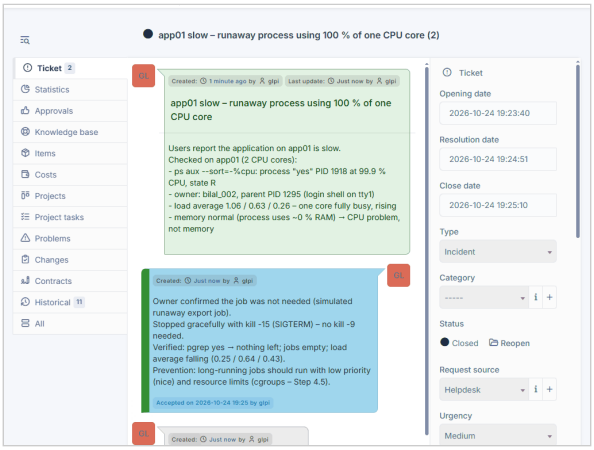
docker ps

📸 **Evidence – Incident ticket #2 created (Type Incident, Urgency/Impact Medium):**



ticket created

📸 Evidence – solution added and approved, ticket closed (opened 19:23:40, solved 19:24:51, closed 19:25:10 UTC):



ticket closed

Lesson: in a ticket, **Requester = who** reported it (a person), **Items = what** is broken (a computer from the inventory). In a company everyone works under a personal named account, not the built-in `glpi` super-admin.

Key understanding – the lab result (in my own words)

On app01 I first installed `htop` and `stress-ng`. The first attempt hung at `0% [Working]` with repeated `Ign:` lines because **gw01 was switched off**: gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT), so it is a single point of failure, and after powering it on the install worked. `nproc` showed **2 cores**. I created a runaway job with `yes > /dev/null &` (job [1], PID 1918). `ps aux --sort=-%cpu | head -5` put `yes` at the top with 99.9 % CPU, owner `bilal_002` (shown as `bilal_0+` because the name was cut off), state `R` (running), almost no memory (a CPU problem, not a memory problem) and a growing CPU TIME; without the minus in `--sort=-%cpu`, ps sorts lowest first and shows only idle processes. `htop` showed one core at 100 % and the other at 0 %, so `yes` fills exactly one of the two cores. `top` showed a load average of `1.06, 0.63, 0.26`: about 1.0 over the last minute (one core fully busy) and still rising over 5 and 15 minutes. `ps -eo user,pid,ppid,stat,%cpu,cmd` showed PID 1918 with PPID 1295, and `echo $$` printed 1295, so the parent of `yes` was my bash. `ps -ef --forest` showed the whole family: systemd (PID 1) → `/bin/login` (911, because I worked on the VMware console tty1; over SSH it would be `sshd`) → `-bash` (1295) → `yes` (1918). I stopped it gracefully with `kill -15` (SIGTERM): bash reported `[1]+ Terminated`, `pgrep yes || echo "gone"` printed "gone", `jobs` was empty, and `uptime` showed the load falling to `0.25, 0.64, 0.43`, the 1-minute value dropping first. I also learned to read the output above a success message: when I mistyped `pgrep` without a name, it failed with an error, and `||` printed "gone" anyway. Finally I recorded the incident in GLPI as ticket #2 – Type Incident, Requester = the person who reported it, Items = the computer that is broken – with the findings, the solution and a prevention note (low priority with `nice`, limits with cgroups), and closed it. `docker ps` on svc01 showed that GLPI is a real app + database pair: the `glpi/glpi` container publishes port 80, while its **MySQL 8.0** database shows only `3306/tcp`, not published, so users reach the app and only the app reaches the database.

Key understanding – the concept (in my own words)

1. The problem. The most common complaint about an application server is *"the application is slow"*. Very often the cause is one program using all of the server's CPU. To fix it, I must understand where the data lives, which job can cause the problem, and how to find and stop that job safely.

2. Where the data lives: files and records. Applications use two kinds of data. **Files** (uploads, photos, PDFs, CSV exports, backups) are kept on a **file server** and shared over the network with **NFS** (SMB on Windows); the application opens them as if they were on its own disk. **Records** (orders, customers, tickets)

are rows in tables inside a **database**, such as MariaDB or PostgreSQL. The application always talks to a database with **SQL**, both to write ("save this order") and to read ("show me today's orders"), and never with NFS. A server whose main job is running database software is called a **database server**. One database program can hold many separate databases (for example `shop`, `hr` and `glpi`), and two different database programs can even run on the same server, because each uses its own port (MariaDB 3306, PostgreSQL 5432). Most applications come as an **app + database pair**: the app contains the pages, buttons and **business logic** (the rules of what happens, for example "when an order is placed, reduce the stock"), and the database stores the data.

3. How a real company makes reports, and how it goes wrong. Users work in the application (a web shop, ERP or MES) on the **application server**. It writes and reads the live data in the **primary database**, always with SQL and only on the primary. The primary first writes every change into its **change log**, a numbered diary on disk (PostgreSQL: the **WAL**, write-ahead log; MariaDB: the **binlog**), and then changes the table. It streams that log **continuously over the network** to a **replica**, a read-only copy of the **same kind** of database (PostgreSQL → PostgreSQL, MariaDB → MariaDB), which **replays** the same changes on its copy, normally less than a second behind. This is **replication**. Every night an automatic **ETL job** (Extract, Transform, Load), started by a timer or cron, runs on a separate **job server**. It **extracts** millions of rows from the **replica**, **transforms** them (cleans and combines them) and **loads** them into a **data warehouse**, a separate database used only for reports. Moving data between different kinds of database like this is ETL, not replication. **Power BI** reads the data warehouse and shows dashboards to managers, such as sales, stock and KPIs. Typical jobs: *"export today's orders to a CSV file for accounting"*, *"create the monthly sales report"*, *"send new customers to another system every hour"*. This design keeps the heavy job away from the live system, so users feel nothing. If a company skips it, the heavy job slows the application in one of two ways. With a **direct effect**, the job runs **on the application server itself** and uses its CPU. With an **indirect effect**, the job reads the **primary database**, which becomes so busy that the application waits for its answers. A replica protects performance but is **not a backup**: if someone deletes data by mistake, the deletion is copied to the replica within seconds as well.

4. The Acme platform. Acme is smaller. **stor01 is a file and backup server, not a database server**: it holds `/data/app` (the customer uploads, on the LVM volume `lvdata`) and `/backups` (copies made by the nightly backup timer). app01 reads and writes these files through the NFS mount `/mnt/app`. The application and any export job **run on app01**, so a stuck job there is a **direct effect**: it uses app01's CPU, and the application becomes slow. Acme's real app + database pair is GLPI on **svc01**: every ticket, user and inventoried computer is a row in its **MySQL 8.0** database (a close relative of MariaDB), and GLPI reads and writes them with SQL. Only GLPI's port 80 is published to the network; the database port 3306 is reachable only by the GLPI container inside Docker, so users reach the app and only the app reaches the database.

5. Runaway job, hung job and the load average. A job causes trouble when it is **stuck in a loop**. A loop is a part of a program that repeats the same steps, such as "read the next row, write it, repeat", until it reaches its stop point, for example the last row. Because of a bug, the stop point never comes, so the job repeats forever at 100 % CPU, like a nightly export still running at 10 a.m. That is a **runaway job**: busy and out of control. A **hung job** is different: it uses almost no CPU because it is stuck **waiting**, like my Ctrl+Z waiting on NFS in Phase 3. A normal scheduled job, like my nightly backup, starts, finishes and stops. One **CPU core** runs one process at a time, and `nproc` shows how many cores a server has. The **load average** is the number of processes that want the CPU at the same moment, both running and waiting. `top` shows it as three numbers, the averages over the last **1, 5 and 15 minutes**. With 1 core, a load of 1.0 means fully busy, and 2.0 means one process is working while another waits. `3.0, 1.0, 0.2` means "busy only since a minute ago"; `3.0, 3.0, 3.0` means "busy for at least 15 minutes".

6. Processes, PID and PPID. Every running program is a **process** with its own number, the **PID**. The **PPID** is the PID of the process that **started** it, its parent. Processes form a family tree, shown by `ps -ef --forest`. **systemd** (PID 1) starts all services at boot, so services such as `sshd` have PPID 1 and no shell. When I log in over SSH, sshd starts a copy of itself for my session, and that copy starts my **shell**, bash, the program that reads the commands I type and runs them. Every command I start, for example `yes` or `systemctl restart sshd`, gets its **own PID**, and its **PPID is my bash**. Processes started by the same bash therefore share the same PPID.

7. Finding and stopping the job on app01, always in this order. Simulate → Look → Identify → Stop politely → Check → (kill -9 only as a last resort) → Record. Simulate: I first installed `htop` (a clearer live process viewer) and `stress-ng` (a test tool that creates CPU, memory or disk load on purpose, used later for limits). `yes > /dev/null &` creates a runaway job: `yes` prints the letter "y" without end, so it alone uses one full core; `>` sends this output to `/dev/null`, a special file where everything disappears and nothing is stored (it is not the `/bin` folder, which holds programs); and `&` runs it in the background. **Look:** `nproc` shows the number of cores, and `ps aux --sort=-%cpu | head -5`, `top` and `htop` list the processes using the most CPU. `yes` appears at the top with about 100 %, and the load average rises. **Identify:** `ps -eo user,pid,ppid,stat,%cpu,cmd` shows the **owner** (the user who started it), the PID, the PPID, the state (`R` = running), the CPU use and the **command** (`cmd`, the command line that started it, which shows what the program is). `ps -o user= -p $(pgrep yes)` shows only the owner, and `pgrep yes` shows only the PID. **Stop politely:** `kill -15` sends **SIGTERM**, a **graceful** stop: the program is asked to finish and clean up. **Check:** `pgrep yes || echo "gone"` prints "gone" when nothing is left (`||` means "if the first command finds nothing, run the second"). **Last resort:** `kill -9` sends **SIGKILL**, a **forceful** stop. The kernel removes the process at once and its clean-up never runs, so leftovers such as a lock file or a half-written file can remain; I use it only if a process ignores SIGTERM. **Record:** in a company, before stopping anything, I check who owns the

process and whether it is important, asking the owner if needed. Afterwards I write a GLPI ticket: what happened, who owned the process, what I did and the result.

4. Step 4.2 – pipes: output → input → output

💡 **The story:** a pipe (`|`) passes the output of one program straight into the next as its input – no file in between. Admins use it every day, and it saves real disk space when moving big data: compress and send in one go, instead of first writing a temporary archive.

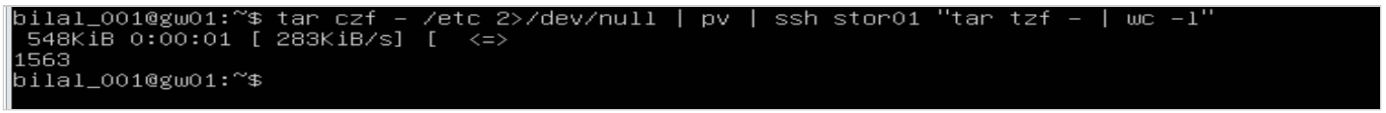
```
command 1 ──output──► | ──input──► command 2 ──output──► | ──input──► command 3 ──output──► screen
```

4.1 Hands-on – stream a compressed copy to another server (gw01 → stor01)

```
# gw01
sudo apt install -y pv
tar czf - /etc 2>/dev/null | pv | ssh stor01 "tar tzf - | wc -l"
```

Piece	Meaning
`tar czf - /etc`	**c**reate an archive, g**z**ip-compress it, **f**ile `-` = the pipe instead of a file
`2>/dev/null`	Hide the "permission denied" messages for root-only files
`pv`	**Pipe viewer** – shows the amount, speed (and % / ETA when the size is known)
`ssh stor01 "…"`	Carries the stream over the network and runs the command on stor01
`tar tzf - | wc -l`	On stor01: lis**t** what arrives and count the lines (`tar xzf - -C /data` would unpack = a real copy)

📸 **Evidence – 548 KiB streamed in 1 s; stor01 counted 1,563 files; nothing was stored on either server. `pv` shows `<=>` instead of % because a stream's size is not known in advance:**



tar pv ssh

Everyday pipe chains (`a | b | c`):

```
ps aux | sort -k3 -nr | head -3          # top 3 CPU users
grep bash /etc/passwd | wc -l            # how many users have a bash shell
pv shop.sql.gz | gunzip | mysql shop     # restore a dump with progress, no unpacked temp file
```

`|` (one bar) = pipe. `||` (two bars) = "or": run the second command only if the first fails – a different thing.

4.2 Reference only – named pipe (FIFO)

A FIFO is a **pipe with a name** – a tube between two programs **on the same server**. It **stores nothing** (size 0, type `p` in `ls -l`), data is gone once read (first in, first out), and the writer **waits** until a reader is there. It is needed only when two programs **cannot** be joined with `|` in one command line – a tool that accepts only a file name, or two programs started separately. Rare in daily admin work (examples: the Nagios/Icinga command file, some database export tools), so this part was kept as a reference and not run.

```
mkfifo /tmp/jobs                                                  # create the named pipe
ls -l /tmp/jobs                                                   # prw-r--r-- … 0 … /tmp/jobs
while read job < /tmp/jobs; do echo "processing: $job"; done      # terminal 1: the reader (worker)
echo "build-report" > /tmp/jobs                                   # terminal 2: the writer
rm /tmp/jobs                                                      # remove it

# the same transfer as above, as two separately started parts
mkfifo /tmp/stream
tar czf /tmp/stream /etc &                                        # writes "a file" — really the pipe
pv /tmp/stream | ssh stor01 "tar tzf - | wc -l"                   # reads it
rm /tmp/stream
```

Key understanding (in my own words)

A **pipe** (`|`) makes the output of one command the input of the next – **output → input → output** – with no file in between, like an assembly line, and only the last output reaches the screen. In `ps aux | sort -k3 -nr | head -3`, `ps` lists all processes, `sort` orders them by CPU and `head` keeps the top three. In real work pipes save time and disk space: on gw01, `tar czf - /etc | pv | ssh stor01 "tar tzf - | wc -l"` packed and compressed `/etc` into the pipe (`tar` packs many files into one stream: **c** create, **x** extract, **t** list, **z** gzip, **f -** = the pipe instead of a file), `pv` (**pipe viewer**) showed how much data passed and how fast (548 KiB in 1 second; a moving `<=>` instead of a percentage, because a stream's size is not known in advance), `ssh` carried the stream to stor01, and there `tar tzf -` listed what arrived and `wc -l` counted 1,563 files – nothing was stored on either server. With `tar xzf - -C /data` instead, stor01 would have

unpacked it, which is a real copy to a new server without any temporary archive; the pipe saves the temporary copy, but the target still needs space for what is finally stored there. `pv` is also used to show progress and time left for long jobs like restoring a database dump (`pv shop.sql.gz | gunzip | mysql shop`). One bar `|` is a pipe; two bars `||` mean "or" – run the second command only if the first fails. A **named pipe (FIFO)** is a pipe with a name in the filesystem: a tube between two programs on the same server that stores nothing (size 0, type `p` in `ls -l`), hands data over in order and only once, and makes the writer wait until a reader is there. It is needed only when two programs cannot be joined with `|` in one command line – for example a tool that accepts only a file name, or two programs started separately. It is rare in daily admin work, so I kept it as a reference: when I see a `p` at the start of an `ls -l` line, I know it is a pipe, not a normal file.

5. Next steps

Continued in Day 8.

1. VMware snapshot `40-automation` if not done yet.
2. Steps 4.3–4.5: signals and `trap` (why `kill -9` leaves a stale lock file), zombie processes, cgroups (`MemoryMax`, `CPUQuota`) – then the first incident drill.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation's *Connect to Server*.

Project 2 – Acme Analytics Linux Platform

Day 8 · Phase 4: signals, lock, trap, a real queue system with a worker, L1 → L2 escalation

Muhammad Bilal Javed · 25 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	stor01, svc01 (GLPI)
Status	Phase 4 🟡 – Steps 4.1, 4.2 ✅ (Day 7) · Step 4.3 🟡 – concept ✅, archive queue system Steps 1–7 ✅ (queue, producer, worker service, FIFO, request at any time, stop mid-job, L1 → L2 escalation with GLPI ticket #3); Step 8 (L3 Problem) and clean-up next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 7.

---

1. Step 4.3 – the concept: two copies, signals, lock and trap

💡 **In plain words:** jobs that run by themselves – nightly backups, exports – must never run **twice at the same time**, and when they are stopped they must not leave broken files behind. Three tools solve this: a **lock**, **signals** and **trap**.

Concept	Question it answers	Key line
Lock	Can a second copy start while the first is running?	`if [ -e "$LOCK" ]` · `flock -n` · a systemd service runs only once
Signal	How is a process stopped?	`kill -15` (SIGTERM) · Ctrl+C (SIGINT) · `kill -9` (SIGKILL)
trap	What is cleaned up when it is stopped?	`trap cleanup SIGINT SIGTERM`
`.part` file	How do I avoid a broken file that looks finished?	write to `file.part`, then `mv` when complete

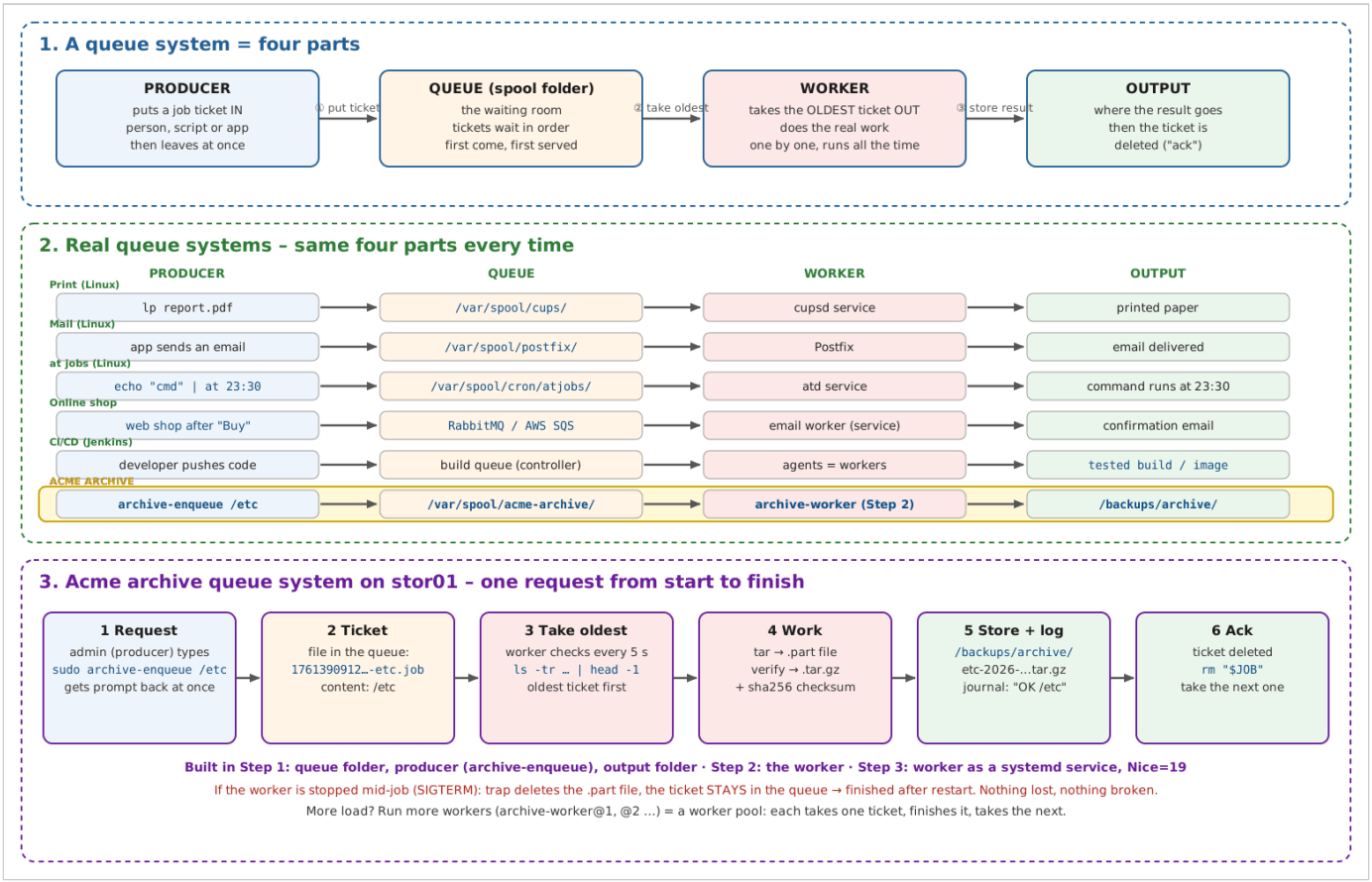
Key understanding (in my own words)

A nightly backup or export started by a **timer or cron** must never run **twice at the same time**: if the timer's backup runs from 02:00 to 03:00 and I start it by hand at 02:05, two copies would write to the same place and produce a broken result. The protection is a **lock** – a "someone is working on this right now" sign. Some programs have a lock **built in** (apt and dpkg – which is why a second apt, often Ubuntu's automatic updates running in the background, answers *"Could not get lock"*), but simple tools like `cp` or `rsync` and my own scripts have **none**, so I add it myself: a lock file checked at the start, `flock -n` (a kernel lock released automatically even after `kill -9`), or simply a systemd service, which never runs twice. A lock only stops a **second copy from starting**; it never stops anyone from terminating the job, and once the first copy has finished and removed its lock, the job can run again normally. A job is stopped by a **signal** from the kernel: **SIGTERM** (`kill -15`, "please stop"; `systemctl stop` sends it), **SIGINT** (Ctrl+C) or **SIGKILL** (`kill -9`, "stop now"). Stopping affects resources in two ways: **CPU and memory** are freed by the kernel whichever signal is used, but files the job created on **disk** – its lock, temporary files, a half-written archive – stay behind unless clean-up code removes them. **Linux processes have no clean-up by default**: the default action of SIGTERM is "die immediately", and SIGTERM only **gives the program a chance** to clean up. rsync, apt and databases have that code built in, while `cp`, `scp` and a plain Bash script do not – stopping a 20 GB `cp` halfway leaves a broken half-file that looks complete. **`trap`** adds clean-up code to a script: `trap cleanup SIGINT SIGTERM` means "on SIGTERM or Ctrl+C, first run my `cleanup` steps, then exit", and it runs only the steps I write. Trap does not stop or restart the process – the signal stops it, the trap cleans the disk before it ends, so the next start begins clean. **SIGKILL cannot be trapped**, so after `kill -9` the leftovers stay; a leftover lock is a **stale lock**, and the fix is always: prove nothing is running (`pgrep -af <job>`), remove the lock by hand, start the job again. Good jobs also write into a temporary **`.part` file** and rename it only when it is complete and verified, so a half-written file never looks finished.

2. Step 4.3 – the concept: queue systems, producers and workers

💡 **In plain words:** a **lock** says "busy, go away". A **queue with a worker** says "wait in line, I'll do it for you": everyone drops a request and leaves at once, and one program works through the requests one by one, at low priority, without losing any.

📸 **Architecture – a queue system has four parts; Linux already has several (print, mail, `at`), and the same pattern runs online shops and Jenkins:**



queue systems

Concept	Question it answers
Scheduler (timer, cron)	**When** does a fixed job start? – driven by time
Socket	Start a service **live** for each connection – the caller waits for the answer
Queue	In **what order** do waiting jobs run? – FIFO, first in, first out
Producer	Who **puts** a job ticket in the queue? – a person, a script or an application
Worker	Who **does** the jobs, one by one, any time? – a program that runs all the time
nice / ionice	How much **CPU / disk** may it use? – a different question from the order

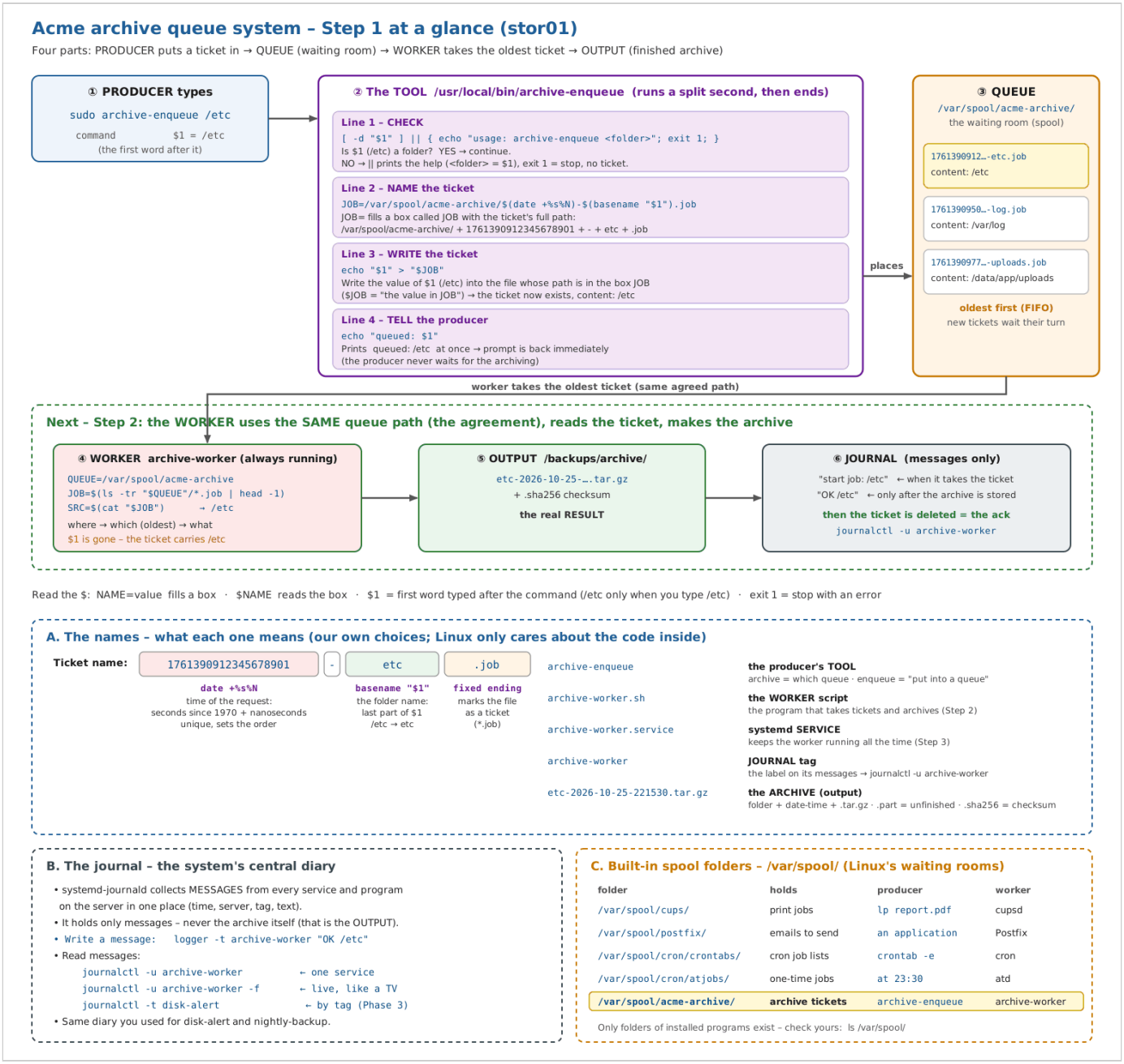
Key understanding (in my own words)

A **queue system** has four parts working together. A **producer** – a person, a script or an application – creates a **job ticket**, a small instruction such as *"archive /etc"*, places it in the **queue** and **gets control back at once**: it never waits for the job to be finished. The queue is a waiting room on disk called a **spool** folder; Linux keeps its own under **`/var/spool/`** – print jobs from `lp` wait in `/var/spool/cups/` for the worker `cupsd`, emails wait in `/var/spool/postfix/` for Postfix, `at` jobs wait in `/var/spool/cron/atjobs/` for `atd`. A **worker**, a program that runs all the time (usually a systemd service), takes the tickets **in order of arrival** – the oldest first, **FIFO**, first in, first out – does the work, stores the result in the agreed **output** place, and only then deletes the ticket (the **ack**, "done"), so if it fails or is stopped halfway the ticket stays and nothing is lost. Linux does not build a queue by itself: someone must place the tickets, and the worker must be written to take them out. Developers write workers; admins run and manage them – a systemd service with `Restart=on-failure`, more copies (a **worker pool**) when the load grows, logs in the journal, a safe stop with SIGTERM. Workers exist for three reasons: **nobody waits**, **the load stays under control**, **nothing is lost**. The same pattern runs a web shop (the shop puts *"send the confirmation email for order 501"* into RabbitMQ or AWS SQS and an email worker sends it) and **Jenkins** (the controller keeps the build queue, *"Waiting for next available executor"*, and agents are the workers). A **timer** is driven by **time** – one fixed job at a fixed time; a **socket** answers each **live connection** at once while the caller waits; a **worker** is driven by **stored requests** – any job, any time, one by one, while the requester has already left. Real automation grows step by step: do it by hand, script it, schedule it, and finally let a worker do it for everyone – repeating the same proven steps like a tireless user, without deciding anything new as an AI agent would. **`nice`** answers a different question from the queue: the queue decides the **order**, nice (-20 most important … 19 politest) decides **how much CPU** a running job gets.

3. Lab – the Acme archive queue system on stor01

💡 **The story:** at Acme, admins and scripts request archives (“archive /etc before I change the config”, “archive the uploads”). Instead of running heavy `tar` jobs by hand, they drop a **ticket** into a queue, and one **archive worker** turns the tickets into verified archives one by one, at low priority, so the customer application is never slowed down.

📸 **Architecture – one request from start to finish, the names, the journal and the built-in spool folders:**



step 1 overview

Part	Path
Producer’s tool	`/usr/local/bin/archive-enqueue`
Queue (spool folder)	`/var/spool/acme-archive/` – tickets like `1761390912345678901-etc.job`, content `/etc`
Worker	`/usr/local/bin/archive-worker.sh`, run as `archive-worker.service`
Output	`/backups/archive/` – `etc-2026-10-25-234228.tar.gz` + `.sha256`
Messages	the journal – `journalctl -u archive-worker`

3.1 Step 1 – the queue and the producer

```
# stor01
sudo mkdir -p /var/spool/acme-archive /backups/archive
sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }
JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
sudo chmod +x /usr/local/bin/archive-enqueue
```

Line	Meaning
`[ -d "$1" ] || { echo "usage: …"; exit 1; }`	$1 = the first word typed after the command (the folder); `-d` asks "is it a folder?"; if not, `||` prints the help (`<folder>` describes $1) and `exit 1` stops – no ticket
`JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job`	JOB= fills a box with the ticket's path: queue folder + time of the request (seconds since 1970 + nanoseconds – unique, sets the order) + folder name (`basename` keeps the last part) + `.job`
`echo "$1" > "$JOB"`	Write the folder path into the ticket file ($JOB = the value in the box) – the instruction for the worker
`echo "queued: $1"`	Tell the producer at once – the prompt is back immediately

📸 **Evidence:**

```
bilal_003@stor01:~$ sudo mkdir -p /var/spool/acme-archive /backups/archive
bilal_003@stor01:~$ sudo -v
bilal_003@stor01:~$ sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }

JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
bilal_003@stor01:~$ sudo bash -n /usr/local/bin/archive-enqueue
bilal_003@stor01:~$ sudo chmod +x /usr/local/bin/archive-enqueue
bilal_003@stor01:~$ _
```

step 1

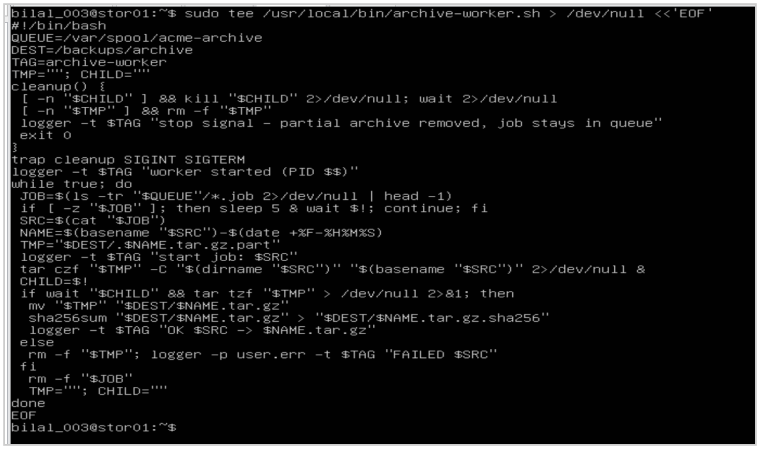
Lesson: my first version had `data` instead of `date`. `bash -n` did not catch it – it checks only the grammar of a script, not whether its commands exist. Only a test run proves a script works.

3.2 Step 2 – the worker

The full script is in the screenshot below. The lines that carry the concept:

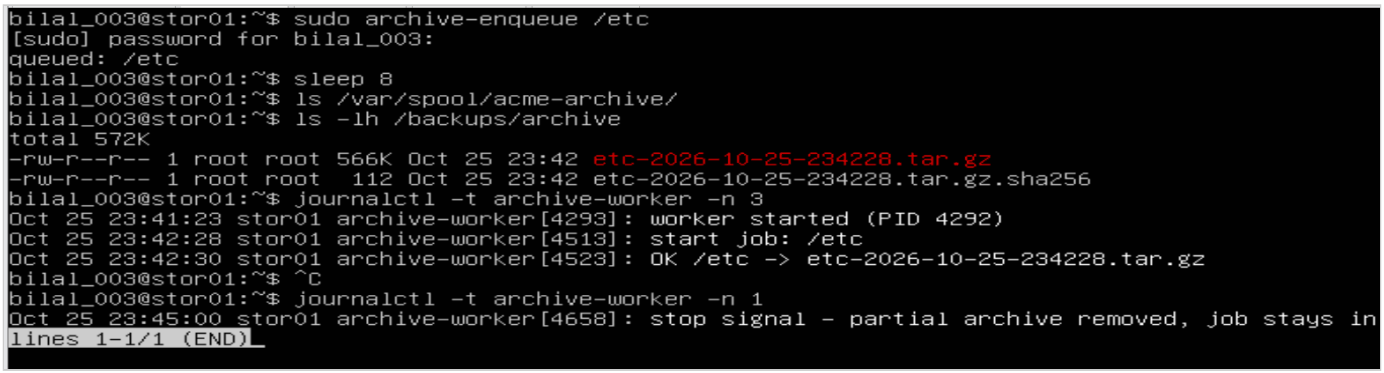
```
QUEUE=/var/spool/acme-archive                          # WHERE – the agreed queue folder (same path as the producer)
DEST=/backups/archive                                  # OUTPUT
trap cleanup SIGINT SIGTERM                            # TRAP – on SIGTERM/Ctrl+C: stop tar, delete the .part file
while true; do                                         # WORKER – runs forever
  JOB=$(ls -tr "$QUEUE"/*.job 2>/dev/null | head -1)   # WHICH – the oldest ticket first (FIFO)
  SRC=$(cat "$JOB")                                    # WHAT – read the folder from the ticket
  TMP="$DEST/.$NAME.tar.gz.part"                       # SAFE FILE – work into a .part file
  logger -t $TAG "start job: $SRC"                     # JOURNAL – message only
  mv "$TMP" "$DEST/$NAME.tar.gz"                       # OUTPUT – only a complete, verified archive gets the real name
  logger -t $TAG "OK $SRC -> $NAME.tar.gz"             # JOURNAL – "OK" only after the archive is stored
  rm -f "$JOB"                                         # ACK – delete the ticket
done
```

📸 **Evidence – the worker script:**



worker script

📸 Evidence – manual test: worker running in one terminal; in the other `queued: /etc` at once, the queue empty again (ack), `etc-2026-10-25-234228.tar.gz` (566 K) + `.sha256` in the output, the journal *worker started* → *start job* → *OK* within 2 seconds; Ctrl+C → the trap's *"stop signal"* message:



enqueue archive journal trap

Lessons: I first typed the worker by hand with five mistakes – `backups/archive` without the leading `/`, `[ -n "$CHILD"]` without the space before `]`, `SIGNIT`, `sha56sum` and, worst, **`*.JOB` instead of `*.job`**: Linux is case-sensitive, so the worker would have waited forever without any error. Long scripts are pasted, not retyped. The number in `archive-worker[4513]` is the PID of each `logger` call; the worker's own PID is the one in the message (4292).

3.3 Step 3 – the worker as a service, at low priority

```
sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
sudo systemctl daemon-reload
sudo systemctl enable --now archive-worker
systemctl status archive-worker --no-pager
ps -o pid,ni,cmd -C archive-worker.sh
```

Setting	Meaning
`ExecStart=`	The worker program
`Restart=on-failure`	Restart it automatically if it crashes
`Nice=19` / `IOSchedulingClass=idle`	Lowest CPU priority / use the disk only when nobody else needs it (= `nice -n 19`, `ionice -c3`)
`WantedBy=multi-user.target` + `enable`	Start at every boot – a link in `multi-user.target.wants/`

📸 Evidence – the unit; `verify` finds nothing in my file (the yellow line is Ubuntu's own `snapd.service`):

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
bilal_003@stor01:~$ sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
/lib/systemd/system/snapd.service:23: Unknown key name 'RestartMode' in section 'Service', ignoring
bilal_003@stor01:~$ _
```

service unit

📸 **Evidence – `active (running)`, Main PID 4839, 1.6 MB; the cgroup holds the worker and its child `sleep 5` (empty queue, waiting); NI 19:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now archive-worker
Created symlink /etc/systemd/system/multi-user.target.wants/archive-worker.service → /etc/systemd/sy
stem/archive-worker.service.
bilal_003@stor01:~$ systemctl status archive-worker
● archive-worker.service - Acme archive worker (processes the archive queue)
     Loaded: loaded (/etc/systemd/system/archive-worker.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-10-25 23:53:46 UTC; 22s ago
   Main PID: 4839 (archive-worker.)
      Tasks: 2 (limit: 964)
     Memory: 1.6M
        CPU: 130ms
     CGroup: /system.slice/archive-worker.service
             ├─4839 /bin/bash /usr/local/bin/archive-worker.sh
             └─4860 sleep 5

Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
bilal_003@stor01:~$ ps -o pid,ni,cmd -C archive-worker.sh
    PID  NI CMD
   4839  19 /bin/bash /usr/local/bin/archive-worker.sh
bilal_003@stor01:~$
```

service running

Key understanding – the lab so far (in my own words)

On stor01 I built the **Acme archive queue system**. In **Step 1** I created the **queue** `/var/spool/acme-archive/` (the waiting room for job tickets), the **output** `/backups/archive/` and the **producer's tool** `archive-enqueue` in `/usr/local/bin`, the folder for an admin's own commands. When I type `sudo archive-enqueue /etc`, `$1` – a box filled with the first word after the command – becomes `/etc`; `[ -d "$1" ]` checks it is a folder (otherwise `||` prints the help and `exit 1` stops); `JOB=` builds the ticket's path from the queue folder, the time of the request (`date +%s%N`) and the folder name (`basename`); `echo "$1" > "$JOB"` writes the folder into that ticket file, and *"queued: /etc"* comes back at once. The tool's name is only a label for humans – "enqueue" is the standard word for "put into a queue" – and what it does is decided only by its code. In **Step 2** I created the **worker** `archive-worker.sh`. It knows where to look because the **same queue path** is written at its top – producer and worker simply agree on it, as `cupsd` reads its spool folder from its settings file – and it reads the folder from the ticket with `cat`, because `$1` exists only for the split second the producer runs. In a manual test it turned my ticket into `etc-2026-10-25-234228.tar.gz` with a `.sha256` checksum within 2 seconds, wrote *start job* and *OK* to the **journal** (the server's central diary, which holds only messages, never the archive itself), deleted the ticket (the ack), and on Ctrl+C its trap logged *"stop signal"* before exiting. In **Step 3** I made it a **systemd service** with `Restart=on-failure`, `Nice=19` and `IOSchedulingClass=idle`; it now runs all the time as PID 4839 at **NI 19**, its cgroup showing it waiting (`sleep 5`) for the next ticket. Because a service runs only once, systemd itself is the lock – no second worker can start.

3.4 Step 4 – three jobs in the queue

```
# Terminal 1 (stor01 console): journalctl -u archive-worker -f
# Terminal 2 (ssh stor01 from gw01):
df -h /backups
sudo archive-enqueue /data/app/uploads
sudo archive-enqueue /etc
sudo archive-enqueue /var/log
ls /var/spool/acme-archive/
journalctl -t archive-worker -n 10
```

📸 **Evidence – three tickets waiting, named by request time so the oldest comes first:**

```
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  701M  2.1G  26% /backups
bilal_003@stor01:~$ sudo archive-enqueue /data/app/uploads
[sudo] password for bilal_003:
queued: /data/app/uploads
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sudo archive-enqueue /var/log
queued: /var/log
bilal_003@stor01:~$ ls /var/spool/acme-archive
1792974483564978473-uploads.job  1792974503083839724-etc.job  1792974511474086151-log.job
bilal_003@stor01:~$ ps -o pid,ni,cmd -C tar
    PID  NI CMD
bilal_003@stor01:~$
```

three tickets

📸 **Evidence – the live `-u` view shows *start job: /data/app/uploads* but not its *OK*:**

```
bilal_003@stor01:~$ journalctl -u archive-worker -f
Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
```

journal -u

📸 **Evidence – `journalctl -t` (by tag) shows every message: uploads 00:28:08 → OK 00:29:04 (56 s), `/etc` 1 s, `/var/log` 11 s – strictly in arrival order (FIFO), one at a time:**

```
bilal_003@stor01:~$ journalctl -t archive-worker -n 10
Oct 25 23:42:28 stor01 archive-worker[4513]: start job: /etc
Oct 25 23:42:30 stor01 archive-worker[4523]: OK /etc -> etc-2026-10-25-234228.tar.gz
Oct 25 23:45:00 stor01 archive-worker[4658]: stop signal - partial archive removed, job stays in qu>
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6586]: OK /data/app/uploads -> uploads-2026-10-26-002808.tar.>
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
Oct 26 00:29:16 stor01 archive-worker[6622]: OK /var/log -> log-2026-10-26-002905.tar.gz
lines 1-10/10 (END)
```

journal -t

Lesson: `logger` runs as a very short-lived process; sometimes it ends before the journal can link the message to the **service**, so `journalctl -u archive-worker` can miss a line. **`journalctl -t archive-worker`** filters by the **tag** set with `logger -t` and always shows them all.

3.5 Step 5 – a request at any time: worker vs timer

```
        sudo archive-enqueue /etc        # "archive /etc NOW – before I change the config"
        journalctl -t archive-worker -n 2
```

📸 **Evidence – queued, started and finished in the same second (00:34:12) – no waiting for a 02:00 timer:**

```
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sleep 3
bilal_003@stor01:~$ journalctl -t archive-worker -n 2
Oct 26 00:34:12 stor01 archive-worker[6922]: start job: /etc
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
bilal_003@stor01:~$ _
```

any time

3.6 Step 6 – stop the worker in the middle of a job (the trap test)

```
        sudo archive-enqueue /data/app/uploads
        sleep 5
        sudo systemctl stop archive-worker          # SIGTERM while tar is compressing 700 MB
        ls -la /backups/archive/                    # no .part file
        ls /var/spool/acme-archive/                 # the ticket is still there
        sudo systemctl start archive-worker
```

📸 **Evidence – after the stop: only complete archives in the output, no `.part` file:**

```
bilal_003@stor01:~$ sudo archive-enqueue /data/app/uploads
queued: /data/app/uploads
bilal_003@stor01:~$ sudo systemctl stop archive-worker
bilal_003@stor01:~$ ls -la /backups/archive
total 729364
drwxr-xr-x  2 root root       4096 Oct 26 00:36 .
drwxr-xr-x 12 root root       4096 Oct 25 10:51 ..
-rw-r--r--  1 root root     579024 Oct 25 23:42 etc-2026-10-25-234228.tar.gz
-rw-r--r--  1 root root        112 Oct 25 23:42 etc-2026-10-25-234228.tar.gz.sha256
-rw-r--r--  1 root root     579075 Oct 26 00:29 etc-2026-10-26-002904.tar.gz
-rw-r--r--  1 root root        112 Oct 26 00:29 etc-2026-10-26-002904.tar.gz.sha256
-rw-r--r--  1 root root     579075 Oct 26 00:34 etc-2026-10-26-003412.tar.gz
-rw-r--r--  1 root root        112 Oct 26 00:34 etc-2026-10-26-003412.tar.gz.sha256
-rw-r--r--  1 root root   10962843 Oct 26 00:29 log-2026-10-26-002905.tar.gz
-rw-r--r--  1 root root        112 Oct 26 00:29 log-2026-10-26-002905.tar.gz.sha256
-rw-r--r--  1 root root  734122617 Oct 26 00:28 uploads-2026-10-26-002808.tar.gz
-rw-r--r--  1 root root        116 Oct 26 00:29 uploads-2026-10-26-002808.tar.gz.sha256
bilal_003@stor01:~$ _
```

no part file

📸 **Evidence – the ticket stayed in the queue; the journal shows the trap's message:**

```
bilal_003@stor01:~$ ls /var/spool/acme-archive
1792974963837982875-uploads.job
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
Oct 26 00:36:08 stor01 archive-worker[7057]: start job: /data/app/uploads
Oct 26 00:36:43 stor01 archive-worker[7081]: stop signal - partial archive removed, job stays in qu>

bilal_003@stor01:~$ sudo systemctl start archive-worker
```

ticket stays

📸 **Evidence – the full story in the journal: start 00:36:08 → *Stopping…* → *stop signal – partial archive removed, job stays in queue* → *Deactivated successfully* (clean exit) → restart 00:38:24 (new PID 7094) → the same job → *OK* 00:39:32:**

```
bilal_003@stor01:~$ journalctl -u archive-worker -f
Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
Oct 26 00:34:12 stor01 archive-worker[6922]: start job: /etc
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
Oct 26 00:36:08 stor01 archive-worker[7057]: start job: /data/app/uploads
Oct 26 00:36:43 stor01 systemd[1]: Stopping Acme archive worker (processes the archive queue)...
Oct 26 00:36:43 stor01 archive-worker[7081]: stop signal - partial archive removed, job stays in que
ue
Oct 26 00:36:43 stor01 systemd[1]: archive-worker.service: Deactivated successfully.
Oct 26 00:36:43 stor01 systemd[1]: Stopped Acme archive worker (processes the archive queue).
Oct 26 00:36:43 stor01 systemd[1]: archive-worker.service: Consumed 2min 1.131s CPU time.
Oct 26 00:38:24 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 26 00:38:24 stor01 archive-worker[7095]: worker started (PID 7094)
Oct 26 00:38:24 stor01 archive-worker[7102]: start job: /data/app/uploads
Oct 26 00:39:32 stor01 archive-worker[7125]: OK /data/app/uploads -> uploads-2026-10-26-003824.tar.g
z
```

stop restart ok

📸 **Evidence – the queue empty again after the restart:**

```
bilal_003@stor01:~$ ls /var/spool/acme-archive
1792974963837982875-uploads.job
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
Oct 26 00:36:08 stor01 archive-worker[7057]: start job: /data/app/uploads
Oct 26 00:36:43 stor01 archive-worker[7081]: stop signal - partial archive removed, job stays in qu>
bilal_003@stor01:~$ ls /var/spool/acme-archive
bilal_003@stor01:~$ ls -lh /var/spool/acme-archive
total 0
bilal_003@stor01:~$ _
```

queue empty

3.7 Step 7 – a deliberate escalation: L1 finds it, L2 fixes it

💡 **The story:** a colleague left manual database dumps in `/backups`, filling it. The archive worker fails. As **L1** I find the cause and collect evidence, but I am not allowed to delete someone else's data or resize volumes – so I **escalate**. Then, as **L2 – Linux Storage**, I get the owner's approval, fix it and close the ticket.

```
        # create the problem
        sudo fallocate -l $(df --output=avail -B1 /backups | tail -1) /backups/db-dump-manual.sql
        sudo archive-enqueue /etc                          # still OK — see the lesson below
        sudo fallocate -l $(( $(stat -f -c '%f' /backups) * $(stat -f -c '%S' /backups) )) /backups/db-dump-manual-2.sql
        sudo archive-enqueue /var/log                      # now FAILED

        # L1 — look, identify, do not fix outside the role
        journalctl -t archive-worker -n 3
        df -h /backups
        sudo du -sh /backups/* | sort -h | tail -3
        ls -l /backups/db-dump-manual*.sql
```

📸 **Evidence – L1: `FAILED` `/var/log`, `/backups` 100 %, the two dump files (owner root, 00:44 and 00:52):**

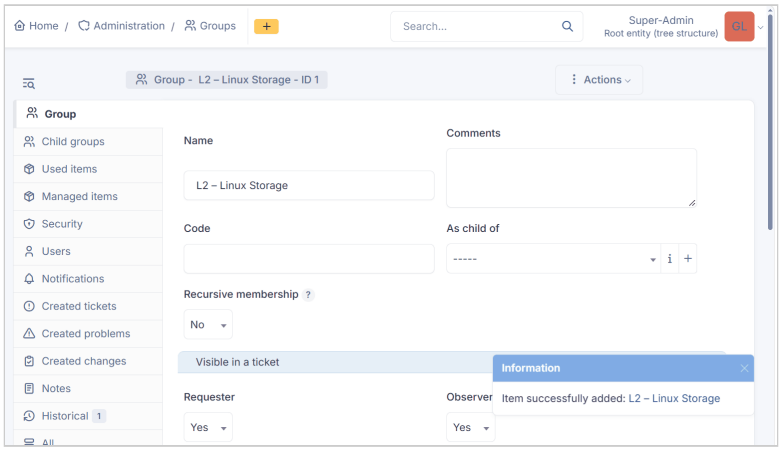
```
bilal_003@stor01:~$ sudo fallocate -l $(df --output=avail -B1 /backups | tail -1) /backups/db-dump-m
anual.sql
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sleep 5
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  2.8G     0 100% /backups
bilal_003@stor01:~$ sudo fallocate -l $(($(stat -f -c '%f' /backups)* $(stat -f -c '%S' /backups)))
/backups/db-dump-manual-2.sql
fallocate: fallocate failed: No space left on device
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  2.9G     0 100% /backups
bilal_003@stor01:~$ sudo archive-enqueue /var/log
queued: /var/log
bilal_003@stor01:~$ sleep 5
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:44:38 stor01 archive-worker[7408]: OK /etc -> etc-2026-10-26-004437.tar.gz
Oct 26 00:53:25 stor01 archive-worker[7872]: start job: /var/log
Oct 26 00:53:25 stor01 archive-worker[7879]: FAILED /var/log
bilal_003@stor01:~$ sudo du -sh /backups/* | sort -h | tail -3
669M    /backups/db-dump-manual.sql
701M    /backups/2026-10-21
1.4G    /backups/archive
bilal_003@stor01:~$ ls -l /backups/db-dump-manual*.sql
-rw-r--r-- 1 root root 160464896 Oct 26 00:52 /backups/db-dump-manual-2.sql
-rw-r--r-- 1 root root 701198336 Oct 26 00:44 /backups/db-dump-manual.sql
bilal_003@stor01:~$
```

l1 evidence

Lesson – the 5 % root reserve: after the first dump `df` already showed **Avail 0 / 100 %**, yet the `/etc` archive still succeeded. ext4 keeps about **5 % reserved for root**; `df`'s "Avail" and "Use%" count only the space normal users can have, and the worker runs as root. Only after the second file filled **all** free blocks (`stat -f`) did the job fail. In a real incident this is why root services often keep working a little longer while users already see *"No space left on device"*.

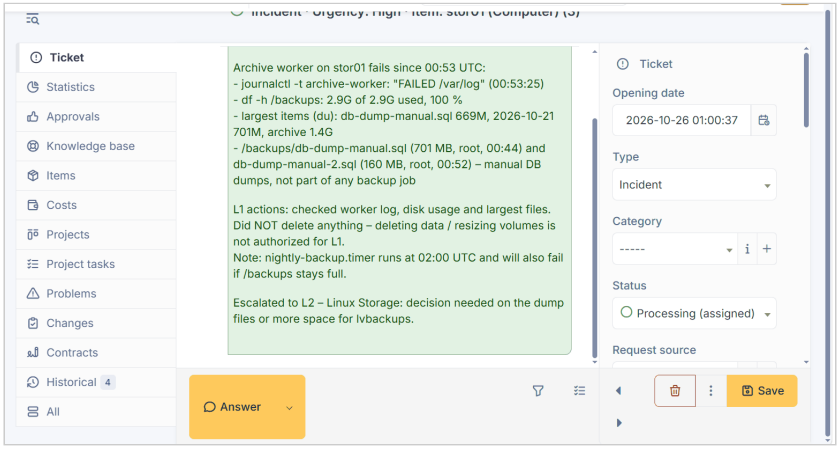
L1 in GLPI: a new group **L2 – Linux Storage**, and an **Incident** with Urgency **High**, Item **stor01**, the evidence, the L1 actions (*"did NOT delete anything – not authorized"*), a note that the 02:00 nightly backup would also fail, and **Assigned to: group L2 – Linux Storage**.

📸 **Evidence – the L2 group:**



group

📸 **Evidence – ticket #3 escalated: Incident · Urgency High · Item stor01 · Processing (assigned) to the L2 group:**

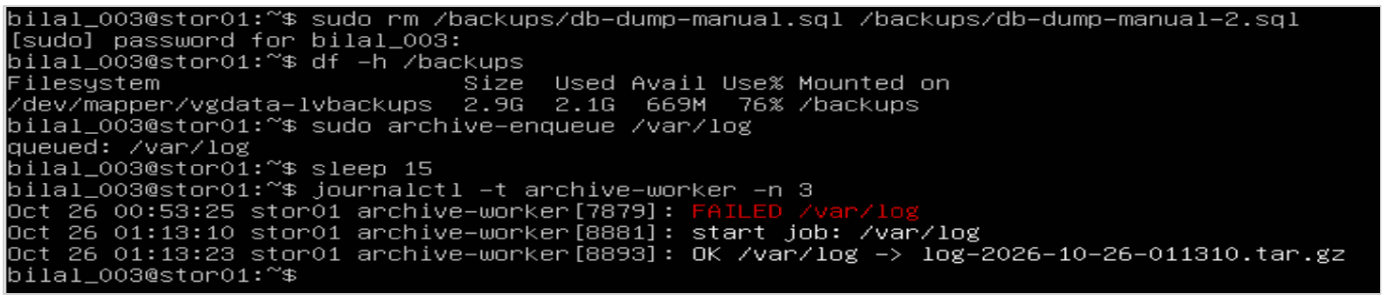


ticket 3

L2: a followup recording the owner's approval (*"both dumps are leftovers from a manual test – deletion approved"*), then the fix:

```
sudo rm /backups/db-dump-manual.sql /backups/db-dump-manual-2.sql
df -h /backups                          # 76 %, 669 MB free
sudo archive-enqueue /var/log           # re-queue the failed job
journalctl -t archive-worker -n 3
```

📸 **Evidence – space back, the failed job re-queued and `OK /var/log` at 01:13:23:**



l2 fix

Then the **solution** in ticket #3 – cause, decision, fix, verification, prevention – and the ticket was closed.

Role	Did	Did not
L1 – service desk	Found the cause (journal, `df`, `du`, `ls`), wrote the evidence, escalated to the right group	Delete data or change storage – not authorized
L2 – Linux Storage	Got the owner's approval in writing, removed the dumps, re-ran the job, verified, documented and closed	Fix the root cause in the design – that is L3's work (next)

Lesson – a lost job: the `/var/log` ticket disappeared after `FAILED`, because the worker deletes the ticket after a failure too. Real queue systems move failed jobs to a **failed queue** (a *dead-letter queue*) so they can be retried. That is a design improvement for **L3**.

Key understanding – Steps 4 to 7 (in my own words)

> With the worker running as a service, I queued three requests – `/data/app/uploads` (about 700 MB), `/etc` and `/var/log`. Each `archive-enqueue` answered *"queued"* at once, and the worker processed them strictly **in arrival order (FIFO)** and **one at a time**: uploads in 56 seconds, `/etc` the moment it finished, then `/var/log`; the later tickets simply waited. `journalctl -u archive-worker` missed one *OK* line, because a message from a very short-lived `logger` process cannot always be linked to the service, while `journalctl -t archive-worker` – filtering by the tag – showed every message. A request at any time (*"archive /etc now"*) was queued, started and finished within the same second, something a 02:00 timer cannot do. Then the real test: I stopped the worker with `systemctl stop` while it was compressing 700 MB. systemd sent SIGTERM, the worker's **trap** logged *"stop signal – partial archive removed, job stays in queue"*, the service was *Deactivated successfully*, `/backups/archive/` contained **no `.part` file**, and the uploads ticket was **still in the queue** – because the ticket is deleted only after success. After `systemctl start`, the new worker took that same ticket and finished it: nothing broken, nothing lost. Finally I practised a deliberate **escalation**. Two forgotten manual database dumps filled `/backups` – the first only up to the **5 % root reserve**, so `df` showed 100 % while the root-run worker could still write, and only the second made the job fail with *FAILED /var/log*. As **L1** I found the cause with `journalctl`, `df`, `du` and `ls`, did **not** delete someone else's data or resize the volume, and opened a High-urgency GLPI incident with all evidence, assigned to the group **L2 – Linux Storage**. As **L2** I recorded the owner's approval as a followup, removed the dumps (76 % used again), re-queued the failed job (*OK /var/log*), and wrote the solution – cause, decision, fix, verification, prevention – before closing the ticket. The failed `/var/log` ticket had been deleted by the worker, which showed a real design gap: production queues keep failed jobs in a **dead-letter queue** for retry. **L1's job is not to fix everything, but to find the cause, collect clear evidence, stay within its authority and hand over well; L2 fixes on the servers; L3 removes the root cause.**

4. Next steps

1. Step 4.3 Step 8: **L2 → L3** – a GLPI **Problem** for the root cause (dumps stored on the backup volume, no space monitoring on `/backups`, failed jobs deleted instead of kept), linked to incident #3, assigned to **L3 – Engineering**, status **Pending** with a workaround. Then Step 9: clean up `/backups/archive`.
2. Step 4.4 zombie processes (optional) · Step 4.5 cgroups (`MemoryMax`, `CPUQuota`) · the first incident drill · VMware snapshot `50-processes`.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation's *Connect to Server*.

Project 2 – Acme Analytics Linux Platform

Day 9 · Phase 4: Step 4.3 clean-up · Step 4.4 how processes are born – PID, fork, exec, wait, orphan, zombie

Muhammad Bilal Javed · 26 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	stor01 (clean-up), app01 (process lab)
Status	Phase 4 🟡 – Steps 4.1, 4.2 ✅ · Step 4.3 ✅ Steps 1–7 and Step 9 clean-up (Step 8, the L3 Problem in GLPI, still open) · Step 4.4 ✅ how processes are born (hands-on) + zombies (concept and reference – rare in practice, so no separate lab) · next: Step 4.5 cgroups
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 8.

1. Step 4.3 Step 9 – clean-up after the queue lab (stor01)

💡 **In plain words:** after the escalation drill of Day 8 the backup volume must be back to normal, and the real nightly backup must still work.

🔧 **Technical:** the nightly backup timer ran by itself at **02:00:07** and finished successfully; the test archives in `/backups/archive/` were removed; `/backups` is back to **26 %**.

```
# stor01
journalctl -u nightly-backup.service --since "today" --no-pager | tail -5
sudo rm -r /backups/archive/*
df -h /backups
```

📸 **Evidence:**

```
bilal_003@stor01:~$ journalctl -u nightly-backup.service --since "today" --no-pager | tail -5
Oct 26 02:00:07 stor01 systemd[1]: Starting Nightly backup of /data/app...
Oct 26 02:00:07 stor01 nightly-backup[11231]: backup of /data/app to /backups/2026-10-26 finished
Oct 26 02:00:07 stor01 systemd[1]: nightly-backup.service: Deactivated successfully.
Oct 26 02:00:07 stor01 systemd[1]: Finished Nightly backup of /data/app.
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  701M  2.1G  26% /backups
bilal_003@stor01:~$
```

Nightly backup at 02:00:07 succeeded; /backups back to 26 %

2. Step 4.4 – the concept: where processes and PIDs come from

2.1 What is a process and a PID?

💡 **In plain words:** a **process** is a program that is running right now – my bash, `ls`, `tar`, sshd, the archive worker. Every process gets a **PID** (its own number) and a **PPID** (the number of its parent, the process that started it). **Every process has a parent**, all the way up to **systemd, PID 1**.

2.2 The PID pool

The **kernel** hands out PIDs like a **ticket machine**: the next number goes to whatever starts – systemd, kernel threads, services, my shell, every command. There is **one pool for everything**, up to `/proc/sys/kernel/pid_max` (about 4 million on Ubuntu, 32,768 on older systems). Numbers are given in **increasing order**; a freed number is **not reused straight away** – only when the counter reaches pid_max does it wrap around to low numbers, skipping the ones still in use. In `/proc/loadavg` (e.g. `0.01 0.07 0.03 1/238 1823`) the fields are: load over 1/5/15 minutes · running/total tasks (threads included) · the last PID given.

```
cat /proc/sys/kernel/pid_max      # size of the pool
cat /proc/loadavg                 # last number = the PID given most recently
ps -e --no-headers | wc -l        # how many PIDs are in use right now
```

2.3 How a process is born – fork, exec, wait

Word	Meaning	Picture
fork	the parent makes a **copy of itself** – a new process with a **new PID**	a cell divides in two
exec	"**replace me**" – the process turns into another program, **same PID**, no new process	same person, new clothes
wait	the parent **reads the child's exit note** when it finishes → the **PID is freed**	collecting the receipt

When I type `ls`, bash **forks**, the copy **execs** into `ls`, `ls` finishes and leaves an exit note, bash **waits** (reads it) – that is why the prompt only returns after `ls` has ended.

2.4 From boot to my shell on app01

```
kernel → systemd (1) → getty@tty1 —exec→ login (911) —fork→ bash (1295) → my commands
```

tty1 is not a process – it is the terminal (the VMware console screen and keyboard). systemd starts the service **getty@tty1**, which shows "app01 login:". When I type my **username** (before the password), getty **execs into login** – the same process (911) changes its program. After the correct password, login **forks** and the copy becomes **my bash (1295)**.

The kernel starts **only systemd**; systemd starts getty and every other service. **getty is a process**, **tty1 is the terminal** (tty1, tty2 … – Ctrl+Alt+F2 opens tty2). After the password **login stays alive in the background** as bash's parent (pstree shows `login(911)──bash(1295)`) – I only see bash.

```
while I work:  ls ends   → leaves an exit note → bash reads it  → back to the prompt
when I exit:   bash ends → leaves an exit note → login reads it → login ends
               → systemd starts a fresh getty → new "login:" screen (new PIDs)
```

A living process has no exit note – it is written only when the process ends, and each level reports to the one above it.

2.5 Why fork + exec? bash works through a representative

```
bash (1295) ── asks the KERNEL "copy me" (fork) ──► representative (new PID 4810)
     │                                    exec ──► becomes ls (still 4810)
     │                                finishes → leaves the exit note
     └─ told (SIGCHLD) → READS the note (wait) → 4810 freed → prompt back
```

exec **replaces** the process that runs it – if bash exec'd into `ls` directly, bash itself would become `ls` and be gone when `ls` ends (the terminal would close). So bash **never does the work itself**: it asks the **kernel** to make a copy of it (environment, folder, open files) with a **new PID**; the copy execs into the command and keeps that PID; bash stays alive, is told when the child finishes (SIGCHLD) and **reads the exit note**. The child only *leaves* the note – the **parent reads** it. Safe proof: run `bash`, then `exec ls` – the inner bash disappears and `echo $$` shows the first bash again.

- **Built-ins** (`cd`, `echo`, `exit`) are done by bash itself – no new PID (`type cd` → "shell builtin", `type ls` → `/usr/bin/ls`).
- **Not every program forks** – only one that starts other programs: bash (commands), systemd (services), sshd (logins), the archive worker (→ `tar`). `ls` starts nothing, so it is the last link.

---

3. Hands-on on app01 – watching processes being born

3.1 Who am I?

```
echo $$                    # $$ = the PID of this shell
```

📸 **Evidence:**

echo $$ prints 1295, the PID of my bash

3.2 My family tree

```
pstree -ps $$              # -p = show PIDs, -s = show all parents
```

📸 **Evidence:**

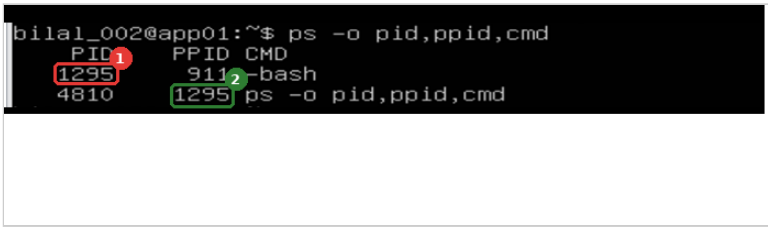
systemd(1) → login(911) → bash(1295) → pstree(4808)

Every command I run is a **child of my shell**.

3.3 Watch a birth

```
        ps -o pid,ppid,cmd          # ps = process status, -o = which columns
```

📸 **Evidence:**



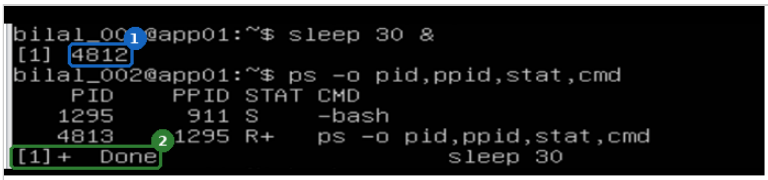
1295 911 -bash, and ps with PPID 1295

`1295 911 -bash` = “process 1295 is bash, its **parent** is 911 (login)”; the minus marks a **login shell**. `ps` has PPID **1295** – bash forked, the copy exec’d into `ps`.

3.4 A child in the background – and a good parent

```
        sleep 30 &                  # & = run in the background, prompt comes back at once
        ps -o pid,ppid,stat,cmd
```

📸 **Evidence:**



sleep 4812 in the background; [1]+ Done – no zombie

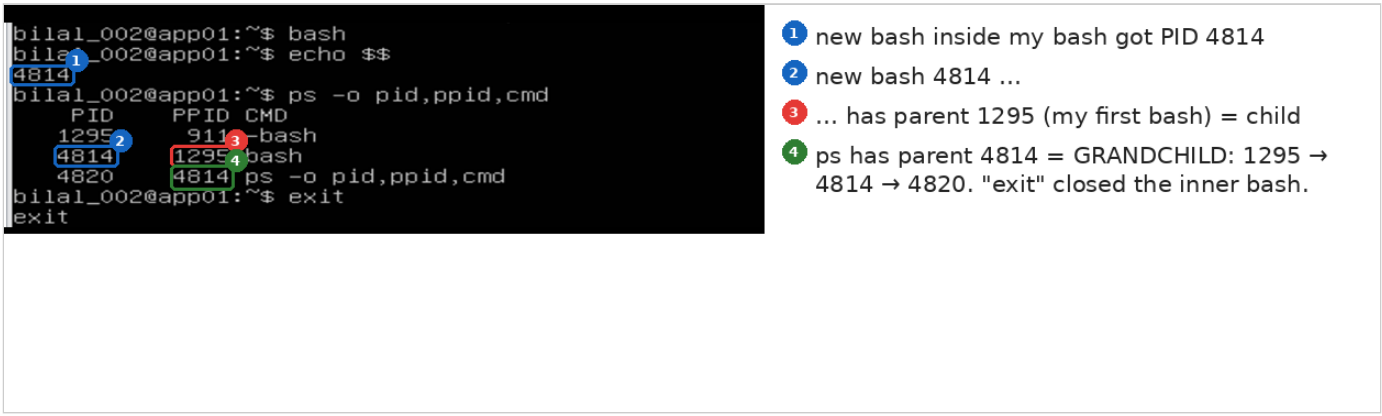
`[1]+ Done` (job 1, `+` = most recent) means bash **read the child’s exit note** – the PID was freed at the moment sleep finished (Linux tells bash with **SIGCHLD**); “Done” is only bash’s report at the next prompt. **No zombie: bash is a good parent.**

STAT	R	S	T	Z	+	s
meaning	running	sleeping (normal)	paused (Ctrl+Z)	zombie	foreground	session leader

3.5 A child of a child

```
        bash                        # a new bash inside my bash — waits for me until exit
        echo $$
        ps -o pid,ppid,cmd
        exit
```

📸 **Evidence:**



new bash 4814 with parent 1295; ps 4820 with parent 4814

Chain of three generations: **1295 → 4814 → 4820**.

3.6 exec – the copy turns into another program (reference)

```
bash -c 'echo "my PID is $$"; exec sleep 30' &     # -c = run these commands, then end
ps -o pid,ppid,cmd                                 # sleep has the SAME PID that echo printed
```

`bash -c` starts a new child bash that runs the commands in the quotes and ends by itself; `;` separates the two commands; `exec sleep 30` turns that bash into `sleep` **without a new PID**. The same trick is used by getty → login, and by Docker start scripts (`exec python app.py`) so the app becomes PID 1 and receives SIGTERM directly.

3.7 Orphan – the parent dies, the child keeps running

```
bash -c 'sleep 60 &'       # the inner bash starts sleep, then ends at once
ps -o pid,ppid,stat,cmd -C sleep
```

📸 **Evidence:**

sleep 60 with PPID 1 and STAT S – an orphan adopted by systemd

PPID 1 – systemd adopted it; **STAT S** – still alive. That is an **orphan, not a zombie**.

---

4. When is a PID freed – and when does it stay?

Case	Child	Parent	What I see	Fix
Normal	finished	read the exit note	`[1]+ Done`, gone	nothing
Orphan	**alive**, keeps working	died → **systemd adopts**	PPID **1**, STAT S/R	nothing
Zombie	**dead** (finished)	alive, **never read the note**	STAT **Z**, <defunct>	**restart the parent** – `kill -9` does nothing

	Services (sshd, cron, archive-worker)	Commands (ls, cp, tar)
When the PID is taken	at boot, first → usually small numbers	later → bigger numbers
Parent	systemd, PPID 1	my bash
How long	days, until stopped; a restart gives a new PID	seconds or minutes; a new PID every run

When do zombies happen in real life? Any process can become a zombie – but only **after it has finished**, and it depends on the **parent**, never the child: (1) a badly written program that forgot `wait()`, (2) a parent that is stuck (e.g. waiting on a dead NFS share), (3) a container without an init process (fix: `docker run --init`). Every zombie keeps its PID, so thousands of them can empty the pool → `fork: Resource temporarily unavailable`.

---

5. Zombies – essentials

A zombie is rare in daily operations, so this step is kept as **concept + reference** instead of a full lab. What an admin must be able to do: **know it, recognise it, fix it**.

1. Every process leaves an **exit note** when it ends (its exit code – `echo $?`), and the **parent must read it**.
2. **Not read → zombie**: STAT **Z**, <defunct>.
3. A zombie holds only a **PID** – no CPU, no memory.

4. **Find it:** `ps -eo pid,ppid,stat,cmd | grep -w Z` (or the "zombie" count in `top`) – its **PPID** points to the parent.
5. **Fix it:** restart the **parent** – never `kill -9` the zombie (it is already dead).

📸 Reference (example output from the lab guide – PID numbers differ on a real server):

EXAMPLE - what app01 should show (your PID numbers will be different)

```
bilal_002@app01:~$ ps -eo pid,ppid,stat,cmd | grep -E "STAT| Z|defunct"
    PID    PPID STAT CMD
   5013    5012 Z    [python3] <defunct>
   5016    1295 S+   grep --color=auto -E STAT| Z|defunct
bilal_002@app01:~$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   1295     911 Ss   -bash
   5012    1295 S    python3 /home/bilal_002/zombie.py
   5013    5012 Z    [python3] <defunct>
   5020    1295 R+   ps -o pid,ppid,stat,cmd
```

1. the zombie child, PID 5013
2. its parent is 5012 = python3 zombie.py
3. STAT Z = zombie
4. <defunct> = "dead". The process is finished, only its exit note waits
5. family view: bash 1295 → python3 5012 (alive, sleeping) → 5013 (Z, dead)

Example: a zombie in ps – STAT Z, defunct, PPID points to the parent

EXAMPLE - what app01 should show (your PID numbers will be different)

```
bilal_002@app01:~$ kill 5012
[1]+  Terminated              python3 ~/zombie.py
bilal_002@app01:~$ ps -eo pid,ppid,stat,cmd | grep defunct
   5027    1295 S+   grep --color=auto defunct
bilal_002@app01:~$ top -bn1 | head -3 | grep Tasks
Tasks: 110 total,   1 running, 109 sleeping,   0 stopped,   0 zombie
bilal_002@app01:~$ rm ~/zombie.py
```

1. stop the PARENT (5012), the real fix
2. only the grep line itself is left → the zombie is GONE. systemd (PID 1) adopted it and read its exit note at once
3. top: "0 zombie"

Example: after stopping the parent the zombie is gone – top shows 0 zombie

Points to ponder – key clues

Clue	What it tells me
PID vs PPID	the process itself vs who started it – the PPID is how I find the responsible parent
PID 1 = systemd	starts every service and adopts every orphan
`ls` = fork + exec + wait	bash asks the kernel for a copy (new PID), the copy becomes `ls`, bash reads its exit note
fork vs exec	fork = new copy, new PID · exec = same process, new program, same PID
Built-ins (`cd`, `echo`)	run inside bash – no new PID (`type cd`)
STAT Z, `<defunct>`	a finished child whose parent never read the note
"No CPU, no memory"	a zombie only blocks a PID – harmless alone, dangerous by the thousand
`fork: Resource temporarily unavailable`	PIDs are used up – look for a zombie leak, even SSH logins can fail
`kill -9` does nothing	the zombie is already dead – act on the **parent**
Zombie vs orphan	zombie = dead child, living parent · orphan = living child, dead parent → PPID 1
Process tree	`pstree -p` or `ps -ef --forest` shows who belongs to whom

6. Key understanding (in my own words)

Every running program is a **process** with a **PID** (its own number) and a **PPID** (its parent's number). The kernel hands out PIDs like a **ticket machine**, in **increasing order**, up to `pid_max` (about 4.2 million on Ubuntu); a freed number is reused only after the counter wraps around. At boot the kernel starts **only systemd (PID 1)**, and systemd starts every service – including **getty**, a process that watches the terminal **tty1** and shows "login:". When I type my username, getty **execs** into **login** (same PID); after the password, login **forks** a copy, the copy execs into my **login shell** `-bash`, and login waits in the background as its parent. bash never runs a program itself: it asks the **kernel** to make a **copy** of it with a **new PID** (**fork**), the copy turns into the command and keeps that PID (**exec**), and bash stays alive; built-ins like `cd` and `echo` are the exception – bash does them itself. When a child ends, it automatically leaves an **exit note** (its exit code, `echo $?`); the kernel tells the parent with **SIGCHLD**, the parent **reads** the note (**wait**), and the PID is freed. `exit` closes the chain the same way: bash ends, login reads its note and ends, and systemd starts a fresh getty. An **orphan** is a child that is **still running** after its parent died – systemd adopts it (PPID 1). A **zombie** is a child that has **finished** but whose parent **never read the note** – a bug (no `wait()`), a stuck parent, or a container without init. It shows **STAT Z, `<defunct>`**, uses no CPU or memory and only holds its PID – rare, but thousands of zombies can use up all PIDs, and then nothing new can start, not even an SSH login. `kill -9` cannot remove a zombie because it is already dead; **the fix is the parent** – find it through the zombie's PPID, restart that service, and systemd adopts the zombie and cleans it up, then open a ticket for the developers.

7. Next

1. Step 4.5 – cgroups: limit CPU and memory of a process and of a service (app01, stor01).
2. First incident drill – “disk full, but nothing is found” (stor01).
3. Step 4.3 Step 8 – the L3 Problem in GLPI, linked to ticket #3 · snapshot `50-processes`.

Project 2 – Acme Analytics Linux Platform

Day 10 · Phase 4: Step 4.5 concept – cgroups, cache and swap, container networking

Muhammad Bilal Javed · 1 November 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	gw01 (PID pool check) · app01 (lab Step 1 – the cgroup tree)
Status	Phase 4 🟡 – Steps 4.1–4.4 ✅ · Step 4.5 🟡 concept ✅ (cgroups, nice vs limits, throttling, MemoryHigh/MemoryMax, cache and swap, Docker/Kubernetes limits, who creates the box, container networking) · lab Step 1 ✅ (cgroup tree on app01, where limits can be set and where they are stored) – Steps 2–7 next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot or diagram. Terms are explained in the glossary. Previous day: Day 9.

1. Warm-up – the PID pool on gw01

```
cat /proc/sys/kernel/pid_max      # 4194304 — size of the PID pool
cat /proc/loadavg                 # 0.01 0.07 0.03 1/238 1823
ps -e --no-headers | wc -l        # 209 processes
```

`/proc/loadavg` = load over 1 / 5 / 15 minutes (almost zero – gw01 is idle) · **1/238** = 1 task running out of 238 (threads included) · **1823** = the PID handed out most recently. `ps -e` counts **processes** only (209), so it is lower than 238.

2. Step 4.5 – the concept: cgroups

💡 **In plain words:** `nice` decides **who goes first**; a **cgroup** decides **how much at most**. Every service runs in its own **box**, and a limit on the box is a hard ceiling for every process inside it – so one bad job cannot freeze the whole server.

2.1 nice vs cgroup limits

	nice	cgroup limits
works on	each **process**	the whole **box** (service)
means	priority – who goes first, no ceiling	hard ceiling – how much at most
in a unit file	`Nice=19` = start value for every process of the service (children inherit it; `renice -n 10 -p <PID>` changes one)	`CPUQuota=` · `MemoryHigh=` · `MemoryMax=` · `TasksMax=` – shared by all processes in the box

2.2 Who creates the box

- **Linux service: systemd** reads the unit file, starts the service, creates its box (`/sys/fs/cgroup/system.slice/<name>.service/`) and writes the limits. **One service = one box.**
- **Docker:** `dockerd` → `containerd` (the manager) → `runc` (the builder: **namespaces** = isolation, **cgroup** = limits).
- **Kubernetes:** `kubelet` → `containerd` → `runc` – the same Linux cgroups underneath.

2.3 What each limit does when it is exceeded

Limit	Over the limit
`CPUQuota=30%`	**throttled** – the box runs only 30 ms of every 100 ms, like mobile data after the monthly limit: slower, never killed. Other services in their own boxes are **not** slowed.
`MemoryHigh=150M` (soft)	the box is **slowed down**: step 1 – the **cache** (copies of files kept in RAM; originals on disk) is deleted from RAM · step 2 – idle **program memory** is moved from RAM to **swap**, a reserved area **on the disk** (Ubuntu `/swap.img`, Windows `pagefile.sys`). Normal files on the disk are never touched; nothing is killed.
`MemoryMax=200M` (hard)	the kernel's **OOM killer** stops a process **in this box only**
`TasksMax=20`	a new fork **fails** inside this box only ("Resource temporarily unavailable") – protects the PID pool
`MemoryMin=` / `MemoryLow=`	the opposite: a **guaranteed** / **protected** minimum for the box

2.4 The archive worker as an example (stor01)

```
        [Service]
        ExecStart=/usr/local/bin/archive-worker.sh
        Nice=19                 # priority — per process
        IOSchedulingClass=idle
        CPUQuota=30%            # limits — for the whole box
        MemoryHigh=150M
        MemoryMax=200M
        TasksMax=20
```

Set them in the unit file (`daemon-reload` + `restart`) or with `sudo systemctl set-property archive-worker CPUQuota=30% MemoryMax=200M`; check with `systemctl show archive-worker -p Nice -p CPUQuotaPerSecUSec -p MemoryMax -p TasksMax`.

📸 **Diagram:**

cgroups – how a service gets its box and its limits

Project 2 · Phase 4 · Step 4.5 · example: archive-worker.service on stor01

① unit file (what I write)

```
[Service]
ExecStart=/usr/local/bin/archive-worker.sh

# priority – per PROCESS
Nice=19
IOSchedulingClass=idle

# limits – for the whole BOX
CPUQuota=30%
MemoryHigh=150M
MemoryMax=200M
TasksMax=20
```

reads

② systemd (PID 1)

starts the service
creates the box · writes the limits

creates + applies limits

③ the box (cgroup)

`/sys/fs/cgroup/system.slice/archive-worker.service/`

archive-worker.sh	tar czf …	gzip
PID 812 · nice 19	PID 4211 · nice 10 (renice)	PID 4212 · nice 19 (child)

one service = one box · all its processes and their children live here · Nice=19 is only the START value
each process can get its own nice (renice -n 10 -p 4211) · the limits are shared by the whole box

`CPUQuota 30%` — over → THROTTLED slowed down: 30 ms of every 100 ms	`MemoryHigh 150M` — over → slowed down soft limit · drop cache / swap · no kill
`MemoryMax 200M` — over → OOM KILLER stops a process in THIS box only	`TasksMax 20` — over → new fork FAILS "Resource temporarily unavailable"

④ other services = other boxes

`sshd.service` · `nightly-backup.service` · `glpi` …
never lose memory or CPU because of this box
→ one bad job cannot freeze the whole server

⑤ real example – compressing uploads at 14:00 while users work

1. **admin queues a big job** — `sudo archive-enqueue /data/app/uploads`
2. **the worker starts tar + gzip** — children inherit nice 19 and live in the SAME box
3. **gzip wants 100 % CPU** — → throttled to 30 % · job ~3× slower, but finishes
4. **other services NOT slowed** — sshd, GLPI, backup = own boxes · use the free 70 %
5. **gzip memory explodes > 200M** — → OOM killer stops gzip only · job FAILED, trap cleans .part
6. **urgent? admin speeds tar up** — `sudo renice -n 10 -p 4211 (box limit stays)`

Without the box: one compression job could slow the whole server for everyone. With it: the job is slower, everyone else is fine.

⑥ what MemoryHigh does – first drop CACHE, then use SWAP (same idea on Linux and Windows)

RAM before	process memory (really in use)	cache = files kept in RAM (original on disk)	free
RAM after step 1	process memory (really in use)	cache deleted from RAM → room again	
RAM after step 2	active memory	RAM → SWAP	idle program memory moved from RAM to swap – slow, nothing killed

Swap = a reserved area ON THE DISK (Ubuntu: /swap.img) used as extra slow RAM · check: free -h (RAM + swap) · swapon --show (where) · Windows: pagefile.sys

⑦ what "throttled" means – CPUQuota=30% (like mobile data after the limit: slower, not stopped)

30 ms run	70 ms paused	30 ms run	70 ms paused	30 ms run	70 ms paused	30 ms run	70 ms paused

every 100 ms the box may run only 30 ms → the job still finishes, about 3× slower · top shows ~30 %

⑧ the same cgroup limits in Docker and Kubernetes

	CPU ceiling (→ throttled)	memory ceiling (→ killed)	guaranteed minimum
systemd unit	`CPUQuota=30%`	`MemoryMax=200M`	– (`MemoryHigh` = soft limit)
docker run	`--cpus=0.3`	`--memory=200m`	–
docker-compose.yml	`cpus: 0.3`	`mem_limit: 200m`	–
Kubernetes YAML	`limits: cpu: "300m"`	`limits: memory: "200Mi"`	`requests: cpu / memory`

⑨ who creates the box – and the network – in Linux, Docker and Kubernetes

Linux service	systemd PID 1 – starts the service	→ cgroup one box per service			network: **the host's own network**
Docker	dockerd Docker engine	→ containerd manages containers	→ runc builds the container	→ namespaces + cgroup isolation + limits	network: **Docker networking**
Kubernetes	kubelet agent on each node	→ containerd manages containers	→ runc builds the container	→ namespaces + cgroup isolation + limits	network: **CNI plugin**

containerd = the manager (images, start/stop) · runc = the builder: namespaces = its own view of processes, network, hostname, files · cgroup = its limits
runc creates an EMPTY network namespace – the IP and connections come from Docker networking (bridge docker0, -p ports) or a Kubernetes CNI plugin (Calico, Flannel)

⑩ final paragraph (corrected) – nice, cgroups, memory, swap

nice sets the priority of each process (-20 highest, 19 lowest) – who goes first, with no ceiling. A cgroup is a box that systemd creates for every service: one service = one box, and all its processes and their children live in it. In the unit file, Nice=19 is only the start value for each process (renice can change one), while CPUQuota, MemoryHigh, MemoryMax and TasksMax are limits shared by the whole box. Over CPUQuota the box is throttled – it runs only e.g. 30 ms of every 100 ms, like mobile data after the limit: slower, never killed – and other services in their own boxes are not slowed. The CPU works only with RAM, so files read from disk are kept as copies in RAM, the cache. Above MemoryHigh (soft limit) the kernel first deletes this cache from RAM – nothing is lost, the originals are on the disk – and then moves idle program memory from RAM into swap, a reserved area on the disk (Ubuntu: /swap.img; Windows: pagefile.sys). Normal files on the disk are never touched; the box only gets slower. If it still reaches MemoryMax (hard wall), the kernel's OOM killer stops a process in this box only. TasksMax caps the number of processes and protects the PID pool. Docker (--memory, --cpus, mem_limit) and Kubernetes (requests = guaranteed, limits = hard ceiling, OOMKilled) use the same cgroups. Check RAM and swap with free -h, where swap lives with swapon --show, and the boxes live with systemd-cgtop. Who creates the box: on Linux systemd; in Docker dockerd → containerd → runc; in Kubernetes kubelet → containerd → runc. containerd manages, runc builds the container with namespaces (isolation) and a cgroup (limits); the IP comes from Docker networking or a CNI plugin.

cgroups – how a service gets its box and its limits

2.5 The same limits in Docker and Kubernetes

	CPU ceiling (→ throttled)	memory ceiling (→ killed)	minimum / soft
systemd unit	`CPUQuota=30%`	`MemoryMax=200M`	`MemoryMin=` guaranteed · `MemoryHigh=` soft
`docker run`	`--cpus=0.3`	`--memory=200m`	`--memory-reservation` (soft only – no guarantee)
docker-compose.yml	`cpus: 0.3`	`mem_limit: 200m`	`deploy.resources.reservations`
Kubernetes YAML	`limits: cpu: "300m"`	`limits: memory: "200Mi"` (→ OOMKilled)	`requests:` = guaranteed, used to choose the node

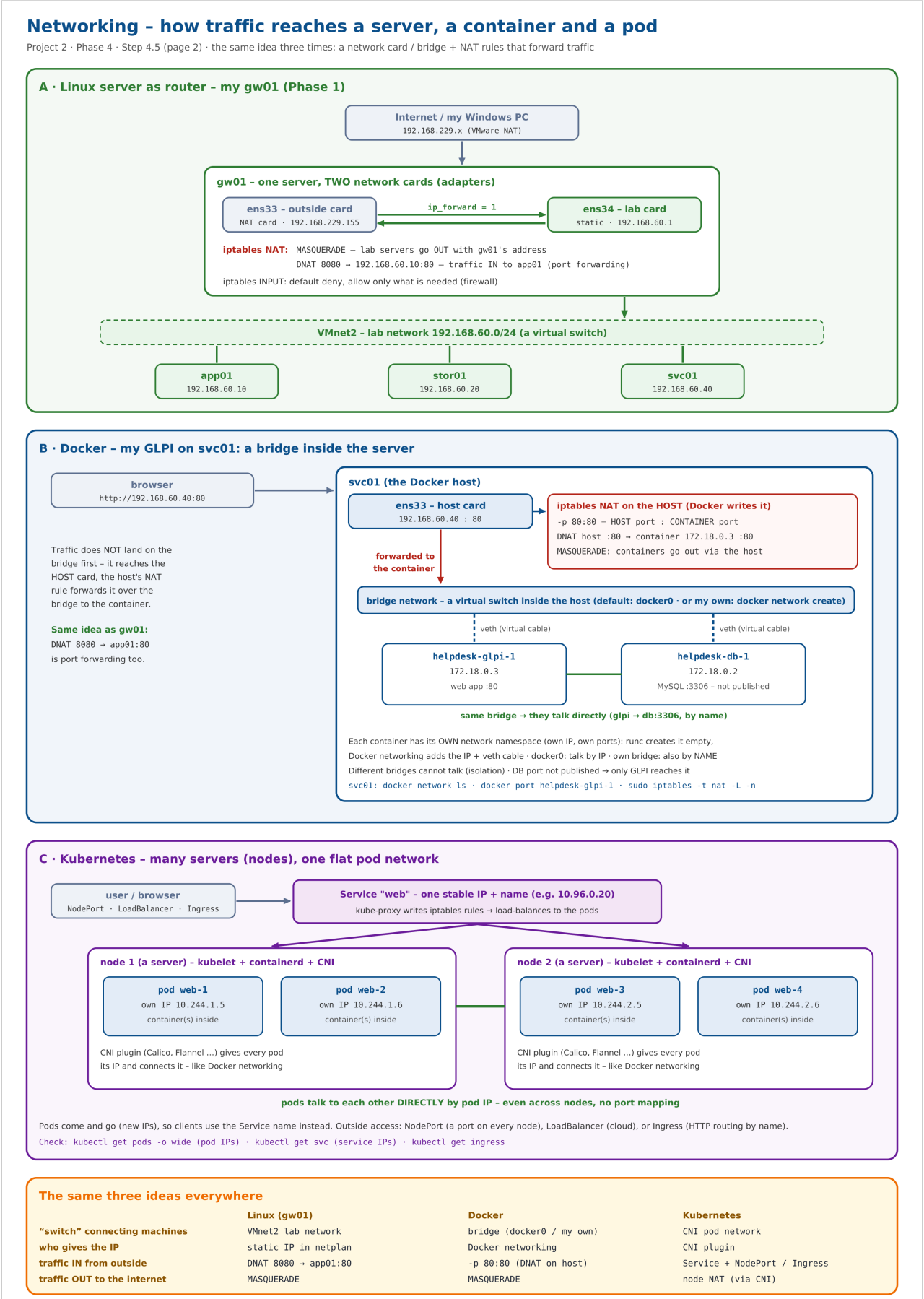
The Dockerfile only builds the image – limits are set when the container runs. `300m` = 300 millicores (0.3 of a core); `200Mi` ≈ 200 MB.

---

3. Container networking – the same idea as gw01

- **Linux (gw01):** two network cards (ens33 outside, ens34 lab), `ip_forward = 1`, **MASQUERADE** for traffic out, **DNAT 8080 → app01:80** for traffic in.
- **Docker (GLPI on svc01):** traffic reaches the **host card** first; the host's iptables NAT rule written by Docker (`-p 80:80` = HOST port : CONTAINER port) forwards it over a **bridge** (default `docker0` or my own `docker network create`) and a **veth** virtual cable to the container's own IP. Containers on the same bridge talk directly (on my own bridge also by name); the MySQL port is not published, so only GLPI reaches it.
- **Kubernetes:** a **CNI plugin** gives every pod its own IP; pods talk directly, even across nodes. A **Service** gives one stable IP/name and **kube-proxy** load-balances to the pods; outside access via NodePort, LoadBalancer or Ingress.

📸 **Diagram:**



Networking – Linux, Docker and Kubernetes

4. Key understanding (in my own words)

nice sets the priority of each process (-20 highest, 19 lowest) – who goes first, with no ceiling. A **cgroup** is a box that **systemd** creates for every service: one service = one box, and all its processes and their children live in it. In the unit file, `Nice=19` is only the start value for each process (`renice` can change one),

while `CPUQuota`, `MemoryHigh`, `MemoryMax` and `TasksMax` are limits shared by the whole box. Over **CPUQuota** the box is **throttled** – it runs only e.g. 30 ms of every 100 ms, like mobile data after the limit: slower, never killed – and other services in their own boxes are not slowed. The CPU works only with **RAM**, so files read from disk are kept as copies in RAM, the **cache**. Above **MemoryHigh** (soft limit) the kernel first deletes this cache from RAM – nothing is lost, the originals are on the disk – and then moves idle program memory from RAM into **swap**, a reserved area on the disk (Ubuntu `/swap.img`, Windows `pagefile.sys`); normal files on the disk are never touched, the box only gets slower. If it still reaches **MemoryMax** (hard wall), the kernel's **OOM killer** stops a process in this box only. **TasksMax** caps the number of processes and protects the PID pool. **Docker** (`--memory`, `--cpus`, `mem_limit`) and **Kubernetes** (requests = guaranteed, limits = hard ceiling, `OOMKilled`) use the same cgroups: on Linux systemd creates the box; in Docker `dockerd → containerd → runc`; in Kubernetes `kubelet → containerd → runc`. containerd manages, runc builds the container with **namespaces** (isolation) and a **cgroup** (limits); the IP comes from Docker networking or a CNI plugin. Traffic to a container always reaches the **host's card** first, and a **NAT rule** forwards it – exactly like my gw01's `DNAT 8080 → app01:80`.

---

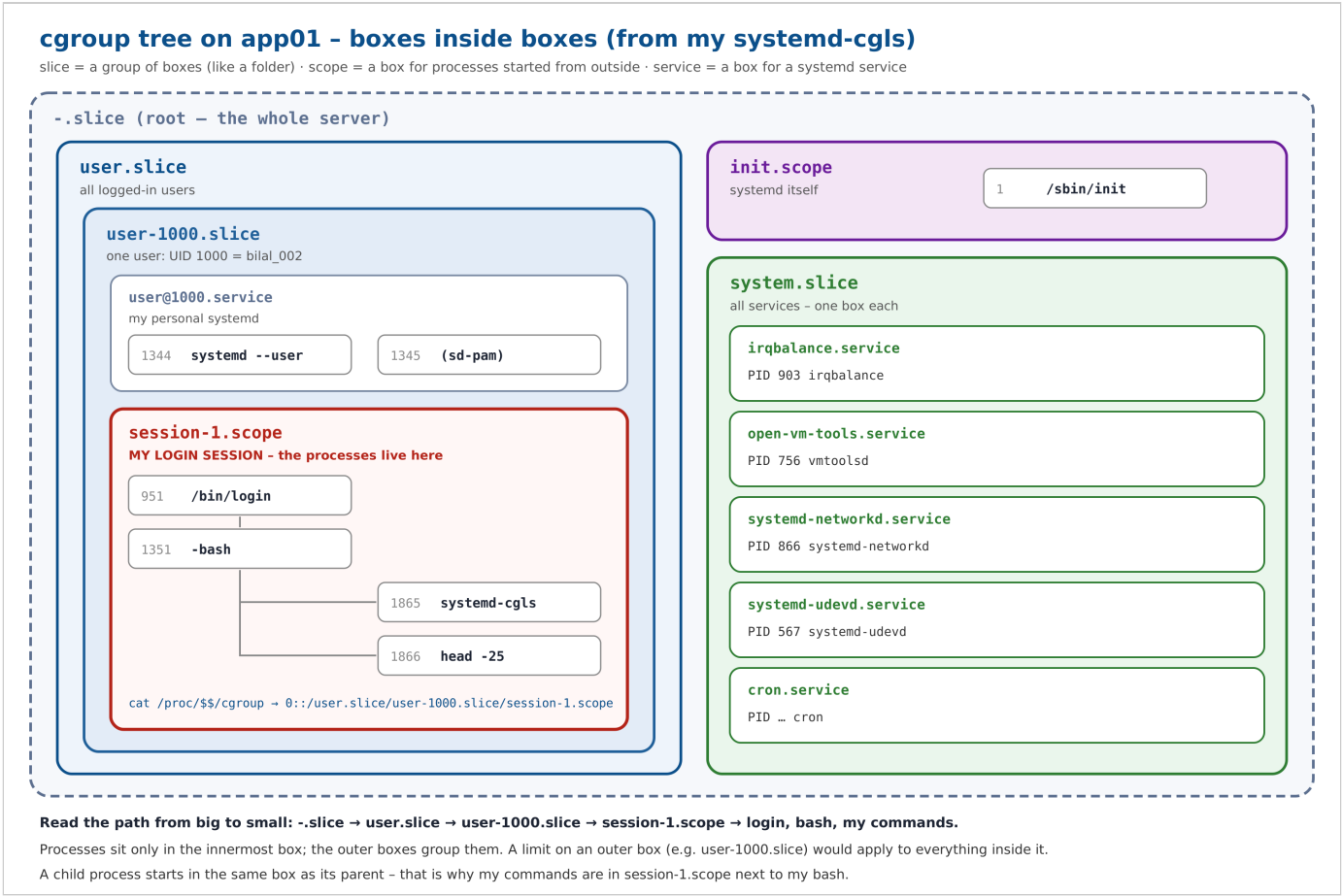
5. Lab Step 1 (app01) – the cgroup tree

```
free -h                 # RAM 1.9Gi · swap 1.8Gi
swapon --show           # /swap.img – the swap file on the disk
cat /proc/$$/cgroup     # 0::/user.slice/user-1000.slice/session-1.scope
systemd-cgls            # the whole tree of boxes
```

💡 **In plain words:** the server is one tree of boxes. Directly under the root, **side by side**, are `init.scope` (only PID 1 – systemd), `system.slice` (all services, one box each) and `user.slice` (all logged-in users). My shell lives in the innermost box: `user.slice → user-1000.slice → session-1.scope`.

- **slice** = a folder that groups boxes · **service** = a box for what systemd starts · **scope** = a box for what someone else starts (e.g. my login)
- processes sit only in the innermost boxes; a child starts in its parent's box

📸 **Diagram – my real tree on app01:**



cgroup tree on app01

---

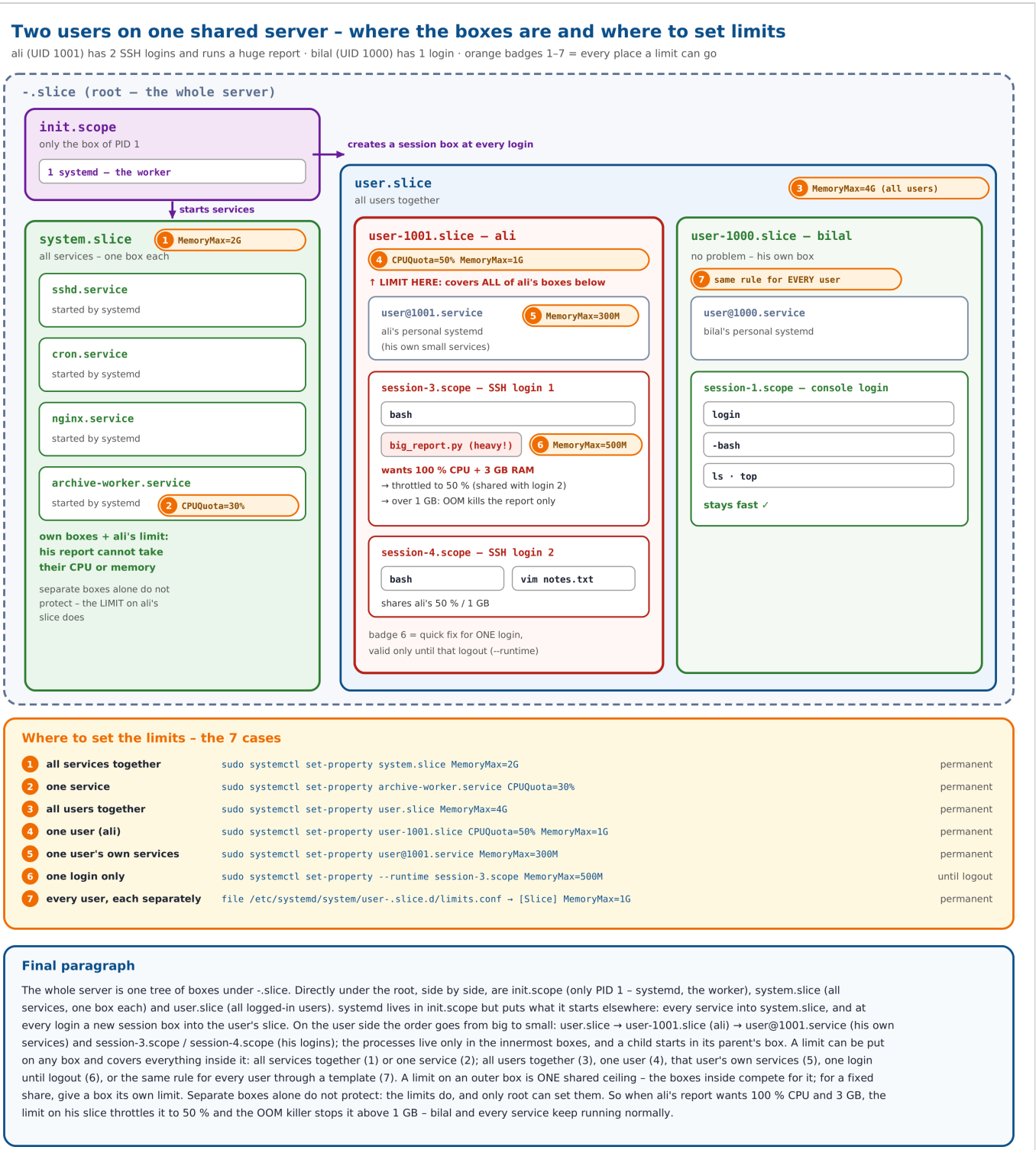
6. Where a limit can be set – two users on one server

Example: **ali** (UID 1001) runs a huge report in one of his two SSH logins; **bilal** (UID 1000) works normally. A limit on ali's slice throttles his report to 50 % CPU and the OOM killer stops it above 1 GB – bilal and all services keep running.

#	level	command
1	all services together	`sudo systemctl set-property system.slice MemoryMax=2G`
2	one service	`sudo systemctl set-property archive-worker.service CPUQuota=30%`
3	all users together	`sudo systemctl set-property user.slice MemoryMax=4G`
4	one user	`sudo systemctl set-property user-1001.slice CPUQuota=50% MemoryMax=1G`
5	one user’s own services	`sudo systemctl set-property user@1001.service MemoryMax=300M`
6	one login only (until logout)	`sudo systemctl set-property --runtime session-3.scope MemoryMax=500M`
7	every user, each separately	file `/etc/systemd/system/user-.slice.d/limits.conf`

A limit on an outer box is **one shared ceiling** – the boxes inside compete for it.

📸 **Diagram:**



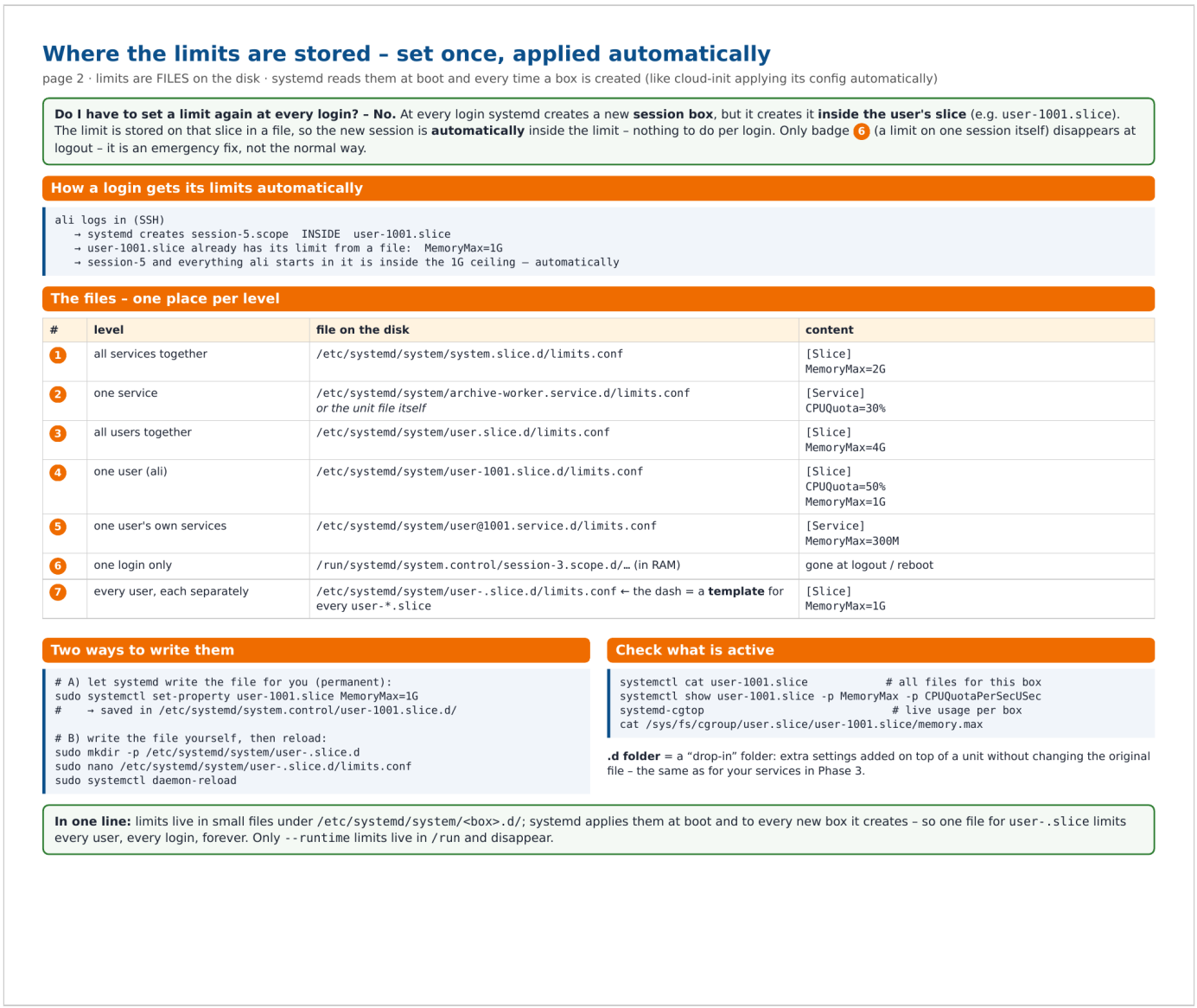
Two users – where to set limits

7. Where the limits are stored – set once, applied automatically

Do I have to set a limit again at every login? – No. systemd creates every new session box **inside the user's slice**, and the slice already has its limit from a file – so each login is automatically limited (like cloud-init applying its config). Only a `--runtime` limit (stored in `/run`, in RAM) disappears at logout.

- system side: `/etc/systemd/system/system.slice.d/limits.conf` · `/etc/systemd/system/archive-worker.service.d/limits.conf`
- user side: `/etc/systemd/system/user-1001.slice.d/limits.conf` · `/etc/systemd/system/user-.slice.d/limits.conf` (template for every user)
- `set-property` writes the file for me (in `/etc/systemd/system.control/`); or I write it myself and run `sudo systemctl daemon-reload`
- check: `systemctl cat user-1001.slice` · `systemd-cgtop` · `cat /sys/fs/cgroup/user.slice/user-1001.slice/memory.max`

📸 **Reference page:**



Where the limits are stored – set once, applied automatically

page 2 · limits are FILES on the disk · systemd reads them at boot and every time a box is created (like cloud-init applying its config automatically)

Do I have to set a limit again at every login? – No. At every login systemd creates a new **session box**, but it creates it **inside the user's slice** (e.g. `user-1001.slice`). The limit is stored on that slice in a file, so the new session is **automatically** inside the limit – nothing to do per login. Only badge 6 (a limit on one session itself) disappears at logout – it is an emergency fix, not the normal way.

How a login gets its limits automatically

```
ali logs in (SSH)
   → systemd creates session-5.scope  INSIDE  user-1001.slice
   → user-1001.slice already has its limit from a file:  MemoryMax=1G
   → session-5 and everything ali starts in it is inside the 1G ceiling – automatically
```

The files – one place per level

#	level	file on the disk	content
1	all services together	`/etc/systemd/system/system.slice.d/limits.conf`	`[Slice]` `MemoryMax=2G`
2	one service	`/etc/systemd/system/archive-worker.service.d/limits.conf` *or the unit file itself*	`[Service]` `CPUQuota=30%`
3	all users together	`/etc/systemd/system/user.slice.d/limits.conf`	`[Slice]` `MemoryMax=4G`

4	one user (ali)	`/etc/systemd/system/user-1001.slice.d/limits.conf`	`[Slice]`
			`CPUQuota=50%`
			`MemoryMax=1G`

5	one user's own services	`/etc/systemd/system/user@1001.service.d/limits.conf`	`[Service]` `MemoryMax=300M`
6	one login only	`/run/systemd/system.control/session-3.scope.d/…` (in RAM)	gone at logout / reboot
7	every user, each separately	`/etc/systemd/system/user-.slice.d/limits.conf` ← the dash = a **template** for every `user-*.slice`	`[Slice]` `MemoryMax=1G`

Two ways to write them

```
# A) let systemd write the file for you (permanent):
sudo systemctl set-property user-1001.slice MemoryMax=1G
#    → saved in /etc/systemd/system.control/user-1001.slice.d/

# B) write the file yourself, then reload:
sudo mkdir -p /etc/systemd/system/user-.slice.d
sudo nano /etc/systemd/system/user-.slice.d/limits.conf
sudo systemctl daemon-reload
```

Check what is active

```
systemctl cat user-1001.slice            # all files for this box
systemctl show user-1001.slice -p MemoryMax -p CPUQuotaPerSecUSec
systemd-cgtop                            # live usage per box
cat /sys/fs/cgroup/user.slice/user-1001.slice/memory.max
```

.d folder = a "drop-in" folder: extra settings added on top of a unit without changing the original file – the same as for your services in Phase 3.

In one line: limits live in small files under `/etc/systemd/system/<box>.d/`; systemd applies them at boot and to every new box it creates – so one file for `user-.slice` limits every user, every login, forever. Only `--runtime` limits live in `/run` and disappear.

Where the limits are stored

In my own words: The whole server is one tree of boxes under `-.slice`: `init.scope`, `system.slice` and `user.slice` side by side. A limit can go on any box and covers everything inside it – all services, one service, all users, one user, a user's own services, one login, or every user through the `user-.slice` template. Limits are small files in `/etc/systemd/system/<box>.d/`; systemd applies them at boot and to every new box it creates, so I set them once and every login is covered automatically.

---

8. Next – the rest of the cgroups lab

1. ~~**Step 1 (app01):** `free -h · swapon --show · stress-ng · cat /proc/$$/cgroup · systemd-cgls`.~~ ✅
2. **Step 2 (app01):** memory limit – `systemd-run --scope -p MemoryMax=100M -p MemorySwapMax=0 stress-ng --vm 1 --vm-bytes 300M` → OOM in the journal.
3. **Step 3 (app01):** CPU limit – `CPUQuota=20%` → `top` and `systemd-cgtop` show ~20 %.
4. **Step 4 (stor01):** permanent limits on `archive-worker.service`.
5. Then: first incident drill · L3 Problem in GLPI · snapshot `50-processes`.

Project 2 – Acme Analytics Linux Platform

Day 11 · Phase 4: hard + soft memory limits · how the OOM killer chooses · disk, RAM, mounts, live vs permanent

Muhammad Bilal Javed · 3 November 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01 (hard-limit test, OOM scores, soft-limit test, clean-up check)
Status	Phase 4 🟡 – Steps 4.1–4.4 ✅ · Step 4.5: concept ✅ · lab Step 1 ✅ · **lab Step 2 ✅ hard limit (OOM killer) and soft limit (swap, no kill)** · how the OOM killer chooses ✅ · foundations ✅ (boot, page cache, mounts, live vs permanent) – CPU limit next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot or diagram. Terms are explained in the glossary. Previous day: Day 10.

---

1. Lab Step 2 (app01) – a hard memory limit and the OOM killer

💡 **In plain words:** I gave a test program a box with a 100 MB memory wall and no swap, then asked it to use 300 MB. The kernel stopped it at the wall – inside its own box only. app01 and every service next to it kept running.

🔧 **Technical:**

```
sudo systemd-run --scope -p MemoryMax=100M -p MemorySwapMax=0 stress-ng --vm 1 --vm-bytes 300M --timeout 1s
sudo journalctl -k --since "5 min ago" | grep -i -E "oom|killed" | tail -5
```

The pattern behind every test:

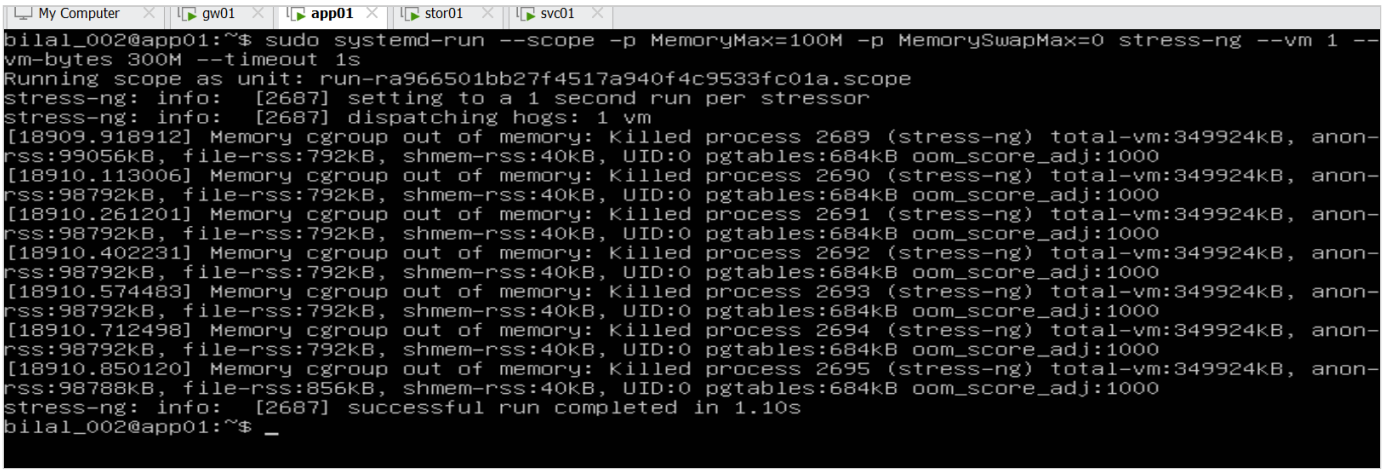
```
sudo systemd-run --scope  -p <LIMIT>   stress-ng <LOAD>  --timeout <TIME>
     └─ make a box ──┘  └ the limit ┘  └── fake load to test the limit ──┘
```

Part	Meaning
`systemd-run --scope`	a temporary box for my command – output on my screen, the box is deleted when the command ends
`-p MemoryMax=100M` · `-p MemorySwapMax=0`	hard memory wall of 100 MB · no escape to swap
`stress-ng --vm 1 --vm-bytes 300M`	1 memory worker that tries to use 300 MB (`--cpu` = CPU load, `--fork` = new processes)
where the box goes	with `sudo` the request goes to the system systemd (PID 1) → default place **system.slice**

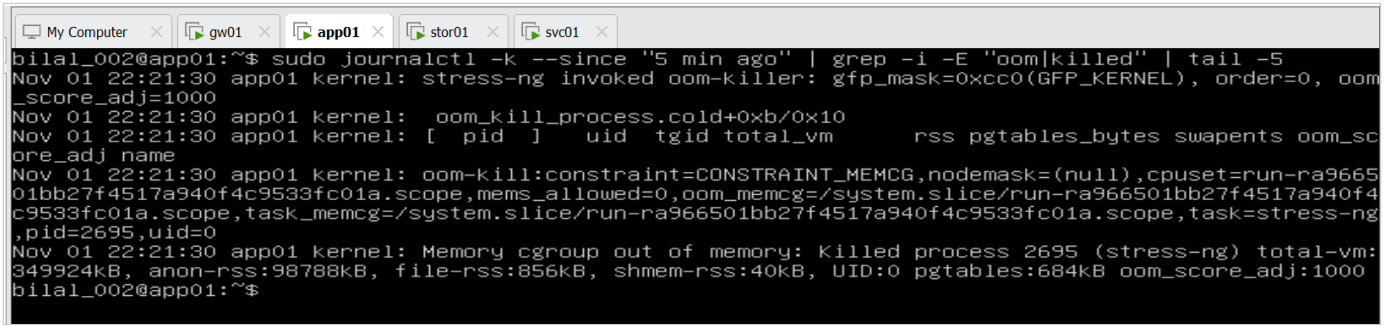
What happened:

Line	Meaning
`Running scope as unit: run-ra9665….scope`	systemd made the temporary box
`total-vm:349924kB`	the worker wanted about 340 MB
`anon-rss:98792kB`	it really got about 96 MB of RAM – right at the 100 MB wall
`Killed process 2689 … 2695` (7 times)	the OOM killer stopped the worker; stress-ng restarted it, each time with a new PID
`[2687] successful run completed`	the parent was never killed – only its workers
`constraint=CONSTRAINT_MEMCG`	killed because of the **box limit**, not because app01 ran out of RAM
`oom_memcg=/system.slice/run-ra9665….scope`	the box that hit the limit – only this box was touched

📸 **Evidence:**



OOM test – stress-ng killed at 100 MB



Journal proof – CONSTRAINT_MEMCG

Clean-up check before the next test: `pgrep -a stress-ng` (nothing = no test running) · `systemctl list-units --all | grep -E "run-|mytest"` – the `run-….mount` lines are normal mounts under `/run`, not old tests; old test boxes would end in `.scope` or be called `mytest.service`. Leftovers: `sudo pkill stress-ng` · `sudo systemctl stop mytest` · `sudo systemctl reset-failed`.

Same idea in containers: a Kubernetes pod shows `OOMKilled`, Docker kills a container over `--memory`.

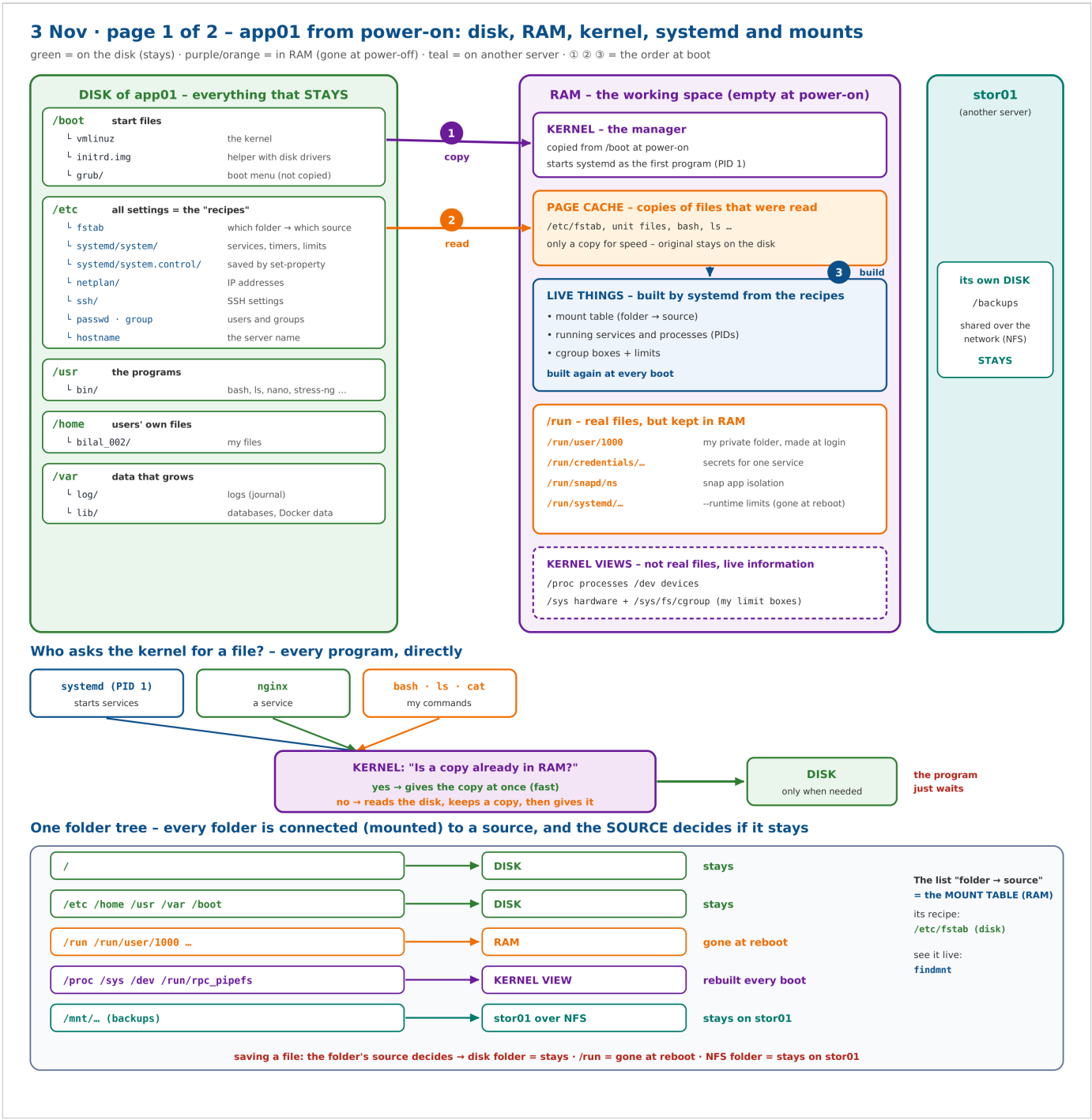
---

2. How Linux uses the disk and RAM – from power-on

💡 **In plain words:** everything is stored on the disk; RAM starts empty. The kernel is copied into RAM and becomes the manager, systemd reads its settings from `/etc`, and builds the live things in RAM.

1. ① **Copy:** two files from `/boot` are copied into RAM – `vmlinuz` (the kernel) and `initrd.img` (a small helper with disk drivers). The rest of `/boot` and all other files stay on the disk.
2. The kernel starts **systemd** as the first program (PID 1).
3. ② **Read:** systemd reads its recipes in `/etc` (fstab, unit files, limits). Reading = the kernel puts a **copy of that one file** in RAM (the page cache); the original stays on the disk.
4. ③ **Build:** from the recipes systemd builds the live things in RAM – the mount table, the running services and the cgroup boxes. They are built again at every boot.

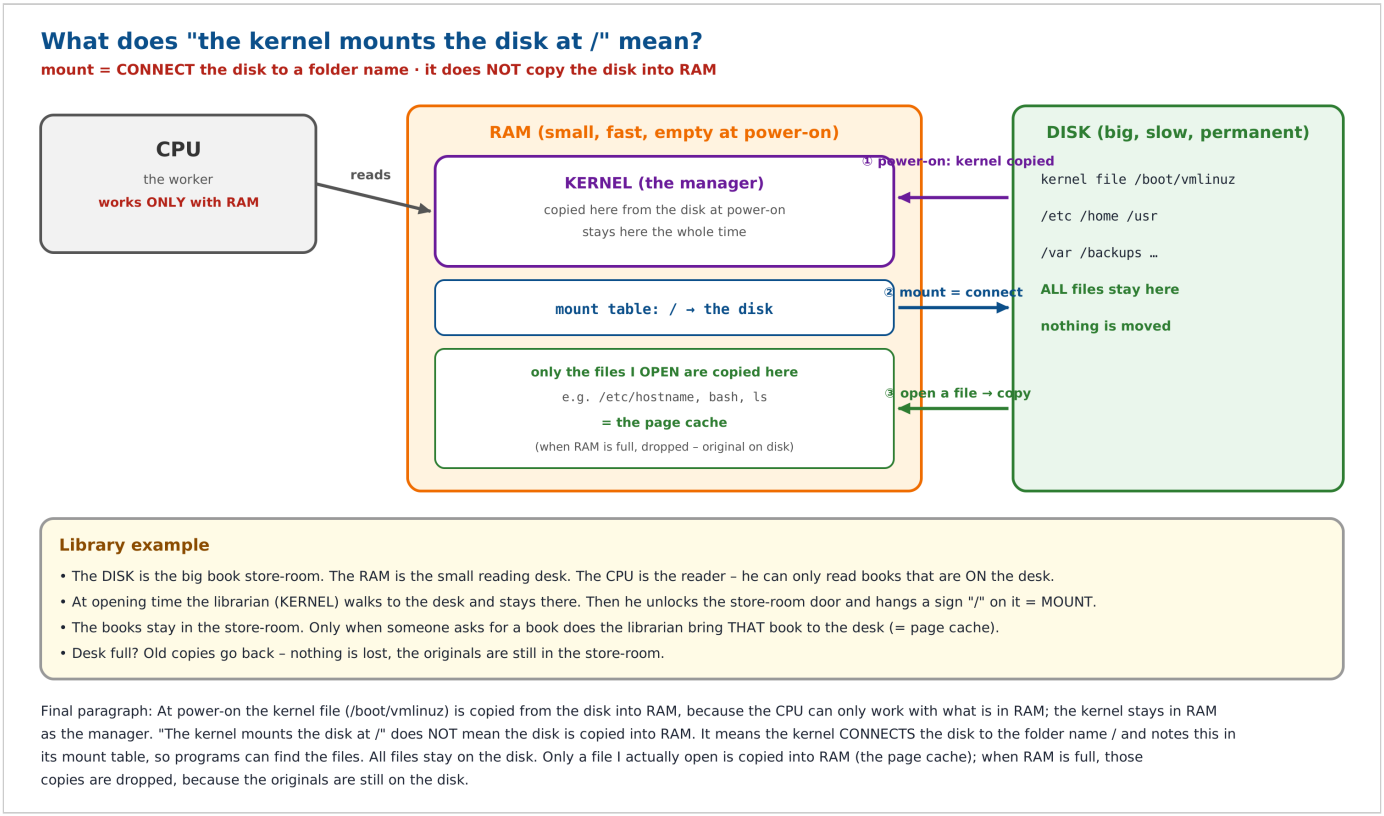
📸 **Diagram – the whole picture on one page:**



app01 from power-on – disk, RAM, kernel, systemd and mounts

2.1 Mount = connect, not copy

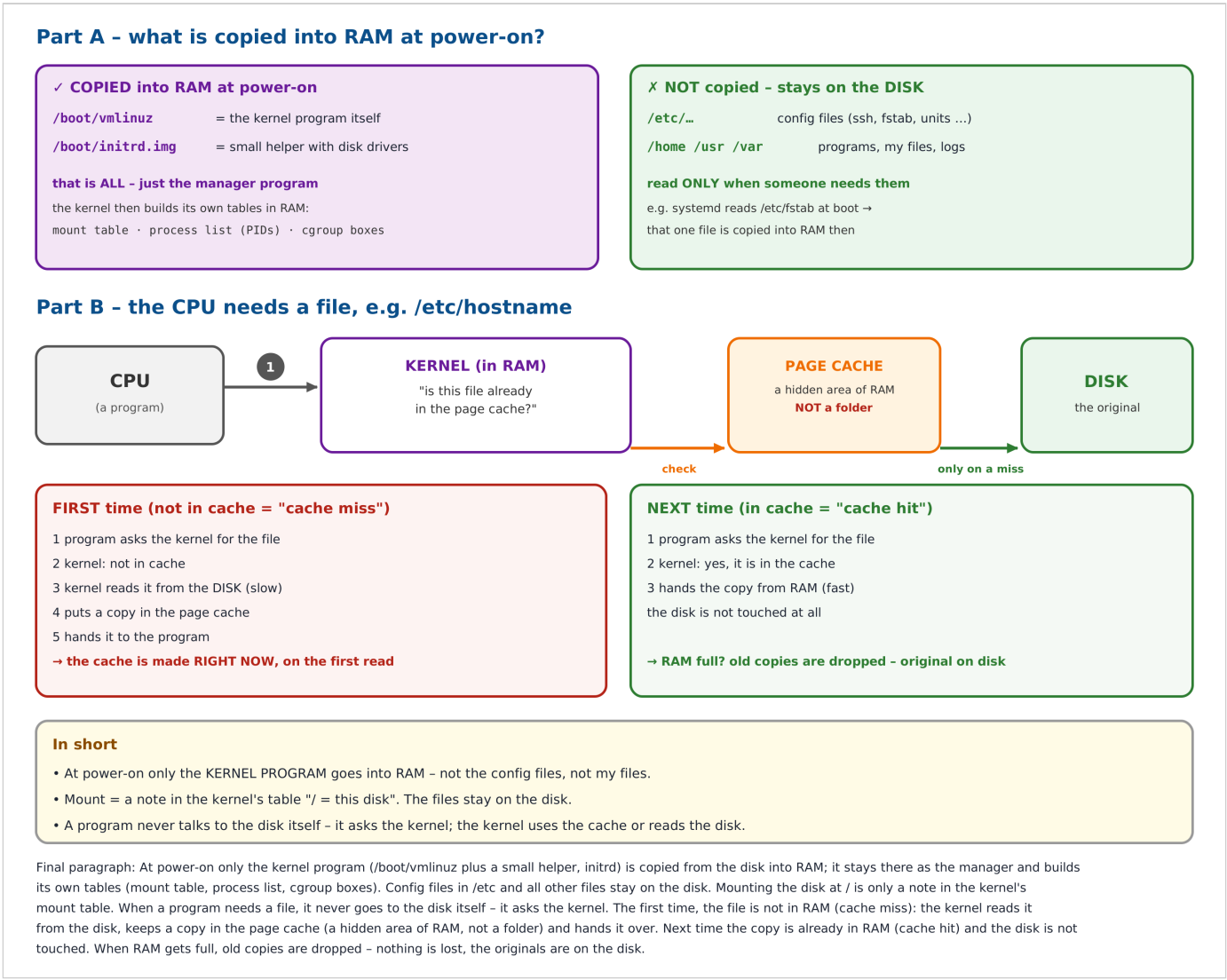
A mount connects a **folder** to a **source**. The files stay where they are; the kernel only notes "/ = this disk" in its mount table (in RAM).



What mount means – the library example

2.2 When a program needs a file

Every program – systemd, nginx, or my own `ls` – asks the **kernel**. Copy already in RAM → given at once (fast). Not in RAM → the kernel reads the disk, keeps a copy, then gives it. The program simply waits. The page cache is a hidden area of RAM, not a folder.



How a file is read – cache miss and cache hit

2.3 One folder tree – the source decides if it stays

Folders	Source	After reboot
`/ /etc /home /usr /var /boot`	disk	stays
`/run` (e.g. `/run/user/1000`)	RAM (tmpfs)	gone
`/proc /sys /dev /run/rpc_pipefs`	kernel view – live information, not real files	rebuilt
`/mnt/…` backups	stor01 over NFS	stays on stor01

systemd tracks every mount as a `.mount` unit named after its folder with `/` → `-` (a real dash = `\x2d`), e.g. `run-user-1000.mount` = `/run/user/1000`. Check: `findmnt`.

---

3. Live test vs permanent – where I write it decides

💡 **In plain words:** the work always happens in RAM. Saving a file in a disk folder is **not** lost – the kernel writes it to the disk within seconds. What is lost at reboot is a **live change with no file**. `/etc` is special because of its job: it holds the settings programs read every time they start.

The change	Live – works at once, gone at reboot	File on the disk – stays
IP forwarding (gw01)	`sysctl -w net.ipv4.ip_forward=1` (= `/proc/sys/…`)	`/etc/sysctl.conf`
a service	`systemd-run --unit=mytest …`	`/etc/systemd/system/mytest.service` + `enable`
a limit	`systemd-run -p … · set-property --runtime`	`set-property` · `…service.d/limits.conf`
a mount	`mount 192.168.60.20:/backups /mnt`	`/etc/fstab`
an IP address	`ip addr add …`	`/etc/netplan/…yaml`

Admin habit: try it live → check it works → write it into the file so it stays. A cgroup box with no recipe file comes back after reboot with the default settings and no limit – that is why my `systemd-run` tests left nothing behind.

📸 **Diagrams:**

Two kinds of change – which one survives a reboot?

saving a file in a disk folder is NOT lost · only changes that live ONLY in RAM are lost

KIND 1 – I SAVE A FILE in a disk folder → STAYS

e.g. nano /home/bilal_002/notes.txt · edit /etc/fstab · a new unit file in /etc/systemd/system/

I press save
(in nano, cp, echo >)

→

copy in RAM
(page cache, updated)

→

kernel writes it to the DISK
automatically, within a few seconds

→ the file is ON THE DISK. After logout or reboot it is still there. The RAM copy was only for speed.

(only a sudden power cut in those few seconds could lose it – a normal shutdown writes everything first)

KIND 2 – a LIVE CHANGE with no file → LOST at reboot

the command changes only the kernel's live tables in RAM – nothing is written to any file on the disk

`sudo renice -n 10 -p 1234`	priority of one process
`sudo systemd-run --scope -p MemoryMax=100M …`	temporary box + limit
`sudo systemctl set-property --runtime …`	limit kept in /run (RAM)
`sudo ip addr add 192.168.60.99/24 dev ens33`	extra IP address
`sudo mount 192.168.60.20:/backups /mnt`	a mount typed by hand

To make it PERMANENT:

write it into a RECIPE FILE

- limit → unit file / drop-in
or set-property (no --runtime)
- Nice= → in the unit file
- IP → /etc/netplan/
- mount → /etc/fstab

→ next boot: systemd rebuilds everything from the recipes only. A box with no recipe comes back with the DEFAULT settings (no limit).

This is why my systemd-run tests left nothing behind – they were live changes only.

My wording – corrected

- Not all green (disk) things come into RAM – only the files a program actually needs, when it needs them.
- Working "in RAM" does NOT mean my saved work is lost: a file saved in /home, /etc or /usr is written to the disk.
- /usr holds the programs (bash, ls). I run them from there; my own files go in /home.
- What is lost at reboot: changes that exist ONLY in RAM – live commands without a file, and everything in /run.
- cgroups: the boxes are rebuilt every boot from the recipe files – no recipe = default box with no limit. Correct!
- Mount = connect a folder to a source (disk, RAM, another server). Saving into that folder puts the file on that source.

Final paragraph: The kernel is copied into RAM and starts systemd. Whenever systemd or any other program needs a file, the kernel copies only that file from the disk into RAM (page cache); the original stays on the disk. There are two kinds of change. KIND 1 – saving a file in a disk folder (/home, /etc …): the kernel writes it to the disk within seconds, so it is still there after logout or reboot. KIND 2 – a live change with no file (renice, systemd-run, set-property --runtime, ip addr add, a hand-typed mount): it lives only in the kernel's tables in RAM and is lost at reboot. To make a live change permanent, write it into a recipe file on the disk (unit file or drop-in, /etc/netplan, /etc/fstab). At the next boot systemd rebuilds everything – including the cgroup boxes – only from those recipes; a box without a recipe comes back with the default settings and no limit.

Two kinds of change

The work always happens in RAM – WHERE you write it decides if it stays

same change, two places: the LIVE way (try it now, gone at reboot) or the FILE in /etc (stays)

the change	LIVE – in RAM / kernel works at once · gone at reboot	FILE in /etc – on the disk read at every boot · stays
IP forwarding on gw01 (Phase 1)	`sudo sysctl -w net.ipv4.ip_forward=1` = writes into /proc/sys/net/ipv4/ip_forward	`/etc/sysctl.conf or /etc/sysctl.d/…conf` net.ipv4.ip_forward = 1
a service run a program as a service	`sudo systemd-run --unit=mytest …` temporary service, gone at reboot	`/etc/systemd/system/mytest.service` + systemctl enable → starts every boot
a limit CPU / memory of a box	`systemd-run -p MemoryMax=100M …` set-property --runtime (→ /run)	`set-property (no --runtime) or` …/xyz.service.d/limits.conf
a mount connect a folder to a source	`sudo mount 192.168.60.20:/backups /mnt` typed by hand	`/etc/fstab` one line per mount
an IP address on a network card	`sudo ip addr add 192.168.60.99/24 dev ens33` typed by hand	`/etc/netplan/…yaml` + sudo netplan apply

/proc /sys /dev /run – what are they for?

- to SEE the system live: processes (/proc), hardware and cgroup boxes (/sys), devices (/dev), running state (/run)
- to CHANGE something live – it works at once but is gone at reboot → perfect to TRY a setting first
- **admin habit: try it LIVE → check it works → then write it into the FILE in /etc so it stays**

Final paragraph: The work always happens in RAM, and what stays depends on where I write it. Most settings can be changed in two ways: LIVE, through a command that changes the kernel's tables or the folders /proc, /sys and /run – it works at once but is gone at reboot – or in a FILE in /etc on the disk, which is read at every boot and therefore stays. IP forwarding on gw01 is the perfect example: sysctl -w (or /proc/sys/net/ipv4/ip_forward) is live, /etc/sysctl.conf is permanent. The same pattern fits services (systemd-run vs a unit file in /etc/systemd/system plus enable), limits (--runtime vs a drop-in), mounts (mount command vs /etc/fstab) and IP addresses (ip addr vs /etc/netplan). /proc, /sys, /dev and /run are for seeing and changing the system live, which makes them good for trying a setting first; the admin habit is: try it live, check it works, then write it into the file so it stays.

Live vs file

3 Nov · page 2 of 2 – live test vs permanent, and my memory-limit test on app01

the big idea of the day: the work always happens in RAM – WHERE you write it decides if it stays

A · Every setting has two ways: LIVE (a test, gone at reboot) or a FILE in /etc (stays)

the change	LIVE – works at once · gone at reboot	FILE on the disk – read at every boot · stays
IP forwarding (gw01)	`sysctl -w net.ipv4.ip_forward=1 (/proc/sys/…)`	`/etc/sysctl.conf`
a service	`systemd-run --unit=mytest …`	`/etc/systemd/system/mytest.service + enable`
a limit	`systemd-run -p MemoryMax=… · set-property --runtime`	`set-property · …service.d/limits.conf`
a mount	`mount 192.168.60.20:/backups /mnt`	`/etc/fstab`
an IP address	`ip addr add …`	`/etc/netplan/…yaml`
a saved file	– (saving in /home, /etc … already goes to disk)	any folder on the disk = stays

admin habit: ① try it LIVE → ② check it works → ③ write it into the FILE so it stays
/proc /sys /dev /run = see and change the system live (kernel views + RAM) – good for trying first · /etc = the settings programs read at start

B · The test pattern – systemd-run makes a temporary box, -p puts a limit on it, stress-ng makes fake load

```
sudo systemd-run --scope -p <LIMIT> stress-ng <LOAD> --timeout <TIME>
```

- **--scope** – box for MY command: output on my screen, deleted when the command ends
- **-p <LIMIT>** – MemoryMax = kill · MemoryHigh = slow + swap · CPUQuota = throttle · TasksMax = max processes
- **stress-ng <LOAD>** – `--vm 1 --vm-bytes 300M` = 1 worker 300MB · `--cpu 1` = busy core · `--fork 4` = forks
- **where does the box go?** – with sudo → the system systemd → system.slice (the default place)

C · My test on app01: MemoryMax=100M, no swap, stress-ng wants 300M → killed inside its box only

1. **systemd-run** – temporary box – `run-ra9665….scope`
2. **stress-ng worker** – wants ~340 MB – `total-vm 349924kB`
3. **hits the wall** – real RAM ~96 MB – `anon-rss 98792kB`
4. **OOM killer** – kills the worker – 7× · new PID each time
5. **parent survives** – PID 2687 – "successful run completed"
6. **box deleted** – command ended – nothing left behind

proof in the journal: constraint=CONSTRAINT_MEMCG · oom_memcg=/system.slice/run-ra9665….scope
CONSTRAINT_MEMCG = killed because of the BOX limit, not because app01 ran out of RAM · only that box was touched – services next to it kept running
same idea as a Kubernetes pod "OOMKilled" or Docker --memory

D · Useful checks

`pgrep -a stress-ng` – is a test still running?	`systemctl list-units --all | grep mytest` – old named test boxes?
`sudo pkill stress-ng · systemctl stop mytest` – stop leftovers	`sudo systemctl reset-failed` – clean old failed boxes
`sudo journalctl -k --since "5 min ago" | grep -i killed` – OOM kills in the kernel log	`findmnt · free -h · cat /etc/fstab` – mounts · RAM/swap · mount recipe

Final paragraph – the whole day in one place

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (vmlinuz with its helper initrd.img) is copied from /boot into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in /etc – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, /run in RAM is gone at reboot, /proc /sys /dev are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and WHERE I write it decides if it stays: a live change (sysctl -w, systemd-run, --runtime, mount, ip addr) is a test that is gone at reboot; a file on the disk – usually in /etc – is read at every boot and stays. My test proved it: systemd-run made a temporary box in system.slice with MemoryMax=100M, the OOM killer stopped stress-ng at about 96 MB inside that box only (CONSTRAINT_MEMCG), and the box disappeared when the test ended.

Page 2 of the overview – live test vs permanent and the OOM test

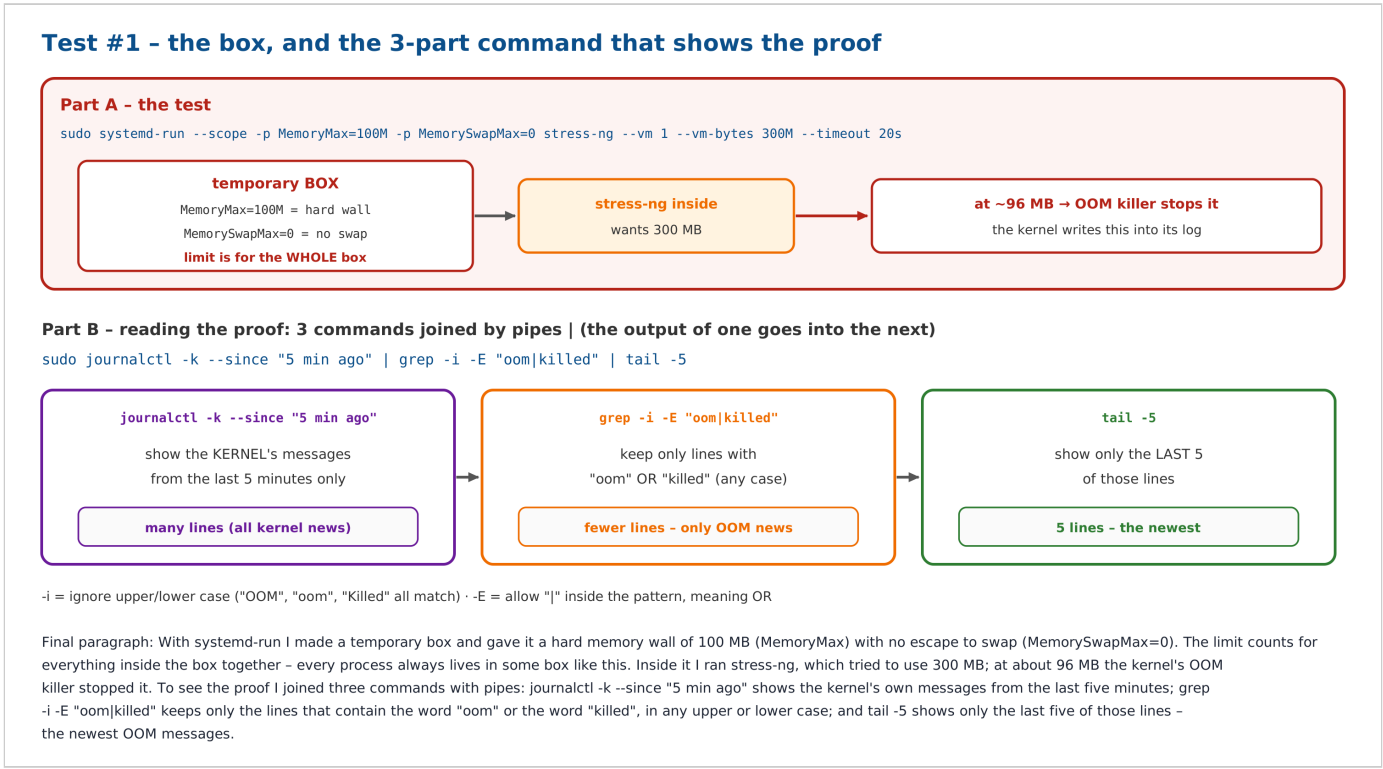
---

4. Evening session – test #1 repeated, how the OOM killer chooses, test #2 (soft limit)

4.1 Test #1 again – and how to read the proof

💡 **In plain words:** the limit is for the **whole box** – everything inside shares it. The proof command is three small commands joined by pipes: the output of one goes into the next.

Part	Meaning
`journalctl -k --since "5 min ago"`	the kernel's own messages from the last 5 minutes
`grep -i -E "oom|killed"`	keep only lines with the word "oom" **or** "killed" – `-i` any upper/lower case, `-E` lets `|` mean OR
`tail -5`	only the last 5 of those lines – the newest

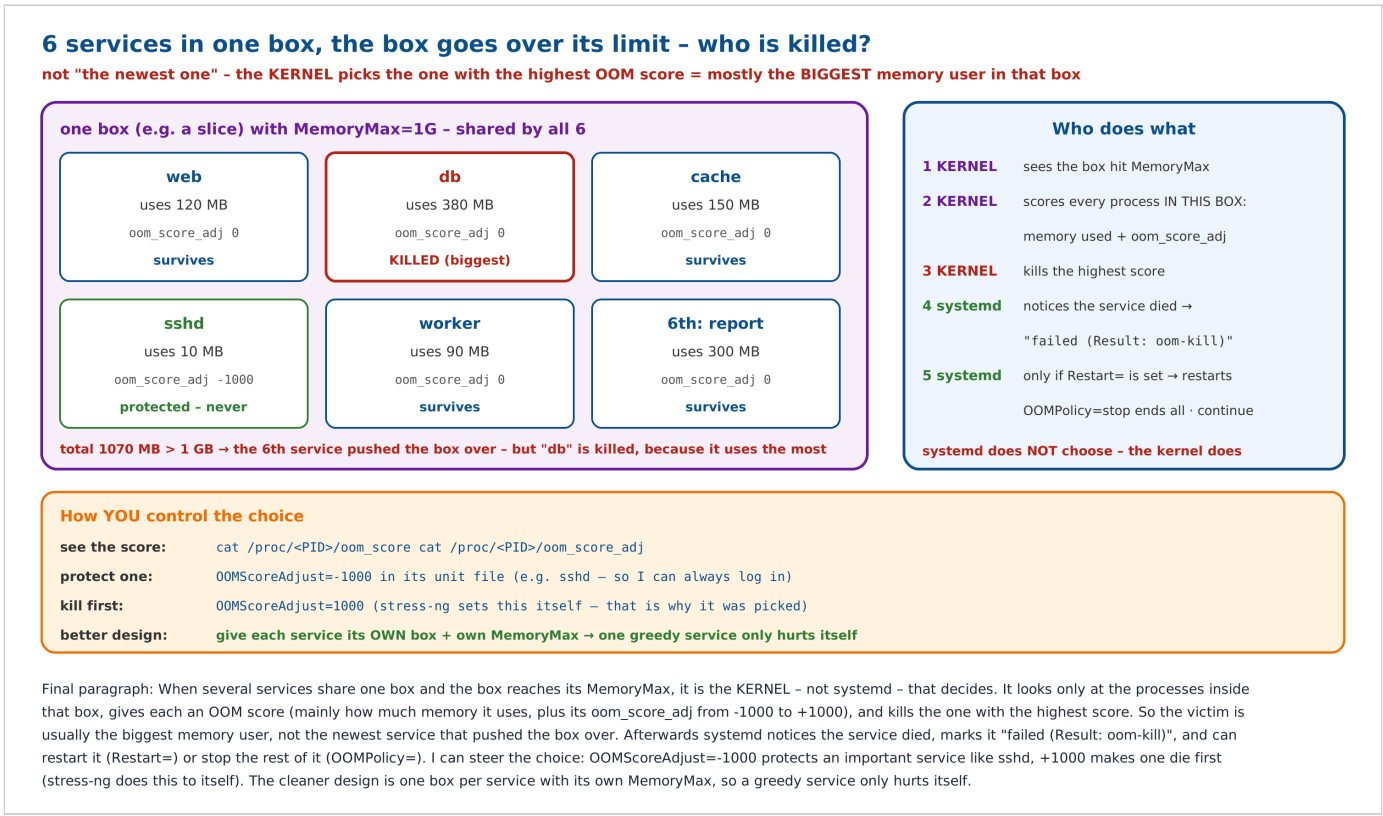


Test #1 – the box and the proof command

4.2 Many services in one box – who is killed?

💡 **In plain words:** when a shared box reaches `MemoryMax`, the **kernel** (not systemd) scores every process **in that box** and kills the highest score – usually the **biggest memory user**, not the newest service that pushed the box over. It is predictable, not luck.

- final score = the memory part (measured by the kernel) + `oom_score_adj` (-1000 … +1000, set by me – adj = adjustment)
- afterwards systemd marks the service `failed (Result: oom-kill)`; it restarts it **only if** `Restart=` is set; `OOMPolicy=stop` ends the whole service, `continue` lets its other processes run
- protect an important service: `OOMScoreAdjust=-1000` (or -500) in its unit file · make one die first: `+1000` (stress-ng does this to itself)
- protect only the few that really matter – if everything is protected, nothing can be freed
- best design: one box per service with its own `MemoryMax`



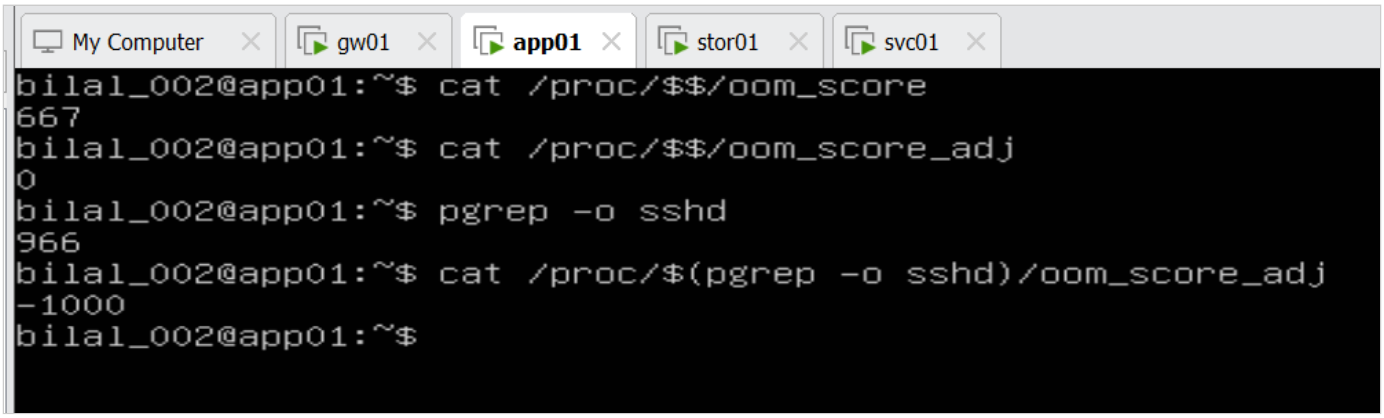
Who gets killed – kernel and systemd roles

4.3 Checking the scores on app01

```
cat /proc/$$/oom_score                    # 667  — my bash, normal
cat /proc/$$/oom_score_adj                # 0
pgrep -o sshd                             # 966  — sshd main PID
cat /proc/$(pgrep -o sshd)/oom_score_adj  # -1000 — protected by Ubuntu
```

On this kernel the scale is shifted: a process using almost no memory shows about **667**, one using nearly all memory approaches **1333**, and `-1000` brings it to **0** (never killed). `$(...)` runs the inner command first and puts its answer into the outer one. `<PID>` in a command is a placeholder for a real number.

📸 Evidence:



oom_score check on app01

Reading my result on app01 – the oom_score scale on this kernel

0 = never killed · ~667 = a normal small process · up to ~1333 = uses almost all memory → killed first

sshd: 0
adj -1000 = protected

my bash: 667
adj 0 · tiny memory · normal

~1333
uses almost all RAM

Why 667 and not a small number? On newer kernels the score is shifted: (1000 + memory share) × 2/3.

A process using almost nothing sits at ~667. Using more memory pushes it up; oom_score_adj moves it down (protect) or up (kill first).

So 667 for bash = "normal, nothing special" – the kernel compares this number between processes in the same box.

`cat /proc/$$/oom_score`	**667**	my bash – normal
`cat /proc/$$/oom_score_adj`	**0**	no adjustment
`pgrep -o sshd`	**966**	sshd main PID
`cat /proc/$(pgrep -o sshd)/oom_score_adj`	**-1000**	protected – never killed

My idea – correct: balance a high score with oom_score_adj (adj = ADJUSTMENT, not adjective)

memory part (the kernel measures it – I cannot change it) + oom_score_adj (I set it: -1000 … +1000) = final oom_score

an important service with a high score → give it a NEGATIVE adj → its final score drops → the kernel picks someone else

`permanent: OOMScoreAdjust=-500 in its unit file · live test: echo -500 | sudo tee /proc/<PID>/oom_score_adj`

careful: if everything is protected, nobody can be killed when memory is full – protect only the few that really matter (e.g. sshd)

Final paragraph: oom_score is the number the kernel uses to choose whom to kill when a box runs out of memory – the highest dies first. On this kernel the scale is shifted, so a normal process using almost no memory shows about 667, and a process using nearly all memory approaches 1333. My bash shows 667 with oom_score_adj 0 – completely normal. sshd (PID 966) has oom_score_adj -1000: Ubuntu protects it, so its score is 0 and it is never killed – that is why I can always log in, even when memory runs out. The final score = the memory part the kernel measures + oom_score_adj (adj = adjustment), which I can set from -1000 to +1000. So if an important service has a high score, I give it a negative adjustment – permanently with OOMScoreAdjust= in its unit file – its final score drops and the kernel chooses another process. I protect only the few services that really matter, because if everything is protected nothing can be freed. $(...) runs the command inside first and puts its answer into the outer command.

The oom_score scale

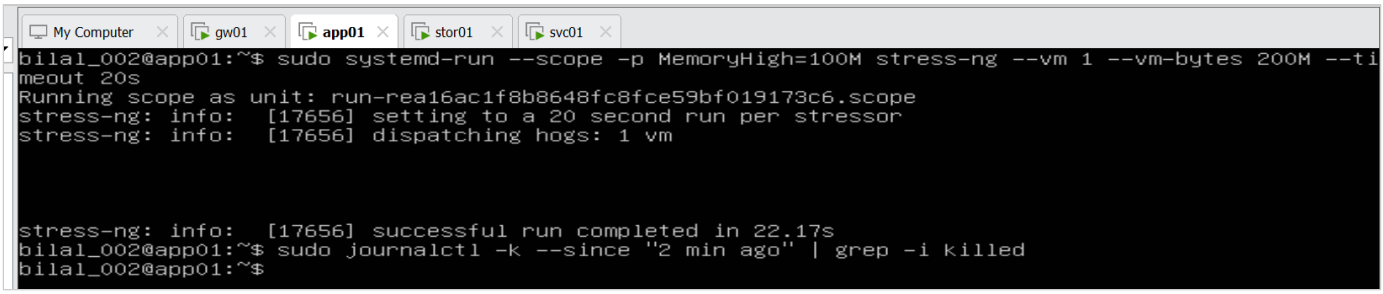
4.4 Test #2 – soft limit: slowed down, not killed

```
sudo systemd-run --scope -p MemoryHigh=100M stress-ng --vm 1 --vm-bytes 200M --timeout 20s
sudo journalctl -k --since "2 min ago" | grep -i killed     # prints nothing
```

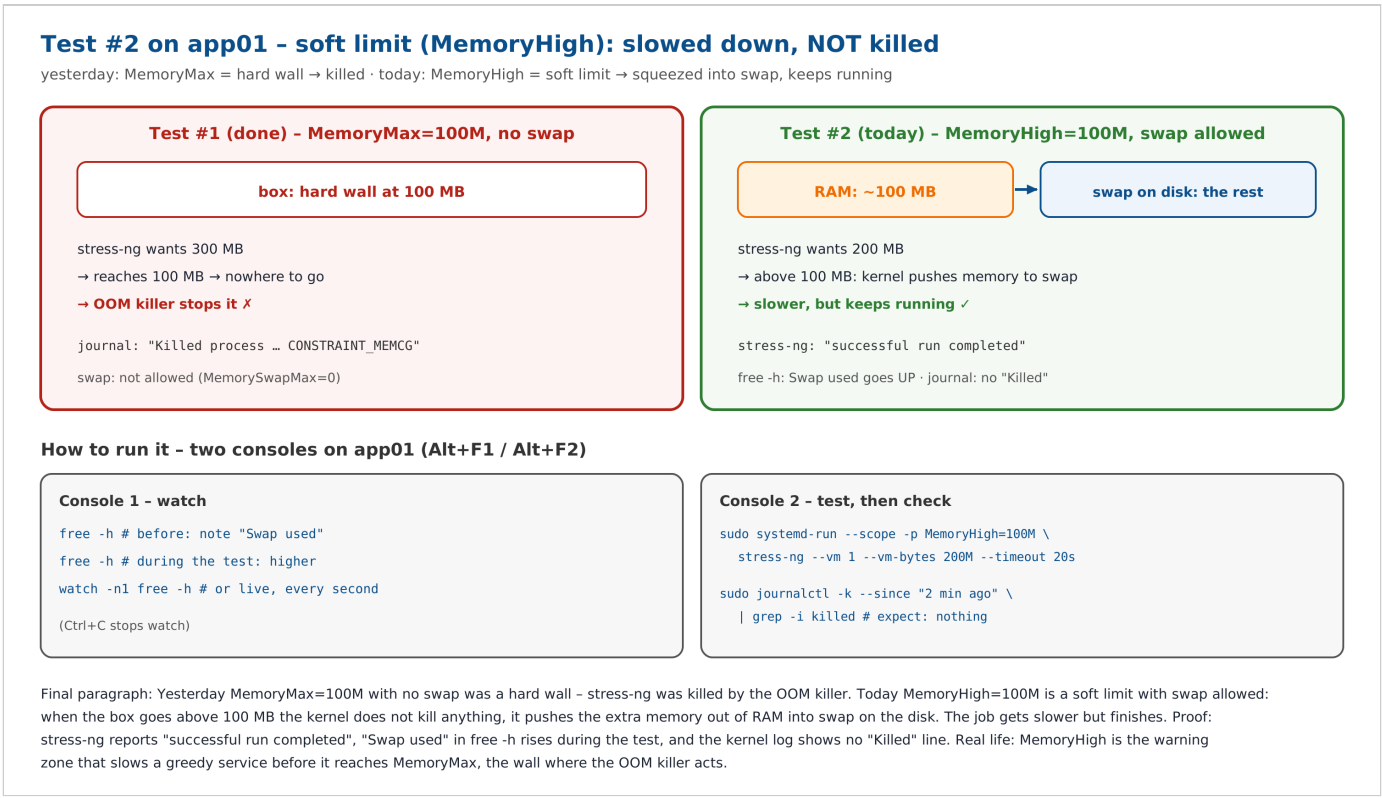
What I saw	Meaning
`successful run completed in 22.17s`	not killed – it ran to the end
22.17 s instead of 20 s	the slowdown: the kernel pushed memory above 100 MB into swap
no `Killed` line	no OOM kill at all

First try failed with `systemd-run: unrecognized option '--vm'` – an extra `-p` before `stress-ng` made systemd-run read `stress-ng` as a limit. Rule: one `-p` per limit, then the program.

📸 **Evidence:**



Test #2 – soft limit result



Test #2 – soft vs hard limit

5. Key understanding (in my own words)

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (`vmlinuz` with its helper `initrd.img`) is copied from `/boot` into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in `/etc` – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, `/run` in RAM is gone at reboot, `/proc`, `/sys` and `/dev` are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and **where I write it decides if it stays**: a live change (`sysctl -w`, `systemd-run`, `--runtime`, `mount`, `ip addr`) is a test that is gone at reboot; a file on the disk – usually in `/etc` – is read at every boot and stays. My test proved it: `systemd-run` made a temporary box in system.slice with `MemoryMax=100M`, the OOM killer stopped stress-ng at about 96 MB inside that box only (`CONSTRAINT_MEMCG`), and the box disappeared when the test ended. In the evening I learned how the kernel chooses: when a shared box is full it kills the process with the highest OOM score – mostly the biggest memory user, adjusted by `oom_score_adj` (-1000 protects, like sshd on app01; +1000 kills first). systemd only marks the service failed and restarts it if `Restart=` is set. With `MemoryHigh=100M` (test #2) the same stress-ng was not killed: the kernel pushed the extra memory into swap, the job took 22 s instead of 20 s and finished. MemoryHigh = slow down, MemoryMax = kill.

6. Next – the rest of the cgroups lab

1. ~~Step 1 (app01): the cgroup tree.~~ ✅ (Day 10)
2. ~~Step 2 (app01): memory limit → OOM killer.~~ ✅
3. ~~Soft limit (app01): MemoryHigh → slowed, not killed.~~ ✅
4. **Step 3 (app01):** CPU limit – `CPUQuota=20%` → `top` and `systemd-cgtop` show ~20 %.

5. **Step 4 (stor01):** permanent limits on `archive-worker.service`.
6. Then: first incident drill · L3 Problem in GLPI · snapshot `50-processes`.

Project 2 – Acme Analytics Linux Platform

Day 12 · Phase 4: CPU limit (CPUQuota) · the 3 limit tests on one page

Muhammad Bilal Javed · 4 November 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01 (one session 3 Nov 22:15 → 4 Nov 01:30: hard limit, OOM scores, soft limit, CPU limit)
Status	Phase 4 🟡 – Steps 4.1–4.4 ✅ · Step 4.5: concept ✅ · lab: cgroup tree ✅ · hard memory limit ✅ · soft memory limit ✅ · **CPU limit** ✅ – permanent limits on stor01 next
Author	Muhammad Bilal Javed

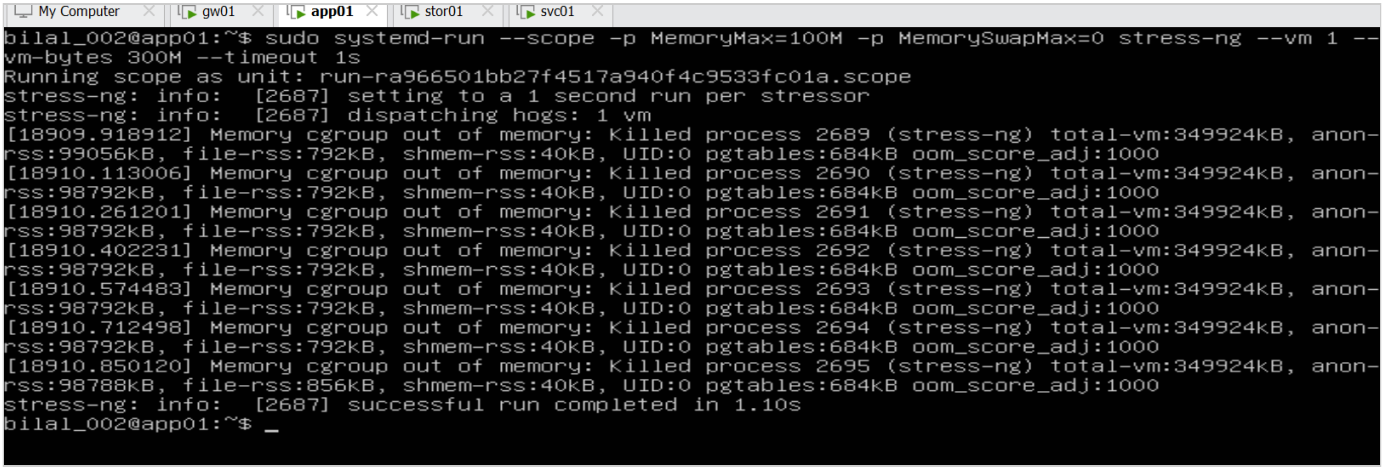
> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot or diagram. Terms are explained in the glossary. Previous day: Day 11.

---

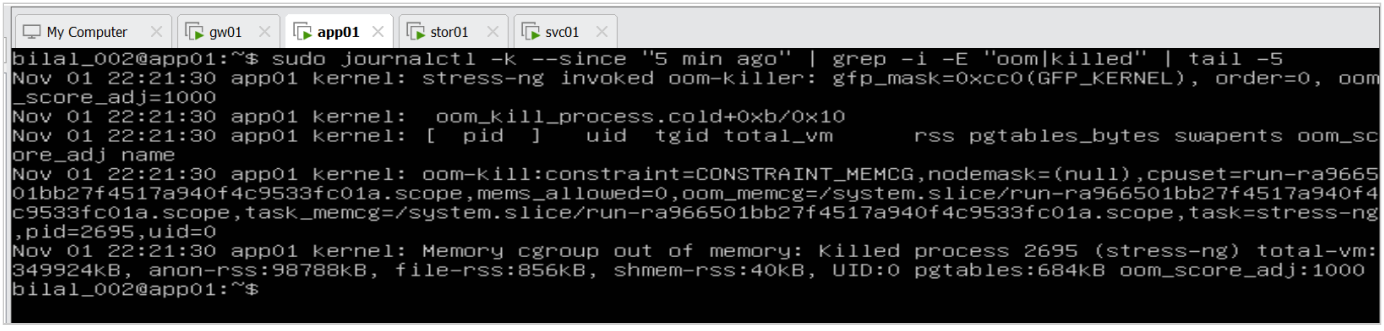
0. This session from the start (3 Nov 22:15 → 4 Nov 01:30) – tests 1 and 2

This session began on the evening of 3 November and continued past midnight. Tests 1 and 2 are documented in full in Day 11, section 4; the screenshots are repeated here so the whole session is in one place.

Test #1 – hard limit (MemoryMax=100M, no swap): KILLED

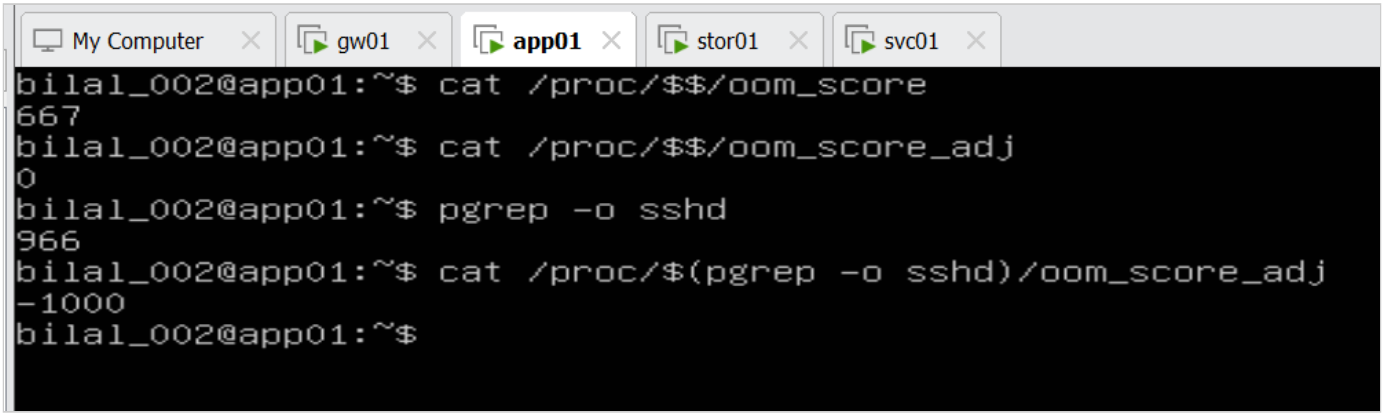


Test #1 – stress-ng killed at 100 MB



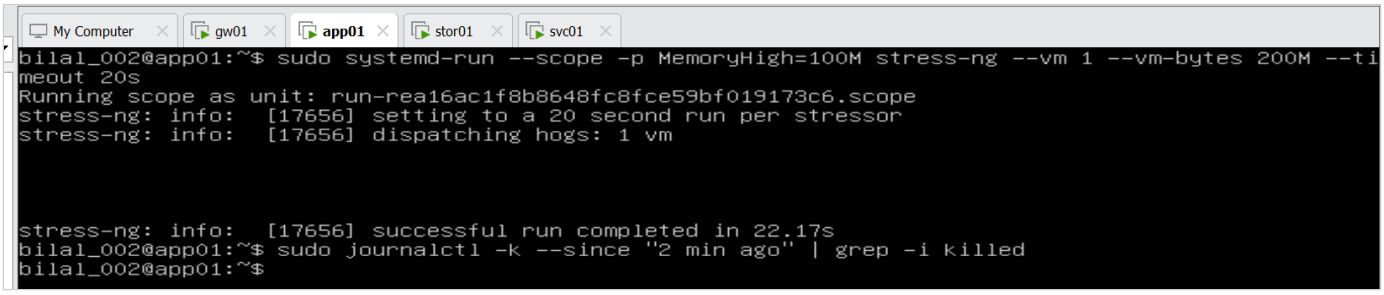
Test #1 – journal proof CONSTRAINT_MEMCG

Who is killed when a box is full – the OOM score checked on app01 (my bash 667, sshd protected at -1000)



oom_score check on app01

Test #2 – soft limit (MemoryHigh=100M): SLOWED, not killed (22.17 s instead of 20 s, no `Killed` line)



Test #2 – soft limit result

---

1. Test #3 (app01) – CPU limit: throttled, never killed

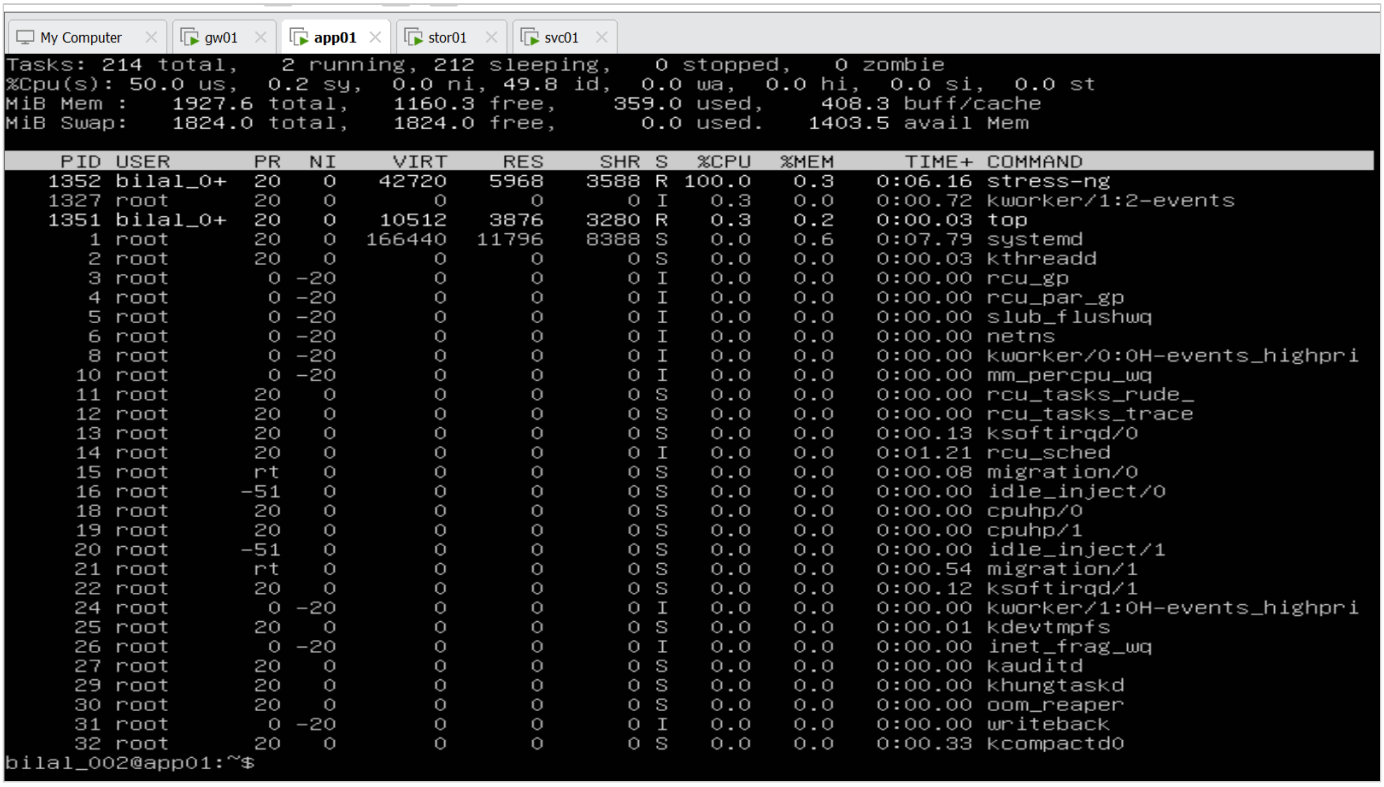
💡 **In plain words:** CPU is not like memory. With `CPUQuota=20%` the box may use the CPU for 20 ms in every 100 ms; when its share is used up it simply **waits** for the next 100 ms – like mobile data after the monthly limit: slower, not cut off. Nothing is killed.

🔧 **Technical:**

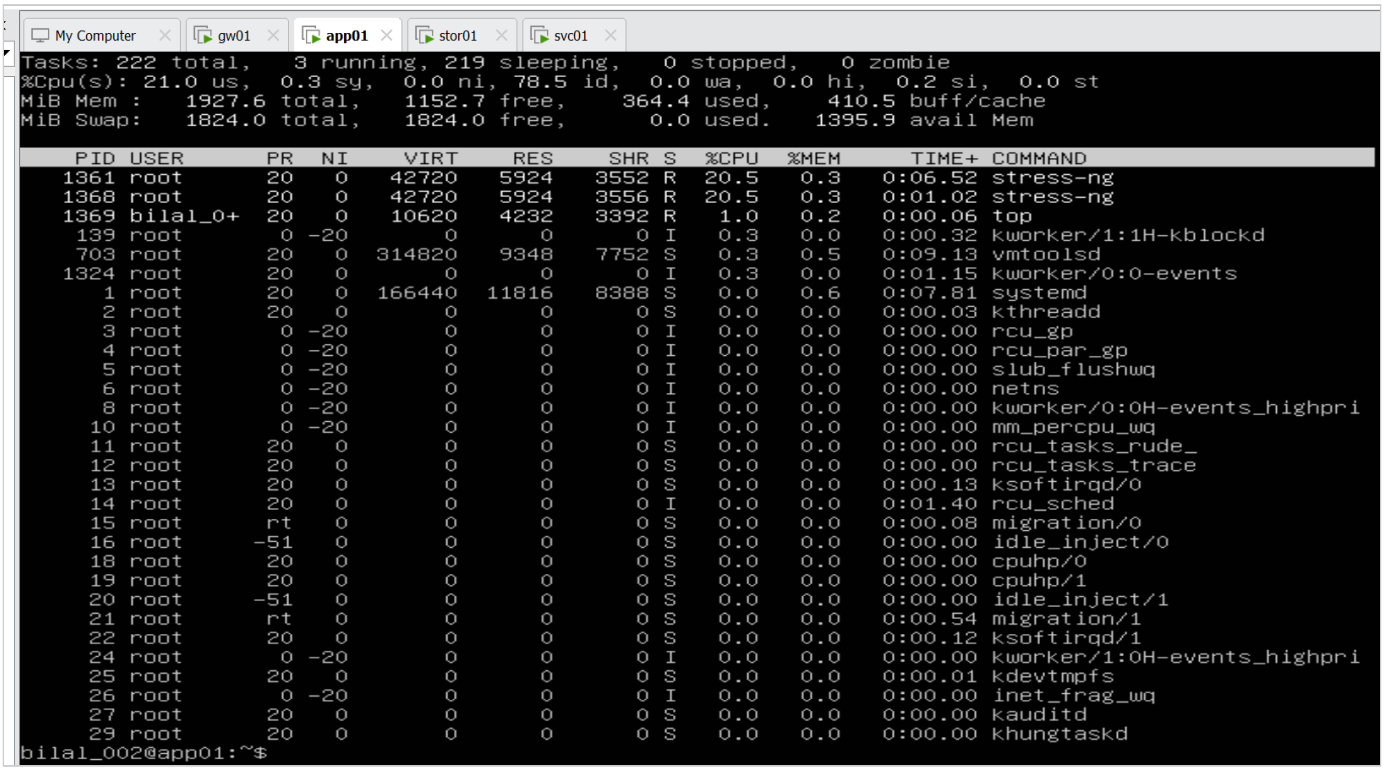
```
        # A) baseline — no limit
        stress-ng --cpu 1 --timeout 30s & sleep 2; top

        # B) with the limit
        sudo -v
        sudo systemd-run --scope -p CPUQuota=20% stress-ng --cpu 1 --timeout 60s & sleep 2; top

        # proof from the box itself (only while the test runs)
        sudo systemd-run --scope -p CPUQuota=20% stress-ng --cpu 1 --timeout 30s & sleep 2; cat /sys/fs/cgroup/
system.slice/run-*.scope/cpu.max
```

What I saw	Meaning
A: stress-ng **100.0 %**, `%Cpu(s): 50.0 us`	one full core busy – app01 has 2 cores, so one core = 50 % of the whole server
B: stress-ng **20.5 %**, `78.5 id`	throttled to the limit – the server is mostly idle, the job waits its turn
B: **two** stress-ng (PID 1361, 1368), **each 20.5 %**	test B was started twice → two boxes, each with its **own** 20 % – a limit belongs to its box
`cpu.max` → `20000 100000`	20 000 µs (20 ms) of CPU in every 100 000 µs (100 ms) = 20 %

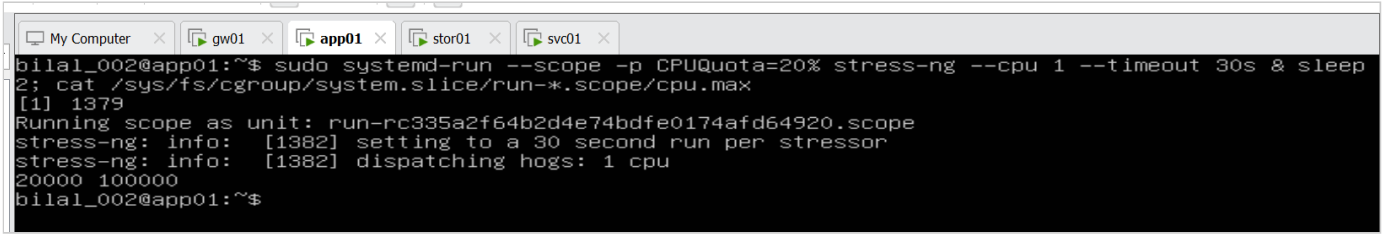
📸 **Evidence:**



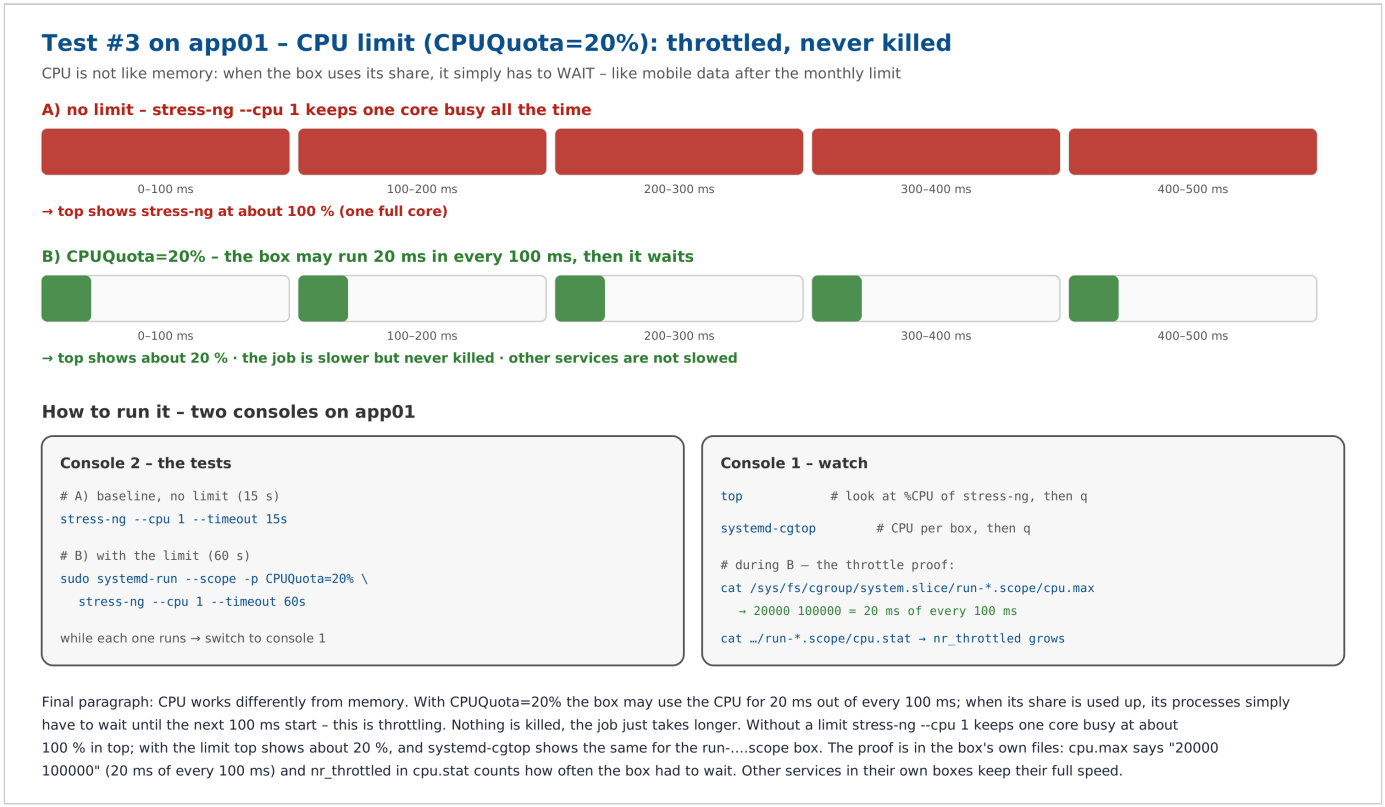
No limit – stress-ng at 100 %



CPUQuota=20% – stress-ng at 20.5 %



cpu.max – 20000 100000



Test #3 – how CPU throttling works

1.1 What went wrong on the way – and the fix

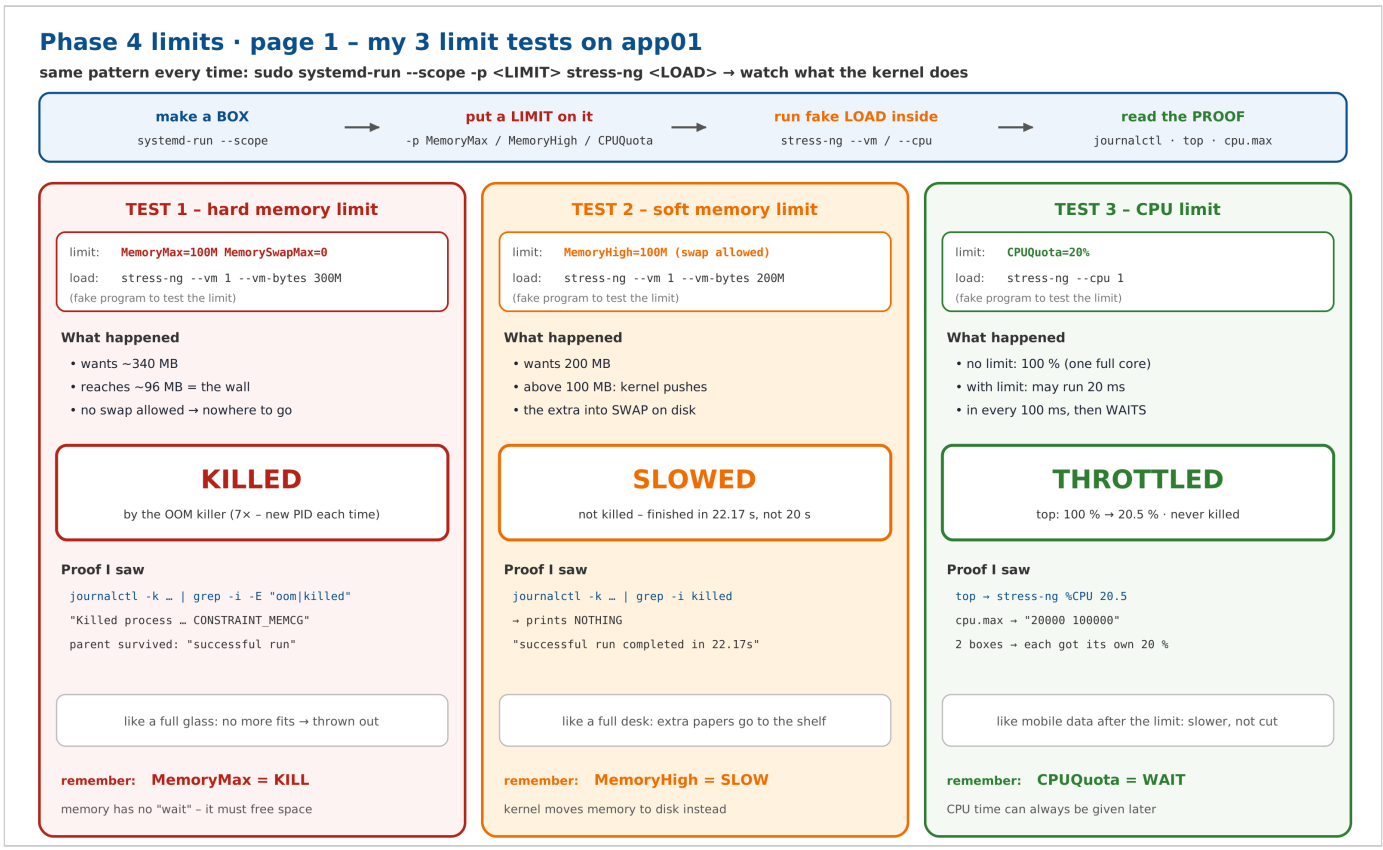
Problem	Cause	Fix
`top` showed no stress-ng, 99.8 % idle	`top` was opened after the test had finished – the stress-ng messages printed over the prompt and made it look busy	start and watch in one line: `cmd & sleep 2; top`
background sudo could get stuck	sudo asks for a password, which a background job cannot read	`sudo -v` first
`cat …/system.slice-run-*.scope/cpu.max`: No such file	a dash instead of a slash – and the box folder is deleted when the test ends	`system.slice/run-*.scope` while the test runs · Tab completes paths
app01 switched off by accident	–	nothing to clean up: the test box was a live change in RAM and was gone after the restart
switching consoles	–	click inside the VM, Alt+F1 / Alt+F2 (or `sudo chvt 2`)

2. The 3 limit tests – revision sheet

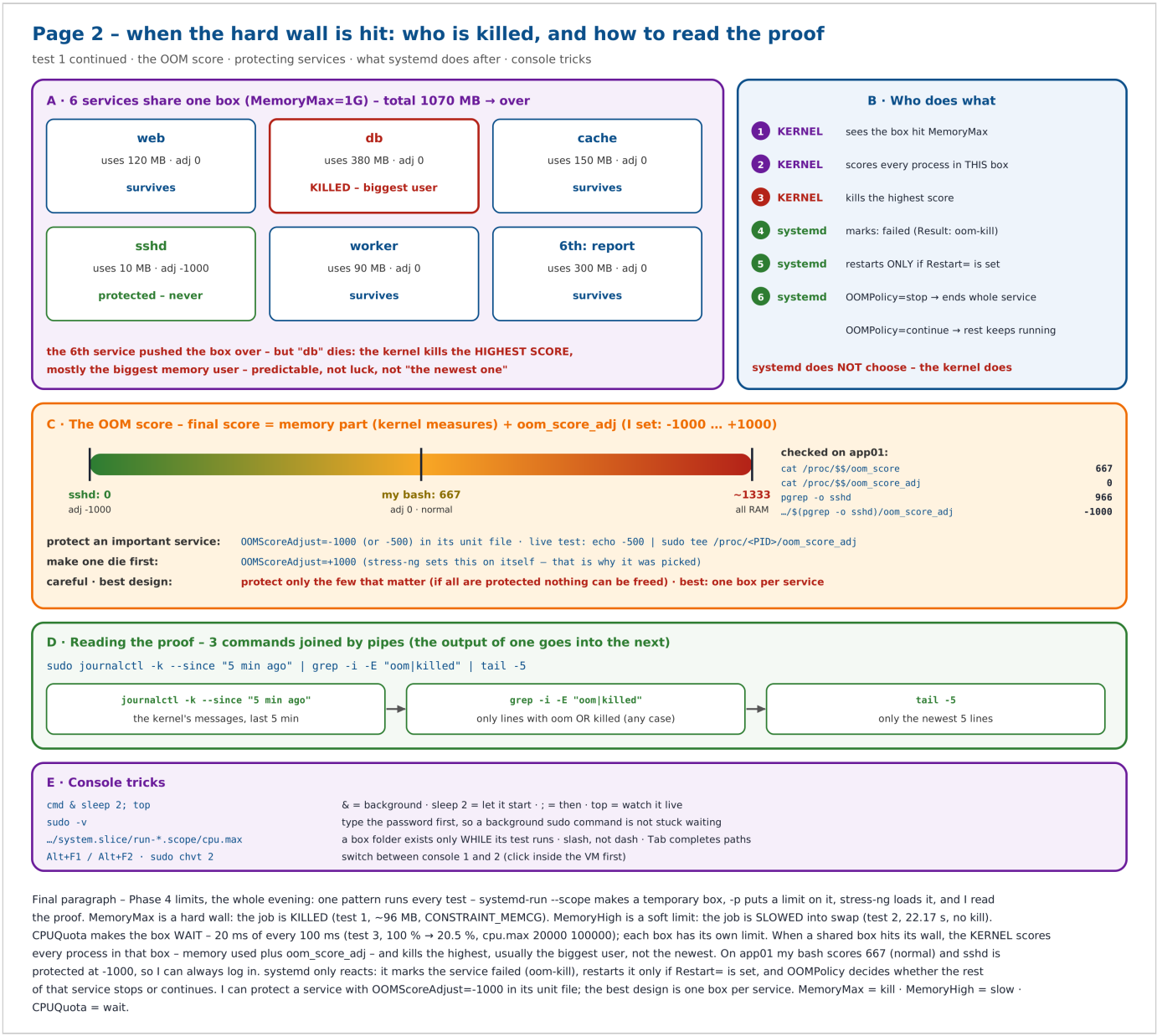
	Test 1	Test 2	Test 3
limit	`MemoryMax=100M` + no swap	`MemoryHigh=100M`	`CPUQuota=20%`
load	`stress-ng --vm 1 --vm-bytes 300M`	`stress-ng --vm 1 --vm-bytes 200M`	`stress-ng --cpu 1`
result	**KILLED** by the OOM killer at ~96 MB	**SLOWED** into swap – 22.17 s instead of 20 s	**THROTTLED** – 100 % → 20.5 %
proof	`CONSTRAINT_MEMCG` in the kernel log	no `Killed` line	`top`, `cpu.max` = `20000 100000`
remember	MemoryMax = **kill**	MemoryHigh = **slow**	CPUQuota = **wait**

Memory cannot wait – too much memory must be freed (killed or swapped). CPU time can always be given later – too much CPU only waits.

📸 **The 2-page revision sheet** (page 2: who the OOM killer chooses, the OOM score, reading the proof, console tricks):



Phase 4 limits – page 1



Phase 4 limits – page 2

3. Key understanding (in my own words)

All three tests use one pattern: `systemd-run --scope` makes a temporary box, `-p` puts a limit on it, stress-ng creates fake load inside, and I read the proof. `MemoryMax` is a hard wall – the job is **killed** (test 1). `MemoryHigh` is a soft limit – the job is **slowed** into swap (test 2). `CPUQuota=20%` lets the box run 20 ms in every 100 ms (`cpu.max` = `20000 100000`): without a limit stress-ng used 100 % of one core, with it `top` showed 20.5 % – **throttled**, never killed, and two boxes each got their own 20 % because a limit belongs to its box. When a shared box hits its memory wall, the kernel kills the process with the highest OOM score (memory used + `oom_score_adj`), and systemd only reacts. MemoryMax = kill · MemoryHigh = slow · CPUQuota = wait.

4. Next

1. **Step 4 (stor01):** permanent limits on `archive-worker.service` – written into a file, so they survive a reboot.
2. First incident drill · L3 Problem in GLPI · snapshot `50-processes` → Phase 4 complete.

4 November – finishing Phase 4

stor01 and GLPI · 15:35 – 18:15 (Germany) · the final file for today, to revise

#	What	Result
1	Step 4 – permanent limits for archive-worker on stor01 (set-property → drop-in files)	✅ **proven after reboot**
2	Incident drill – "deleted the file, but the disk is still full" (lsof +L1)	✅ **500 MB freed**
3	GLPI Incident ticket #4 for the drill	✅ **solved and closed**
4	GLPI L3 Problem behind ticket #3 – archive worker without limits	✅ **solved with Step 4**
5	VMware snapshot 50-processes on all 4 VMs	✅ **Phase 4 complete**

Today in one paragraph: The archive worker on stor01 now has permanent limits – CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 – written by `systemctl set-property` (without --runtime) as drop-in files on the disk, so they came back after a reboot. In the incident drill a deleted 500 MB file stayed on the disk because a running process still held it open: df showed it, du did not, `lsof +L1` found PID 5468, and stopping it freed the space. Both went into GLPI the ITIL way: Incident ticket #4 for the drill (fix it now) and an L3 Problem for the root cause behind ticket #3 (fix it for good), and a snapshot "50-processes" closes Phase 4.

Inside: **Part 1** Step 4 file (with a refresher of tests 1–3) · **Part 2** incident drill · **Part 3** GLPI ticket, Problem, snapshot · **Part 4** Step 4 explained: CPUQuota in time, NFS/ backup sharing, TasksMax vs restart limits.

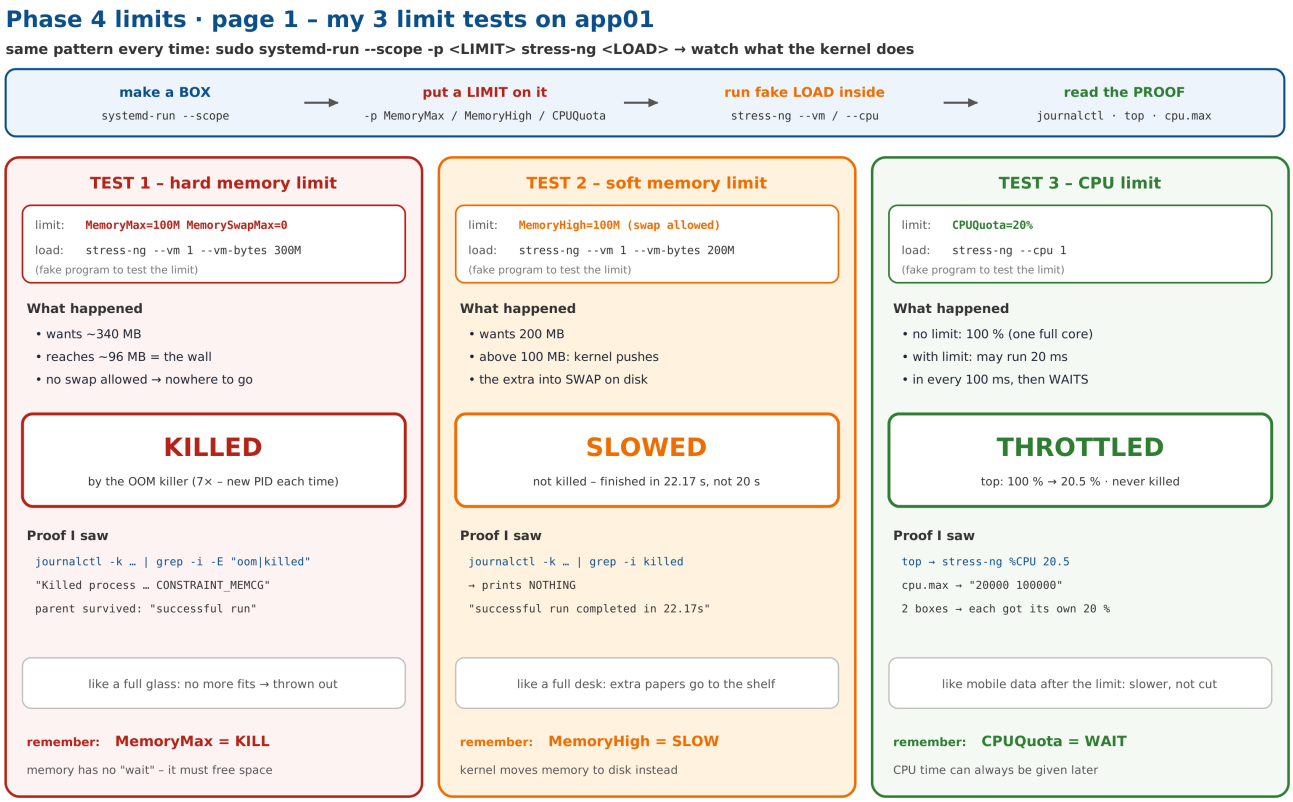
Step 4 on stor01 – permanent limits for archive-worker

Phase 4 · 4 November 2026 · every command with its own screenshot · refresher of tests 1–3 first

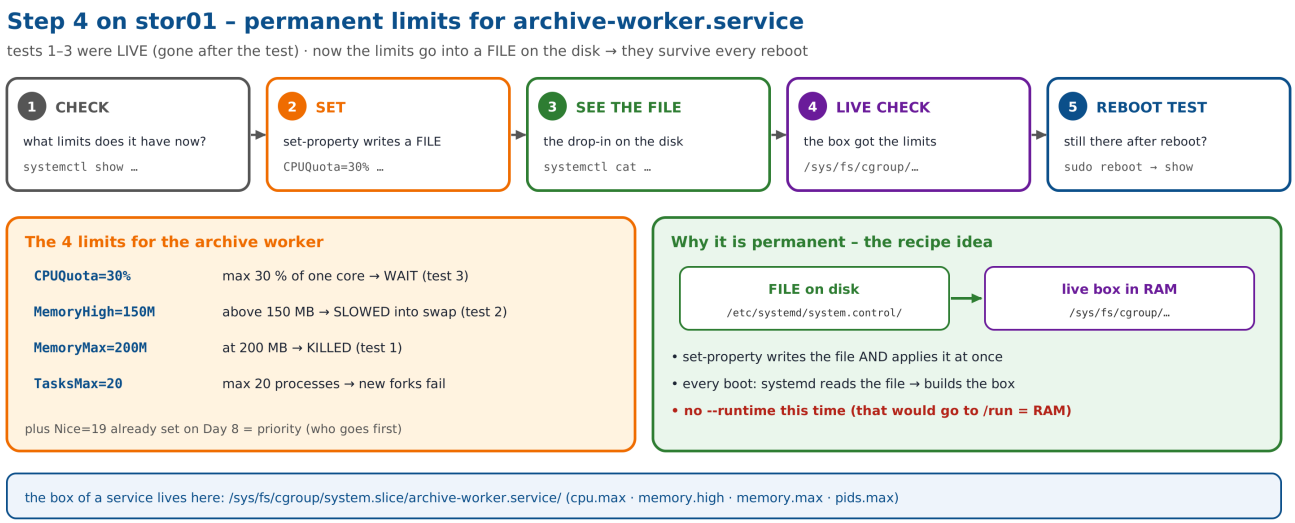
Refresher – what tests 1–3 taught (read this first)

	limit	what happens	remember
Test 1	`MemoryMax=100M`	hard wall – the OOM killer stops the job inside the box	**KILL**
Test 2	`MemoryHigh=100M`	soft limit – extra memory pushed to swap, job slower, not killed	**SLOW**
Test 3	`CPUQuota=20%`	box may run 20 ms of every 100 ms, then waits – throttled	**WAIT**

Tests 1–3 used `systemd-run --scope` = a **temporary** box, a **live change** in RAM – gone when the test ended. Today the limits go on a **real service** and into a **file on the disk** – the "recipe" that systemd reads at every boot. That is what makes them permanent.



Today's plan – 5 steps



Final paragraph: Tests 1 to 3 were live changes – the temporary boxes and their limits disappeared when each test ended. For a real service the limits must survive a reboot, so they go into a file. sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20 does two things at once: it applies the limits to the running box and writes them as a drop-in file under /etc/systemd/system.control/ on the disk. At every boot systemd reads that file and builds the box again with the same limits. I check the file with systemctl cat, the live box under /sys/fs/cgroup/system.slice/archive-worker.service/, and prove it is permanent with a reboot. CPUQuota makes the worker wait, MemoryHigh slows it, MemoryMax kills it, TasksMax stops it creating too many processes – and Nice=19 from Day 8 still decides that it goes last.

Step 1 – check what the worker has now

```
systemctl status archive-worker --no-pager
```

My Computer | gw01 | app01 | **stor01** | svc01

```
bilal_003@stor01:~$ systemctl status archive-worker --no-pager
● archive-worker.service - Acme archive worker (processes the archive queue)
     Loaded: loaded (/etc/systemd/system/archive-worker.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-11-04 14:36:57 UTC; 2min 34s ago
   Main PID: 1019 (archive-worker.)
      Tasks: 2 (limit: 964)
     Memory: 1.0M
        CPU: 689ms
     CGroup: /system.slice/archive-worker.service
             ├─1019 /bin/bash /usr/local/bin/archive-worker.sh
             └─1639 sleep 5

Nov 04 14:36:57 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Nov 04 14:36:57 stor01 archive-worker[1024]: worker started (PID 1019)
```

What it shows: the worker is **active (running)** since boot (enabled = starts automatically). Its box is `/system.slice/archive-worker.service` with 2 processes inside: the worker script (PID 1019) and the `sleep 5` it runs while waiting for jobs. **Tasks: 2 (limit: 964)** – 964 is the default, not a limit I chose. Memory 1.0 MB – tiny.

```
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
```

```
bilal_003@stor01:~$ systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -
p TasksMax -p Nice
CPUQuotaPerSecUSec=infinity
MemoryHigh=infinity
MemoryMax=infinity
TasksMax=964
Nice=19
```

Before: `infinity` = no CPU or memory limit · TasksMax 964 = the default · `Nice=19` = the low priority I set on Day 8. Priority says *who goes first*; it does not cap anything.

Step 2 – set the 4 limits, permanently

```
sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M
MemoryMax=200M TasksMax=20
```

No output = success. **No `--runtime`** this time – with it the limits would go to `/run` (RAM) and vanish at reboot. Without it, set-property does two things at once: **applies** the limits to the running box and **writes** them to files on the disk.

limit	meaning for the worker
`CPUQuota=30%`	max 30 % of one core – a big archive job cannot take the CPU from NFS or backups (test 3 = wait)
`MemoryHigh=150M`	above 150 MB it is slowed into swap (test 2)
`MemoryMax=200M`	at 200 MB the OOM killer stops it (test 1) – `Restart=on-failure` in its unit file then starts it again
`TasksMax=20`	max 20 processes – a bug that forks endlessly cannot fill the PID pool

Step 3 – see the files on the disk (the recipe)

```
systemctl cat archive-worker
```

```
bilal_003@stor01:~$ systemctl cat archive-worker
# /etc/systemd/system/archive-worker.service
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
```

Part 1 – my original unit file `/etc/systemd/system/archive-worker.service` from Day 8 – unchanged: the script, `Restart=on-failure`, `Nice=19`, `IOSchedulingClass=idle`.

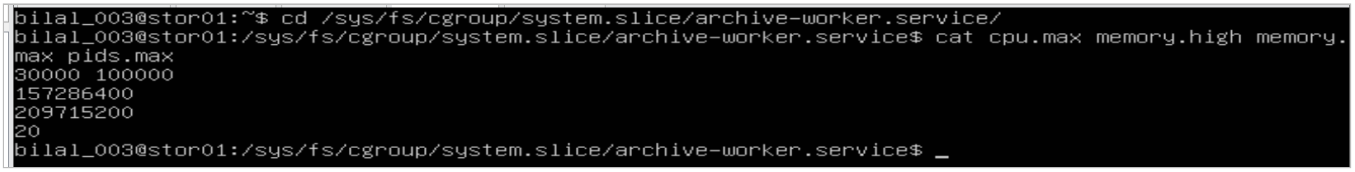
```
# /etc/systemd/system.control/archive-worker.service.d/50-CPUQuota.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
CPUQuota=30%

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryHigh.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryHigh=157286400

# /etc/systemd/system.control/archive-worker.service.d/50-MemoryMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
MemoryMax=209715200

# /etc/systemd/system.control/archive-worker.service.d/50-TasksMax.conf
# This is a drop-in unit file extension, created via "systemctl set-property"
# or an equivalent operation. Do not edit.
[Service]
TasksMax=20
bilal_003@stor01:~$ _
```

Part 2 – four new drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, one per limit (`50-CPUQuota.conf` …). They are added *on top of* the unit file without changing it. "Do not edit" – to change a limit, run set-property again.

Memory is stored in **bytes**: 157286400 = 150 × 1024 × 1024 = 150 MB · 209715200 = 200 MB.

Step 4 – check the live box in the kernel

```
cd /sys/fs/cgroup/system.slice/archive-worker.service/
cat cpu.max memory.high memory.max pids.max
```

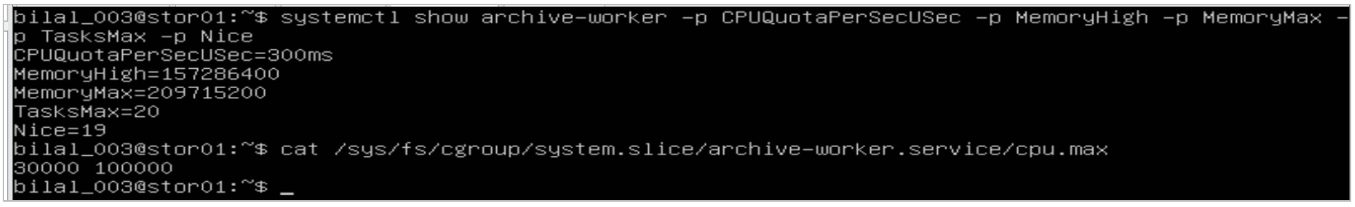


file	value	meaning
`cpu.max`	30000 100000	30 ms of every 100 ms = 30 %
`memory.high`	157286400	150 MB soft limit
`memory.max`	209715200	200 MB hard wall
`pids.max`	20	max 20 processes

The file on the disk (step 3) is the **recipe**; these values in `/sys/fs/cgroup` are the **live box** in RAM that the kernel enforces.

Step 5 – the reboot test (proof that it is permanent)

```
sudo reboot
# after logging in again:
systemctl show archive-worker -p CPUQuotaPerSecUSec -p MemoryHigh -p MemoryMax -p
TasksMax -p Nice
cat /sys/fs/cgroup/system.slice/archive-worker.service/cpu.max
```



after reboot	meaning
`CPUQuotaPerSecUSec=300ms`	30 % = 300 ms of CPU per second – same limit, new way of writing it
`MemoryHigh=157286400` · `MemoryMax=209715200`	150 MB soft · 200 MB hard – still there
`TasksMax=20` · `Nice=19`	process cap and priority – still there
`cpu.max 30000 100000`	the kernel's live box was rebuilt with the limit

Proof: RAM was emptied by the reboot, yet the box came back with every limit – because systemd read the drop-in files on the disk at boot and built the box again. ✅ Permanent.

Final paragraph: Tests 1 to 3 were live changes – their temporary boxes and limits disappeared when each test ended. For the real archive worker on stor01 the limits must survive a reboot, so they go into files. Before, the worker had no CPU or memory limit

(`infinity`), the default TasksMax 964 and Nice=19 from Day 8. `sudo systemctl set-property archive-worker.service CPUQuota=30% MemoryHigh=150M MemoryMax=200M TasksMax=20` – without `--runtime` – applied the limits at once and wrote four drop-in files under `/etc/systemd/system.control/archive-worker.service.d/`, next to my unchanged unit file; `systemctl cat` shows both. The kernel's live box in `/sys/fs/cgroup/system.slice/archive-worker.service/` confirms `cpu.max 30000 100000`, memory.high 150 MB, memory.max 200 MB and pids.max 20. Now the worker waits above 30 % CPU, is slowed above 150 MB, is killed at 200 MB – and restarted by `Restart=on-failure` – and can never run more than 20 processes, while Nice=19 still makes it go last. At every boot systemd reads the drop-ins and builds the box again: the reboot test proved it: after the restart `systemctl show` still reported 300ms, 150 MB, 200 MB, 20 tasks and Nice=19, and cpu.max was again 30000 100000.

Step 4 explained – how the 4 limits really work

questions from revising: how does CPUQuota=30% work in time? what does TasksMax count? how are restarts limited?

CPU throttling – CPUQuota=30% on stor01, everything on one page

one CPU, three jobs: the archive worker · the NFS share for app01 · the nightly backup (or any other job running in the background)

① Who gets the CPU

WITHOUT a limit

archive job 92%

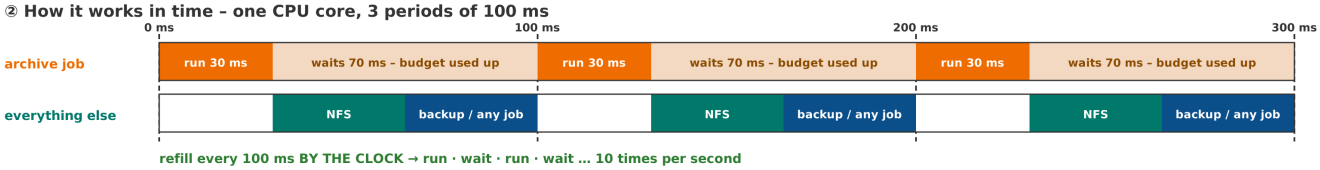
the archive job takes ~92 %
app01 waits for NFS · the backup crawls
the others only get tiny gaps

WITH CPUQuota=30%

archive 30% | NFS 35% | backup / any job 35%

the archive job never gets more than 30 %
NFS and the backup keep their share
the archive job just takes longer

② How it works in time – one CPU core, 3 periods of 100 ms



refill every 100 ms BY THE CLOCK → run · wait · run · wait … 10 times per second

③ The facts to remember

`cpu.max = 30000 100000`
30 000 µs (30 ms) budget in every 100 000 µs (100 ms) period
runs at FULL speed
until its 30 ms are used up – the CPU never slows down
then it WAITS
only until the next 100 ms period – not until the backup ends
the 70 ms go to
whatever else needs the CPU: NFS, backup, sshd, any job
nothing else running?
the CPU stays idle – CPUQuota is a HARD cap (30 % stays 30 %)
never killed
CPU can always be given later – too much CPU only waits

④ Pocket money that refills every "day"

every day you get 300 (= 30 ms every 100 ms)
you spend at your NORMAL speed until it is gone
then you have 0 → you wait until tomorrow
tomorrow: 300 again → spend → wait → …

300 | wait — day 1 = 100 ms
300 | wait — day 2 = 100 ms
300 | wait — day 3 = 100 ms

the speed of spending never changes – only the waiting

⑤ Proven in test 3 (app01): no limit → stress-ng 100 % · CPUQuota=20% → 20.5 % · cpu.max 20000 100000 · never killed
and on stor01 (Step 4): archive-worker cpu.max 30000 100000 – still there after reboot

Final paragraph: On stor01 the archive worker, the NFS share for app01 and the nightly backup share one CPU. Without a limit one big archive job could take about 92 % of it, so app01 would wait for NFS and the backup would crawl. CPUQuota=30% gives the archive box a budget of 30 ms in every 100 ms period (cpu.max 30000 100000). The job runs at its normal full speed until the 30 ms are used up, then it must wait – not until the backup ends, only until the next 100 ms period, when the clock refills its budget. In the other 70 ms the CPU works for whatever else needs it: NFS, the backup or any other job; if nothing else is running the CPU stays idle, because CPUQuota is a hard cap. Like pocket money paid every day, the spending speed never changes – only the waiting. This repeats 10 times per second, so the archive job gets 30 % on average and simply takes longer; nothing is killed. Test 3 proved it (100 % → 20.5 % with 20 %), and on stor01 the 30 % limit survived a reboot.

Two lines made clear – who shares stor01's CPU, and what TasksMax really counts

① "A big archive job cannot take the CPU from NFS and backups"

NFS (not NFC) = the network share on stor01 · all three jobs run ON stor01 and share ITS CPU – it is not about the CPU of another server

WITHOUT a limit

stor01 CPU (100 %)

archive job 92%

the archive job takes almost everything
→ app01 waits for the NFS share (slow)
→ the nightly backup runs very slowly
(backup + NFS fight for the last few %)

WITH CPUQuota=30%

stor01 CPU (100 %)

archive 30% | NFS 35% | backup 35%

the archive job may use at most 30 %
→ NFS keeps answering app01 quickly
→ the backup still gets its share
(the archive job just takes longer – fine)

② TasksMax is NOT "how many restarts" – it is "how many processes at the SAME time"

TasksMax=20

max 20 processes/threads ALIVE at once in the box

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20

the 21st fork fails: "Resource temporarily unavailable"
protects stor01's PID pool from a fork bug

restarts – a DIFFERENT setting

`Restart=on-failure` → restart it when it crashes / is killed
`StartLimitBurst=5` → at most 5 starts …
`StartLimitIntervalSec=10s` … within 10 seconds

more often → systemd gives up: "start request repeated too quickly" – the service stays failed

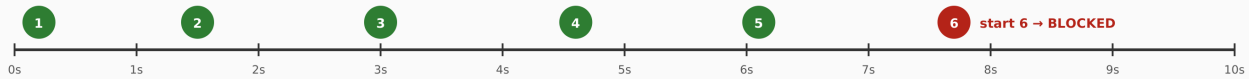
(5 in 10 s are the systemd defaults)

Final paragraph: stor01 does three jobs on the same CPU – it shares /backups over NFS to app01, it runs the nightly backups, and it runs the archive worker. Without a limit one big archive job could take almost all of stor01's CPU, so app01 would wait for the NFS share and the backups would crawl. CPUQuota=30% lets the worker use at most 30 % of a core, so NFS and the backups always keep their share; the archive job only takes longer. TasksMax=20 is something else: it is the maximum number of processes or threads alive in the worker's box at the same time – the 21st fork fails, which protects the PID pool. It does not count restarts. Restarts are set by Restart=on-failure (restart when it dies) and limited by StartLimitBurst and StartLimitIntervalSec (by default 5 starts in 10 seconds, then systemd gives up and the service stays failed).

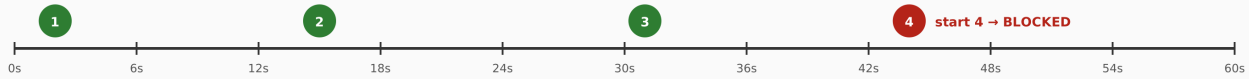
StartLimitBurst + StartLimitIntervalSec – the brake on endless RESTARTS (not threads)

one rule: "at most BURST starts within INTERVAL seconds" – two examples with different numbers

Example A – the DEFAULT (what every service has if I set nothing): 5 starts in 10 seconds



Example B – MY OWN choice in the unit file: 3 starts in 60 seconds



Example B written in the unit file

```
[Unit]
StartLimitIntervalSec=60 ← INTERVAL: 60 s
StartLimitBurst=3 ← BURST: 3 starts

[Service]
Restart=on-failure ← restart when it dies
```

Check and recover

```
systemctl show archive-worker -p StartLimitBurst \
  -p StartLimitIntervalUSec
```

archive-worker has nothing set → shows the default 5 · 10s

blocked? fix the cause, then:

```
sudo systemctl reset-failed archive-worker
```

Final paragraph: Restart=on-failure starts a service again when it dies. To stop an endless crash–restart loop, systemd counts the STARTS (not threads or processes) inside a time window: StartLimitBurst is how many starts are allowed, StartLimitIntervalSec is the length of the window. If I set nothing, the default is 5 starts in 10 seconds (example A); in the unit file I can choose my own numbers, for example StartLimitIntervalSec=60 and StartLimitBurst=3 = at most 3 starts per minute (example B). One start too many is blocked with "start request repeated too quickly" and the service stays failed until I fix the cause and run systemctl reset-failed. TasksMax is different: it counts processes running at the same moment.

Incident drill on stor01 – "I deleted the file, but the disk is still full"

Phase 4 · 4 November 2026 · a deleted file still held open by a running program · every command with its own marked-up screenshot

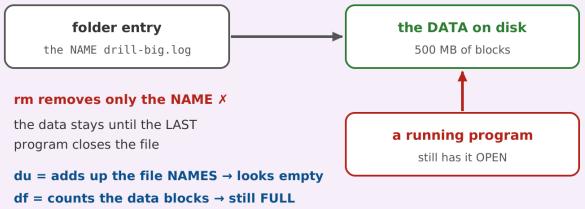
Incident drill on stor01 – "I deleted the file, but the disk is still full"

a classic real ticket and interview question: a program still holds a deleted file open, so its space is NOT freed

Ticket (client words): "/backups on stor01 was nearly full. I deleted a big old log file – but df still says it is full!"

real-life cause: logrotate or a person deletes a log file while the program that writes it keeps running

Why the space is not freed



rm removes only the NAME ✗

the data stays until the LAST program closes the file

du = adds up the file NAMES → looks empty

df = counts the data blocks → still FULL

The drill – 4 parts

1 BUILD the problem
make a 500 MB file · a program holds it · delete it

2 SEE the symptom
df: still used · du: nothing there

3 FIND the cause
sudo lsof +L1 → "(deleted)" + the PID

4 FIX + TICKET
stop that program → space free · GLPI ticket

How admins fix it in real life

- best: restart the service that holds the file (systemctl restart ...) – it closes the old file and opens a new one
- if it must not restart: empty the deleted file through the process: : > /proc/<PID>/fd/<FD> – frees the space at once

Final paragraph: In Linux, deleting a file with rm only removes its name from the folder. The data blocks are freed only when the last program that has the file open closes it. So if a running program still holds a deleted file – typically a log file removed while the service keeps writing – df still counts the space as used, while du, which adds up the file names it can see, shows nothing. The tool to find it is sudo lsof +L1: it lists open files whose name is gone, marked "(deleted)", with the PID and file descriptor of the program holding them. The fix is to restart or stop that program (or empty the file through /proc/<PID>/fd/<FD> if it must keep running), and the space comes back at once. In the drill I build this on stor01's /backups, find it, fix it and record it as a GLPI ticket.

The incident drill – final corrected version, in plain words

an INCIDENT drill (a disk-space mystery), not a restore drill · /backups was used only because it is a separate disk with easy numbers

1 look at the disk — `df -h /backups`
how full is the /backups disk? → 60K used

2 make a big file — `sudo fallocate -l 500M …`
fallocate = FILE ALLOCATE = reserve 500 MB for one empty file (no stress, just space) → 501M used

3 a process holds it OPEN — `sleep 3600 < file &`
the KEY step: process sleep (PID 5468) holds the file open on its file descriptor 0 – it plays a service and its log

4 delete the name — `sudo rm file`
only the NAME is gone – USED stays 501M, because the process still holds the file open

5 prove the mystery — `sudo du -sh /backups`
du = disk usage · -s = summary (one total) · -h = human-readable → only 56K (du counts names)

6 find + fix — `sudo lsof +L1 → kill 5468`
lsof = list open files · +L1 = less than 1 name left = deleted but still open → shows the PID of the PROCESS → stop it → 60K

Words – say it right

fallocate	file ALLOCATE (reserve), not "locate"
the PROCESS has the PID	the file has no PID
process HOLDS it open	a file has no "parent" – that word is for processes
df vs du	df counts disk blocks · du counts file names

Same effect with a copy A → B

cp is reading file X · someone deletes X
→ cp still finishes the copy · X's space stays used
→ freed when cp closes X – after seconds, nobody notices

a SERVICE keeps its log open for weeks
→ the space stays "lost" until the service is restarted

Final paragraph: This was an incident drill, not a restore drill. df -h /backups showed the disk almost empty (60K). sudo fallocate (file allocate) reserved 500 MB for one empty file – no stress, just space – so used went to 501M. The key step: sleep 3600 < file & – a process (PID 5468) holding the file open on its file descriptor 0, like a service holds its log. sudo rm removed only the name, so used stayed at 501M, while du -sh (disk usage, summary, human-readable) counted only 56K, because du adds up the names it can find and df counts the blocks. lsof +L1 (list open files with fewer than one name) showed the deleted file and the PID of the process holding it – the file itself has no PID and no parent. Stopping that process freed the 500 MB; on a real server I would restart the service. The same happens when a file is deleted while cp is copying it, but a copy closes the file within seconds – a service keeps its log open for weeks, which is why this becomes a real incident.

1 – Build the problem

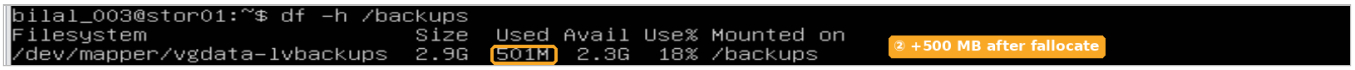
```
df -h /backups
```

① **Before:** /backups (an LVM volume, 2.9 GB) uses only **60K** – it is empty, 1 %.

```
sudo fallocate -l 500M /backups/drill-big.log
```

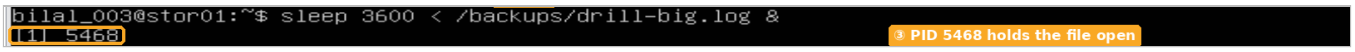
No output = success. `fallocate` reserves 500 MB on the disk at once – our big "log file".

```
df -h /backups
```



② **After fallocate: 501M** used, 18 % – the new file takes the 500 MB.

```
sleep 3600 < /backups/drill-big.log &
```



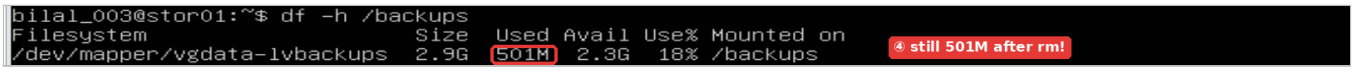
③ **A program holds the file open:** `sleep` uses the file as its input and runs for an hour in the background – job **[1]**, **PID 5468**. It stands in for a real service keeping its log file open.

```
sudo rm /backups/drill-big.log
```

No output = the file name is deleted.

2 – See the symptom

```
df -h /backups
```



④ **The problem:** still **501M** used after `rm`! `df` counts the data blocks on the disk – they are still there.

```
sudo du -sh /backups
```

⑤ **The contradiction:** `du` adds up the file names it can find – the name is gone, so it sees only **56K**. When df and du disagree, think "deleted but still open".

3 – Find the cause

```
sudo lsof +L1
```

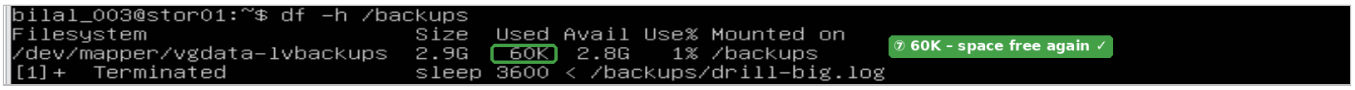
⑥ **The cause:** `lsof +L1` lists open files that have no name left (fewer than 1 link). Marked: **PID 5468** (sleep) · **524288000** bytes = 500 MB · **(deleted)**. FD `0r` = its input, opened for reading · NLINK `0` = no names left.

4 – Fix it

```
kill 5468
```

Stop the program that holds the file (the PID from lsof).

```
df -h /backups
```



⑦ **Fixed:** back to **60K**, 1 % – the kernel freed the 500 MB as soon as the last program closed the file. `[1]+ Terminated` confirms the background job ended.
On a real server I would not kill a service blindly – I would **restart** it (`sudo systemctl restart <service>`) so it closes the old log and opens a new one; if it must keep running, `: > /proc/<PID>/fd/<FD>` empties the deleted file in place.

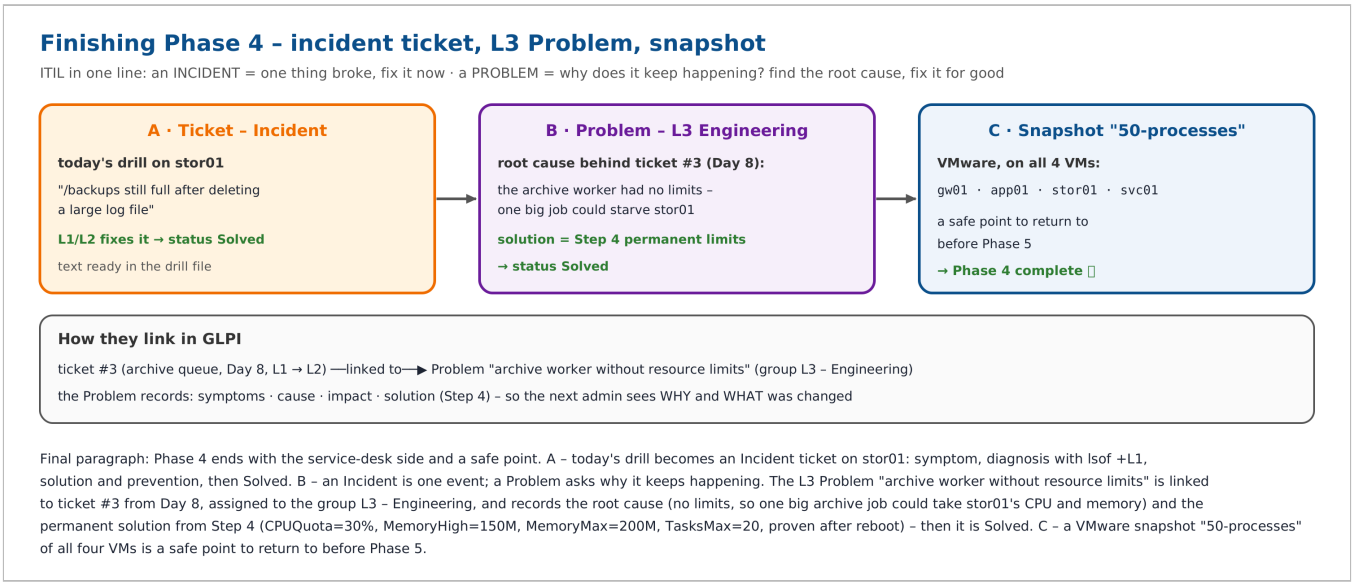
5 – Record it as a GLPI ticket (text ready to paste)

Title: stor01 – /backups still full after deleting a large log file
Type: Incident · **Category:** Storage · **Urgency:** Medium · **Asset:** stor01
Description: User deleted /backups/drill-big.log (500 MB) but `df -h /backups` still shows 501M used (18 %). `du -sh /backups` shows only 56K.
Diagnosis: `sudo lsof +L1` → process `sleep`, PID 5468, holds /backups/drill-big.log (deleted), 524288000 bytes, NLINK 0. The space is not freed while a process keeps the deleted file open.
Solution: Stopped the holding process (kill 5468). `df -h /backups` → 60K used, 1 %. In production: restart the service that holds the file, or empty it via /proc/<PID>/fd/<FD>.
Prevention: rotate logs with logrotate using `copytruncate` or a service reload, never delete open log files by hand; check `lsof +L1` when df and du disagree.

Final paragraph: This was an incident drill, not a restore drill. `df -h /backups` showed the disk almost empty (60K). `sudo fallocate` (file allocate) reserved 500 MB for one empty file – no stress, just space – so used went to 501M. The key step: `sleep 3600 < file &` – a process (PID 5468) holding the file open on its file descriptor 0, like a service holds its log. `sudo rm` removed only the name, so used stayed at 501M, while `du -sh` (disk usage, summary, human-readable) counted only 56K, because du adds up the names it can find and df counts the blocks. `lsof +L1` (list open files with fewer than one name) showed the deleted file and the PID of the process holding it – the file itself has no PID and no parent. Stopping that process freed the 500 MB; on a real server I would restart the service, and prevent it with proper log rotation. The same happens when a file is deleted while `cp` is copying it, but a copy closes the file within seconds – a service keeps its log open for weeks, which is why this becomes a real incident. The incident is recorded in GLPI as ticket #4.

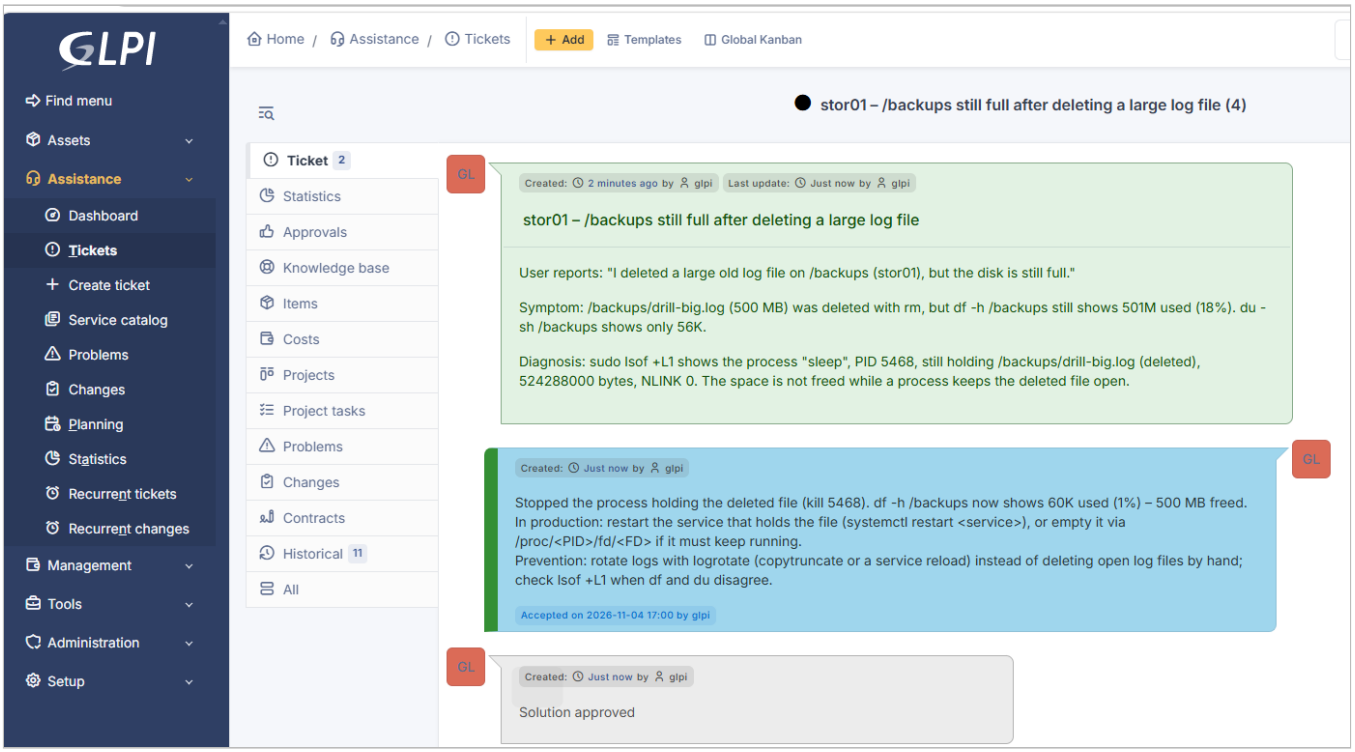
Closing Phase 4 – Incident ticket, L3 Problem, snapshot

4 November 2026 · GLPI on svc01 (http://192.168.60.40) · ITIL: Incident = fix it now · Problem = find the root cause, fix it for good



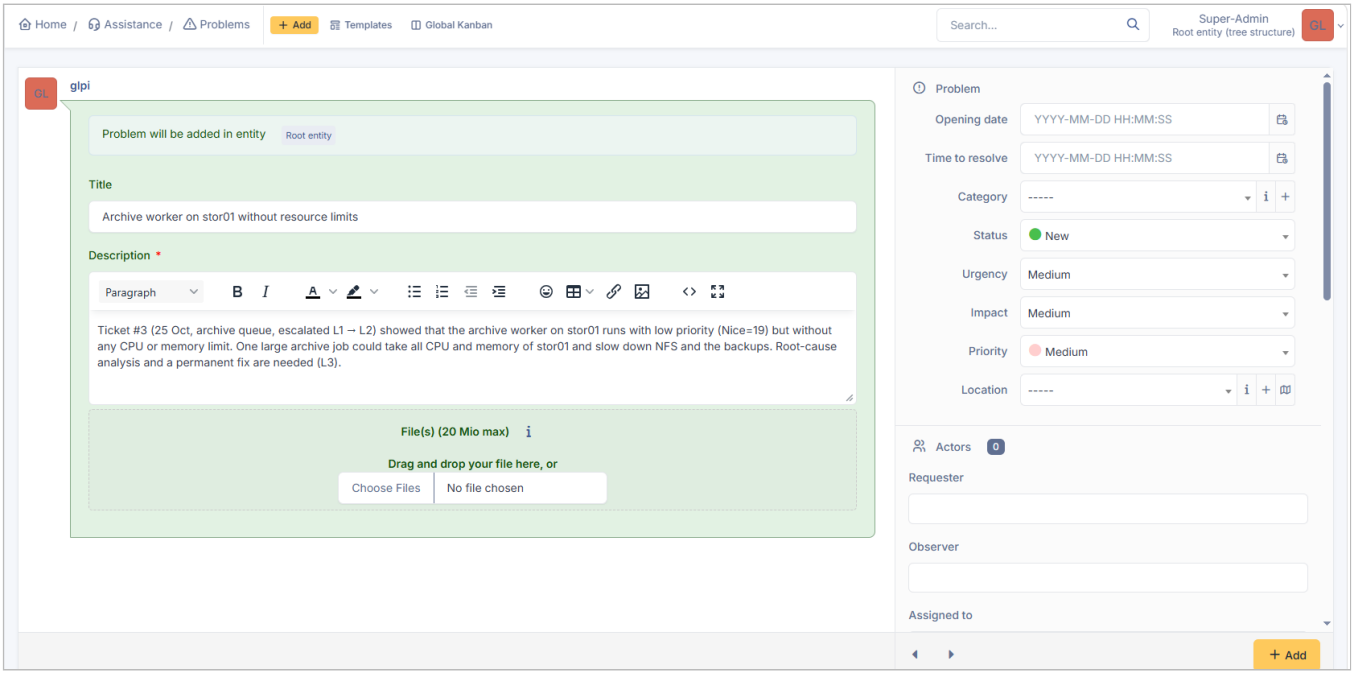
A – Incident ticket #4 for today's drill – Solved and closed

Title *stor01 – /backups still full after deleting a large log file* · Type Incident · Category Storage & Backup · Requester **alice** (Requester = who reported it) · Assigned to group **L2 – Linux Storage** · Item **stor01** (Item = what is broken). The description holds the symptom and the lsof diagnosis; the solution holds the fix and the prevention. The solution was accepted and the ticket closed (black dot). In a company alice would approve it herself.

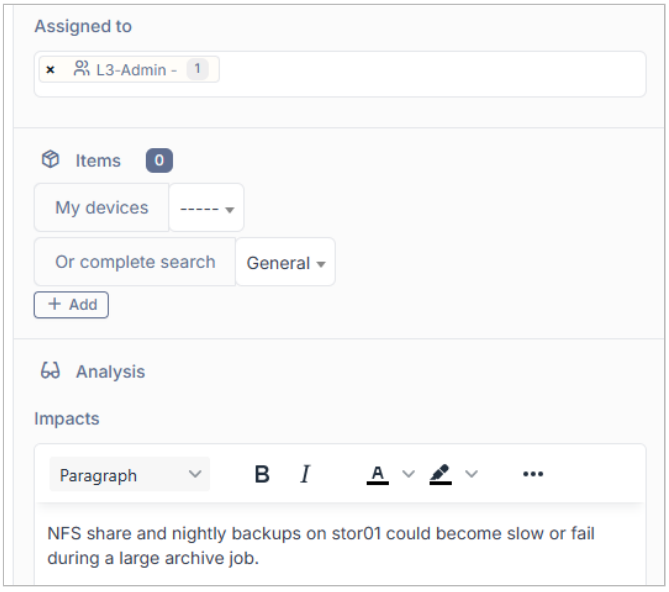


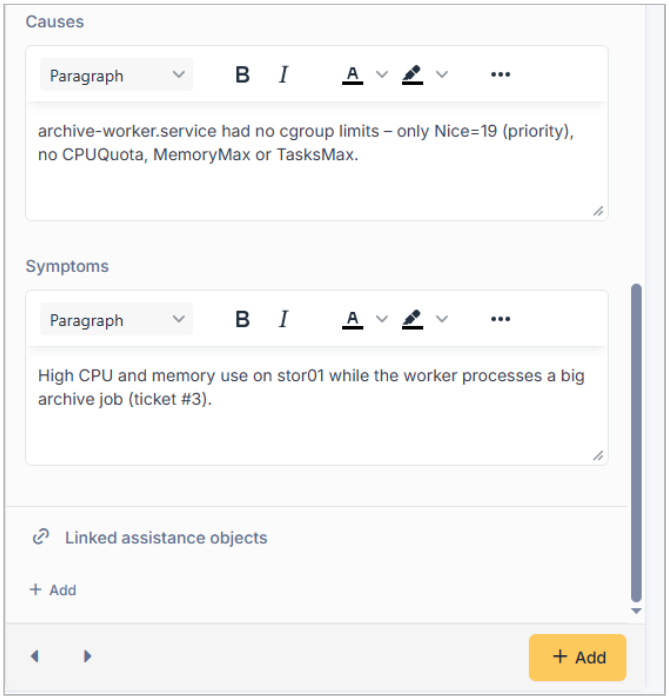
B – L3 Problem: "Archive worker on stor01 without resource limits"

An Incident is one event; a Problem asks why it happens. This Problem is the root cause behind **ticket #3** (25 Oct, archive queue, escalated L1 → L2). Assigned to group **L3-Admin**. Note: it must be created under **Assistance → Problems** – the same text under Tickets would create a ticket.



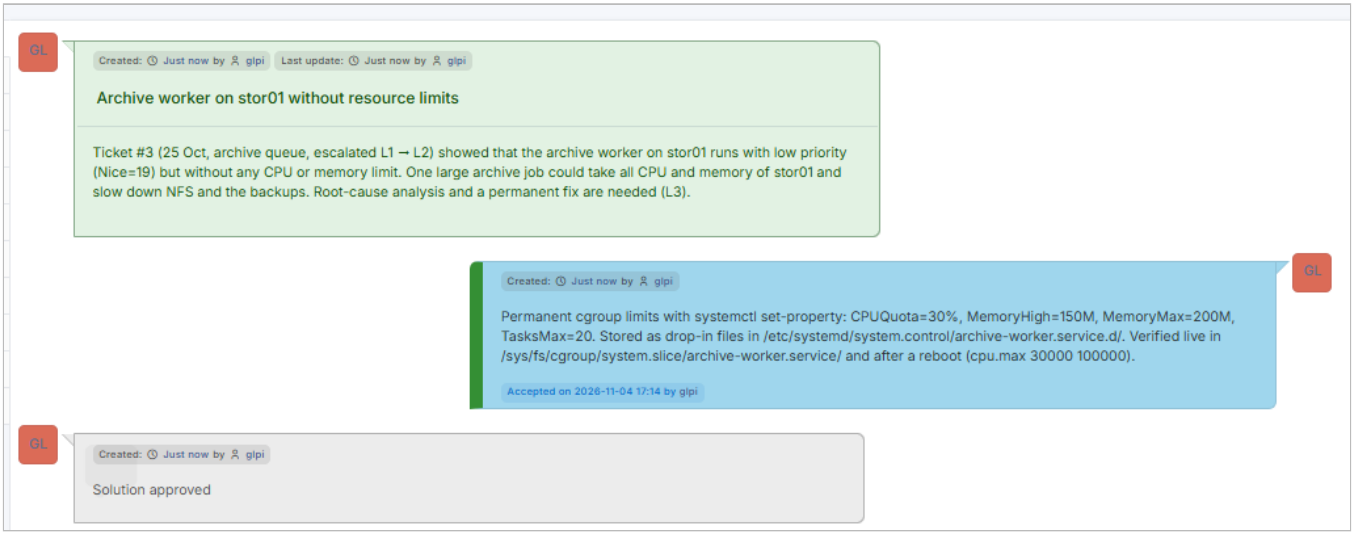
Analysis – the three boxes are part of the Problem form:





Impacts: NFS and backups could become slow · Causes: no cgroup limits, only Nice=19 · Symptoms: high CPU and memory during a big archive job.

B – Solution: the Step 4 limits – approved



The solution records exactly what was changed and how it was verified: `CPUQuota=30%`, `MemoryHigh=150M`, `MemoryMax=200M`, `TasksMax=20`, drop-in files in `/etc/systemd/system.control/archive-worker.service.d/`, checked in `/sys/fs/cgroup` and after a reboot. Ticket #3 and stor01 are linked to the Problem.

C – Snapshot "50-processes"

VMware snapshot **50-processes** on all four VMs (gw01, app01, stor01, svc01) – description *Phase 4 complete: processes, signals, limits, incident drill, L3 Problem*. A safe point to return to before Phase 5.

Final paragraph: Phase 4 ends on the service-desk side. Today's drill became Incident ticket #4: requester alice, group L2 – Linux Storage, item stor01, with the symptom (df 501M, du 56K), the diagnosis (lsof +L1, PID 5468, deleted file still open), the solution and the prevention – solved, approved and closed. Behind ticket #3 from Day 8 I opened an L3 Problem, "Archive worker on stor01 without resource limits", assigned to L3-Admin, with its analysis – impact on NFS and backups, cause no cgroup limits (only Nice=19), symptom high CPU and memory during big jobs – and the permanent solution from Step 4: CPUQuota=30%, MemoryHigh=150M, MemoryMax=200M, TasksMax=20 as drop-in files, verified after a reboot. An Incident fixes one event; a Problem removes the cause so it does not come back. A snapshot "50-processes" of all four VMs closes Phase 4.

Before Phase 5 (logging) – the 6 things from Phases 1–4 you will use again

5-minute revision · left = what you already know · right = where it comes back in Phase 5 (journald · rsyslog · logrotate)

1 systemd services — Phase 3

- every program runs as a .service unit
- systemctl status / start / restart / enable
- unit file + drop-ins in /etc/systemd/system/

→ systemd-journald and rsyslog ARE services · each service's output lands in the journal → journalctl -u nginx

2 journalctl — Phases 3–4

- journalctl -u <service> = one service's log
- journalctl -k = kernel messages (the OOM kills!)
- --since "5 min ago" · | grep -i killed · | tail -5

→ the main tool of Phase 5 – filter by service, time, priority (-p err), boot (-b), follow live (-f)

3 RAM vs disk – live vs permanent — Phase 4

- /run = RAM → gone at reboot
- /var = disk → stays
- a setting is permanent only if it is in a FILE on the disk

→ journal in /run/log/journal = lost at reboot · in /var/log/journal = kept → journald.conf Storage=persistent

4 deleted file still open — Phase 4 drill

- rm removes only the name
- a service holding its log keeps the space
- df vs du · lsof +L1 · restart the service

→ exactly why logrotate exists: rotate logs safely (copytruncate or a service reload) instead of deleting them

5 disks and space — Phases 2–3

- df -h · du -sh · LVM volumes
- disk-full alert timer from Phase 3
- /var/log can fill a disk

→ logs grow forever → logrotate (daily, rotate 7, compress) + journald limits (SystemMaxUse=)

6 network + the 4 servers — Phase 1

- gw01 .1 · app01 .10 · stor01 .20 · svc01 .40
- ports, firewall (iptables) on gw01
- SSH between the servers

→ rsyslog sends logs from app01/gw01/svc01 to stor01 = the central log server (UDP/TCP port 514)

Where Phase 5 is going – logs from every server end up on stor01

Final paragraph: Phase 5 is about logs, and it builds on six things I already know. Services (Phase 3): journald and rsyslog are services, and every service writes into the journal. journalctl (Phases 3–4): -u for one service, -k for the kernel like the OOM kills, with --since, grep and tail. RAM vs disk (Phase 4): a journal in /run is lost at reboot, one in /var/log/journal is kept – permanent only when written to the disk. The deleted-but-open file (Phase 4 drill): the reason logrotate rotates logs safely instead of deleting them. Disks and space (Phases 2–3): logs grow forever, so they need rotation, compression and size limits. Network and the four servers (Phase 1): rsyslog will send logs from gw01, app01 and svc01 over port 514 to stor01, the central log server.

Project 2 – Acme Analytics Linux Platform

Day 14 · Phase 5 starts: how Linux runs a program and reads a file

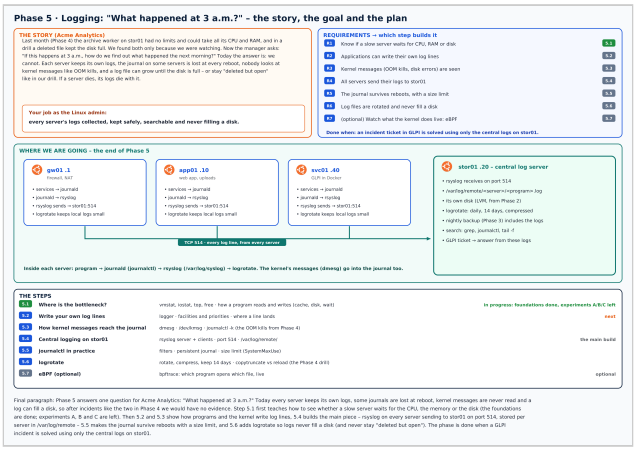
Muhammad Bilal Javed · 6–7 November 2026 · study copy

Phase	5 – Logging (journald, rsyslog, logrotate) · Step 5.1: CPU, memory or disk – where is the bottleneck?
Hosts worked on	app01
Status	Step 5.1 🟡 – foundations ✅ (user/kernel space, system calls, process cards, file tables, inodes, map cache, page cache, CPU states, vmstat columns) · warm-up on app01 ✅ · experiments A, B, C ⏳

How to read this report: every section starts with the picture, then the commands with the real output from app01 under them, then a plain-language explanation. Each picture ends with a *final paragraph* that contains all corrections. New terms are in the glossary.

---

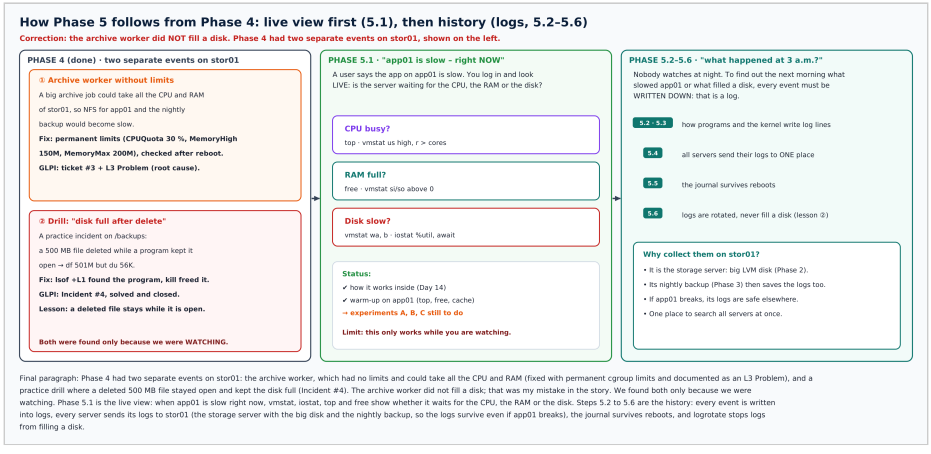
0. Phase 5 – the story, the goal and the plan



Phase 5 – story and plan

The story: Phase 4 had two separate events on stor01: the archive worker had no limits and could take all the CPU and RAM (fixed with permanent limits, ticket #3 + L3 Problem), and in a drill a deleted file kept the disk full (Incident #4). We found both only because we were watching. Now the manager asks: *"If this happens at 3 a.m., how do we find out what happened the next morning?"* Today each server keeps its own logs, some journals are lost at reboot, kernel messages are never read and a log can fill a disk. Phase 5 collects every server's logs on stor01, keeps them safely and stops them from filling a disk. It is done when a GLPI incident is solved using only the central logs.

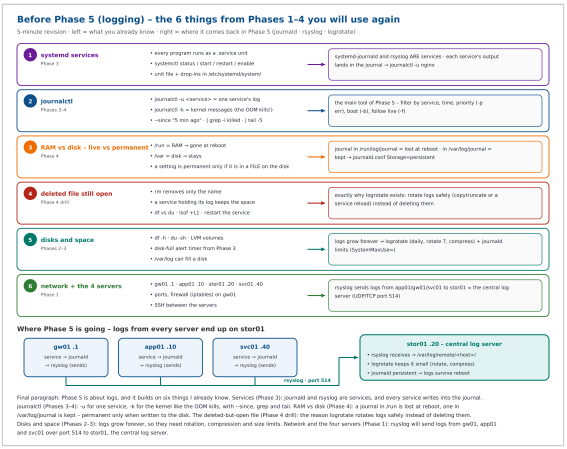
How Phase 5 follows from Phase 4



Phase 4 → Phase 5

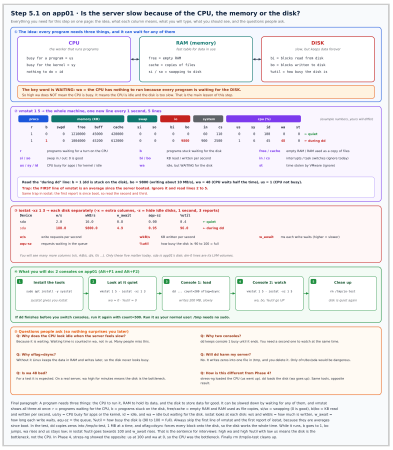
Step **5.1** is the live view: when app01 is slow *right now*, `vmstat`, `iostat`, `top` and `free` show whether it waits for the CPU, the RAM or the disk. Steps **5.2–5.6** are the history: nobody watches at 3 a.m., so every event must be written into logs, and the logs are collected on stor01 (the storage server with the big LVM disk and the nightly backup, so they survive even if app01 breaks).

Before Phase 5 – quick revision



Before Phase 5 – revision

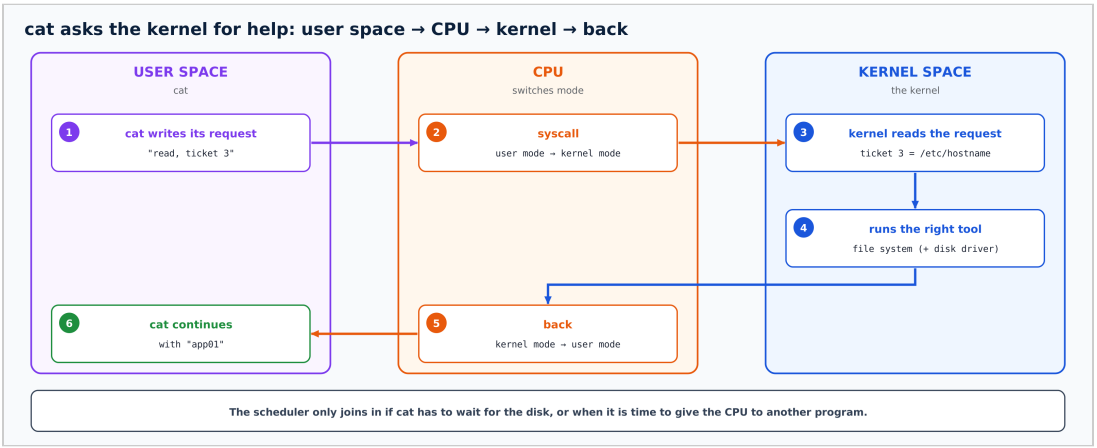
Phase 5 is about logs. Before collecting logs, Step 5.1 answers a simpler question first: **when a server is slow, which part is waiting – the CPU, the memory or the disk?** To read the answer from `vmstat` and `iostat`, you need to know what really happens inside Linux when a program runs and reads a file. That is what this day covers.



Step 5.1 on one page

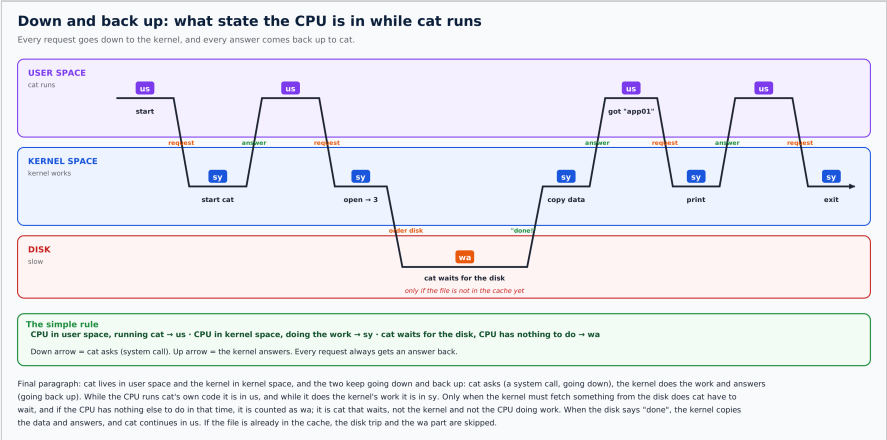
---

1. User space, kernel space and the system call



user space → CPU → kernel → back

- **User space** is where programs live, each in its own walled room in RAM (its code and its data). The CPU runs them in **user mode** – counted as `us`.
- **Kernel space** is where the kernel lives. Only the kernel may touch the disk, the network or other programs. The CPU runs it in **kernel mode** – counted as `sy`.
- A program can only ask the kernel for help through a **system call** (open, read, write, fork, exec, exit). It writes a request number and the details into the CPU's registers and runs the `syscall` instruction; the CPU switches to kernel mode, the kernel does the job and the CPU switches back.



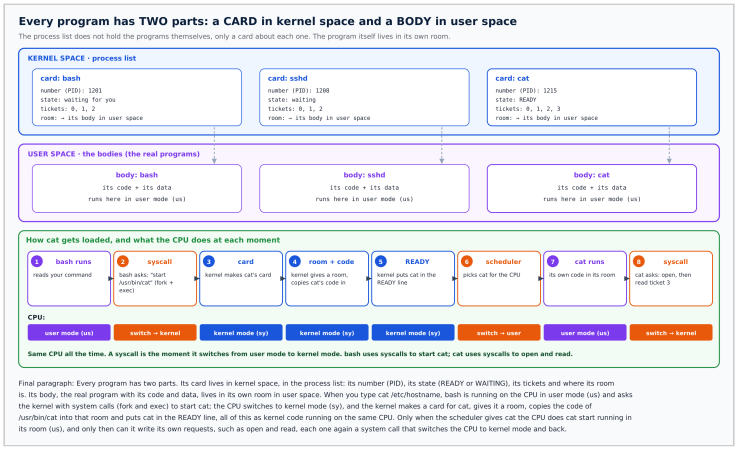
Down and back up – us, sy, wa

CPU time	Meaning
us	running a program's own code (user mode)
sy	running kernel code for it (kernel mode)
wa	idle, because a program is waiting for the disk
id	idle, nothing to do

Waiting is **never** counted as us.

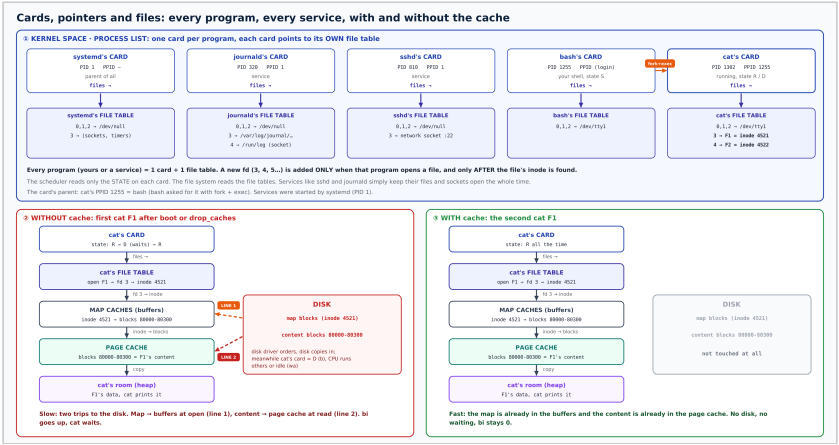
---

2. Every program = a card in kernel space + a body in user space



Card and body

- When you type `cat F1`, **bash** asks the kernel with `fork` + `exec`. `fork` makes a **new card** (a new PID, parent = bash); `exec` puts cat's code into its new room. bash keeps its own card and waits.
- The card (real name: `task_struct`) holds the **PID**, the **parent PID**, the **state** and a pointer to the program's **file table** (fds 0, 1, 2 = keyboard, screen, errors).
- States: **R** = can run · **S** = sleeping (e.g. waiting for typing) · **D** = waiting for the disk.
- Every program and every service (systemd, journald, sshd, bash, cat) has its own card and its own file table.



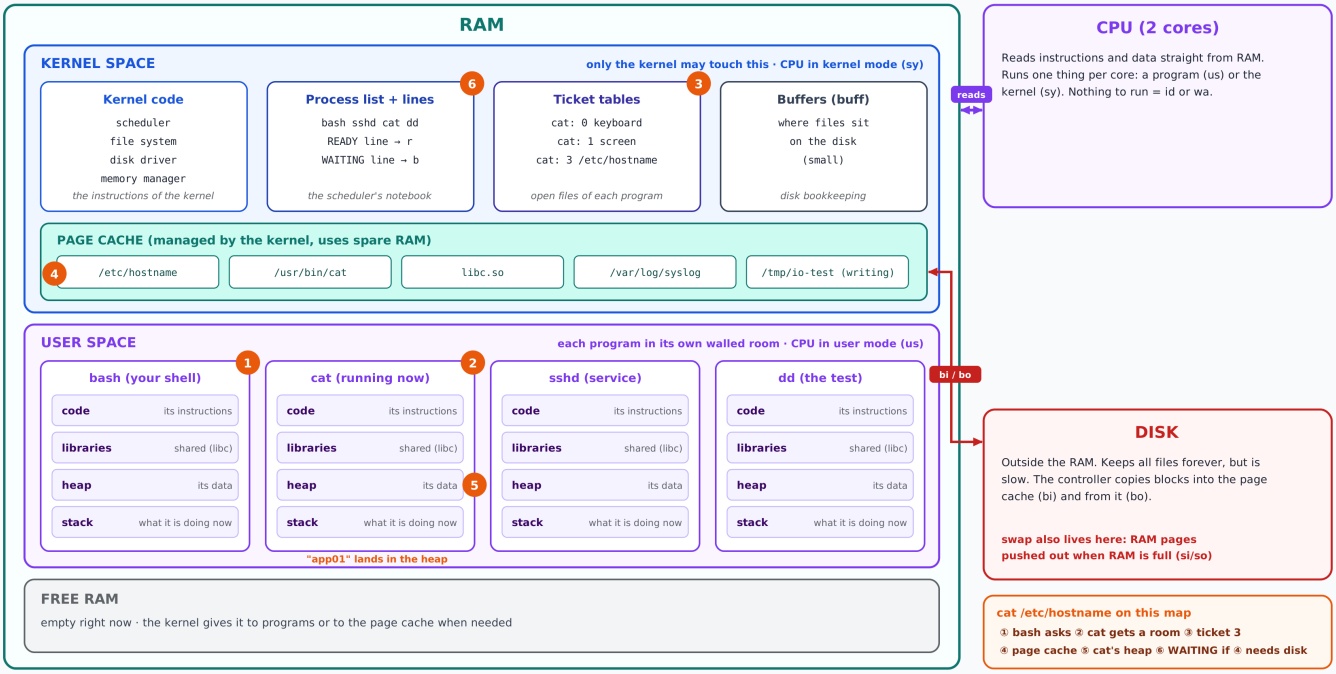
Cards, pointers and files – with and without the cache

---

3. Where everything lives in RAM

The RAM of app01 (about 1.9 GB): where everything lives

One map for all the words: kernel, scheduler, page cache, programs, tickets, free RAM. The CPU reads all of it directly. The disk is outside.



How this map shows up in free -h and vmstat

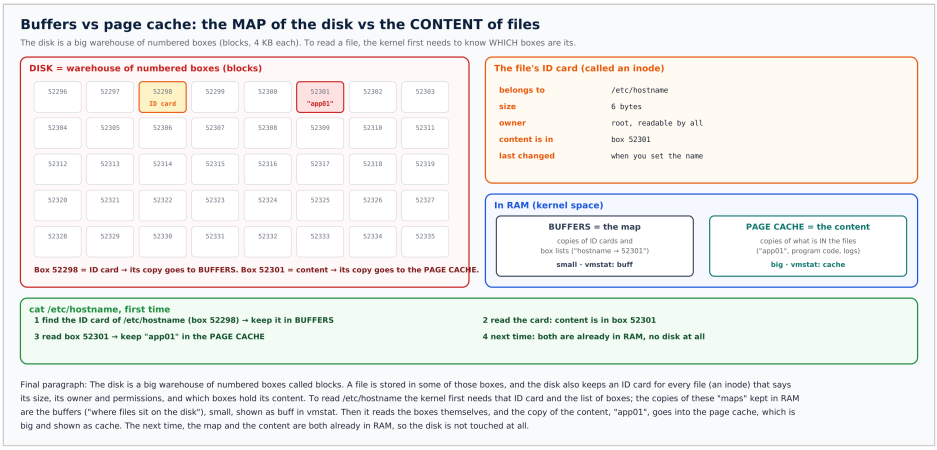
`free -h: used = kernel + programs · buff/cache = buffers + page cache · free = free RAM · available = free + cache that can be dropped`

`vmstat: buff = buffers · cache = page cache · free = free RAM · r / b = the scheduler's two lines · si / so = swap on the disk`

`Low "free" is normal: Linux fills spare RAM with cache on purpose. Look at "available".`

Final paragraph: The RAM of app01 has three parts. Kernel space, at the top, belongs to the kernel alone: it holds the kernel's code (the scheduler, the file system, the disk driver and the memory manager), the scheduler's notebook (the list of all processes with the READY line, r, and the WAITING line, b), the ticket tables (which files each program has open, like cat's ticket 3 for /etc/hostname) and the small buffers. The page cache is spare RAM that the kernel fills with copies of files, such as /etc/hostname, /usr/bin/cat and libc. User space holds every program in its own walled room, with its code, the shared libraries, its heap (its data) and its stack. The rest is free RAM. The CPU reads all of it straight from RAM, in user mode for program code (us) and in kernel mode for kernel code (sy). The disk is outside the RAM; its controller copies blocks into and out of the page cache (bi and bo), and swap is the part of the disk that holds RAM pages pushed out when RAM is full (si and so). For cat /etc/hostname: first bash, in its own room, asks the kernel to start cat; then cat gets a room, the kernel writes ticket 3 into cat's ticket table, the kernel finds the copy in the page cache (or fetches it from the disk while the scheduler puts cat in the WAITING line), and then copies "app01" into cat's heap. In free -h, buff/cache is the page cache plus buffers, and "available" is the number to watch, because the cache can be dropped whenever programs need RAM.

The RAM of app01

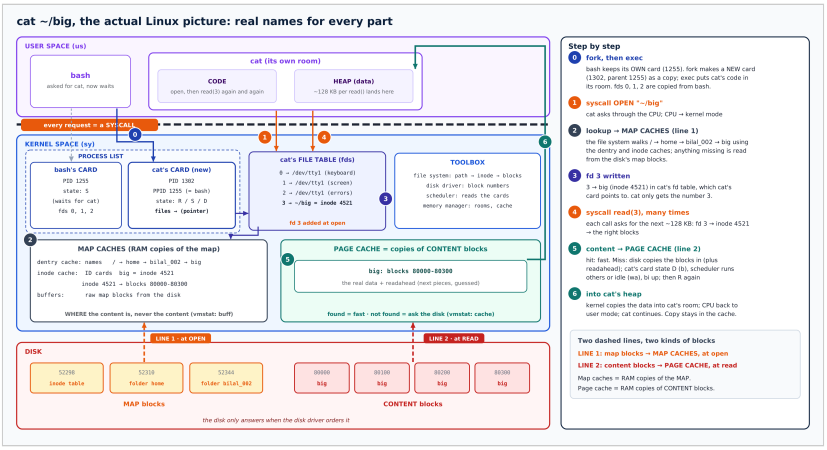


Buffers vs page cache

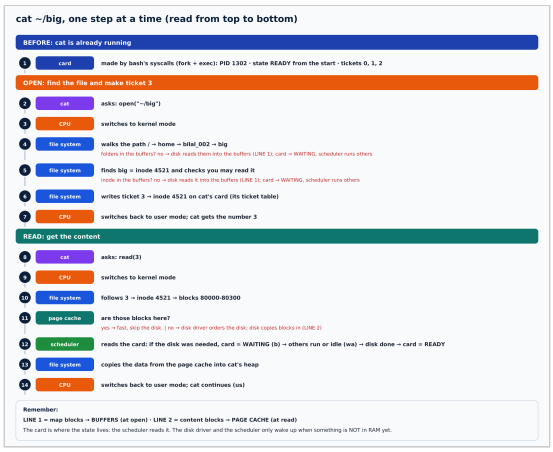
- **Map cache** (buffers, inode cache, dentry cache): RAM copies of the *map* – folder entries (name → inode number) and inodes (owner, size, **list of content blocks**). It holds **where** the content is, never the content.
- **Page cache**: RAM copies of file **content**.
- The disk keeps everything for good; the **disk driver** only fetches what RAM is missing.

---

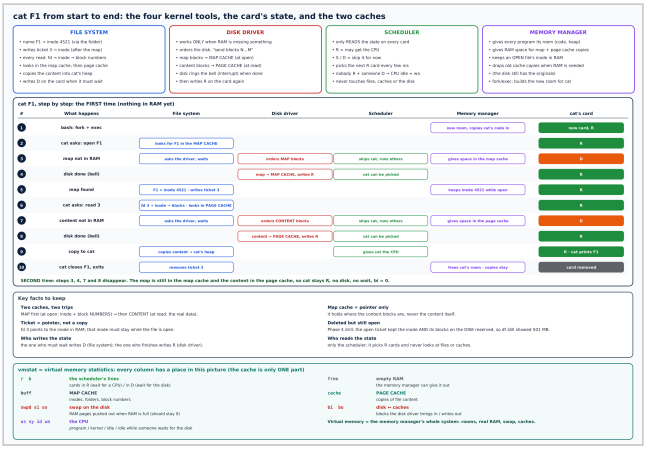
4. `cat F1` from start to end – who does what



Open and read – the actual Linux picture



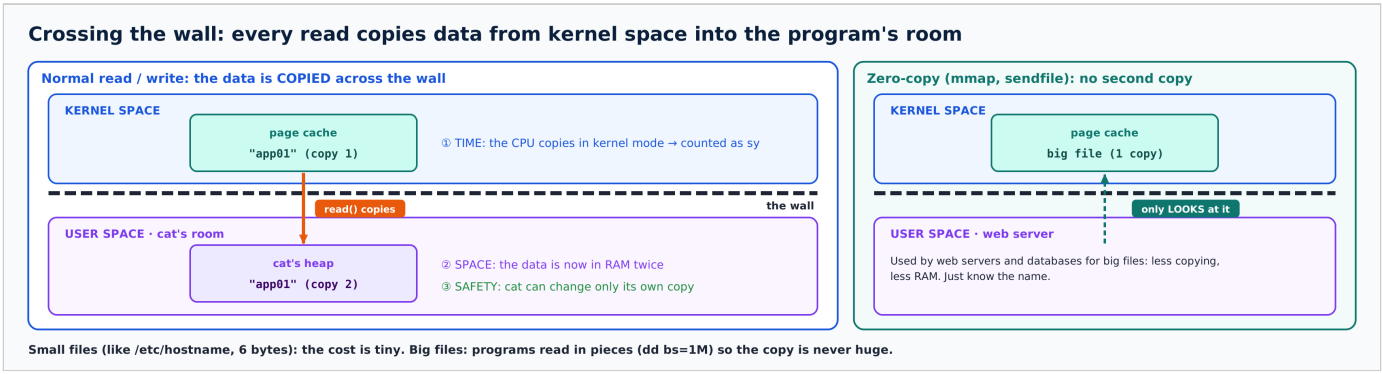
cat F1 step by step



The four kernel tools, the card's state and vmstat

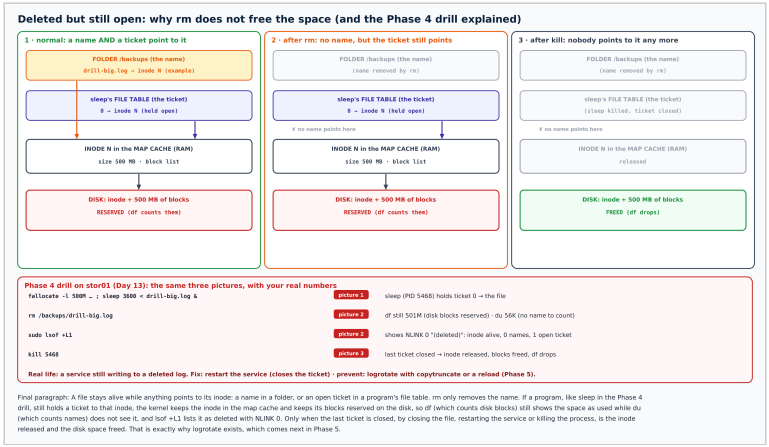
1. **open F1** – the file system looks up the map: folder → `F1 = inode 4521` → inode with the block list. Missing map blocks are fetched from the disk into the map cache (**line 1**).
2. Only **after** the inode is found is the ticket written: `3 → inode 4521` in cat's file table.
3. **read(3)** – fd 3 → inode → block numbers → page cache. Missing content blocks are fetched from the disk into the page cache (**line 2**).
4. While the disk works, the file system writes **D** on cat's card; the scheduler (which only reads the states) runs other programs or the CPU is idle (`wa`). When the disk rings its bell, the disk driver writes **R** again.
5. The content is copied from the page cache into cat's heap and cat continues. The second time, everything is already in RAM: no disk, no wait.

Every read and write crosses the wall as a **copy** – a little `sy` time and the data sits in RAM twice, but each program can only change its own copy:



Crossing the wall

5. Link to Phase 4: deleted but still open



Deleted but still open – explained

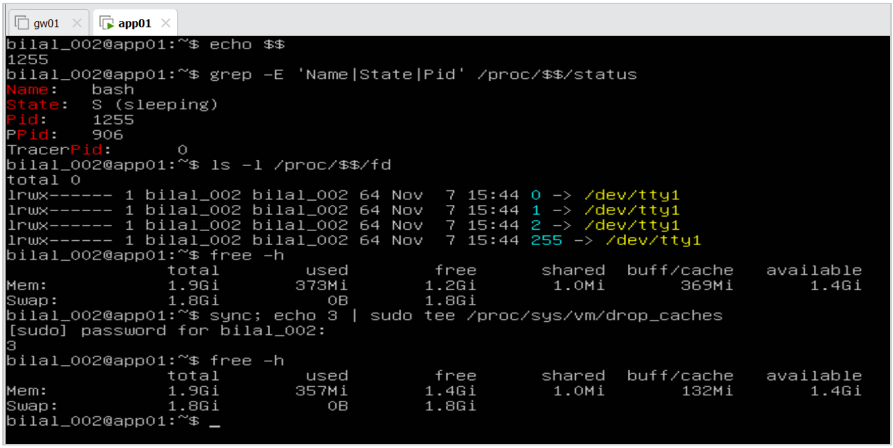
This explains the **first incident drill of Phase 4** (Day 13, section 2):

Drill step on stor01	What happened inside
`sleep 3600 < /backups/drill-big.log &`	sleep's file table holds a ticket (fd 0) to the file's inode
`rm /backups/drill-big.log`	only the **name** is removed; the ticket still points to the inode
`df` 501M · `du` 56K	the inode stays in the map cache and its blocks stay **reserved on the disk**; `du` counts names, `df` counts disk blocks
`lsof +L1` → NLINK 0 "(deleted)"	the inode is alive with 0 names and 1 open ticket
`kill 5468`	the last ticket is closed → inode released, blocks freed, `df` drops

In real life this is a service still writing to a deleted log – and the reason **logrotate** exists (later in Phase 5).

6. Warm-up on app01 – seeing it live

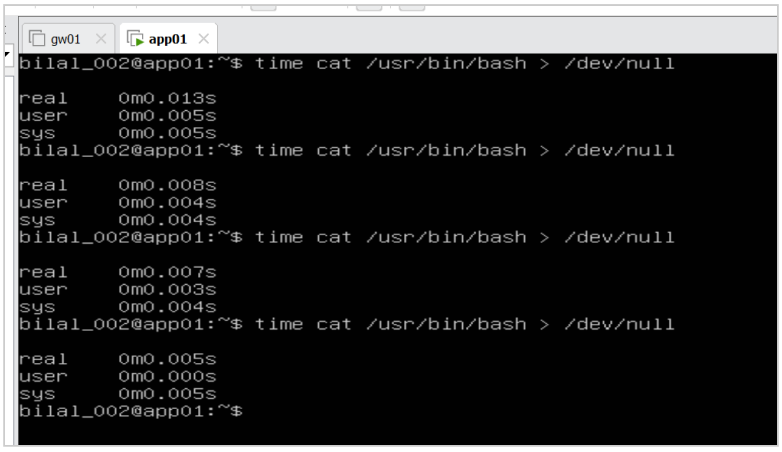
```
echo $$
grep -E 'Name|State|Pid' /proc/$$/status
ls -l /proc/$$/fd
free -h
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
free -h
```



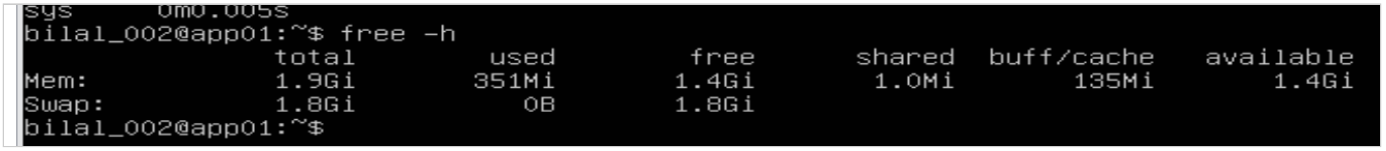
bash's card, tickets, cache emptied

bash = PID 1255, state S (waiting for typing), parent 906 (the login program); tickets 0, 1, 2 point to the console `/dev/tty1` (255 is bash's own extra ticket). Dropping the caches took buff/cache from **369 Mi to 132 Mi** – the rest is in use by running programs.

```
time cat /usr/bin/bash > /dev/null     # 4 times
free -h
```



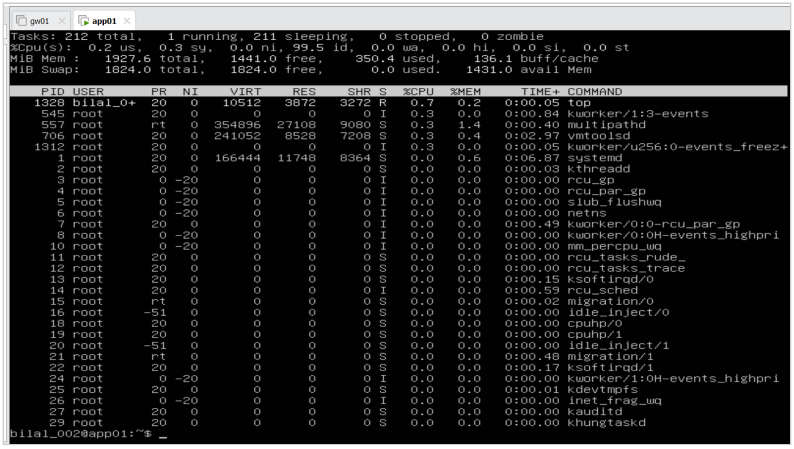
time cat – first from disk, then from cache



buff/cache grew again

First read **13 ms** (from the disk), then 8, 7, **5 ms** (from the page cache); buff/cache grew from 132 to 135 Mi. The gap is small because the file is small and parts of bash were still in RAM.

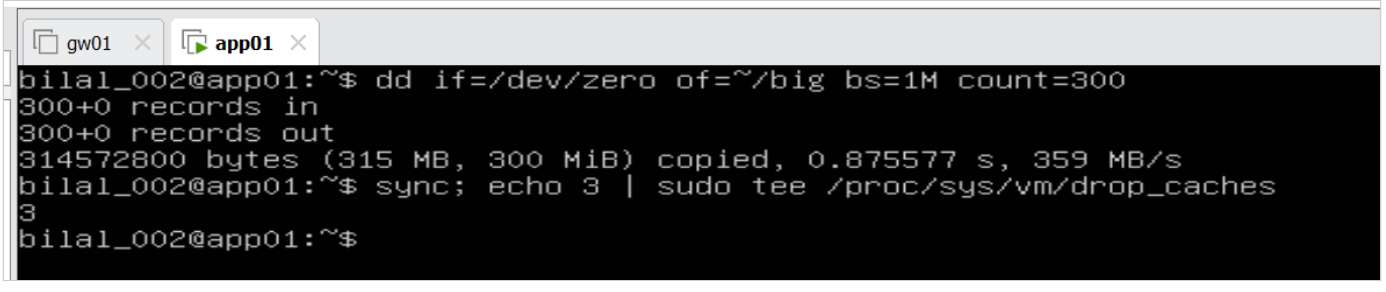
```
top
```



top – 212 cards, 1 running

212 tasks: 1 running, 211 sleeping – 212 cards in the process list (systemd PID 1, vmtoolsd, multipathd, many kernel `kworker` helpers); CPU **99.5 % id, 0.0 wa**: nobody ready, nobody waiting for the disk.

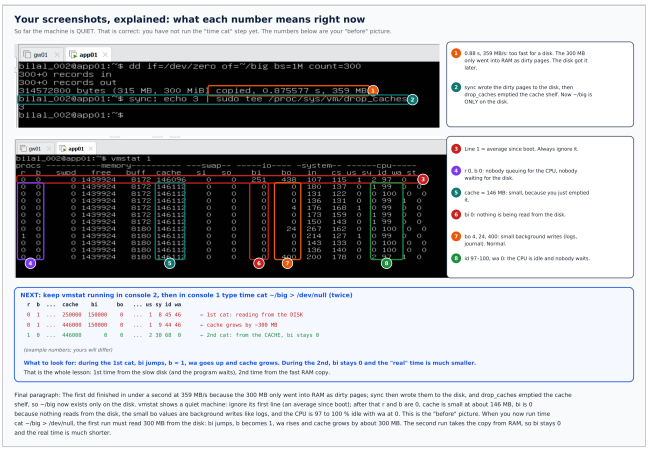
```
dd if=/dev/zero of=~/big bs=1M count=300
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
vmstat 1          # second console
```



dd 300 MB in 0.88 s

```
bilal_002@app01:~$ vmstat 1
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 0  0      0 1439924   8172 146096    0    0   251   438  107  115  1  2 97  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  180  137  0  1 99  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  131  122  0  0 100  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  136  131  0  0 99  1  0
 0  0      0 1439924   8172 146112    0    0     0     4  176  168  1  0 99  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  173  159  0  1 99  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  150  143  0  1 99  0  0
 0  0      0 1439924   8180 146112    0    0     0    24  267  162  0  0 100  0  0
 1  0      0 1439924   8180 146112    0    0     0     0  214  127  1  0 99  0  0
 0  0      0 1439924   8180 146112    0    0     0     0  143  133  0  0 100  0  0
 0  0      0 1439924   8180 146112    0    0     0     0  136  140  0  0 100  0  0
 0  0      0 1439924   8180 146112    0    0     0   400  200  178  0  2 97  1  0
```

vmstat – the quiet “before” picture



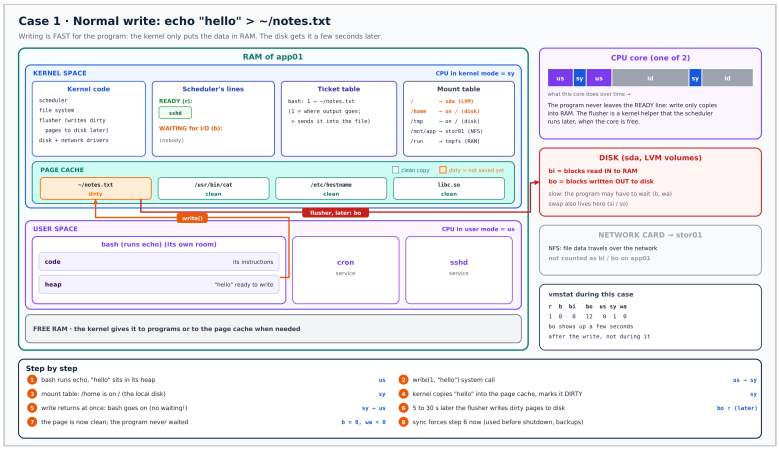
The screenshots explained

dd wrote 300 MB in 0.88 s (359 MB/s) – only into RAM as dirty pages; `sync` wrote them to the disk and `drop_caches` emptied the cache. vmstat shows the quiet “before” picture: skip line 1 (average since boot), then r 0, b 0, bi 0, small bo from background logs, id 97–100, wa 0.

---

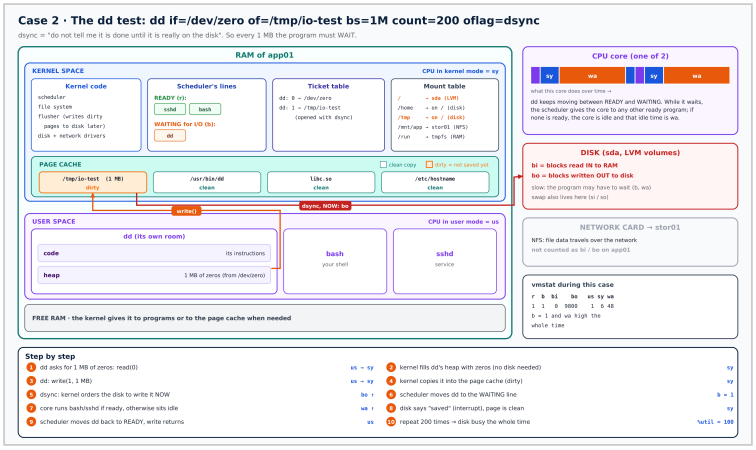
7. Writing, compressing and mounts on the same map

Case 1 – normal write: RAM first (dirty pages), disk later (`bo` later).



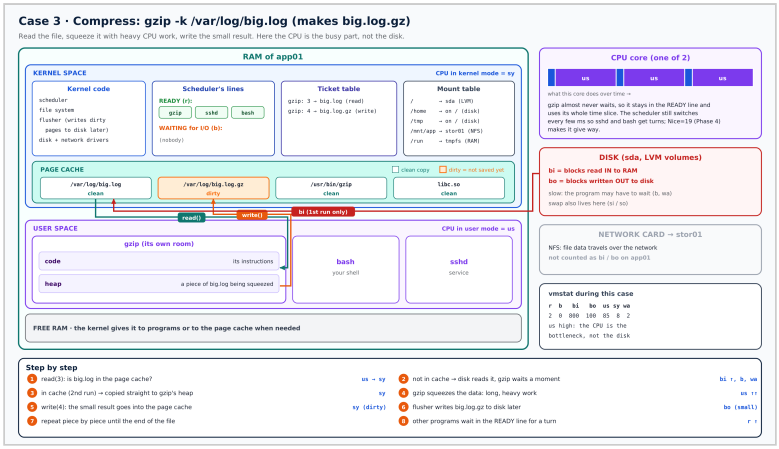
Case 1 – normal write

Case 2 – the dd test: `oflag=dsync` forces every MB to the disk, so dd waits (`b`, `wa`, `bo`).



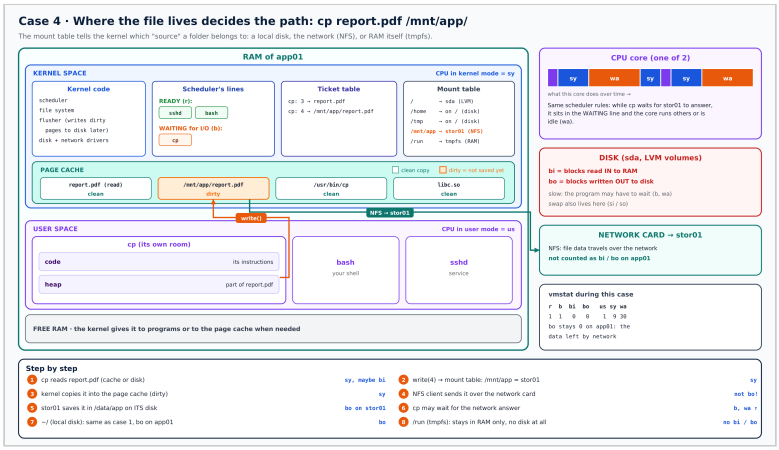
Case 2 – the dd test

Case 3 – compress: the CPU works (us high, r up), the disk is quiet.



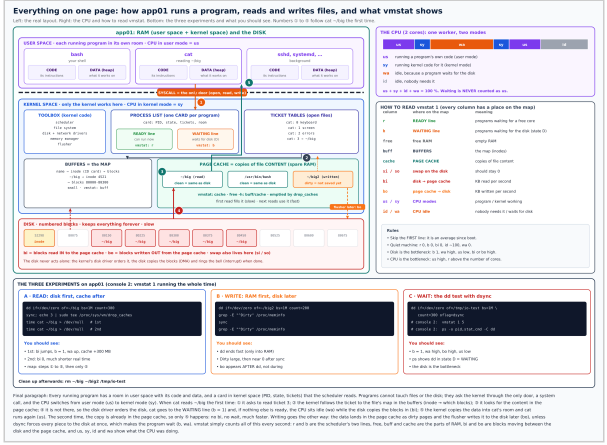
Case 3 – compress

Case 4 – mounts: the folder decides the path – local disk, NFS to stor01 (bo shows on stor01), or tmpfs (RAM only).



Case 4 – mounts

All four cases and the vmstat columns on one page:



Everything on one page

8. Next – Step 5.1 experiments

With `vmstat 1` running in a second console on app01:

- **A · read:** `time cat ~/big > /dev/null` twice → 1st: `bi` up, `b` = 1, `wa` up, cache +300 MB · 2nd: `bi` 0, much faster.
- **B · write:** `dd` without dsync, `grep ^Dirty /proc/meminfo`, `sync` → dirty pages first, `bo` later.
- **C · wait:** `dd … oflag=dsync` with `vmstat 1 5`, `iostat -xz 1 5` and `ps -o pid,stat,cmd -C dd` → `b` = 1, `wa` high, dd in state **D**.

Clean up afterwards: `rm ~/big ~/big2 /tmp/io-test`.

Final understanding (one paragraph)

Every running program has a body in user space (its code and data) and a card in kernel space (PID, parent, state R/S/D, and a pointer to its own file table); bash makes cat's card with the system calls fork and exec, and every request after that also goes through a system call that switches the CPU from user mode (us) to kernel mode (sy). When cat opens F1, the file system first finds the map – the folder entry (name → inode number) and the inode (owner, size, block list) – from the map cache, or from the disk if it is missing, and only then writes ticket 3 → inode in cat's file table. When cat reads 3, the file system follows the inode to the block numbers and takes the content from the page cache, or, if it is missing, the disk driver fetches the content blocks into the page cache while the file system marks cat's card D, the scheduler (which only reads the states) runs other programs or leaves the CPU idle (wa), and the disk driver marks the card R again when the disk rings its bell; finally the content is copied into cat's heap. The second time, everything is already in RAM. Writes go the other way: into the page cache as dirty pages, to the disk later (bo), unless dsync forces them at once and makes the program wait. A file stays alive while any name or any open ticket points to its inode – which is why, in the Phase 4 drill, the deleted log still filled the disk until the process holding it was killed. vmstat simply counts all of this: r and b are the scheduler's lines, free, buff and cache are parts of RAM, si and so are swap, bi and bo are blocks moving between the disk and the caches, and us, sy, id and wa show what the CPU was doing.